

Detailed robotics programme: M1-M24

This programme consists of 24 modules, each 15 pages long. This volume brings them together in 365 pages, with a contents list and PDF bookmarks for navigation. The practical work focuses on measuring, calculating, designing and testing robotic systems.

The plan develops the topics and goals of M1-M24 in the supplied roadmap.pdf. Every module specifies assigned reading, essential concepts, equipment and software. Worked mechanics examples connect the theory to leg, body and finger motion, and to robot locomotion.

Start with sensor data and a single joint. Progress through mathematics, electronics, leg mechanics, control, ROS and simulation. After perception and autonomous behaviour, build a quadruped and extend the tested design into a hexapod. The hand examples teach shared principles; building the hand and working with tactile sensors belong to the separate OPT stage.

How to use this edition

Before each module, check the equipment and software list: identify what you already have, what the experiments require, and how to verify that the environment is ready. Read the robot motion fundamentals on page 15; in M1, the introduction also appears on page 13. Explain the concepts in your own words and reproduce the calculation. Each reading list identifies the relevant sections. This work is included in the module's 75 hours.

Use PDF bookmarks to move between sections. The ZIP archive contains 24 separate documents, M01-M24. The contents tables give the original schedule dates and each module's starting page in this volume. This English edition retains the detailed tasks, worked examples, references, definitions and equipment stages of the edited Russian edition.

Study hours

Programme component	Hours	Included work
24 modules at 75 hours each	1,800	Theory, practical exercises, verification and recording the results.
G1-G4 at 35 hours each	140	Subsystem integration, fault correction and readiness checks before the next stage.
OPT after M17	120	Building the robotic hand and working with tactile sensors.
Complete programme	2,060	11 Oct 2026-22 Dec 2028; the original scheduling rules are retained.

These documents provide the detailed plans for M1-M24. The G1-G4 checkpoints and the OPT hand project have separate time allocations. Completing the full programme requires the modules and these additional stages.

What counts as progress

Support each learning outcome with working software, a calculation, a measurement or a repeatable experiment. The stated criteria guide the learning process; verify the allowable loads and operating conditions of the actual hardware separately. Record simulation results and physical robot tests as distinct forms of evidence.

Contents: M1-M12

Module start and end dates follow the supplied roadmap. The final column gives the starting page in this volume. The module itself explains the sequence of study sessions and practical work.

M	Module	Dates	Page
M1	Robot software foundations	11 Oct 2026 03 Nov 2026	6
M2	Applied mathematics: coordinates, calibration and motion	03 Nov 2026 29 Nov 2026	21
M3	Probability, statistics and optimisation for robotics	29 Nov 2026 27 Dec 2026	36
M4	Electronics from first principles and robot signal processing	27 Dec 2026 31 Jan 2027	51
M5	CAD, mechanics and manufacture of a three-joint leg	31 Jan 2027 24 Feb 2027	66
M6	Embedded controller, buses, timing and fault tolerance	25 Feb 2027 22 Mar 2027	81
M7	Actuators, joint control and system identification	04 Apr 2027 30 Apr 2027	96
M8	Robot geometry: forward and inverse kinematics	02 May 2027 03 Jun 2027	111
M9	Rigid-body dynamics and physical modelling	03 Jun 2027 06 Jul 2027	126
M10	Classical control: digital controllers and frequency analysis	07 Jul 2027 08 Aug 2027	141
M11	State space, estimation and constrained optimisation	08 Aug 2027 12 Sep 2027	156
M12	ROS 2: architecture, hardware interfaces and diagnostics	12 Sep 2027 14 Oct 2027	171

Checkpoint G1 follows M6, and G2 follows M12. Before moving on, verify the required outputs from earlier work. Resolve faults in the essential software or test bench before building on it.

Contents: M13-M24

Module start and end dates follow the supplied roadmap. The final column gives the starting page in this volume. The module itself explains the sequence of study sessions and practical work.

M	Module	Dates	Page
M13	Simulation, digital twins and hardware-in-the-loop testing	26 Oct 2027 21 Nov 2027	186
M14	Computer vision and three-dimensional perception	21 Nov 2027 19 Dec 2027	201
M15	Localisation, mapping and SLAM	19 Dec 2027 20 Jan 2028	216
M16	Motion planning, decision-making and autonomous behaviour	21 Jan 2028 15 Feb 2028	231
M17	Manipulation, human-robot interaction and industrial applications	15 Feb 2028 19 Mar 2028	246
M18	Machine-learning fundamentals for robotics	30 Apr 2028 04 Jun 2028	261
M19	Walking robots: state estimation, gait and footholds	04 Jun 2028 06 Jul 2028	276
M20	Support-force and whole-body motion control	07 Jul 2028 31 Jul 2028	291
M21	Robot learning: imitation, RL and transfer from simulation	01 Aug 2028 03 Sep 2028	306
M22	Quadruped robot: assembly, calibration and initial trials	03 Sep 2028 01 Oct 2028	321
M23	Six-legged robot: scaling, terrain and reliability	01 Oct 2028 01 Nov 2028	336
M24	Systems engineering, final verification and project release	02 Nov 2028 27 Nov 2028	351

The separate 120-hour OPT stage follows M17. M22 builds the quadruped; M23 develops a hexapod from the tested design. The programme concludes with checkpoints G3 and G4.

Schedule and readiness to progress

Organising study sessions

The original schedule allocates 2 hours each weekday, Monday to Friday, and 10 hours of practical work on Sunday; Saturday is free. During a holiday block, only its first Sunday remains a study day. Breaks are excluded from study time. In M1, one of the programme's 12 additional hours is assigned to 2 November 2026.

The remaining modules use three 20-hour blocks and a final 15-hour block: 35 hours of study and analysis, and 40 hours of practice. Complete these blocks in sequence on the available days, allowing for holidays. They need not coincide with four calendar weeks.

One module may end on the day the next begins. Divide that day's available time between them; count each hour only once. The cover pages retain the original dates. This edition does not replace the roadmap with a new hour-by-hour calendar. Allocate additional hours and Sunday sessions according to the overall plan without counting them twice.

Checkpoints

Stage	Dates	Main readiness question
G1	22 Mar-04 Apr 2027	Have electronics, mechanics, measurements and fault responses been checked before connecting the actuator?
G2	15-25 Oct 2027	Are the leg model, controller and ROS interfaces consistent, and has their integrated operation been demonstrated?
G3	27 Nov-10 Dec 2028	Have autonomous behaviour, perception, data, trained models and their intended operating conditions been evaluated?
G4	10-22 Dec 2028	Have all requirements been met, and can the final result be reproduced?

The source plan contains two sequencing mismatches. G2 requires HIL tests that are developed later in M13; that requirement remains open until the tests are completed. G3 appears after the AI modules and serves as a final audit, so each model must be evaluated before it is used. The decision to buy a Jetson or hardware for learned locomotion belongs to G3. Successful simulation alone does not establish readiness for that purchase.

What to retain at the end of each module

Save the versions of code, CAD models and configurations, raw data with units, and commands for repeating experiments. Mark completed requirements and unresolved issues. For hardware, record power settings, loads and fault responses. If parts have not arrived, continue modelling and preparing documentation, and mark the corresponding hardware tests as pending.

Robot motion fundamentals: learning sequence

The sections on robot motion connect the individual subjects into an understanding of the complete robot. In M2-M24, the model and exercise appear on page 15. M1 introduces the foundations on pages 13 and 15. The reading and calculations are included in each module's study hours.

Stage	Essential understanding	Evidence of understanding
M1-M3	Links, joints, configuration and state; vectors, coordinates, derivatives and measurement error.	Draw a mechanism, calculate a point's position and velocity, and assess the effect of sensor error.
M4-M7	Motion sampled at fixed intervals; statics, torques, transmissions and actuator power.	Explain the sampling-rate choice, calculate the torque under load, and estimate heating.
M8	Forward and inverse kinematics, coordinate frames, Jacobians, singularities and forces.	Calculate foot position from joint angles, solve IK, and verify velocities and work.
M9	Inertia, energy, Newton-Euler and Lagrangian methods, forward and inverse dynamics, and contact.	Derive and verify equations of motion; explain forces and accelerations.
M10-M13	Feedback, state, stability, physical models and model validation.	Trace how a command produces measured motion, and validate the model.
M14-M17	Perception geometry, body motion, trajectories, finger contact and grasping.	Express a target in the robot frame, calculate a trajectory, and verify a grasp.
M18-M21	Identification, centre of mass, support forces, floating-base dynamics and learning.	Explain balance, feasible forces and the physical meaning of a learned model's commands.
M22-M24	Measured loads, support transitions, hexapod locomotion and verification of calculations.	Compare predictions with measurements and explain the model's limits.

Practical progression: from one link to a step

Begin with a link whose base is fixed. Calculate leg position from joint angles, link velocities and the acting forces. Account for inertia and prescribe motion over time. Then introduce ground contact, body movement and changes of support. Apply the same principles to a finger, adding checks of grasp forces and transmission behaviour.

When to move on

Before the next topic, check that you understand the notation, coordinate frames, units and modelling assumptions. Solve a simple example by hand, then verify it with software or an experiment. Assess separately whether the target pose is reachable, whether the actuator can supply the required torque, and how accurately the controller follows the command.

M1 Robot software foundations

M1 • 11 Oct 2026 - 03 Nov 2026 • 75 hours

Python, C++ and Rust: a body, one joint, observation and fault investigation

Deliverable: telemetry-core v1.0. One repository with a shared IMU and joint data structure, three command-line utilities (CLIs), a Python analyser and a main C++ processor. A generator produces data; the test bench displays roll and joint-angle error, detects invalid rows and interrupted transmission, and saves a log for repeating the experiment. This prepares you for legs and fingers; actuators and physics simulation are not yet needed.

Block	Dates in 2026	Analysis	Practice	Deliverable
1	11-16 October	10 h	10 h	Environment, snapshot and Python
2	18-23 October	10 h	10 h	C++ and logger
3	25-30 October	10 h	10 h	Faults, tests and Rust
4	1-3 November	5 h	10 h	Streaming, timeout and replay
Total	M1	35 h	40 h	75 hours

Calendar. Four Sundays of 10 h and 17 weekdays of 2 h give 74 h. One of the programme's 12 additional hours is assigned to **Monday 2 November: 3 h**; the source does not date this hour. Saturdays are free. In the source, M2 also starts on 3 November, with no boundary specified by hour; its calendar is not recalculated. Detailed dates and tasks are on pp. 11-12.

Work rhythm and scope

Weekdays: 120 min - restore context 10, read 45, work through an example 35, explain 20, record results 10. Sunday: five 2 h blocks - readiness criterion 10, implementation 90, verification 20 min. Breaks of 15-30 min and lunch are outside the 75 h. Reading, glossary work and setup are included.

Keep one task on a board labelled 'Next / In progress / Verified'. Log the date, minutes, scenario, random-number generator seed, axes, units, commit, parameters and command. After 30 min without progress, simplify the example; after 60 min, describe the problem and change sources. Update README at each block's end. The main processor is C++; cut extensions before core tests.

Acceptance checks: evidence to save

Provide an axes and joint-zero diagram, rest and 15° roll recordings, q_{ref} and q plots, and a NO_RESPONSE event without guessing its cause. Check all three CLIs and count gaps, rejected and dropped separately. The watchdog must trigger during silence; duplicates must not refresh its timer, and FAULT must not clear itself. Include raw-recording replay; GDB, ASan/UBSan, and TSan or its limitations; a clean Ubuntu build; timing measurements, CLI medians, p50/p95/p99; queue sizes and deliberately slow logging.

Save six scenarios, expected results calculated by hand, plots, events, a report and commands. Total your hours and identify three skills, two mistakes and unfinished tasks; repeat the experiment independently after AI assistance. Multiple joints will require names and aligned timing. Balance, kinematics, state estimation, physical constraints and torque control come later.

Motion fundamentals are compulsory from M1: link, joint, degree of freedom, configuration and state; the distinction between kinematics, dynamics and control. Explanations are on p. 13 and the first mechanism calculation on p. 15. It belongs to the 'axes, units, RobotState' block on 11 October.

A body and one-joint test bench

A shared state snapshot: from reading fields to observing a robot

Mechanism and coordinates

The model has a body with an IMU and one revolute joint, such as a leg's knee or a finger joint. Use the IMU accelerometer readings, namely [specific force](#). Its axes align with the [body axes](#): x forwards, y left, z upwards, forming a right-handed frame. Measure specific force in m/s², joint angle in radians and current in amperes. Draw and label the joint zero and positive direction in README.

A snapshot contains one generator step's data. All fields share the timestamp `t_us`; this is a test-bench simplification. Real IMUs and encoders may have different rates and clocks. Here, current `i_a` is merely a recorded signal; [it is not actuator torque](#). `q_ref_rad` is the requested angle and `q_rad` the encoder measurement.

```
seq,t_us,ax,ay,az,q_ref_rad,q_rad,i_a
0,0,0.0,0.0,9.81,0.0,0.0,0.2
1,10000,0.0,0.0,9.81,0.01,0.008,0.2
```

Data and error contract

Use unquoted UTF-8 CSV with the exact header and 8 fields. `seq` and `t_us` are ASCII integers in $0 \dots 2^{63}-1$. Other fields are finite decimal numbers; signs, decimal points and exponents are allowed. Reject hex, underscores and NaN/Inf. Trim surrounding whitespace and require complete parsing. Both angles of the teaching joint are limited to $[-1; 1]$ rad.

For accepted snapshots, `seq` and `t_us` increase strictly. An invalid row does not update the accepted state. A `seq` gap is a diagnostic event, not a reason to reject the next snapshot. Resetting the counter requires a new recording; M1 does not require automatic reset detection.

Check	Result
Header, file	Stop with code 2 if the header is wrong or the file cannot be read.
Row or fields	Reject as blank, field_count, number, nonfinite, joint_range, sequence or timestamp, in that priority order.
CLI result	Code 0: all rows valid; 1: some rejected. JSON to stdout, reasons to stderr.
Summary	rows_total, accepted, rejected, errors_by_reason; first/last_t_us; mean_ax/ay/az; max_abs_joint_error. Empty metrics: null.

Experiment set

Create separate files of 1 000 snapshots at 10 000 µs intervals: **rest; static 15° roll; smooth joint motion; no response after seq=200; seq=40...49 gap; bad rows**. Use a separate streaming test with a pause to test real clock progression. Verification files contain no noise; noise with seed=42 is an additional experiment.

All language implementations read the same files. Calculate expected results for small experiments by hand. Integers and error reasons must match exactly; metrics use an absolute tolerance of 10^{-9} for these data. Shared layout: `python/`, `cpp/`, `rust/`, `scenarios/`, `fixtures/`, `tests/`, `reports/`.

Linux and reproducible experiments

Learn to run and investigate a robotics experiment

Topics to study

- **Files and the shell.** Paths, working directory, permissions, environment variables, quoting, stdin/stdout/stderr, redirection, pipes and exit codes. Commands: pwd, ls, cd, mkdir, cp, mv, less, head, tail, rg, chmod, env.
- **Processes.** PID, foreground and background execution, ps, top, SIGINT/SIGTERM. Learn how to stop the generator while allowing the logger to close its file, and why kill -9 prevents normal cleanup.
- **Python environment.** Interpreter, virtual environment, pyproject.toml, [uv.lock](#), uv run and dependency restoration. The project needs numpy, matplotlib and pytest.
- **Git.** Index, commit, diff, branch and merge; Pro Git 2.1-2.5, 3.1-3.2. Preserve the history of scenario parameters and changes to sensor-data processing.

Workstation setup

On an Apple Silicon Mac, use the planned Ubuntu 24.04 ARM64 in UTM for primary runs. If Ubuntu is already available, use it. You need a terminal, Git, uv/Python, a C++ compiler, CMake, GDB, clang-format and Rustup/Cargo. These experiments require neither ROS nor a physics simulator.

Record versions, architecture and commands in README. Add .venv, build, target, caches and large results to .gitignore. Keep small verification recordings in Git; generate large ones. For every experiment, save the scenario, axis and unit configuration, seed, commit and result.

Practical tasks

- L1 Run an experiment.** Generate a stationary-robot recording, analyse it, and save the JSON summary and errors separately. Repeat from another directory using explicit paths. Then supply a missing file: this must produce a clear error rather than an empty successful report.
- L2 Process lifecycle.** Make the generator emit one row every 10 ms with an explicit flush; actual intervals may vary. Pipe it into a simple Python logger. Stop the generator with SIGINT, wait for EOF and check the last saved row. The final multithreaded logger comes later.
- L3 Configuration history.** Create a branch to correct the y-axis sign in the tilt scenario. Change the data and expected result together, explaining why in the commit. Create a threshold-value conflict in a separate practice configuration file, resolve it and rerun the relevant test.
- L4 Replay.** In a project copy, restore dependencies from the lockfile and repeat the 'no response' experiment. The command, data version and result must match. A VM snapshot helps restore the environment, but does not replace project history and an experiment description.

Connection to later robotics work

These skills matter when a sensor or driver runs as a separate process, an experiment runs on an onboard computer, and a fault must be reproduced on a laptop. Save a reproducible experiment folder and explain what happens when each process stops.

Python and motion diagnostics

Five tasks using body and joint data

Study before practice

Python: functions, dataclass, dict/list, generators, with, modules, exceptions, argparse, pathlib and JSON. [NumPy](#): arrays, shape/dtype, columns, masks, abs, mean, max, isfinite, sin/cos and arctan2. [Matplotlib](#): Figure/Axes, multiple lines, labels, legends and savefig. Apply each construct to a RobotState snapshot.

P1 Generate data for known motion

For rest, set $f=(a_x, a_y, a_z)=(0, 0, 9.81)$, $q_{\text{ref}}=q=0$, $i=0.2$. For static roll $\phi=15^\circ$: $f=(0, g \cdot \sin\phi, g \cdot \cos\phi)$, $g=9.81$; convert ϕ to radians for sin/cos. The joint is stationary in both experiments. These are ideal data with known answers.

For joint motion, set $q_{\text{ref}}=\min(0.8, 0.002 \cdot \text{seq})$, $q=\max(0, q_{\text{ref}}-0.02)$, $i=0.2$. In the 'no response' variant, after $\text{seq}=200$ hold q at its value at 200 while q_{ref} keeps changing. Record this boundary in the configuration. This is a predefined motion trace, not a physical motor model.

P2 Read and validate a snapshot

Implement `parse_row` and sequence validation separately. Stream the file using `with`, recording rejection reasons and maintaining `accepted+rejected=rows_total`. Distinguish an unrepresentable number, a packet gap and mechanically suspicious behaviour: these are different events. Update the last valid state only after all checks.

P3 Plot body data

Plot a_x , a_y , a_z and the specific-force magnitude against time. In static experiments, estimate roll as $\phi_{\text{est}}=\text{atan2}(a_y, a_z)$: expect 0° and 15° , with a 0.1° tolerance without noise. Add an artificial 2 m/s^2 to a_y for the stationary body and demonstrate the false tilt. Conclude that [linear acceleration](#) and vibration interfere with accelerometer-only tilt estimation; yaw cannot be determined this way.

P4 Detect a joint that does not respond

Plot q_{ref} and q , then $e=q_{\text{ref}}-q$. Find $\max|e|$ and $\text{mean}|e|$. Flag **NO_RESPONSE** if, in a 10-snapshot window with consecutive seq values, every $|e|>0.1$ rad and $\max(q)-\min(q)\leq 0.005$ rad. Restart the window after a gap. These are teaching thresholds; 10 samples specify a sample set, not an exact duration.

Compare the event with the scenario label. q alone cannot distinguish a frozen encoder, a jammed mechanism and a disconnected motor: report 'no expected response'. Plot current alongside as an additional observation, but do not treat it as proof of the cause.

P5 Find transmission gaps

Remove $\text{seq}=40\dots 49$ from a clean file: 990 of 1 000 snapshots remain, with 10 seq numbers missing. Next corrupt one row without removing it: `rejected=1`. A number missing among accepted snapshots does not prove transmission packet loss: validation may have rejected a row. Account for these cases separately.

Deliverables: a JSON summary, body and joint plots, `events.json` with seq and reason, and expected results calculated by hand for every experiment. Python analyses events; C++ and Rust implement the shared contract, not the entire analysis.

Modern C++ and resource management

Object lifetime, resource ownership and porting the data parser

Study in this order

- **Building a programme.** Source, header, translation unit, object file and linking. Examine one compilation error and one linking error in the sensor processor. Study declarations and definitions, include guards or pragma once.
- **Values and references.** Initialisation, automatic lifetime, scope, const, passing by value and by const T&. Compare pointers and references. Understand why a reference to a local variable becomes invalid.
- **RAII.** The constructor establishes a valid state; the destructor releases the resource. A file closes both on function exit and on an exception. Usually express ownership with an object, `std::vector` or `std::unique_ptr`; use `shared_ptr` only when shared ownership is actually needed.
- **Standard library.** `std::string`, `std::vector`, `std::array`, iterators, range-for, algorithms and `std::optional`. Determine when modifying a vector invalidates references. Study `string_view` as a non-owning view with a limited lifetime.
- **Copying and moving.** Understand copy/move behaviour before studying `std::move`. It permits move selection but does not itself transfer bytes. Do not build a complex class hierarchy for a `RobotState` structure containing a few numbers.

Practical tasks with robot data

C1 State snapshot. Create `RobotState` with the contract's eight fields. Put the parser, structure and CLI in separate files. Pass a snapshot by value and by const reference; explain who owns it and why a consumer must not retain a reference to a reused sensor buffer.

C2 Data reception. Port snapshot validation from Python, including the angle range and seq ordering. Run rest, motion and corrupted recordings. Accepted rows, counters, mean ax/ay/az and `max|q_ref-q|` must match; `NO_RESPONSE` analysis remains in Python.

C3 Logger resource. Wrap the output file in a RAII object. Save 10 snapshots; exit via return, then via an exception after the fifth. Verify that the object is destroyed and the error remains visible. Check stream state on explicit close: object destruction alone does not prove a successful disk write.

C4 Sensor buffer. In a separate experiment, retain a `string_view` of an IMU row, then reuse the original buffer. Show the changed contents; separately reproduce an invalid reference after vector reallocation. Fix the transfer by parsing the row into a `RobotState` that owns its data before reusing the buffer. Save ASan diagnostics where the error is detected.

Readiness for the second checkpoint deliverable

C++ produces the counters and means required by the contract. Explain each object's owner and why the main code has no dangling references. C++17 is sufficient for M1; record the standard and features used in CMake. Template metaprogramming, custom allocators and complex design patterns are unnecessary.

CMake, debugging and diagnostics

Reproduce an error, find its cause and verify the fix

Topics to study

In the official [CMake](#) Tutorial, study building an executable and a library, separating subdirectories, target commands, and Testing and CTest. Understand the relationship between `add_library`, `add_executable`, `target_link_libraries`, `target_include_directories` and `target_compile_features`. Distinguish PRIVATE, PUBLIC and INTERFACE using one dependency example; do not study the entire CMake language.

The project needs a telemetry library, a CLI and a test build target. Enable `-Wall -Wextra -Wpedantic` for GCC/Clang. Use Debug for debugging and Release for speed comparisons. `clang-format` establishes a consistent style; formatting does not establish programme correctness.

```
cmake -S cpp -B build/debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug
ctest --test-dir build/debug --output-on-failure
cmake -S cpp -B build/release -DCMAKE_BUILD_TYPE=Release
cmake --build build/release
```

These commands assume a configured CMakeLists.txt. Describe GoogleTest integration and test-data paths in README; pin the dependency version for reproducible builds.

Debugging a concrete scenario

D1 Incorrect units. In a practice analyser version, pass 15 as radians instead of 15°. In [GDB](#), set a breakpoint at the `q_ref` check, inspect the value and stack, and practise `run`, `next`, `step`, `print` and `backtrace`. The range check must reject this command; fix the conversion and add a test. Small unit errors may remain within range, so this check alone is insufficient.

D2 Corrupted snapshot. In a separate faulty C++ version, access `q` after receiving a row with too few fields. Examine the crash stack and diagnostic-tool message, then check the length before accessing fields. Save the input file and test. Study core dumps using this example; if dump saving is disabled, debug directly in GDB.

Separate diagnostic builds

Check	Practice experiment	Save
ASan and UBSan	Reused sensor buffer, invalid field index and integer timestamp overflow.	Report excerpt, cause and corrected version.
TSan	Reader and monitor update shared RobotState without a mutex, then fix it.	Command, environment and race-check result.
Regular tests	Repeat the checks on the corrected Debug build.	Passing report and a test for the original error.

[ASan](#) and [UBSan](#) are usually compatible; build [TSan](#) separately. Both compiler and linker need the flags: `-g` and, where necessary, `-fno-omit-frame-pointer`. Do not measure speed with diagnostics enabled. If the environment does not support TSan, record that limitation and retain queue tests; race checking remains incomplete.

Robot fault tests

Verify the CSV format and the evidence behind conclusions about the robot

Testing-tool topics

[pytest](#): assertions, fixtures, tmp_path, raises and parametrize. [GoogleTest](#): TEST, ASSERT/EXPECT, TEST_F and execution through CTest. In Rust, use unit tests and cargo test. A Python programme that launches all three CLIs checks the shared files and expected.json; separate Python tests check robot-behaviour diagnostics.

Scenario	Expectation	What it checks
S1 Rest and roll	For error-free static data, roll 0°/15° within 0.1°; magnitude 9.81.	Axes, units and method limitations.
S2 Motion	Smooth recording within range; $\max e \leq 0.02$ rad with numerical tolerance; no NO_RESPONSE.	Distinction between command and measurement.
S3 No response	After q is held at seq=200, NO_RESPONSE occurs under rule P4.	Fault indication without an unsupported conclusion about its cause.
S4 Missing records	Remove seq=40...49: accepted=990, rejected=0, accepted-sequence gap 10.	Missing snapshots counted separately from bad rows.
S5 Invalid input	Blank row, 7/9 fields, NaN/Inf, 1_000, hex, angle 1.01 rad, repeated seq/t_us.	Rejection reason; accepted state remains unchanged.
S6 Stream disappearance	More than 100 ms since a new snapshot; FAULT and simulated sending disabled.	Periodic checking without new messages.

T1 Calculate expected answers by hand

Save the generator configuration and a small example with its expected answer for each scenario. Separately convert 0°, 90° and 180° to 0, $\pi/2$ and π ; the working joint remains limited to ± 1 rad. The 90°/180° cases test the conversion function and must not become joint commands.

Test ± 1 rad, the 0.1 rad threshold, an empty file and CRLF. NO_RESPONSE requires cases with 9 and 10 threshold exceedances, followed by a seq gap. Calculate expectations independently of the code under test.

T2 Introduce errors deliberately

Remove the degrees-to-radians conversion, refresh the watchdog on an invalid packet, and allow NaN in q. A test must detect each defect. Correct the code and explain how the error affects the represented position of a leg or finger.

T3 Controlled time and streaming

Supply now manually in the watchdog test: last snapshot at 0 ms; tick(100) remains valid; tick(101) produces FAULT. A duplicate at 90 ms does not refresh the valid-snapshot receipt time. Do not wait real seconds in unit tests. Separately test the queue for EOF, a full buffer, write failure and termination of both threads.

Scope: about 20-25 compact cases. Test the parser in each language; CLI equivalence through a shared programme; sensor events in Python; and the queue and watchdog in C++. Three complete system implementations are not required.

Rust as a practical tool

Understand ownership and errors by validating familiar robot recordings

Required Rust Book chapters

Chapter 3: selected language constructs; chapter 4: ownership, references and borrowing; chapter 5: structs; chapter 6: enum and match; chapter 8: vector, string and hash map as needed; chapter 9: Result and error handling; chapter 11: testing. From chapters 12-13, use only CLI arguments, file reading and iterators. A conceptual overview of chapter 16 on threads is sufficient in M1.

Identify who owns a String, where a value moves, how long a reference lives, why mutable borrowing restricts other access, and when Result or Option is needed. Begin with owning structures and simple borrows; defer complex lifetime annotations.

Minimum implementation scope

- **R1.** Create a Cargo project, a RobotState structure and a ParseError enum. Write `parse_row(&str) -> Result<RobotState, ParseError>`. Check finite values and joint range, and return a clear reason. A corrupted snapshot must not become robot state; `unwrap()` is unsuitable for handling input failures.
- **R2.** Pass an IMU snapshot row by value, then try logging the same String and read the compiler error. Change the parameter to `&str`. Separately create conflicting mutable and immutable borrows, then fix their usage scope.
- **R3.** Read a leg or finger log through `BufRead` and produce the shared JSON summary and exit codes. You may use `serde/serde_json` for JSON with a pinned `Cargo.lock`. Do not spend time escaping JSON manually.
- **R4.** Use cargo test to check a bad IMU row, an invalid angle and a normal snapshot; connect the shared verification programme. Run `cargo fmt --check` and `cargo clippy`, reading every warning. Use `cargo build --release` for measurements.

A substantive comparison of Rust and C++

Question	C++	Rust
When is a resource released?	Usually when a RAI object's lifetime ends.	Usually when the owner's lifetime ends, through Drop.
How is an invalid row represented?	An explicit result structure or the chosen exception policy.	Result with an error variant and match or the ? operator.
What happens to unsafe access?	Some errors are detected by analysis, tests and runtime diagnostic tools.	The safe Rust compiler rejects many borrowing errors.

Save two Rust diagnostic examples and the C++ RAI experiment. Explain that ownership checking constrains how code can be written, but does not eliminate logic errors, deadlocks or the need for tests. Compare performance using your own measurements.

Readiness criterion. The Rust utility passes the shared checks, and you can explain ownership errors in your own words. Implement the main processor once, in C++.

Data streaming and update monitoring

Distinguish slow disk writes from an interrupted robot stream

Programme model and study topics

Study threads, shared memory, mutexes, RAII locks and [condition_variable](#) with a predicate. A reader thread receives snapshots; a writer thread logs them through a queue of $K=64$. `latest_state` stores the accepted snapshot and `host_received_at`. Read and update all shared state under one mutex; perform file I/O outside the lock.

On normal EOF, close the queue: set closed, wake threads, write the remaining entries and join. A writer error wakes a blocked reader and ends the experiment with an error. The monitor runs in the main thread. Disable the watchdog for file processing; in streaming mode, normal EOF disables commands and stops the monitor before draining the queue.

Q1 Logger overload

In **blocking** mode, the reader waits for space; a slow disk also slows reception of new snapshots. In **drop newest** mode, a new record cannot enter a full queue, but the reader continues updating `latest_state`. The log can therefore lose rows while the monitor still sees new snapshots. Report missing log entries and reception delays separately, explaining their causes.

For verification file processing, use blocking mode without losses. For streaming, compare $K=4$ and $K=64$ with a slowed writer. Check saved seq order and capacity. After normal completion, `accepted=enqueued+dropped` and `enqueued=processed`. Count invalid rows separately; after an error, also report unprocessed snapshots.

Q2 Monitor updates independently of packet reception

Roughly every 10 ms, the main thread calls `tick(now)` and checks `now-last_new_valid_receive`. After **more than 100 ms**, it logs FAULT and disables simulated command sending. Before the first snapshot, state is INIT and commands are disabled; timing starts when reception begins. A first valid snapshot before timeout changes INIT to OK. Duplicates, old seq values and invalid rows do not refresh the time. The 10/100 ms limits are teaching settings, not real-time guarantees.

In the streaming test, stop updates while the reader keeps waiting. The watchdog must trigger without another message. A new valid snapshot after failure does not automatically clear FAULT: allow reset separately, and only with a current accepted snapshot. No commands are sent to actual actuators; test the `command_allowed` variable and event log.

Q3 Required checks

- Update RobotState as a whole: angle and time belong to one snapshot. Run TSan if supported.
- Test empty input, EOF with queued entries, a slow writer and write failure. Hold the writer with an explicit signal to fill the queue.
- Source silence, duplicates and NaN cause FAULT when the valid snapshot becomes too old. Test the 100/101 ms boundary with controlled clocks.

The watchdog measures time since receipt of a new snapshot at the host. It does not establish measurement freshness: data may have been delayed before reaching the reader. Clock synchronisation and source-delay estimation will be needed later.

Replay and processing latency

Repeat a failure and check whether monitor updates arrive on time

E1 Reprocess a motion recording

Save complete original scenario files and a separate cleaned log of accepted snapshots. Original data are needed to reproduce input errors. Write numbers without losing precision: in C++, use `max_digits10` for double or preserve the original fields. Compare values and ordering, not CSV bytes.

Fast mode reads the file without waiting. Reanalysing raw data produces the same rejected and NO_RESPONSE events at the same seq values; analysing the cleaned file produces the same accepted states. Snapshots dropped by the queue are missing from the cleaned log but remain in the original scenario file. They cannot be recovered from the cleaned log alone.

E2 Check timing characteristics

Metric	Measurement boundary and meaning for the robot
Source interval	Difference between consecutive <code>t_us</code> values; 10 000 μ s in the clean scenario. Gaps differ from host interval variation (jitter).
Age at monitoring	<code>now_host-last_new_valid_receive_host</code> . The watchdog uses this to detect channel silence.
Queueing and writing	<code>t_done-t_enqueue</code> on one monotonic clock. Shows logger queue accumulation.
Three-CLI runtime	From launch to completion on one file: includes runtime startup, parsing, reading and summary generation.
p50 p95 p99	Sort <code>n</code> latency values; take element <code>ceil(p×n)</code> , counting from 1; <code>p=0.50/0.95/0.99</code> . Empty sample: null.

Clocks: Python [perf_counter_ns](#), C++ [steady_clock](#), Rust `Instant`. Subtract readings from the same clock: do not subtract `t_us` from host `now`. A 'nanosecond' unit does not imply that accuracy; a VM provides no hard deadline guarantee.

E3 Take measurements

- Generate one file of 100 000 motion snapshots; record SHA-256, seed and versions. First verify equivalence of the three CLIs. Use Release builds of C++/Rust without diagnostic tools.
- Use an external Python programme for 2 warm-ups and 10 measurements of each CLI, alternating languages. Save individual runtimes, median and range. Compare the implementations without claiming general language superiority.
- For C++, compare normal and slowed writers, `K=4/64` and blocking/drop newest. Show maximum queue size, dropped, latencies and watchdog events. Collect at least 10 000 individual latency measurements for p99.
- In a 1-2 page report, explain whether history was retained, whether the monitor received updates, what limited processing, and whether the data allow the failure to be repeated.

Optional: replay 100-1000 snapshots at a specified rate: `deadline_s=start_s+(t_us-first_t_us)/(1 000 000×speed)`, `speed>0`. Wait until each specified absolute time and record lateness. Control clocks in watchdog tests.

M1 first-half schedule

11-23 October • 20 hours per block • one test bench

Sunday has five 2-hour practice blocks. Weekdays cover reading, analysis and small checks; save results as experiment records. Dates and total workload follow the original plan with the clarification on page 1.

Date	Hrs	Task and deliverable
Sun 11.10	10	Body and joint. Block 1: environment, repository and L1. Block 2: axes, units, RobotState. Block 3: rest, tilt and motion generator. Block 4: Python parser and summary. Block 5: P3/P4 plots and manual verification. Deliverable: first experiment with plots.
Mon 12.10	2	Linux and streaming. 45 min paths and redirection; 45 min L2, pipe and EOF; 30 min shutdown checks. Deliverable: source can be stopped while preserving the log.
Tue 13.10	2	Python and snapshots. 45 min dataclass/exceptions; 45 min streaming reads; 30 min invalid fields and seq order. Deliverable: accepted-state update scheme.
Wed 14.10	2	IMU data. 45 min NumPy/axes; 45 min rest, roll and false tilt from added acceleration; 30 min plotting. Deliverable: understand accelerometer atan2 limitations.
Thu 15.10	2	Experiment version. 45 min branches and history; 45 min L3, axis-sign correction; 30 min L4 and lock. Deliverable: repeatable experiment with known parameters.
Fri 16.10	2	Joint diagnostics. 45 min P4 and the 9/10-sample boundary; 45 min P5, gaps and rejections; 30 min expectations. Deliverable: Python distinguishes the chosen scenarios.
Sat 17.10	0	Rest; no required tasks.
Sun 18.10	10	C++ processor. Block 1: compilation and C1. Block 2: C2 parser and summary. Block 3: C3 RAI logger. Block 4: C4 buffer and CMake. Block 5: comparison with Python. Deliverable: C++ validates and records snapshots.
Mon 19.10	2	Snapshot ownership. 60 min values, references and const; 40 min sensor buffer and consumer; 20 min conclusions. Deliverable: RobotState ownership diagram.
Tue 20.10	2	RAII and logging. 60 min lifetime, copy and move; 40 min write failure or exception; 20 min conclusions. Deliverable: explanation of file closure and error checks.
Wed 21.10	2	STL and buffers. 45 min vector/string_view; 45 min C4 and ASan; 30 min correction. Deliverable: snapshot survives input-row reuse.
Thu 22.10	2	Building the test bench. 60 min CMake targets; 40 min Debug/Release; 20 min commands. Deliverable: separate state library, CLI, logger and tests.
Fri 23.10	2	Error investigation. 45 min GDB; 45 min D1/D2, units and incorrect snapshot length; 30 min test. Deliverable: input, cause, correction and evidence.

By 23 October: both implementations accept and reject the same snapshots; the logger preserves values. The report shows axes, units and joint limits. Resolve discrepancies before porting the contract to Rust.

M1 second-half schedule

25 October - 3 November • 35 hours for faults, streaming and verification

Date	Hrs	Task and deliverable
Sat 24.10	0	Rest.
Sun 25.10	10	Faults and Rust. Block 1: scenarios/expected.json and S1-S5. Block 2: pytest/GoogleTest and T1/T2. Block 3: Cargo, RobotState and Result. Block 4: Rust validator. Block 5: shared verification programme and corrections. Deliverable: three CLIs check one contract.
Mon 26.10	2	Evidence. 45 min parametrisation; 45 min 9/10 exceedances, gaps and wrong units; 30 min T2. Deliverable: identify the test that catches each failure.
Tue 27.10	2	Rust ownership. 60 min ownership and borrowing; 40 min sensor row for analysis and logging; 20 min explanation. Deliverable: two ownership conflicts corrected.
Wed 28.10	2	Invalid state. 45 min Result/Option; 45 min NaN, out-of-range angle and Cargo tests; 30 min RAI/ownership. Deliverable: invalid snapshots do not update state.
Thu 29.10	2	Communication between threads. 60 min mutex/condition_variable; 40 min Q1 and closure; 20 min diagram. Deliverable: latest_state owners and overflow policy are defined.
Fri 30.10	2	Update monitoring. 45 min Q2 and clocks; 45 min T3, 100/101 ms boundaries; 30 min measurement plan. Deliverable: testable watchdog logic before thread integration.
Sat 31.10	0	Rest.
Sun 01.11	10	Integration. Block 1: reader/writer, queue and shutdown. Block 2: latest_state, monitor and FAULT. Block 3: Q3, overload and E1 replay. Block 4: E2/E3, raw measurements. Block 5: report and clean build. Deliverable: telemetry-core v1.0 candidate.
Mon 02.11	3	Review and reserve. 60 min conclusions on gaps, delays and causes; 60 min experiment instructions; 60 min one problematic scenario. Uses one additional programme hour.
Tue 03.11	2	Test-bench verification. 45 min explaining scenarios unaided; 45 min checklist; 30 min conclusions. Deliverable: skills and unmet criteria list.

Fitting integration into the budget

Before the final Sunday, prepare snapshots, a sequential logger, shared checks and a tested tick(now). Connect them with the watchdog in the main thread and reader/writer in worker threads. No separate task system or reactor is needed.

One joint, IMU axes, files and plots are required. Cut extensions first: noise, current plots, paced replay and additional input sizes. A complete robot, physics simulation, ROS, PID and an orientation filter come later. Record failed core tests as unfinished M1 work.

Final review: why can the log fall behind while the monitor receives new snapshots? Support your answer with counters and timestamps.

Assigned reading by task

Programming references and the compulsory first mechanism analysis

The plan and definitions are in English; references include English-language materials. Read the assigned passage before its task, then test the idea in code. You need not buy four books or read them cover to cover. Core reading is identified; consult the reference book when stuck.

Core reading

1. Scott Chacon, Ben Straub - Pro Git, 2nd ed. [English edition](#). Sections 2.1-2.5: creating a repository, recording changes, history, undoing and remotes; 3.1-3.2: branching and merging. For L3/L4: axis-correction branch, threshold conflict and versioning a reproducible experiment.

2. Steve Klabnik, Carol Nichols and the Rust community - The Rust Programming Language. [Rust Book](#). Chapters 4-6 and 9: ownership and references, structs, enum/match and errors for R1/R2. From 11-13: tests, CLI, file reading and iterators for R3/R4. Read chapters 3 and 8 selectively for syntax and collections; skim 16. Defer async, unsafe and FFI.

3. Anish Athalye, Jon Gjengset, Jose Javier Gonzalez - The Missing Semester of Your CS Education, MIT, 2020 course. [Course notes](#). 'Course Overview + The Shell', 'Command-line Environment', 'Debugging and Profiling': redirection, processes, diagnostics and measurement for L1/L2, D1/D2, E3. Use relevant parts of 'Metaprogramming' for builds and dependencies.

C++ reference: specific sections

4. Bjarne Stroustrup - A Tour of C++, 3rd ed. [Author and contents](#). §1.5-1.7: lifetime, const, pointers and references; §3.2 and 3.4: separate compilation and arguments; §6.2-6.3: copy/move and resources for C1-C3. §10.2-10.3 and 12.2: string, string_view and vector for C4. §18.2-18.4: threads, shared data and waiting for Q1-Q3. The book assumes programming experience; begin after P1/P2. Use the C++17 subset; modules and coroutines are unnecessary here.

Documentation for individual steps

[Python Tutorial](#): Control Flow, Data Structures, Modules, Input/Output, Errors and Exceptions, Classes, Virtual Environments for P1/P2. Follow task-page links for NumPy/Matplotlib and testing tools. [REP-103](#) (Units, Axis Orientation) and [REP-145](#) (Data Sources) explain axes and accelerometers; reading them requires no ROS installation.

Motion fundamentals: what the programme describes

A **link** is a mechanism part treated as rigid in the model. A **joint** defines allowed relative link motion: a revolute joint permits an angle, a prismatic joint a displacement. A **degree of freedom** is one independent local motion. A fixed-base leg with an open link chain and three independent revolute joints has three; actuator count and degrees of freedom can differ.

Configuration q determines relative link placement; for a single fixed pivot it is an angle. A second-order mechanical model's state also includes velocity: $x=(q, \dot{q})$, where $\dot{q}$ is angular velocity in rad/s. The same pose at different velocities gives different states. M1 RobotState is an observation snapshot, not the complete physical state.

Kinematics relates coordinates, velocities and accelerations without calculating forces. **Dynamics** explains changes in motion through mass, inertia, forces and torques. **Control** selects an input to achieve a goal. An inverse-kinematics angle does not establish that the motor can perform the motion. Further detail: M8, M9, M10.

Read before P1: Kevin Lynch, Frank Park, Modern Robotics, [§2.1: configuration and degrees of freedom](#) and [§2.2: joints](#). Study definitions, joint types and the mechanism diagram; the general mobility formula comes in M5.

M1 key terms

Definitions for the data schema, resource ownership and stream processing

Telemetry. Transmission of measured quantities and diagnostic events to observe a system; it does not itself control motion.

State snapshot. A consistent set of values for one step; in M1, eight fields share a t_{us} timestamp.

Coordinate frame. An origin and oriented axes relative to which vector components are expressed; here x forwards, y left, z upwards.

Radian. A unit of plane angle: arc length divided by radius. One revolution is 2π rad; $\theta_{rad} = \theta_{deg} \cdot \pi / 180$.

Specific force. Ideal accelerometer output: $f = a - g$; a is motion acceleration and g gravitational acceleration, expressed in the same frame. At rest with z upwards, $f_z \approx +9.81 \text{ m/s}^2$.

Roll. Body rotation about the longitudinal x axis; the static estimate $\text{atan2}(a_y, a_z)$ is distorted by linear acceleration and cannot determine yaw.

Position error. Difference between requested and measured angles, $e = q_{ref} - q$, in radians; these quantities describe the command and observation respectively.

Timestamp. Event time on a chosen clock; source t_{us} and host time cannot be subtracted without aligning their timescales.

Monotonic clock. A timescale that never goes backwards; suitable for intervals within one time source, but it does not specify a calendar date.

Invariant. A condition that must hold in valid programme states; for example, $\text{accepted} + \text{rejected} = \text{rows_total}$ after file processing.

Ownership. Responsibility for a resource's lifetime. The owner ensures release; a non-owning reference does not extend the resource's lifetime.

RAII. Tying a resource to a C++ object's lifetime: the destructor releases it. Object destruction does not prove a successful write.

Borrowing. Rust access through a reference without ownership transfer, limited by lifetime and compatibility rules for mutable and immutable access.

Dangling reference. A reference to an object whose lifetime has ended or whose address has become invalid; using it violates programme correctness.

Mutex. A mutual-exclusion mechanism that protects shared state from simultaneous conflicting access when all threads use it consistently.

Data race. Unsynchronised, conflicting thread accesses to the same memory, with at least one write; in C++, these cause undefined behaviour.

Backpressure. Slowing a producer when a consumer cannot keep up: in blocking mode, the reader waits for space in a full queue.

Watchdog. Periodic checking of the time since a new valid snapshot was received, using the host clock; silence causes FAULT without another message.

Latency. Time between two defined events, such as enqueue and write completion. Specify the boundaries and clock source before measuring.

p99 quantile. A value at or below which approximately 99% of observations lie; here, take element $\text{ceil}(0.99 \cdot n)$ of the sorted sample, counting from 1.

Replay. Reprocessing saved input; raw rows are needed to reproduce rejections that are absent from the cleaned log.

SHA-256 hash. A 256-bit content fingerprint for checking file equality; it does not itself establish authorship or authentic provenance.

Check yourself: why is $a_z \approx g$ at rest; why does constant q not establish a fault's cause; why are dropped, rejected and seq gaps separate counters?

M1 equipment and setup

Prepare before L1, C1 and R1; setup time is included in the 75 hours

Equipment and purchases

Available: 1 MacBook Pro M4, a terminal and the existing 3D printer; choose its profile later using the exact printer model. M1 needs no printing. **Required purchases: none.** No IMU, MCU, motors, batteries, robot or GPU is needed: the Python generator supplies data. Borrow the C++ book from a library if available; purchase is optional.

Install before the first practical session: 11 October

On macOS ARM: an editor such as [VS Code for Apple Silicon](#), and [UTM](#). Use [Ubuntu 24.04 ARM64](#) as the guest OS; choose the arm64 image and ARM virtualisation, rather than x86 emulation. Use an existing suitable Ubuntu installation if available. Allocate VM resources according to the Mac's memory and disk capacity; check free space and responsiveness before experiments.

In Ubuntu ARM64: Git, terminal and shell, ripgrep; [uv](#) and Python; numpy, matplotlib and pytest dependencies in one project. Pin Python and dependencies in pyproject.toml/uv.lock, and record commands in README. Do not install experiment packages in system Python. Save plots as PNG so experiments also run without a GUI.

Before C1: 18 October; before R1: 25 October

C++ in Ubuntu: GCC/G++ or Clang with C++17, CMake, make/Ninja, GDB, clang-format; a pinned GoogleTest version for tests. Ubuntu packages: [build-essential](#), [cmake](#), [gdb](#), [clang-format](#); record compiler choice and versions. Use a separate Debug configuration for ASan/UBSan; build TSan separately and only where the environment supports it.

Rust in the same Ubuntu: [rustup](#), rustc, Cargo, rustfmt and Clippy. Pin the toolchain and Cargo.lock. Add serde/serde_json for R3 as needed. Native macOS builds are an additional check; take comparison measurements for all three CLIs in one Ubuntu environment.

Installation checks and optional additions

Record guest architecture (aarch64) and versions. Make a Git commit; use uv to import NumPy/Matplotlib/pytest and save a plot; build and run a C++17 example through CMake/CTest and stop at a GDB breakpoint; run cargo test, cargo fmt --check and cargo clippy. Restore dependencies from the lockfile in a clean copy and repeat L1/L4. The environment is ready for a task once its corresponding tool passes verification.

Optional: Docker only if isolation beyond the VM becomes necessary; M1 does not need it. Defer ROS 2, Gazebo and the GPU stack to their modules. Apple M4 does not support CUDA. A VM does not guarantee hard real time; check graphics and USB before later hardware experiments. Delivery time for subsequent purchases is outside study hours.

Basis: roadmap.pdf, physical p. 3 for the calendar; pp. 19-21 for M1 objectives. The RobotState schema, scenarios, teaching thresholds and acceptance checks were developed for this detailed plan. Installation and environment checks: p. 15.

First motion calculation: before generator P1

A planar link $L=0.20$ m pivots at the origin: x right, y upwards; q is measured anticlockwise from +x. Endpoint: $p=(L \cos q, L \sin q)$. At $q=0$, obtain (0.20; 0) m; at $q=0.50$ rad, (0.17552; 0.09589) m. Mass is unnecessary for this calculation: it is forward kinematics.

In block 2 on 11 October, replace part of the RobotState analysis with 30 min for definitions and a diagram, and 30 min for calculation and plotting. Save mechanism.md and the figure; check $\|p\|=L$ and $q=-0.50$. Why can q and L alone not determine motor torque? Retain the CSV, [-1;1] rad limit and 75 hours.

Mathematics of robot state and motion

M2 • 03 Nov 2026 - 29 Nov 2026 • 75 hours

What you should achieve by the end of M02

Learn to turn a simple leg or finger problem into quantities, equations and a verifiable calculation. For example: find a force's moment arm from a support point's position; recover an angle from encoder counts; obtain velocity from a trajectory; predict a compliant support's motion from mass, stiffness and damping. The main deliverable is **robot-math**, a collection of numerical and geometric checks reusable in later modules.

The scope is deliberately limited: vectors and matrices, linear systems and least squares, derivatives and integrals, and ordinary differential equations. Introduce the Jacobian as a table of sensitivities and eigenvalues as properties of a simple linear transformation and system. Do not yet implement full 3D robot kinematics, spatial velocities, controllers or a Kalman filter.

Starting model and preparation

Use Python, NumPy, Matplotlib and SciPy in the M01 environment. Create notebooks/01_vectors, 02_calibration, 03_motion and 04_odes; move repeated functions into robot_math/. Each notebook must contain a problem statement, axes diagram, parameters with units, calculation, error plot and conclusion. Keep raw and computed data separately; pin dependency versions and the random-number generator seed.

The shared teaching object is a link of length $L=0.12$ m with one revolute joint and a support point at its tip. Masses, forces and other numbers in this document are calculation-example parameters, not authorisation to test a future design. The second object is a compliant support with mass $m=0.5$ kg, stiffness $k=50$ N/m and viscous damping coefficient $c=1$ N·s/m. M02 investigates motion only through calculation.

Entry diagnostic: within the first two hours

Without references, convert 120 mm to metres, 90° to radians and 10 ms to seconds; find $\sin(\pi/2)$, a rectangle's area and the solution of two linear equations. In Python, create an array of shape (100,3), select one coordinate and calculate its mean. If unit arithmetic or trigonometry is difficult, spend up to 90 minutes of block A addressing that specific gap and reduce decorative plotting work.

Starting criterion: explain in your own words the difference between a number, a vector component and a physical quantity. Record one M01 error that mathematics could detect: a reversed axis sign, milliseconds used instead of seconds, or a false velocity jump from a missing packet. This becomes your first regression example.

Allocating 75 hours and recording calculations

Four study blocks

A - hours 1-20: 10 h of theory and hand calculations, 10 h of practice. Topics: units, vectors, bases, dot and cross products, matrices and rank. Deliverable: force diagrams and a library of verifiable point and vector transformations.

B - hours 21-40: 10 h of theory and 10 h of practice. Topics: Gaussian elimination, overdetermined systems, residuals, QR, conditioning and eigenvalues. Deliverable: encoder calibration with a separate validation set and an unsuccessful experiment example.

C - hours 41-60: 10 h of theory and 10 h of practice. Topics: derivatives, integrals, partial derivatives, gradients and an introductory Jacobian. Deliverable: link-trajectory analysis, work done by a force and sensitivity of the endpoint position to angle.

D - hours 61-75: 5 h of theory and 10 h of practice. Topics: initial-value problems, explicit Euler, semi-implicit Euler, RK4, stability and step-size error. Deliverable: mass-spring-damper, pendulum and convergence report. Total: exactly 35 h of analysis and 40 h of practice.

Session rhythm

Divide each 10 h of theory into five two-hour sessions: 15 min reviewing the preceding idea, 35 min reading, 40 min hand calculations, 20 min Python verification and 10 min recording conclusions. Theory in the final block takes 2+2+1 h. Divide the ten-hour practical into five 2 h sessions: formulation, implementation, cross-checking, error analysis and presentation. Breaks are outside study hours.

The calendar window on the first page comes from the roadmap. A boundary date may belong to two modules; rest days, holidays and integration stages remain as in the original plan. Numbered work steps fill available sessions sequentially without recounting hours. Do not add an extra hour to every module.

When to move from reading to practice

After each section, close the text and explain which quantity you are calculating, why the operation is valid and how you will verify the answer. One correct example and one deliberately faulty example are more useful than a second lecture on the same topic. Use these log fields: robot task; model; assumptions; prediction before running; result; error; next check.

If behind schedule, first remove animation styling and a second language implementation, then reduce parameter sweeps. Retain four notebooks, both dynamic objects, unit checks and method comparisons. Full QR proofs and derivation of RK4 from Taylor series can wait; explanations of residuals, time steps and causes of discrepancies are compulsory.

Vectors, bases and forces on a link

Topics to study

Study point and free-vector coordinates, length and normalisation, force addition, bases and projections. The dot product $\mathbf{a} \cdot \mathbf{b}$ connects components to angle; for unit direction $\mathbf{n}$, the force projection is $\mathbf{F} \cdot \mathbf{n}$. The cross product $\mathbf{r} \times \mathbf{F}$ gives torque about a chosen origin. Measure r in metres, F in newtons and torque in $\text{N} \cdot \text{m}$; its direction follows the right-hand rule.

Study matrix-vector multiplication as a coordinate transformation, matrix product order, transposition and the meaning of an inverse matrix. For planar rotation, use $R(\theta)$ with first row $[\cos\theta, -\sin\theta]$ and second row $[\sin\theta, \cos\theta]$. State the convention explicitly: an active anticlockwise vector rotation in a fixed frame. The reverse transformation uses $R(\theta)^T$; do not confuse the two operations.

Task 1. Loading a finger or shank

Input: $\mathbf{r} = (0.12; 0; 0)$ m, $\mathbf{F} = (0; 0; -5)$ N. First draw a side view and predict the torque direction. Then calculate $\mathbf{r} \times \mathbf{F}$ by hand and with NumPy, find its magnitude and project $\mathbf{F}$ onto the link axis. Rotate the force by 30° in the same plane and repeat. Deliverable: diagram and table of four positions, labelling axes, units and signs.

Check: in the first case, torque is $(0; 0.6; 0)$ $\text{N} \cdot \text{m}$; collinear $\mathbf{r}$ and $\mathbf{F}$ give zero. Doubling the moment-arm length must double torque magnitude. Faulty case: enter 120 instead of 0.12 and explain the exact thousandfold discrepancy. This checks the unit model; it is not a physical motor test.

Task 2. Sensor coordinates on a rotated body

Input: three unit vectors and ten arbitrary points in the plane. Implement R and rotate the set by 0° , 90° , 180° and back. For an offset-mounted sensor, distinguish $\mathbf{p}_{\text{body}} = R \cdot \mathbf{p}_{\text{sensor}} + \mathbf{t}$ from the direction transformation $\mathbf{v}_{\text{body}} = R \cdot \mathbf{v}_{\text{sensor}}$. Do not add translation $\mathbf{t}$ to force, acceleration or direction.

Save a plot of original and transformed axes, a norm test and a reverse-transformation test. For float64 numbers of order one, use a teaching absolute tolerance of 10^{-12} ; also explain the scale choice. Test the zero vector without normalising it and an angle in degrees mistakenly passed as radians. Be able to state by hand the result of rotating $(1, 0)$ by 90° .

Linear systems and encoder calibration

From equation to solution

Study $A \cdot x = b$: each row defines a constraint and each column an unknown's contribution. Work through Gaussian elimination on a 2×2 system, row swaps for a poor pivot, and nonexistent or nonunique solutions. Rank is the number of independent constraints. The inverse matrix is useful conceptually; solve numerically with `solve` or `lstsq` rather than `inverse(A) \cdot b`.

For calibration $q = a \cdot u + b$, u is the encoder count and q is in radians. Construct A from a column of u values and a column of ones. With more observations than parameters, minimise the sum of squared residuals $r = A \cdot x - q$. QR separates the problem into an orthogonal transformation and a triangular system; understand this role and one 3×2 example without writing a general QR implementation.

Task 3. A teaching body with two supports

A 2 kg body rests on two points at horizontal positions $x=0$ and $x=0.3$ m; its centre of mass is at $x=0.12$ m. Write vertical force and torque equilibrium equations; find support reactions by hand and in Python. With $g=9.81$ m/s², reactions must sum to 19.62 N, and opposing moments about the left support must balance.

Move the centre of mass from 0 to 0.3 m. A negative reaction outside the segment means the contact model, which can only push, is invalid. Do not present it as a feasible leg force. Save a reaction-force plot and mark the model's validity boundary; balance control comes later.

Task 4. Find encoder scale and zero

Create 21 u positions evenly spanning the teaching range $-1000 \dots 1000$. Generate q with known $a=0.001$ rad/count, $b=0.03$ rad and small noise with fixed settings; save the ground-truth parameters separately. Reserve some points for validation. Estimate the parameters, plot residual against u and calculate $RMSE = \sqrt{\text{mean}(r^2)}$ in radians. First check noise-free data: parameters must be recovered to machine precision.

Then restrict inputs to around $u=500$ and compare parameter sensitivity to a small change in one observation. A small error within a narrow region does not establish good calibration across the full range. Deliverables: `coefficients.json`, raw CSV, training and held-out error plots, rank of A and an explanation of units. Failure cases: identical u values, an empty set, NaN and a duplicate mistakenly counted as a new independent measurement.

Conditioning and eigenvector directions

Skills needed before M03 optimisation

The condition number describes how much input error can be amplified in the answer for a specified norm and scale. A large value warrants examining the formulation, units and data diversity. It is not a universal good/bad switch: parameters in millimetres and metres have different scales. Compare normalised formulations and always show the actual change in the answer.

Under a matrix transformation, an eigenvector remains on the same line; its eigenvalue is the multiplier. For a diagonal stiffness matrix $K = \text{diag}(40, 100)$ N/m, the directions are obvious. If off-diagonal terms couple the axes, compute the directions. For now, use a symmetric 2×2 matrix and check $K \cdot v = \lambda \cdot v$; leave complex modes, spectral control and general multilink mechanics for later months.

Task 5. Nearly identical support measurements

Use $A = \begin{bmatrix} 1 & 1 \\ 1 & 1 + \epsilon \end{bmatrix}$ and b generated from known $x = (1, 2)$. Set $\epsilon = 10^{-1}, 10^{-4}, 10^{-8}$. For each case, save $\text{cond}(A)$, the residual and the error in recovered x . Add the same small relative perturbation to one b component and repeat. This illustrates two nearly duplicate constraints when determining a robot's state.

Check: a small residual can coexist with a large x error. Demonstrate this numerically, then replace the second equation with an independent one and repeat. Deliverable: six-run table and a conclusion identifying an additional measurement that resolves the problem. Do not insert a pseudoinverse without explanation: it belongs to M03 and cannot create missing information.

Task 6. Stiff and compliant support directions

For $K = \begin{bmatrix} 80 & 20 \\ 20 & 50 \end{bmatrix}$ N/m, use `eigh` to find eigenvalues and normalised eigenvectors. Draw original axes and eigenvector directions. Apply an equal teaching force along each direction and find displacement from $K \cdot x = F$. A greater stiffness λ produces less displacement for the same force magnitude.

Check orthogonality of the two directions, each eigen-equation's residual and reconstruction of the original matrix. Set stiffness along one axis to zero: this model no longer determines static displacement along the free direction. Save it as an expected solve failure caused by a singular system. Additional 20-minute experiment: rotate K using orthogonal R and verify unchanged eigenvalues.

Derivatives, integrals and motion sensitivity

Theory connected to motion

The time derivative of position is velocity; the derivative of velocity is acceleration. For angle q , units are rad, rad/s and rad/s². Study derivatives of sums, products and sin/cos, and the chain rule for $q(t)=A\cdot\sin(\omega t)$, with ω in rad/s. Integrating velocity gives coordinate change; integrating force along a path gives work in joules, and over time gives impulse in N·s. Label the integration variable for every integral.

A partial derivative describes how a function changes when one argument varies and the others remain fixed. The gradient collects these derivatives for a scalar function; a vector function's Jacobian arranges them with outputs in rows and inputs in columns. Do not call every multidimensional array a Jacobian: first specify the function, input/output dimensions and each element's units.

Task 7. Encoder velocity and noise

Generate 5 s of motion $q=0.4\cdot\sin(2\pi t)$ rad with a 0.01 s step. Derive exact velocity and acceleration. Implement forward and central differences; compare central differences only at interior samples and state the time assigned to each forward-difference estimate. Add angle noise and plot velocity error. Compare two time steps with both noise-free and noisy input.

Save q , dq/dt and absolute-error plots with RMSE over the same time window. Noise-free error should decrease for sufficiently small steps, while differentiating a noisy signal too frequently can worsen the estimate. Remove one sample and use actual timestamps: division by constant dt at that point becomes a deliberate error case. M04 covers filtering in detail.

Task 8. Link-endpoint sensitivity

For $p(q)=(L\cdot\cos q, L\cdot\sin q)$, derive $J(q)=(-L\cdot\sin q, L\cdot\cos q)^T$. With $L=0.12$ m and $q=30^\circ$, estimate $\Delta p \approx J\cdot\Delta q$ for $\Delta q=0.1^\circ, 1^\circ, 10^\circ$. Compare with exact $p(q+\Delta q)-p(q)$; the error magnitude shows where the linear approximation loses accuracy. Deliverables: tangential-displacement diagram and error table in millimetres.

Separately integrate a known velocity with the trapezoidal rule and check that the angle returns after a full period. Calculate work done by a constant torque of 0.2 N·m through 30° : $W=\tau\cdot\Delta q$, with angle in radians. Do not confuse torque with energy even though both have dimensional product N·m: their physical meanings and operations differ.

ODEs: a compliant leg and a free pendulum

Formulating an initial-value problem

For a spring-supported mass with displacement measured from equilibrium, $m \cdot x'' + c \cdot x' + k \cdot x = F(t)$. Introduce $v = x'$ and state $y = (x, v)$; then $x' = v$ and $v' = (F - c \cdot v - k \cdot x)/m$. Initial values $x(0)$ and $v(0)$ are compulsory parts of the problem. Measuring from equilibrium removes constant gravity; with another coordinate definition, explicitly restore gravity with the correct sign.

Explicit Euler evaluates the derivative at the step's start. Semi-implicit Euler updates v first, then uses the new v to update x . RK4 evaluates four intermediate derivatives; implement it for general $y' = f(t, y)$. It has a fixed step and fourth-order accuracy for sufficiently smooth problems; library RK45 uses a different scheme and adaptive control, so it is not the same algorithm.

Task 9. Mass-spring-damper

Use the parameters on page 1, $x(0) = 0.02$ m, $v(0) = 0$ and $F = 0$, for 5 s. Implement the three methods and a reference `solve_ivp` calculation with stricter tolerances. For each method, test steps 0.04; 0.02; 0.01 s and compare x and v at common times. In a separate undamped experiment with $c = 0$, track $E = mv^2/2 + kx^2/2$.

Save state, energy and error-versus-step plots. For $c > 0$ without external force, energy must decrease in the continuous model. Energy growth may indicate numerical instability or a sign error. A decay plot alone is insufficient: check equilibrium $x = v = 0$ and compare with the analytical solution for $c = 0$.

Task 10. Pendulum model of a free link

Set $\theta'' = -(g/L) \cdot \sin\theta$, $L = 0.3$ m, $\theta(0) = 0.1$ rad, $\omega(0) = 0$. Measure the period between zero crossings in the same direction and compare with $T \approx 2\pi \cdot \sqrt{L/g}$. Increase the initial angle to 1 rad and demonstrate the small-angle approximation's limitation. Energy $E = mL^2\omega^2/2 + mgL(1 - \cos\theta)$ must remain constant in the ideal continuous model.

Test $\theta = 0$, a negative initial angle and an excessively large step. Reject negative m or L before integration. Save solver settings and an error estimate; agreement between two equally faulty implementations cannot replace an independent analytical reference. Actual foot impacts, joint friction and backlash are not yet part of this model.

The shared robot-math test bench and error checks

Connect all notebooks through one scenario

Scenario: a virtual encoder produces $u(t)$, calibration converts it to $q(t)$, geometry computes the link endpoint, a numerical derivative gives velocity, and an independent pendulum model supplies reference motion. Define each stage's input/output contract. Data include t_s , $encoder_count$, q_rad , p_x_m and p_z_m ; settings include L_m , $a_rad_per_count$ and b_rad .

First generate a trajectory from the ODE, then convert q to u through inverse calibration and quantisation. The analyser must recover motion with the expected precision loss. Combine existing functions; a separate large project is unnecessary. In the report, distinguish model, quantisation, calibration and numerical-differentiation errors: the same final RMSE can have different causes.

Six required scenarios

1. Wrong units. Pass a length in millimetres to an interface expecting metres; a schema or scale check must explain the failure. **2. Zero vector.** Normalisation must return a clear error rather than propagate NaN through the remaining calculation. **3. Singular calibration.** All u values are equal; the report identifies insufficient rank and does not produce an unsupported scale coefficient.

4. Poor conditioning. Nearly identical equations give a sensitive answer; the programme must not present a small residual as proof of accuracy. **5. Missing time sample.** A removed packet must not cause an artificial doubling of velocity through a hard-coded dt . **6. Unstable step.** Detect energy growth in an ideal spring and reduce it by adjusting the method or step.

Numerical and geometric checks

For exact noise-free examples, combine absolute and relative tolerances: $|a-b| \leq atol + rtol \cdot |b|$. Absolute tolerance matters near zero; relative tolerance matters when scale changes. Choose tolerances before viewing results and label their units. For integration methods, the main check is convergence over a series of steps, rather than one fortuitously accurate value.

In `test_report.csv`, record test name, input scenario, expectation, observation, threshold and pass/fail. Expected failures are valid: a test passes if the programme recognises an invalid formulation. Save at least one plot each of a faulty and corrected version. For the final run, remove computed results and recreate them from raw data using one recorded command sequence.

Step-by-step work: hours 1-40

Block A: coordinate and force mathematics

Hours 1-2. Complete the entry diagnostic, create a quantity log and define axis conventions. Solve two unit-conversion examples and resolve a force into components by hand. Deliverable: assumptions.md with a link diagram and correctness criteria.

Hours 3-4. Study length, normalisation, dot/cross and projections. Calculate torque by hand, then check with a library function. **Hours 5-6.** Study matrices, product order and transposition. Draw a rotation's effect on a basis; explain the result before running code.

Hours 7-8. Study points, directions and shifting the coordinate origin. **Hours 9-10.** Solve a simple force system by hand and check dimensions. If these topics are easy, use a more complex force orientation while retaining the same object and a clear manual check.

Hours 11-20 - practical. First 2 h: tasks 1 and 2 by hand; next 2 h: transformation functions; 2 h: basis and torque plots; 2 h: zero vector, degrees and radians, and reverse transformation; final 2 h: tests and a short explanation without code. Finish when every drawing and number can be related to one chosen coordinate frame.

Block B: calibration and numerical stability

Hours 21-22. Apply Gaussian elimination to support reactions, then repeat with solve. **Hours 23-24.** Study rank and overdetermined systems; construct the calibration matrix on paper. **Hours 25-26.** Study residuals, least squares and QR's role; compare a prediction with the observed point.

Hours 27-28. Explore conditioning with two nearly identical equations and understand solution error versus residual. **Hours 29-30.** Study stiffness-matrix eigenvector directions. Avoid proofs for arbitrary-size matrices until you can explain the 2×2 example.

Hours 31-40 - practical. 2 h: supports and reaction plot; 2 h: encoder generator and calibration; 2 h: held-out validation and a narrow range; 2 h: conditioning and eigenvector directions; 2 h: documenting coefficients and tests. Checkpoint: recover parameters from noise-free data, then deliberately create an unidentifiable calibration.

Before proceeding, write three short answers: why inverse is not a replacement for a normal solve call; why a small residual does not guarantee an accurate parameter; why more identical observations do not resolve insufficient input diversity. If you cannot answer, replace some plot styling with another hand-worked example.

Step-by-step work: hours 41-75

Block C: motion as a function of time

Hours 41-42. Review derivatives and velocity units for linear and angular motion. **Hours 43-44.** Study sin/cos and the chain rule; derive reference q , ω and α . **Hours 45-46.** Study forward and central differences, the time assigned to an estimate and boundary handling.

Hours 47-48. Study integration as accumulation, work done by torque and the trapezoidal rule.

Hours 49-50. Derive partial derivatives and one column of the link-endpoint Jacobian. The derivation is short, so reproduce it unaided.

Hours 51-60 - practical. 2 h: trajectory and exact derivatives; 2 h: finite-difference comparison; 2 h: noise and missing time sample; 2 h: $p(q)$ sensitivity and integral check; 2 h: error plots and conclusions. Do not change noise and step size together without a control run, or you cannot explain the RMSE change.

Deliverable: a table of 'method / step / noise / time window / RMSE / maximum error' and two illustrations: the tangent to the endpoint trajectory and angle recovery by integration. Use three different Δq values for the linear approximation; one small perturbation cannot reveal the model's limit.

Block D: dynamics, integration and presentation

Hours 61-62. Convert the second-order equation into a first-order system; check each term's units. **Hours 63-64.** Write explicit Euler, semi-implicit Euler and RK4 updates, and calculate one step by hand for $x=0.02$ and $v=0$. **Hour 65.** Study local and accumulated error, stability, initial conditions and library-solver tolerances.

Hours 66-75 - practical. 2 h: mass-spring and equilibrium check; 2 h: method and energy comparison; 2 h: pendulum, period and large angle; 2 h: combined encoder→angle→position→velocity processing; 2 h: clean reproducible run, six failures and oral explanation.

Reserve the last 30 minutes for a new variant without copying: change link length, initial angle and step size; predict the direction of period change before running. Then explain what changed in the model and what stayed the same. List two omitted real-leg effects and a measurement that would allow each to be added later.

If integrators take extra time, retain one short file with the shared interface `step(f,t,y,h)` and remove interactive elements. Do not cut force-sign checks, the analytical comparison or the step-reduction experiment: these establish that the mathematical model has been checked, beyond merely running its code.

Acceptance checks and handover to later modules

What to save in robot-math

Four completed notebooks; a function module with clear units; input CSVs; parameter configurations; calibration coefficients; test table; norm, residual, derivative and energy plots; and README with execution order. State the formula and seed for synthetic data; store reference values separately. Label the 'raw measurements' field synthetic until a real sensor is used.

Begin each notebook with a robotics question and end with an engineering conclusion. 'The code ran' is not a conclusion. Suitable statements include: 'the length-scale error increased torque a thousandfold'; 'the narrow encoder range prevents stable scale estimation'; 'the chosen Euler step increases this oscillator's energy'. Fill in all numerical results after your own run.

Passing criteria

1. Each of ten practical tasks has an input, result, independent check and edge case. 2. Parameters, columns and plot axes have units. 3. Analytical and numerical answers are compared over the same time interval. 4. ODE results cover at least three step sizes and both objects. 5. Singularity and instability are explicitly identified. 6. A clean run reproduces conclusions without manually editing intermediate files.

Passing many tests does not replace understanding. For the oral review, choose six questions below at random and answer each with a small diagram or calculation. You may consult NumPy syntax; explain the underlying torque, linear-calibration and ODE formulae independently.

Self-check questions

Why does shifting the coordinate origin affect a point but not a direction? When is $\mathbf{r} \times \mathbf{F}$ zero? What does a negative support reaction mean? How does an overdetermined system differ from an inconsistent one? Why are residual and coefficient error different quantities? How do units affect numerical conditioning?

How do you obtain link-endpoint velocity from joint velocity? Why does numerical differentiation amplify noise? What is a 'linear approximation in angle'? Which initial conditions does a pendulum require? Why is energy conservation useful as a test? How does fixed-step RK4 differ from RK45 in `solve_ivp`?

Subsequent use

M03 uses the same calibration and Jacobian for uncertainty estimation. M04 uses the noisy encoder and derivative to study filtering. M05 uses torque, reactions and eigenvector directions as preparation for mechanics. Preserve the functions' public interface, but do not claim real-robot accuracy from synthetic data. Physical test-bench validation follows electronics and mechanical preparation.

Assigned reading: a short route

Read only the specified passages before the corresponding experiment. References are mainly in English; keep explanations, glossary and report in English. Reading and preparation fit within the existing 75 hours; whole books are not required.

Core reading

1. Gilbert Strang, MIT 18.06: Linear Algebra. [Lectures 1-2, 16, 21](#). Equation geometry and elimination support reaction calculations; projections and least squares support calibration; eigenvector directions support task 6. After each section, solve one 2×2 example by hand.

2. Gilbert Strang, Edwin Herman, Calculus Volume 1 (OpenStax). [§3.1-3.6](#): derivative definition, rules, rate of change, trigonometry and chain rule; [§5.2-5.3](#): definite integral and fundamental theorem. Use in tasks 7-8 for link velocity and angle recovery.

3. Haynes Miller, Arthur Mattuck, MIT 18.03: Differential Equations. [Lecture notes 2, 11-12](#): Numerical methods; Modes and the characteristic polynomial; Good vibrations, damping conditions. For tasks 9-10, explain integration step size, natural frequency and damping effects.

References for implementation

4. Kevin Lynch, Frank Park, Modern Robotics - reference. [§3.2.1 Rotation Matrices, Part 1](#). For task 2, reconstruct the geometric meaning of matrix columns and $R^T R = I$; skip exponential coordinates for now.

NumPy linear algebra. [lstsq: Parameters, Returns](#) and [cond: norms](#). Check A,b shapes, rank and residual meaning; an absent residuals array does not establish exact calibration.

SciPy integration. [solve_ivp: y0, t_eval, rtol, atol, success](#); [RK45: method description](#). In tasks 9-10, distinguish output times, internal RK45 steps and your own constant-step RK4 implementation.

Minimum reading deliverable. Four sheets: change of basis; calibration with residuals; derivative and integral; ODE with initial conditions. Each includes your own formula, units and one independent check.

Glossary: terms and definitions

Vector. A geometric quantity with direction and magnitude; its components depend on the chosen basis, while the quantity itself does not.

Basis. A set of linearly independent vectors in terms of which every vector in the space has a unique representation.

Coordinate frame. An origin together with a basis; point coordinates change when the origin shifts or the axes rotate.

Rotation matrix. A matrix describing an orthonormal basis's orientation; a proper rotation satisfies $R^T R = I$ and $\det R = 1$.

Cross product. In three dimensions, a vector perpendicular to both operands; its direction follows the right-hand rule and its magnitude is the parallelogram's area.

Torque. The vector $\mathbf{r} \times \mathbf{F}$ about a chosen point; it describes a force's rotational effect and is measured in $\text{N} \cdot \text{m}$.

Rank. Number of independent matrix columns or rows; it determines how many independent constraints a linear system contains.

Residual. The difference $\mathbf{A} \cdot \mathbf{x} - \mathbf{b}$ for a proposed solution; a small residual alone does not guarantee accurate unknown parameters.

Least squares. Choosing parameters to minimise the sum of squared residuals; fits a model to redundant, usually inconsistent measurements.

Conditioning. Sensitivity of a problem's result to small input changes; depends on its formulation, units and scaling.

Eigenvector direction. Direction of a nonzero vector $\mathbf{v}$ satisfying $\mathbf{A} \cdot \mathbf{v} = \lambda \mathbf{v}$. The matrix scales the vector while preserving the line on which it lies.

Derivative. Limit of the ratio of function increment to argument increment; differentiating position with respect to time gives velocity in m/s .

Integral. Accumulated contribution of a function over an interval; integrating velocity over time gives displacement with consistent initial conditions.

Jacobian. Matrix of partial derivatives of output coordinates with respect to inputs; approximately maps small input changes to output changes.

Ordinary differential equation (ODE). Equation relating an unknown function of one variable to its derivatives; a motion model also requires initial conditions.

Time step. Time interval for one numerical state update; affects trajectory error, stability and computation cost.

Numerical stability. Property of a numerical scheme in which perturbations do not grow unacceptably for the problem and chosen integration step.

Equipment, installation and checks

Complete preparation within the first study blocks in place of another general overview. The budget remains 75 hours; delivery waiting time is excluded. Statuses below specify actions for you to take, not purchases or installations already completed.

Equipment and purchases

Use the MacBook Pro M4, editor, Git, uv/Python, NumPy, Matplotlib and pytest from M01. **No new purchases.** All M02 objects are computational: no motor, IMU, electronics or mechanical parts are needed. Books are available through the supplied links; print copies are optional.

Install now: macOS ARM

Add **SciPy** to robot-math if absent: [official installation guide](#). This is M02's addition for ODEs; the earlier 'SciPy from M01' wording means using the same environment, not an earlier mandatory installation. Pin its version and lockfile.

For the four notebooks, add **JupyterLab and ipykernel** if the editor does not already open notebooks: [Jupyter installation](#). The notebook kernel must use project Python. **SymPy is optional**, only for checking symbolic derivatives: [official instructions](#). Hand derivation remains part of the task.

Installation checks

1. Record Python path, arm64 architecture and library versions. 2. Run `import numpy, scipy, matplotlib`; open a notebook with the project kernel. 3. Solve a 2×2 system and check $A \cdot x = b$. 4. Integrate $y' = -y$, $y(0) = 1$ to $t = 1$ and compare with $\exp(-1)$. 5. Restart the kernel and run all cells in order.

Check numerical error with explicit `rtol/atol` and a comparison tolerance. Save one PNG without an interactive window, one CSV and a pytest result. This establishes environment readiness; verify each model's accuracy separately in the tasks.

Optional work and later modules

You may repeat the short calculation in M01's Ubuntu 24.04 ARM64 in UTM if using that environment. Main work runs natively on the Mac. M02 requires no ROS 2, Gazebo or GPU; do not create a separate numerical-library environment in the VM without a task that needs it.

Ready for M03: dependencies are pinned, notebook→CSV→plot is saved, and robot-math runs cleanly. If installation takes too long, reduce plot styling while retaining mathematical checks and the overall budget.

Motion fundamentals: from joint angle to foot velocity

Compulsory, within 75 hours. Use 1 h from session 49-50 and 2 h of practice 51-60 already allocated to p(q) sensitivity and checks. This connects tasks 2 and 8; it does not add tasks beyond the plan. Before starting, convert 30° to $\pi/6$ rad and explain why m/s differs from rad/s.

Why joint velocity differs from point velocity

Consider a rigid link of length $L=0.12$ m with one revolute joint. Frame A is fixed: its origin is on the joint axis, x points right, y upwards, and positive q is anticlockwise from x. The foot point has coordinates $p_A(q)=(L \cos q, L \sin q)$. Length is constant; backlash and deformation are excluded. This requires geometry, not link mass.

For a small rotation, endpoint displacement is approximately tangent to the circle. By the chain rule, $v_A=dp_A/dt=J(q)\cdot\dot{q}$, where $J=(-L \sin q, L \cos q)^T$, in m/rad. Thus $|v_A|=L|\dot{q}|$ and $p_A\cdot v_A=0$. The Jacobian depends on configuration: equal angular velocities at different q give equal endpoint speed but different directions. Even at constant $\dot{q}$, velocity changes direction: acceleration $a_A=-p_A\cdot\dot{q}^2$ points towards the joint. This last equality assumes $\ddot{q}=0$.

Numerical example and change of fixed frame

At $q=\pi/6$ rad and $\dot{q}=0.5$ rad/s, $p_A=(0.103923; 0.060000)$ m, $v_A=(-0.030000; 0.051962)$ m/s and $|v_A|=0.060000$ m/s. Over a short time $\Delta t=0.01$ s, tangential displacement is approximately 0.6 mm; distinguish distance travelled in this interval from a change in one coordinate.

Let $R_{BA}=[[0,-1],[1,0]]$ describe A's axes in another fixed frame B, and $t_{BA}=(0.20;0.10)$ m be A's origin position in B. Then $p_B=R_{BA}\cdot p_A+t_{BA}=(0.140000;0.203923)$ m, but $v_B=R_{BA}\cdot v_A=(-0.051962;-0.030000)$ m/s. The derivative of a constant translation is zero. A moving B introduces additional terms; that case is not modelled here.

Two checks using existing functions

1. In robot-math, compute the central difference $[p(q+\dot{q}h)-p(q-\dot{q}h)]/(2h)$, $h=10^{-4}$ s, and compare with $J\dot{q}$. For these noise-free float64 values, use a teaching tolerance of 10^{-8} m/s on the error norm. Save a table for $q=0, \pi/6, \pi/2$; check tangency, speed magnitude and inverse coordinate transformation.

2. On the leg diagram, label angle, both frames and the v arrow. Predict doubled speed when L doubles and a reversed arrow when $\dot{q}$ changes sign. Deliverables: one drawing and tests for tasks 2/8. These check kinematics; they do not establish the existence of a motor capable of producing the motion.

Read before calculating: Lynch, Park, Modern Robotics: [chapter 5 introduction, deriving \$\dot{x}=J\dot{q}\$; §3.3.1, changing coordinate frames](#). Spatial velocities are studied in M08.

Uncertainty: how much to trust a sensor

M3 • 29 Nov 2026 - 27 Dec 2026 • 75 hours

This month's deliverable

In M02, you obtained an angle, force or position as a number. M03 adds variability, error sources and conditions under which a conclusion remains valid. Deliver a **portfolio of uncertainty and optimisation work**: a 30-repeat measurement experiment, encoder and IMU noise models, calibration through optimisation, poor-conditioning diagnostics and a report on link-endpoint position uncertainty.

This month's questions are practical: does encoder jitter differ from actual motion; how should two angle estimators be compared; how much independent information does a long recording contain; why did an optimiser produce a plausible but incorrect scale; how far does joint error move a foot or fingertip? Do not yet build a Kalman filter, machine-learning model or QP controller: first learn to check the assumptions they will require.

Shared object and data

Continue with the M02 link $L=0.12$ m, $p(q)=(L \cos q, L \sin q)$ and calibration $q=a \cdot u+b$. Create data/synthetic, data/measured, experiments, calibration, uncertainty and report. In the data manifest, record each file's source, units, random-number generator seed, date, independent-run identifier and how ground truth was obtained. Keep synthetic ground truth separate from algorithm inputs.

The physical experiment needs no actuator: place a stationary IMU in the same fixture thirty times. An existing phone that exports raw readings also works. If no sensor is available, first practise the procedure with synthetic data; the real-measurement section remains incomplete until the experiment is done. Generated noise does not replace measured device repeatability.

Prerequisites and scope

You need array means, matrices, least squares, derivatives and the meaning of the Jacobian from M02. In the first two hours, explain the differences between true value, measurement, model residual and numerical error. Review why the calculated 'velocity' of a stationary encoder can fluctuate. M03 adds ways to describe these fluctuations and assess the robustness of conclusions.

Begin with definitions: a random variable models possible outcomes; a sample is the data actually obtained; a parameter is an unknown distribution characteristic; an estimator is a rule for computing from a sample. One distribution plot does not prove normality, and optimiser success does not establish physically correct calibration. Both distinctions must appear in your checks.

75 hours: experiments, analysis and reproducibility

Four work blocks

A, hours 1-20: 10 h theory + 10 h practice. Random variables, distributions, expectation, variance, covariance, correlation, independence, conditional probability and Bayes' rule. Practical deliverables: encoder quantisation and IMU noise models, and diagnosis involving a rare event.

B, hours 21-40: 10+10 h. Sampling plans, confidence intervals, effect size, outliers, robust summaries, hypotheses, repeatability and data leakage. Deliverables: a 30-repeat protocol, raw recordings and a comparison of two estimators across independent runs.

C, hours 41-60: 10+10 h. Objective functions, constraints, gradients, Hessian meaning, gradient descent, line search, local minima, least squares and regularisation. Deliverables: calibration with `scipy.optimize`, held-out validation and residual analysis.

D, hours 61-75: 5+10 h. SVD, pseudoinverse, poor conditioning, covariance propagation through a Jacobian, Monte Carlo and bootstrap. Deliverable: an endpoint uncertainty region and checks of linear-approximation limits. Total: 35 h of theory and analysis plus 40 h of practice.

Organising short and long sessions

Divide each 10 h of theory into five two-hour sessions and the final 5 h into 2+2+1. Each 2 h session should cover one concept, one hand-worked example and one decision about the forthcoming experiment. Schedule 10 h practical blocks in available Sunday or holiday sessions from the original plan; divide each into five 2 h segments and count breaks separately.

The calendar window follows the original plan. A boundary date may coincide with a neighbouring module; holidays, rest and integration stages are preserved. Allocate step hours sequentially to available sessions, without double counting or adding another 'compensation' hour.

Experiment log

Before collecting data, record the question, primary metric, unit of independent repetition, recording-exclusion criterion and stopping rule. Afterwards, add actual deviations from the protocol. Do not delete an inconvenient recording because it worsens the mean: retain it with a reason and, if necessary, present analysis with and without exclusion.

If time is short, remove optional CVXPY comparison and complex interactive plots. Retain 30 repeats, basic calibration, splitting data by run, SVD diagnostics and Monte Carlo calculation. Another book cannot replace the measurement work. Being able to say 'these data do not yet establish the claim' is a valid laboratory outcome.

Probability and sensor-noise models

Topics to study

Study discrete and continuous random variables, interval probabilities, probability density and cumulative distribution functions. For sensors, use a normal distribution as a working model of small disturbances, a uniform distribution as a quantisation simplification, and a Bernoulli distribution for 'packet lost'. Expectation describes the centre; variance is the mean squared deviation; standard deviation has the same units as the measurement.

Covariance describes joint variation of two channels; correlation normalises it by their standard deviations. Zero correlation alone does not establish independence. Consecutive stationary-IMU samples can be dependent because of internal filters and drift. A thousand samples therefore need not equal a thousand independent experiment repeats.

Task 1. Encoder quantisation

Input: true angle q within $-0.5 \dots 0.5$ rad and quantisation step $\Delta = 0.001$ rad. Compute rounded q_{meas} , error $e = q_{\text{meas}} - q$, a histogram and empirical cumulative distribution function. For uniform coverage of quantisation intervals, compare variance with $\Delta^2/12$. Then take a stationary angle exactly between two codes and explain why this distribution model need no longer describe the observations.

Deliverables: generator, two different input trajectories and a table of mean, standard deviation (SD) and maximum error. Check $|e| \leq \Delta/2$ under the chosen rounding convention, apart from stated numerical tolerances. Failure case: the model saturates beyond range; an error does not become uniform merely because the data came from an encoder.

Task 2. Shared noise on two IMU axes

Generate 5000 synthetic error pairs by adding each axis's independent noise to a shared component. Record the covariance matrix and explain its units: for acceleration, $(\text{m/s}^2)^2$. Plot the point cloud and compare it with independent data having the same per-channel variance. Equal axis standard deviations do not imply equal uncertainty in their sum.

Check covariance symmetry and nonnegative eigenvalues, allowing for rounding. Add slow shared drift and compare the recording's first and second halves. Deliver three point clouds and explain the effect shown by each. The aim is to recognise when constant-noise assumptions fail, rather than force a 'correct' distribution onto the data.

Conditional probability and diagnostic reliability

Bayes' rule for a clear robot event

Study $P(A|B)$, the law of total probability and Bayes' rule using a frequency table. Example: A is a genuine foot-contact failure; B is a threshold diagnostic trigger. Sensitivity $P(B|A)$ and the probability of a real failure after a trigger $P(A|B)$ answer different questions. The latter also depends on event prevalence.

For the teaching example, use failure probability 1%, sensitivity 95% and a 5% false-trigger probability without failure. These are illustrative calculation values, not properties of an actual sensor. Among 10 000 virtual episodes, expect 95 true and 495 false triggers; about 16.1% of all alerts are true. Check using a probability tree or table.

Task 3. Rare events and thresholds

Create an episode generator with a known label and diagnostic feature. Calculate TP, FP, FN, TN, then $\text{precision} = TP/(TP+FP)$ and $\text{recall} = TP/(TP+FN)$. Show how both metrics and trigger frequency change across several thresholds. If a denominator is zero, return 'undefined' rather than silently assign a perfect score.

Repeat with event probabilities 0.1%, 1% and 10%, retaining the feature's conditional distributions. Deliver a table and short note identifying the more consequential error type for this test bench. Here, diagnostics only create log events; this model does not verify a robot's actual protective functions.

Task 4. Why independence needs checking

Generate white noise and a correlated series $e_t = 0.95 \cdot e_{t-1} + \eta_t$. Scale them to approximately equal SD; compare time plots and adjacent-sample correlation. Average over equal-length windows and measure variation of means across non-overlapping windows. Equal sample counts do not guarantee equal improvement.

Control check: shuffle the correlated series and repeat. Shuffling preserves the marginal distribution but destroys temporal structure. Save two plots and explain why randomly splitting rows of one recording can yield an overly optimistic algorithm evaluation. Exact effective-sample-size theory is optional; the meaning of an independent run must be understood now.

Statistics: a real 30-repeat experiment

Write the protocol before recording

Task 5 assesses the repeatability of a stationary IMU repositioned in the same fixture. Choose a device that exports timestamps and raw accelerometer and gyroscope readings; record its settings and available rate. Perform 30 separate placements, keeping the device still each time, waiting the same predetermined settling interval and recording for the same duration. Choose a duration supported by export; the teaching guideline is 2 s after settling.

The repeat unit is one independent run after repositioning, not one CSV row. Record run_id, fixture orientation, time, temperature if available, observed interference and setting changes. Within each run, calculate the axis mean and SD; across 30 runs, examine variation of these means.

Repositioning adds geometric uncertainty, so describe the result as repeatability of the whole procedure.

Calculations and interpretation

For the 30 means, find the mean, median, SD and interquartile range. If assumptions are reasonable, construct a 95% t-confidence interval for the mean: $\text{mean} \pm t \cdot s / \sqrt{n}$. This interval concerns the procedure's mean; it is neither a range covering 95% of future individual readings nor a promise to cover a particular sensor's true angle.

If run independence or stability fails, show a time plot and limit the conclusion. Do not treat 30 as universally sufficient: justify this pilot sample size using available time and desired interval width, then estimate the additional independent repeats needed to halve the interval width under the same assumptions.

Task 6. Outlier robustness and effect size

Compare two constant-level estimates on the same 30 runs: within-recording arithmetic mean and median. Add labelled individual outliers to copies of several recordings. Calculate the change in error relative to the clean copy and paired estimate differences, rather than choose an algorithm from histogram appearance. Express effects in physical units and relate them to the robot task.

Deliverables: protocol.md, 30 raw-data files or one CSV with run_id, a summary table and a plot in test order. Edge cases: empty run, gaps, changed sampling rate and noticeable device motion. Define exclusion criteria before viewing results. Without independent ground truth, compare robustness and repeatability without claiming absolute IMU accuracy.

To introduce hypothesis testing, state H_0 : the mean paired difference between the two estimates is zero. Discuss false rejection of H_0 and missing a real effect; statistical significance does not say whether an effect is useful to the robot. Report the difference in physical units and its interval in the main conclusion, rather than a p-value alone.

Calibration optimisation: from error to parameters

Minimum theory

A loss function answers a specific question: how far do model predictions differ from measurements? Parameters θ may include encoder scale and zero; constraints express meaningful admissible values. The gradient gives the local direction of increasing loss; the Hessian describes local curvature. Begin with a two-variable quadratic bowl that can be drawn and whose derivatives can be checked by hand.

Gradient descent updates $\theta_{\text{new}} = \theta - \alpha \cdot \text{grad}$. Excessive α can increase error; too small an α slows progress. Learn backtracking line search: reduce the step until the objective decreases by the required amount. Distinguish stopping criteria based on step, gradient and loss; a small step may indicate poor scaling rather than a good minimum.

Task 7. Implement descent for scale and zero

Using M02 calibration, scale u to a comparable magnitude and derive the sum-of-squares gradient with respect to a and b . Check it with central finite differences at five parameter points. Run descent from three initial guesses with two step strategies; save the loss curve, iteration count and final coefficients.

On noise-free data, compare with linear `lstsq`. For poorly scaled u , explain elongated contours and slow convergence. Deliverables: short implementation, contour plot and table. Edge case: wrong gradient sign; a test must detect increasing loss and stop the run. Choose the gradient-check threshold relative to function scale and finite-difference step.

Task 8. Constrained nonlinear model

Use the teaching model $q = a \cdot u + b + c \cdot u^2$ with normalised u . Prepare independent calibration and verification series; generate a small known nonlinear component. Pass the residuals vector, not total loss, to `scipy.optimize.least_squares`. Specify bounds, initial guess and parameter scales; compare ordinary least squares and robust loss with labelled outliers.

Regularise c by adding residual $\sqrt{\lambda} \cdot c$ after choosing a dimensionless scale. Compare several λ values on the validation set, then evaluate the result once on the test set. Show a case where the linear model performs better on held-out data despite greater training error. The optimum is an estimate within the chosen model; still check residual dependence on position and motion direction.

SVD, pseudoinverses and uncertainty propagation

Core concepts

SVD factors $A=U \cdot \Sigma \cdot V^T$ into orthogonal input/output directions and stretch factors. A very small singular value indicates a parameter direction weakly represented in the data. A pseudoinverse selects a solution of a singular problem according to a defined rule; it cannot recover information the experiment never measured.

For $p=f(x)$ and small uncertainties near $\bar{x}$, use $\Sigma_p \approx J \cdot \Sigma_x \cdot J^T$. Σ_x is input covariance; evaluate J specifically at $\bar{x}$. Covariance has squared units of the corresponding output quantities. This is a local approximation, so compare with nonlinear Monte Carlo calculation, especially near geometric singularities or with large variability.

Task 9. Weakly determined calibration

Take the calibration matrix for a wide u range and the same matrix with inputs restricted to a narrow band. After applying the same scaling, compare singular values, cond and parameter changes under a small q perturbation. Find the right singular vector for the smallest singular value: it identifies the parameter combination least distinguishable in the data.

Save a plot of the uncertain parameter direction and predict which u values a new experiment should add. Then add extreme positions to the synthetic data and verify improvement. Test: all u values identical; pinv may return a solution, but the report must identify insufficient rank and nonunique physical calibration.

Task 10. Foot-position variability

Use $L=0.12$ m, $q=30^\circ$ and teaching standard uncertainties $\sigma_L=0.0005$ m and $\sigma_q=0.005$ rad. Repeat with large noise $\sigma_q=0.3$ rad; set nonzero correlation $\rho=0.6$ while keeping covariance positive semidefinite. Let $x=(L,q)$, and derive J for $p=(L \cos q, L \sin q)$. First assume independent errors, then add admissible nonzero covariance. Compute Σ_p , its eigenvector directions and a 10 000-point Monte Carlo cloud.

Compare coordinate SDs and covariance from the linear prediction with sample estimates. Repeat for small and large angle noise; record relative discrepancies while avoiding division by nearly zero covariance. The ellipse visualises the assumed uncertainty model, not a guaranteed real-robot position region. Negative sampled length requires revising the distribution model rather than silently deleting inconvenient points.

Integrated laboratory and six validity checks

Assemble the final package

Combine calibration and uncertainty: input `encoder_count` passes through the chosen $q(u, \theta)$ model, then geometry $p(L, q)$. For each validation run, save predicted position, estimated variability and actual error if independent ground truth exists. Synthetic experiments have known ground truth; the real stationary-IMU experiment usually establishes only repeatability.

Create three scenarios: nominal noise, correlated drift and outliers. Split independent series into training (train), validation and test sets. Choose the model, λ and threshold using train and validation; open test only after fixing settings. If you subsequently change the solution, mark the test set as used and prepare a new independent check: repeated tuning removes test independence.

Required error checks

1. Leakage. Compare random row splits with whole-run splits under slow drift; show why the former appears better. **2. Pseudoreplication.** Duplicate each row 100 times; the reporting algorithm must not count these as new independent experiments. **3. Outlier.** Add one large spike and compare mean, median and robust loss without deleting the original.

4. Singularity. Nearly constant u must trigger a weak-identifiability warning regardless of solver success. **5. Nonlinearity.** Large q noise invalidates linear covariance propagation; Monte Carlo must reveal the discrepancy. **6. Nonstationarity.** Different means at the start and end require a separate drift discussion rather than one 'sensor σ '.

Bootstrap: an additional interval check

Resample with replacement from the 30 independent run results and estimate intervals for the median or paired difference. Record resample count and seed; repeat with another seed to observe computational variation. Do not resample correlated individual rows as independent observations. Temporal dependence requires another scheme, such as block bootstrap, which is sufficient to identify as a later topic.

Distinguish observation and interpretation on every report page. A useful format is: 'between-run SD is ... under these conditions; it includes repositioning; no independent absolute reference is available'. Do not fill numerical fields in advance. Save metrics with units, interval bounds, independent-repeat count and specific applicability limits.

Session sequence: hours 1-40

Block A - build a noise model and identify its limits

Hours 1-2. Review error, true value and estimator definitions; prepare the data structure. **Hours 3-4.** Study distributions, expectation and variance; calculate probabilities of three discrete-encoder outcomes by hand. **Hours 5-6.** Study covariance and correlation with two channels. **Hours 7-8.** Study conditional probability and Bayes' rule through a diagnostic table. **Hours 9-10.** Study independence, temporal dependence and the experimental unit.

Hours 11-20 - practice. 2 h: quantisation and $\Delta/2$ check. 2 h: independent and shared IMU noise. 2 h: rare event, TP/FP/FN/TN. 2 h: white and correlated series. 2 h: plots, generator tests and conclusions. For each plot, state the sensor assumption built into the generator and a real effect it does not yet describe.

Hour 20 checkpoint: explain to someone without statistics training why high diagnostic sensitivity can coexist with many false alarms, and why 5000 adjacent samples do not equal 5000 repositionings. If you cannot, revisit the frequency table and temporal-dependence plot.

Block B - make a justified comparison

Hours 21-22. Study sampling and estimation; choose the measured quantity. **Hours 23-24.** Write the 30-repeat protocol and exclusion criteria. **Hours 25-26.** Study SD, standard error and a t-confidence interval for the mean. **Hours 27-28.** Study outliers, median, IQR and effect size. **Hours 29-30.** Study training, validation and test sets, paired comparison and repeatability.

Hours 31-40 - practice. 2 h: fixture preparation and pilot export; 2 h: 30 measurement runs with metadata; 2 h: integrity checks and individual-run summaries; 2 h: intervals and estimator comparison; 2 h: report and excluded-recording review. If longer recordings are necessary, reduce styling and use prepared summary scripts while retaining the independent-repeat count.

Checkpoint: another person can understand the dataset's experiment boundaries, what remained stationary, the coordinate frame used and why some recordings are marked doubtful. Without this information, measurements cannot support rigorous calibration comparisons in later months.

Session sequence: hours 41-75

Block C - optimise and verify calibration

Hours 41-42. Write the loss function for $q=a \cdot u+b$ and check dimensions. **Hours 43-44.** Derive and numerically check the gradient; draw curvature in two parameters. **Hours 45-46.** Study fixed steps and backtracking. **Hours 47-48.** Study nonlinear-model residuals, constraints and initial guesses. **Hours 49-50.** Study regularisation and model selection using validation data.

Hours 51-60 - practice. 2 h: implement descent and compare with `lstsq`; 2 h: `least_squares` with bounds; 2 h: outliers and robust losses; 2 h: several λ values and independent series; 2 h: gradient checks, model saving and residual analysis. For every run, retain status, evaluation count, initial guess, objective value and physical coefficients.

In the last two-hour segment, deliberately create a problem with local minima, such as periodic zero-offset ambiguity in $\sin(u+b)$, and start from different b values. This briefly introduces local minima and period ambiguity; do not add this complexity to the main calibrator. Conclude that results depend on the model and observational information, not merely the algorithm's name.

Block D - uncertainty in the result

Hours 61-62. Study SVD geometry and truncation of small singular values; check a diagonal example by hand. **Hours 63-64.** Derive J with respect to L and q , and propagate Σ . **Hour 65.** Study Monte Carlo, bootstrap, and measurement uncertainty versus computational variation in an estimate.

Hours 66-75 - practice. 2 h: poor calibration, SVD and a revised measurement plan. 2 h: analytical endpoint covariance. 2 h: Monte Carlo and large noise. 2 h: bootstrap by run and consolidated report. 2 h: six error checks, a fresh run and defending conclusions.

Final half-hour task without copying: specify two noise models with equal SD but different cross-axis dependence. Before running, predict which gives greater variation in a chosen foot-position projection. Check using the covariance formula and simulation. If the prediction was wrong, correct the explanation and preserve the mistake as a useful finding.

Do not measure success by method count. The required minimum is one explainable calibrator, one real experiment with justified conclusions, one poor-identifiability example and one error-propagation check. Rust/C++ may read M01 logs, but reimplementing all statistics in three languages is outside the 75 h.

Laboratory acceptance and the future robot

Verifiable deliverables

The package must include ten tasks with initial conditions, a 30-repeat protocol, labelled real and synthetic data, distribution models with limitations, calibration configurations, splits by series, residual plots, SVD diagnostics, covariance and Monte Carlo calculations. Record commands, library versions and report-reproduction order in README.

Every substantial number must answer: 'which data?', 'what unit of repetition?', 'what physical units?' and 'which metric?'. A mean without run count or an interval without a calculation method fails acceptance. Do not report more significant figures than uncertainty supports; preserve full machine precision separately for reproduction.

Check your understanding

Explain variance versus SD; independence versus zero correlation; random spread versus systematic bias; repeatability versus absolute accuracy. Use a table to distinguish $P(\text{failure}|\text{alarm})$ from $P(\text{alarm}|\text{failure})$. Why can you not narrow a confidence interval simply by copying rows?

For optimisation: what goes into residuals; why normalise parameters; why is success insufficient; what does a small singular value reveal; what does regularisation change, and what does it not know about the robot? For uncertainty: where is J evaluated; what are Σ 's units; when does linear propagation fail; which units should bootstrap resample?

Practical assessment: take a new independent synthetic series with a seed unknown to you, calibrate and compare estimates with saved ground truth. Add drift and state which assumptions fail. For real data, show one supported conclusion and one requiring a different reference or experiment.

Handover

Pass M04 the actual sampling rates, gaps, noise summaries, original recordings and corrupted copies for filtering work. Pass M05 length-and-angle error propagation as an example connecting tolerances to foot position. Later filters and multi-joint calibration can reuse these functions, but a sensor's numerical covariance must be remeasured after mounting or settings change.

Keep open questions about temperature drift, vibration dependence, axis correlation, reverse mechanism travel and reference accuracy. For each, describe the smallest future experiment that could test it. This makes the laboratory a foundation for later hardware tests, beyond a collection of one-off plots.

Assigned reading: a short route

Read only the specified passages before the corresponding experiment. References are mainly in English; keep explanations, glossary and report in English. Reading and preparation fit within the existing 75 hours; whole books are not required.

Core reading

1. Allen B. Downey, Think Stats, 3rd online edition. [Chapters 2, 4, 7, 8](#): Distributions; Cumulative Distribution Functions; Relationships between variables; Estimation. Tasks 1-6 need distributions, relationships and estimation uncertainty; adapt the examples to your measurements.

2. NIST/SEMATECH, e-Handbook of Statistical Methods. [§1.3.5.2 Confidence Limits for the Mean](#): interval from 30 independent runs; [§1.3.5.11, skewness and kurtosis](#): noise-shape reference. Write interval assumptions before inserting numbers.

3. Stephen Boyd, Lieven Vandenberghe, Convex Optimization (2004). [§4.1, §6.1, §6.3, §9.2-9.3](#). Problem formulation; norm approximation; regularisation; descent and gradient methods. In tasks 7-8, connect loss, constraints and parameter scale to physical calibration. General-case proofs are optional.

Compact implementation references

SciPy optimize. [least_squares: fun, jac, bounds, x_scale, loss](#). Before task 8, check stopping criteria and output fields. A success status does not replace unit, bound and held-out-data checks.

NumPy. [svd: Returns, Notes](#). For task 9, reconstruct A from U,S,Vh, then compare the pseudoinverse with pinv. Explain what changes when a small singular value is truncated.

SciPy stats. [bootstrap: data, statistic, paired, method](#). Resample independent runs; use paired only for paired comparisons. Optional constrained least-squares comparison: [CVXPY example](#).

Reading sequence. First distributions and the 30-repeat plan, then estimates and intervals, then optimisation and SVD. Deliver one sheet of experiment assumptions and one defining the calibration problem.

Glossary: terms and definitions

Random variable. A numerical function of a possible random-experiment outcome; its distribution specifies probabilities of values or ranges.

Independence. Property of events or variables whereby information about one does not change the probability distribution of the other.

Conditional probability. Probability of A given B: $P(A|B)=P(A \cap B)/P(B)$, provided $P(B)>0$.

Sample. The particular set of observations obtained; under temporal dependence, its size is not the number of independent observations.

Parameter and estimate. A parameter characterises an unknown model; an estimate approximates it from a particular sample using a specified rule.

Bias. Difference between an estimator's expectation and the true parameter. Averaging does not remove constant sensor bias.

Variance. Expected squared deviation from the mean; describes variability and has squared units of the measured quantity.

Standard deviation. Square root of variance; describes individual-value variability in original units, such as rad or m/s².

Standard error. Standard deviation of an estimator's distribution over repeated samples; describes estimation uncertainty rather than individual-reading variability.

Covariance. Mean product of centred deviations of two quantities; its sign describes the direction of their joint linear variation.

Correlation. Normalised covariance; describes linear association, but zero generally does not imply independence.

Confidence interval. Interval produced by a procedure that, over repeated experiments, covers a fixed parameter at the specified frequency.

Bootstrap. Estimation of a statistic's distribution by repeated sampling with replacement. The resampling unit must match the original experiment's independent unit.

Loss function. Scalar measure of model-data discrepancy; its choice determines which errors have greater influence on calibration.

Gradient. Vector of a scalar function's partial derivatives; gives the local direction of greatest increase for the chosen coordinates and metric.

Regularisation. An additional preference or penalty in estimation that limits unstable solutions at the cost of possible systematic bias.

SVD (singular value decomposition). Factorisation $A=U\Sigma V^T$ for a real matrix; singular values describe amplification along mutually orthogonal directions.

Identifiability. Ability to distinguish parameters from available observations and inputs; repeating the same experiment can preserve model ambiguity.

Equipment, installation and checks

Complete preparation within the first study blocks in place of another general overview. The budget remains 75 hours; delivery waiting time is excluded. Statuses below specify actions for you to take, not purchases or installations already completed.

Use existing equipment

Use the MacBook Pro M4 and M02 Python environment with pinned NumPy, SciPy, Matplotlib, pytest and the chosen notebook environment. For physical work, use **an existing phone with raw IMU data or an existing IMU**, a stable fixture and a way to repeat placement. Perform 30 separate placements and runs, not 30 adjacent samples.

Purchases if needed

No new mandatory M03 purchases. A motor and separate new IMU board are unnecessary before M06. If the phone cannot supply the required data, use an available existing sensor; synthetic work can begin immediately, but the real experiment remains open until 30 physical repeats are completed.

Install now

On the phone: an app exporting original accelerometer readings, timestamps and units; for example, [phyphox](#), [Acceleration with g](#). Check sensor availability on your model. Choose readings including gravity; do not describe 'linear acceleration' after hidden processing as a raw channel.

On macOS ARM: no new mandatory packages; use `scipy.stats` and `scipy.optimize` from M02. If the file format requires another package, add only that package and update the lockfile. Install CVXPY solely for the optional constrained-calibration comparison, not as an M03 passing requirement.

Readiness checks

Before the series, export one pilot CSV containing time in seconds, three axes, units, run identifier and sensor-mode name. Check finite values, timestamp ordering, gaps and duration. On a stationary support, raw accelerometer magnitude should be near g; this checks mode and scale, not full calibration.

Open the CSV after restarting the notebook. On synthetic data with a known seed, calculate mean, SD and an interval; recover known a, b with `least_squares` and check SVD matrix reconstruction. Save software versions and the data manifest. Successful optimisation does not replace checking ground truth and run independence.

Before the next stage

The raw-data archive must contain 30 identifiable physical repeats or explicitly mark that part incomplete. Keep export settings and synthetic data separately; assess laboratory readiness against the original M03 criteria without adding hours.

Motion fundamentals: how angle error moves the foot

Compulsory, within 75 hours. Use 1 h from 63-64 and 4 h from block 66-75 already allocated to analytical covariance and Monte Carlo in task 10. No new measurements or equipment are required. Entry check: explain zero-offset error, random variability and standard error of the mean.

Displacement direction depends on the error source

Use the task 10 link: $L=0.12$ m, $q=\pi/6$ rad, in a fixed planar frame with x right and y upwards; $p=(L \cos q, L \sin q)$. Parameters $u=(L, q)$ have small independent zero-mean errors: $\sigma_L=0.0005$ m, $\sigma_q=0.005$ rad. This is a specified teaching model, not a claim about real-encoder accuracy.

A small length error moves the endpoint along the link: $\partial p / \partial L = (\cos q, \sin q)^T$. Angle error acts tangentially: $\partial p / \partial q = (-L \sin q, L \cos q)^T$. Assemble these columns into J to obtain $\delta p \approx J \delta u$. Here u denotes the parameter pair, not `encoder_count` in the calibration model.

Covariance transforms as $\Sigma_p \approx J \cdot \text{diag}(\sigma_L^2, \sigma_q^2) \cdot J^T$. Independent input errors do not imply independent x and y : both outputs depend on L and q . Standard spread along the link is 0.5 mm and across it $L \sigma_q = 0.6$ mm; these are ellipse semiaxes at one standard uncertainty, not a guaranteed foot-position region.

A calculation you can check by hand

At 30° , $\Sigma_{xx}=0.2775$ mm², $\Sigma_{yy}=0.3325$ mm², $\Sigma_{xy}=\Sigma_{yx}=-0.047631$ mm². Thus $\sigma_x \approx 0.5268$ mm and $\sigma_y \approx 0.5766$ mm. The matrix trace is 0.61 mm², the sum $0.5^2 + 0.6^2$; rotating axes changes components but preserves this trace.

Now add a constant zero-offset error $\Delta q = +0.01$ rad. The linear mean shift $J_q \Delta q \approx (-0.600; 1.039)$ mm has length 1.2 mm. The exact change $p(q + \Delta q) - p(q) \approx (-0.605; 1.036)$ mm. Repeating one position can reduce uncertainty in the estimated mean, but does not remove this constant geometric shift.

Verify task 10 and decide what to calibrate

1. Using the same 10 000-point Monte Carlo cloud, compare both variances and covariance with the calculation, then rotate axes by 90° and check trace and eigenvalues. Save a drawing of the link, ellipse and cloud. Do not use relative error of nearly zero covariance as the sole metric; allow for Monte Carlo's own variability.

2. Add Δq separately and show the cloud-centre shift; then restore the task's large $\sigma_q = 0.3$ rad. Explain why a tangent describes the arc less accurately. In the report, state what needs correcting: zero offset, length or random-error model. M03 phone repeatability alone does not calibrate a future leg's geometry.

Read before calculating: NIST/SEMATECH, [§2.5.5: propagation of error, sensitivity coefficients](#); Lynch, Park, [Modern Robotics, chapter 5 introduction: Jacobian](#).

Electronics and signals for a robot sensor node

M4 • 27 Dec 2026 - 31 Jan 2027 • 75 hours

What you will build

M04 delivers a **low-voltage electrical experiment log and a sensor-signal filtering report**. The first part teaches assembly and measurement of simple future-robot circuits: a status indicator, analogue-input divider, digital contact and a switch for a small inductive load. The second explains why functioning electronics do not guarantee usable data: sampling rate, quantisation, aliasing and delay change the observed motion.

These tasks directly support leg and hand design. A foot-contact sensor or limit switch must produce a defined logic level. An analogue position sensor requires a suitable voltage range. A body-mounted IMU sees both useful tilt and vibration. Filtering may suppress jitter, but delayed angles can degrade later feedback. Measure this tradeoff in M04 before programming the M06 controller.

Equipment and monthly scope

You need an adjustable supply with current limiting and a suitable low-voltage output, a multimeter, breadboard, wires, resistors, LEDs, small capacitors, a switch and switching-circuit components. Frequency experiments may use simulation; an oscilloscope or logic analyser allows physical measurement but does not replace circuit understanding. First record available equipment specifications, then select compatible components.

Battery packs and high currents are outside the plan before checkpoint G1. Here, use bench circuits with limited stored energy; leg power drives, loaded motors and whole-robot power come later. A small relay coil with known specifications and unloaded contacts is sufficient for the inductive experiment. Derive current limits and fuse ratings from the actual components rather than copy universal values.

Preparation after M02-M03

Retain M03 noisy encoder and IMU recordings, timestamps and the 30-repeat protocol. Create `circuits/`, `measurements/`, `signals/` and `filter_report/`. Link every measurement to a circuit revision, component values, probe locations and instrument mode. State which results were calculated, simulated or physically measured.

Initial check: distinguish voltage and current, convert mA to A and μF to F, and solve $I=V/R$ for a known resistor. If electronics is entirely new, devote the first two hours to circuit diagrams and the multimeter; use familiar Python for software work. Working ESP32 firmware is not yet required this month.

Allocating 75 hours and laboratory procedures

Four blocks

A, hours 1-20: 10 h theory + 10 h practice. Voltage, current, resistance, power and energy; Ohm's and Kirchhoff's laws; series and parallel branches; multimeter use. Deliverables: LED and loaded divider with predictions and measurements.

B, hours 21-40: 10+10 h. Capacitors, diodes, BJT/MOSFET and flyback diode; pull resistors, logic levels, ground and supply decoupling; wires, connectors, crimping and fuses. Deliverables: verified low-power switch and wiring diagram for a future sensor node.

C, hours 41-60: 10+10 h. Sampling, Nyquist, aliasing and quantisation; low-pass, high-pass and notch filters; moving average, IIR and complementary estimation. Deliverable: reproducible encoder and IMU processing with error checks.

D, hours 61-75: 5+10 h. Frequency response, bandwidth, phase, FFT, vibration spectrum, ADC/PWM and sensor delay. Deliverables: noise-versus-delay table, aliasing experiment and final presentation. Total: 35 h of analysis and 40 h of practice.

Scheduling sessions

Divide ten theory hours into five two-hour sessions and the final 5 h into 2+2+1. Divide each 10 h practical into five parts: circuit and prediction, assembly, measurement, edge cases and documentation. For electrical experiments, calculate and draw wiring first, assemble without power, then measure only after checking.

First-page dates follow the original plan. Boundary dates may overlap a neighbouring module; holidays, rest and integration stages are not recalculated. Work steps fill available sessions sequentially without double counting. Time spent merely waiting for recording or printing does not add study hours.

Required measurement sequence

Before power-up, check polarity, pinout, component limits, common return and accidental shorts. Measure voltage across a circuit section; measure current in series with its branch; measure resistance and continuity only on an unpowered circuit with discharged capacitors. After current measurement, return the probe to the voltage socket. Do not test incorrect ammeter connection by creating a real short circuit.

Before the physical inductive experiment, check the circuit against component documentation and, if inexperienced, review it with someone who can assemble such circuits. Without the necessary review or equipment, perform calculation and simulation and mark the physical work pending. A simulator screenshot cannot replace real-circuit acceptance. If behind schedule, reduce filter variants while retaining basic circuits and delay measurement.

Direct current: indicator and analogue input

Study before assembly

Voltage is potential difference; current is charge flow; resistance relates them in an ohmic element: $V=I \cdot R$. Power $P=V \cdot I$ is measured in watts; energy $E=P \cdot t$ at constant power is measured in joules. Kirchhoff's current law expresses current balance; the voltage law expresses loop-voltage balance. Choose arrow directions beforehand: a negative calculated value indicates the reverse direction.

Study series and parallel connections, open-circuit voltage and loading. An LED is not a constant resistor: its forward voltage depends on current and temperature. Calculate the limiting resistor for permissible current and the worst combination of supply voltage and forward drop, then check resistor power.

Task 1. Node status indicator

Input: supply and LED with manufacturer documentation, and two suitable resistors. A teaching calculation for 5 V, $V_f \approx 2$ V and 5 mA gives about 600 ohm; this illustrates arithmetic, while the actual value depends on your components and margin. Draw the loop, label voltage drops and current, then assemble LED and resistor with the supply disconnected.

After checking the circuit, enable power with the calculated limit. Measure supply, LED and resistor voltages; compute current as V_R/R and, if needed, compare with a correctly connected direct measurement. Deliverables: schematic, photograph, component values and a predicted-versus-measured table. Check voltage balance. Study reversed polarity and a missing resistor in a diagram or simulation without exposing components to damage.

Task 2. Divider as a position-sensor model

Build a divider from two known resistors. Predict $V_{out}=V_{in} \cdot R_2/(R_1+R_2)$, then measure without an added load. Add a third resistor parallel to the lower leg and recalculate using equivalent resistance. This demonstrates the practical meaning of the next device's input resistance.

Compare three positions of a model resistive sensor; plot voltage against divider-leg ratio. Deliver an error table accounting for resistor and instrument tolerances. Edge case: divider output exceeds the future analogue-to-digital converter (ADC) input limit; identify incompatibility during calculation. Do not treat a divider as a complete voltage regulator or universal input protection.

Components, ground and logic levels

From passive components to switching

Study capacitors through $Q=C \cdot V$ and energy $C \cdot V^2/2$, and RC charging/discharging with time constant $\tau=R \cdot C$. A diode conducts mainly in one direction and has current and voltage limits. A BJT is controlled by base current; a MOSFET by gate-source voltage and gate charge. On-state losses and switching transients matter, beyond the component name.

Distinguish $V_{GS(th)}$ from the voltage at which the manufacturer specifies $R_{DS(on)}$: threshold does not mean a fully enhanced power switch. Study the selected package's exact pinout. For a digital input, compare source V_{OH}/V_{OL} with receiver V_{IH}/V_{IL} and absolute limits. A '3.3 V' board label does not make all pins 5 V compatible.

Task 3. Foot contact and pull-up resistor

Model the contact with a normal switch connecting the input to ground. A pull-up resistor connects the input to a suitable logic supply. Before attaching a controller, measure open/closed contact levels with a multimeter and calculate pull-up current. For teaching values 3.3 V and 10 kohm, closed-contact current is about 0.33 mA; verify the final value against the input and wire length.

Try two resistor values and compare prediction with measurement. A broken wire must have a defined logic meaning for future diagnostics, though this need not distinguish it from an open contact. Deliver a state table for 'open, closed, broken wire' and an explanation. First observe or simulate switch bounce; firmware debouncing comes in M06.

Task 4. RC circuits, decoupling and return current

Build an RC circuit in a simulator, apply a step and predict the time to approximately 63% of final voltage. With a suitable instrument, measure the transient in a circuit with little stored energy. Double R or C and compare the timescale. Save a plot and τ -estimation error; account for instrument input resistance.

Draw supply-current paths for the sensor and a separate load. Explain why decoupling capacitors sit near supply pins and why a long shared return wire can introduce interference. Do not disconnect ground while powered to 'demonstrate failure'. Model shared-wire resistance and show the added voltage drop caused by load current.

Protected switching and sensor-bench connections

Task 5. Small inductive load

Choose a low-power, low-voltage relay coil with known rated voltage, resistance and current. Leave relay contacts unloaded. Draw a low-side switch: coil between positive supply and drain or collector, source or emitter to common ground. Connect the flyback diode across the coil so it is reverse-biased in the normal energised state; on a standard positive rail, its cathode connects to the positive side.

First calculate coil current, switch losses, control-level compatibility and diode-rating margins. Study any ULN2003A option through its documentation: connect the clamp-diode common pin correctly, and do not assume the whole breadboard assembly can carry the IC's rated currents. With a MOSFET, ensure a defined off-state gate voltage and check $R_{DS(on)}$ at the actual drive voltage.

Experiment sequence and acceptance

1. Check the circuit on paper and in simulation. 2. Verify components and pinouts. 3. Assemble unpowered after selecting the supply current limit and fuse. 4. Start with static on/off operation. 5. Measure coil supply and switch voltage drop. 6. If an oscilloscope is available and its ground connection is understood, compare the measured transient with the model.

Deliver a log of currents, voltages, calculated losses and turn-off response. The coil must reliably de-energise, current must remain within component limits, and wiring must show no overload signs. Perform the no-flyback-diode experiment only in simulation. A multimeter cannot establish absence of a brief voltage spike; explicitly state if fast transients were not measured.

Task 6. Wiring, connector and fuse

List connection details: wire length and cross-section, contact type, connector current rating, retention method and labels. Calculate $\Delta V = I \cdot R$ across outgoing and return wires. Select fuse rating together with its time-current characteristic, operating and inrush currents, voltage and the weakest section's allowable conditions; 'slightly above current' is insufficient.

On an unpowered sample, inspect crimps visually and apply a light controlled pull according to contact instructions; check continuity of each conductor and isolation from adjacent ones. Prepare a pin→signal→voltage→wire_colour table and photographs of both ends. Edge cases: mirrored connector pinout and reversed polarity. Detect these before powering rather than rely on protection after an error.

Sampling: when a sensor reports different motion

Theory

Sampling converts a continuous signal into values at time intervals. $f_s = 1/dt$ is measured in hertz. A band-limited signal requires a sampling rate above twice its highest frequency, plus practical margin for an antialiasing filter's transition band. Once frequencies have aliased, an ordinary digital low-pass filter applied after recording cannot reliably separate them.

ADC quantisation limits amplitude resolution; sampling limits temporal resolution. Study saturation, missing samples and uneven timestamp intervals separately. PWM is a pulse train with a defined on-time fraction; its frequency differs from command-update frequency and does not automatically produce a precise analogue voltage.

Task 7. Vibration becomes apparent motion

Generate ground truth on a dense time grid: a useful 2 Hz sinusoid plus 70 Hz vibration. Sample at 100 Hz and show the false 30 Hz component; repeat at 250 Hz. All numbers are teaching examples. Compare time plots and spectra, labelling true and observed frequencies.

Next, apply a suitable antialiasing filter to the dense recording before downsampling, and compare with 'downsample first, filter afterwards'. Deliver three spectra and explain operation order. Edge case: a sinusoid at $f_s/2$ with an unsuitable phase; a few samples cannot reliably recover its amplitude. Do not test the Nyquist criterion only at the exact boundary.

Task 8. ADC and actual timestamps

Model conversion of voltage $0 \dots V_{ref}$ to an N-bit ADC integer code with explicit rounding and saturation rules. Calculate the quantisation step for the chosen model, plot error during a slow voltage change and check the smallest distinguishable change in a model joint position.

Add timestamp-interval variation (jitter) and one missing sample. Compare angle derivatives using constant dt and actual intervals; incorporate M01 checks. Deliver a CSV with `raw_code`, `t_s`, `voltage_V`, `angle_rad` and flags. Do not silently treat irregular samples as uniform before FFT: for this laboratory, select a uniform segment or explicitly resample onto a grid and describe the procedure.

Filters for encoders and body tilt

Required concepts

A low-pass filter preserves slow components and attenuates high frequencies; a high-pass filter attenuates DC and slow components; a notch filter suppresses a narrow band. A moving average uses a finite sample window; an IIR uses previous state. A simple low-pass filter is $y[k] = \alpha \cdot x[k] + (1 - \alpha) \cdot y[k-1]$, with $0 < \alpha \leq 1$. Smaller α gives stronger smoothing and slower response.

Study causality: a real-time filter cannot access future samples. Forward-backward processing such as `filtfilt` is useful for recorded-data analysis, but cannot establish a streaming algorithm's delay. Define initial filter state and behaviour after missing samples separately.

Task 9. Encoder noise versus delay

Input: a slow reference trajectory with a stationary section and a step, followed by a copy with noise and outliers. Implement causal moving averages with windows 3, 5, 11 and IIR filters with three α values. Calculate SD on the same stationary section, motion RMSE and time to reach 50% of the step relative to the reference. Distinguish step delay from 10-90% rise time.

Deliver a parameter table, time plots and a chosen tradeoff for the future joint monitor. Edge cases: first packet, NaN, a long gap and restart. NaN handling must allow valid IIR state to recover. State the policy for each case: reject, reset or continue with a flag; the policy must be visible in output data.

Task 10. Complementary tilt estimation

Generate a synthetic body-tilt trajectory in one plane with known $\theta(t)$. One input is angular velocity to integrate, with a small constant bias; the other is a noisy accelerometer angle estimate assuming small translational acceleration. Implement $\theta_est = \beta \cdot (\theta_prev + \omega \cdot dt) + (1 - \beta) \cdot \theta_acc$ and check rad and s units. For the initial discrete approximation, use $\beta = \tau / (\tau + dt)$, with chosen time constant τ in seconds; recalculate β when dt changes and check the response.

Compare each separate estimate and their combination at rest, during tilt and during artificial linear acceleration. The last case must reveal the accelerometer-angle limitation. Do not add full 3D orientation, quaternions or automatic calibration here. Deliver drift/noise plots and describe conditions in which this simplified complementary filter is suitable.

Frequency response and latency budget

What a vibration spectrum means

FFT converts a finite, uniformly sampled recording into frequency components. Study recording length, frequency spacing f_s/N , DC component, window functions and spectral leakage. Spectrum amplitude and power spectral density are different quantities; acceleration PSD has units $(\text{m/s}^2)^2/\text{Hz}$. Welch's method helps estimate a noise spectrum, but window parameters affect resolution.

A filter's frequency response describes amplitude and phase changes at each frequency. For a sinusoid, time delay relates to phase, but one delay constant may not describe an arbitrary signal. Do not approve a filter for future control from one smoothed plot: relate bandwidth and phase to task dynamics.

Task 11. Measure the frequency tradeoff

Pass a set of sinusoids through the task 9 causal filter. After the initial transient, measure amplitude ratio and phase shift; compare with `scipy.signal.freqz`. Add one synthetic vibration peak and calculate PSD with Welch's method. If using a notch filter, first specify centre frequency and suppressed bandwidth, then verify that nearby useful components remain intact.

Deliver amplitude/phase response plots and a three-variant table. For each setting, report stationary SD, motion RMSE, step delay and attenuation of the selected vibration. Define acceptance thresholds from the teaching monitor's requirements; the conclusion does not establish suitability for balancing a real robot.

From ADC to computed state

Draw the timing sequence: physical event → wait for sampling → conversion and internal sensor filter → transmission → processing → publication. List ADC sampling rate, output-message rate, PWM update rate and computation time separately. '100 Hz' does not describe the whole chain. Some times come from manufacturer documentation, some are measured and some remain unknown.

Add a known message delay to the test-bench model and compare physical-change time with filter-output time. If source and receiver clocks are not synchronised, their timestamp difference is not a reliable one-way latency. M04 needs one shared simulation timebase and a separate list of future M06-M07 hardware measurements.

Work sequence: hours 1-40

Block A: understand multimeter readings

Hours 1-2. Inventory equipment, study modes and convert units. **Hours 3-4.** Ohm's law, power and LED calculations. **Hours 5-6.** Kirchhoff's laws and series/parallel branches. **Hours 7-8.** Divider, loading and instrument input. **Hours 9-10.** Draw correct probe placement for all forthcoming measurements and check circuits.

Hours 11-20 - practice. 2 h: unpowered LED assembly and checks; 2 h: LED measurements and voltage balance; 2 h: unloaded and loaded divider; 2 h: repeat measurements and account for tolerances; 2 h: log, photographs and error review. Do not spend the practical buying equipment: prepare it before the relevant session.

At hour 20, choose a new resistor pair and predict divider output before measuring. Explain deviations from nominal-value calculations: tolerances, actual supply, instrument loading or connection errors. Identifying testable causes is more useful than adjusting an answer to an ideal number.

Block B: from contact to small coil

Hours 21-22. Capacitors, diodes and RC transients. **Hours 23-24.** BJT/MOSFET, pinouts and turn-on specifications. **Hours 25-26.** Pull resistors and logic-level compatibility table. **Hours 27-28.** Flyback diode, return currents and decoupling. **Hours 29-30.** Wires, connectors, crimping and protection selection for the actual circuit.

Hours 31-40 - practice. 2 h: pull-up switch and RC. 2 h: inductive-switch schematic, simulation and component checks. 2 h: physical assembly of the checked circuit. 2 h: measurements, static turn-off and log. 2 h: pinout, unpowered connection checks and verification report.

Intermediate acceptance: another person can use the schematic to trace on/off current paths, identify flyback-diode polarity and find protection ratings. Documentation supports all voltages and permissible operating conditions. If hardware is not ready, software block C proceeds independently; physical items remain open without being falsely marked complete.

Work sequence: hours 41-75

Block C: record a signal and verify its interpretation

Hours 41-42. Sampling, f_s and Nyquist; predict an alias frequency by hand. **Hours 43-44.** Quantisation, saturation, timestamps and jitter. **Hours 45-46.** Low-pass, high-pass and notch filters, and causality. **Hours 47-48.** Moving average and IIR; calculate state after one sample. **Hours 49-50.** Planar complementary filter and accelerometer-angle assumptions.

Hours 51-60 - practice. 2 h: 70→30 Hz aliasing and filtering before downsampling. 2 h: ADC model and missing sample. 2 h: causal encoder filters. 2 h: complementary angle and body acceleration. 2 h: error table and restart/NaN tests. Use the same reference when comparing settings.

By hour 60, implement streaming interface `update(sample,timestamp)`. It updates state without future data and returns a value plus diagnostic flag. A separate saved-file analysis may use the whole recording, but its result name and delay conclusions must reflect that.

Block D: frequencies, latency and presentation

Hours 61-62. FFT, windows, resolution and PSD. **Hours 63-64.** Frequency response, bandwidth and phase shift. **Hour 65.** Build the ADC/sensor/transport/filter/PWM latency budget and identify unknown times.

Hours 66-75 - practice. 2 h: spectrum and Welch parameters. 2 h: sinusoidal runs and freqz. 2 h: final noise, error and delay table. 2 h: six error checks and reproduction. 2 h: assemble electronics log and report, give an oral explanation, and list M06 hardware questions.

Six required failure cases: incorrect polarity detected before power-up; mirrored pinout found by continuity checks; ADC range excess identified by calculation and simulation; aliasing not 'fixed' by later filtering; NaN does not permanently corrupt IIR; a long gap is flagged and handled by the chosen policy. The first three require no deliberate damage to the physical test bench.

Final new variant: change f_s by a factor of two and, before running, predict changes in an 11-sample window's delay, the IIR coefficient for the same time constant and the observed alias frequency. Verify the calculation. If the programme simply keeps the old coefficients, explain why filter behaviour in physical seconds changes.

Acceptance: a measured circuit and an explainable filter

Deliverable package

The electronics log includes schematics and revisions, selected components and documentation, resistance/power/protection calculations, probe positions, assembly photographs and measurement tables. The switching circuit shows current paths, flyback diode, turn-off and measurement limitations. Connections include pinout, labels and confirmed unpowered continuity checks.

The signal-processing report includes a reference generator, real M03 recordings, fs metadata and timestamps, causal filters, settings, time plots and spectra, noise/delay metrics, gap handling and tests. Do not present a smoothed real-IMU signal as unknown ground truth; calculate reference RMSE on synthetic data, and assess repeatability and observable properties on real data.

Questions to explain

Why are voltmeters and ammeters connected differently? How does loading change divider output? Why does an LED need current limiting? How does VGS(th) differ from a low-RDS(on) operating condition? Where does coil current flow immediately after turn-off? Why is the common return part of the signal path? How are fuse parameters selected, and why do they depend on wiring?

How does sampling rate differ from PWM frequency? Why cannot digital filtering restore information lost through aliasing? What changes when the window grows? How does step delay differ from rise time? Why is filtfilt not a streaming filter? How does body acceleration distort a simple accelerometer-angle estimate? What are PSD units?

Completion criteria

Passing work includes a correctly assembled and protected low-power circuit after checking, correct multimeter measurements, recognised aliasing, comparison of at least two causal filters and a quantified delay tradeoff. Label physical experiments and simulations separately. Fill blank templates with your own results; the document does not claim accuracy in advance.

Pass M06 the pinout, signal ranges, expected rates, error handling and low-voltage assembly log. M07 uses current calculations and understanding of inductive loads, but its power bench requires its own components, protection and tests after G1. State one sensor-path limitation each for the leg and hand; for example, permissible delay remains undefined until the future loop dynamics are known.

Assigned reading: a short route

Read only the specified passages before the corresponding experiment. References are mainly in English; keep explanations, glossary and report in English. Reading and preparation fit within the existing 75 hours; whole books are not required.

Core reading

1. Tony R. Kuphaldt, Lessons in Electric Circuits. DC volume: chapters 1-3, 6, 16. Selected topics: voltage/current, Ohm's law, measurement, dividers, Kirchhoff's laws and RC transients. Before tasks 1-4, draw instrument connections and calculate the circuit. For task 5, read Semiconductors volume, chapter 3, Inductor Commutating Circuits.

2. Steven W. Smith, The Scientist and Engineer's Guide to Digital Signal Processing. Chapter 3: sampling; chapter 15: moving average. For tasks 7 and 9, compare aliasing, noise and delay. In chapter 15, read Noise Reduction vs. Step Response and Frequency Response; a full DSP course is unnecessary.

3. Anant Agarwal, MIT 6.002: Circuits and Electronics, notes. Lectures 2, 4, 12: Basic circuit analysis method; The digital abstraction; Capacitors and first-order systems. Consult after a simple experiment to understand logic-level tolerances and finite RC settling time.

Documentation for individual experiments

Switching and protection. TI ULN2003A: pinout, typical circuit, power supply or Nexperia AN11158: MOSFET parameters. Read the chosen option for task 5: limits, losses and flyback diode. Every circuit component contributes to permissible operation.

Filtering. Analog Devices: Anti-Alias Filters; SciPy freqz and welch. For tasks 7 and 11, specify fs, window, nperseg and scaling; assess causality and units first.

Bus pull-up reference. TI SLVA689, I2C pull-up calculation. Understand resistance, capacitance and edge-rate relationships; build a real I2C bus in M06.

Understanding check. Before assembly, explain current paths when the switch turns on and off; before filtering, identify information already lost during sampling and the cost of noise reduction.

Glossary: terms and definitions

Voltage. Electric potential difference between two points; measured in volts and always requires a reference point.

Current. Rate of electric-charge transfer through a cross-section; measured in amperes, with sign determined by the chosen direction.

Resistance. Voltage-to-current ratio of an ohmic element under specified conditions; measured in ohms and limits circuit current.

Power. Rate of energy transfer; for an electrical circuit, $P=U \cdot I$ with consistent direction conventions, measured in watts.

Ground. Chosen circuit reference node and current-return path; its conductors have nonzero resistance and inductance.

Voltage divider. Series circuit producing a fraction of input voltage; connected loads change the division ratio and must be included.

Time constant. For a simple RC circuit, $\tau=R \cdot C$ in seconds; during τ , step response covers approximately 63% of its full change.

Inductance. Parameter relating current change to voltage; a coil opposes rapid current changes and stores magnetic energy.

Flyback diode. Diode providing a coil-current path after the switch opens; supply configuration determines its polarity and ratings.

MOSFET. An insulated-gate field-effect transistor; gate-source voltage controls channel conductance, and switching requires changing gate charge.

Pull-up. Resistor between signal line and supply that establishes a high level when the input or open drain is high impedance. At a low level, it limits current.

Fuse. Component that breaks a circuit under a specified current-time load; protects the selected wiring path within its rated characteristics.

Sampling. Representing a continuous signal by samples in time; $f_s=1/\Delta t$ on a uniform time grid.

Aliasing. Frequency ambiguity after sampling in which different continuous signals produce identical samples, causing information loss.

Quantisation. Replacing a continuous value with one of a finite set of levels; introduces amplitude error distinct from temporal sampling.

Causal filter. Filter using only present and past inputs; future samples are unavailable during streaming.

FIR and IIR. FIR has a finite impulse response; IIR usually uses feedback and may respond for an unlimited duration.

Power spectral density (PSD). Distribution of signal power over frequency; integrating a band gives its mean-square contribution under consistent normalisation.

Complementary filter. Combination of estimates with complementary frequency responses, such as slow accelerometer tilt and rapid gyroscope angle changes.

Equipment, installation and checks

Complete preparation within the first study blocks in place of another general overview. The budget remains 75 hours; delivery waiting time is excluded. Statuses below specify actions for you to take, not purchases or installations already completed.

Use existing equipment

Use the MacBook Pro M4 and Python/SciPy from M02-M03, plus sensor recordings or synthetic data for filtering. Reuse listed tools if already available and working.

Buy before the first circuit experiment

1 each: a multimeter measuring DC voltage and resistance, with continuity, diode test and a fused current input; a set of working probes. Supplement this [measurement guide](#) with your model's manual. Connect across a circuit for voltage and in series for current.

1 CC/CV bench supply with adjustable voltage, current limit and independent output switching. A guideline for the shared M04-M07 bench is approximately 0-30 V and up to 5-10 A at the required voltage. These are supply capabilities; calculate each experiment's low-voltage, low-power operating point. Do not route full supply power through a breadboard.

1 low-power circuit kit: breadboard and jumpers; several each of resistors from 330 ohm to 100 kohm, LEDs and buttons; 100 nF and 10-100 μ F capacitors; 1 low-power, low-voltage relay with unloaded contacts, 1 compatible switch and flyback diode. Select final values, powers and voltages from tasks 1-5 and documentation before ordering.

1 connection kit: adjustable soldering iron or station with stand, solder, flux and fume extraction; side cutters, wire stripper, tool for the chosen crimp contacts, wires, connectors, heat-shrink, holder and suitable fuses. Consumable quantities should cover one bench and task 6 samples.

Install and check now

macOS ARM: [LTspice for macOS](#) for RC and switching circuits; verify your macOS version and Apple Silicon support in the selected release. SciPy.signal is already available. Test: RC step response reaches about 63% at $t=RC$, freqz displays a Hz axis, and welch reproduces after restart.

Before power-up: continuity and polarity checks, correct multimeter sockets, calculated supply limits and protection ratings; then test a low-power resistive load according to the supply manual. M04 needs neither a motor nor MCU firmware. An oscilloscope remains optional: simulate fast transients and explicitly state that no physical measurement was made.

Motion fundamentals: what a sampled sensor sees

Compulsory, within 75 hours. Use 1 h from 63-64 and 2 h of practice 66-75, 'sinusoidal runs and freqz'. Use existing generators and the filter from tasks 7, 9 and 11; this is their combined physical interpretation. Before starting, calculate $\Delta t = 1/100 = 0.01$ s and convert 20 ms to seconds.

Motion continues between samples

Let a joint in a fixed plane have angle $q(t) = 0.20 \sin(2\pi \cdot 2t)$ rad relative to the horizontal axis. Its velocity $\dot{q} = 0.80\pi \cos(4\pi t)$ rad/s reaches 2.513 rad/s. For link $L = 0.12$ m, maximum endpoint speed $L|\dot{q}| \approx 0.302$ m/s. The sensor measures q , not foot velocity directly; calculation supplies geometry and differentiation.

The example recording also contains sinusoidal interference $e(t) = 0.01 \sin(2\pi \cdot 70t)$ rad. At $f_s = 100$ Hz, its samples match $-0.01 \sin(2\pi \cdot 30k/100)$: 70 Hz appears as 30 Hz with a different phase. This sequence cannot establish the original continuous oscillation. A later filter may attenuate 30 Hz but cannot restore lost information; antialiasing is needed before sampling or reducing its rate.

Why a smoother angle may arrive too late

Consider a causal five-sample average: $q_f[k] = (q_m[k] + \dots + q_m[k-4])/5$, where q_m is measured angle. Output is issued at $k\Delta t$ although the window centre is two samples earlier. In steady state at 2 Hz, delay is $(5-1)\Delta t/2 = 0.020$ s and phase shift is $-2\pi \cdot 2 \cdot 0.020 = -0.2513$ rad, or -14.4° .

The amplitude of the desired sinusoid is multiplied by $\sin(5\pi f/f_s)/(5 \sin(\pi f/f_s)) \approx 0.98428$ at $f = 2$ Hz. Thus the smoothed angle nearly preserves amplitude but describes earlier motion. Treating it as current gives endpoint position error of scale $|v| \cdot 0.020 \approx 6$ mm during rapid motion. This is a local delay-effect estimate, not an accuracy tolerance for a finished leg.

Checks for the future control loop

1. Compare sampled 70 Hz interference with a negative 30 Hz sinusoid; repeat at $f_s = 250$ Hz. Save overlaid sequences and label true frequency only where known from the generator. A measured spectral peak does not establish a mechanical cause.
2. On clean 2 Hz motion, measure the five-sample filter's amplitude and phase after the transient and compare with freqz. For a unit step with zero prehistory, output first exceeds 0.5 at 20 ms after the first changed input sample. Add these values to the task 11 table; account for transport delay separately.

Do not shift streaming data backwards in time to claim less delay: an earlier timestamp does not speed publication. Feedback suitability depends on the future M10 loop dynamics, not plot smoothness. All motion here is synthetic; no motor is needed.

Read before calculating: Steven W. Smith, DSP Guide: [chapter 3, The Sampling Theorem](#); [chapter 15, Noise Reduction vs. Step Response](#).

A parametric leg: from sketch to measured part

M5 • 31 Jan 2027 - 24 Feb 2027 • 75 hours

M05 outcome

During this month, create **a parametric CAD assembly of a leg with three degrees of freedom, drawings, a bill of materials and test specimens**. This is the first physical part of the future walking robot: links, brackets, shafts, fasteners and the cable route. At this stage the leg may be passive, with joints moved by hand; the actuator test rig and force control belong to later modules.

The tasks require you to justify the design: how link length determines part dimensions, which components transfer the load to the body, how to provide access to a screw, how tolerances affect backlash, and why print orientation changes a part's properties. The same approach will later apply to robotic fingers, but designing the complete hand is outside M05.

Learning object and model boundaries

Use three consecutive revolute joints: the first turns the leg in the horizontal plane, and the second and third bend it. In the zero configuration, align the first joint axis with z and the next two axes with y; label the local coordinate frames. Initial link lengths, for example 35/80/100 mm, are only starting geometry. You may change them after checking the layout; these are not the final dimensions of the future robot.

Until a motor is selected, create a separate parametric component representing its envelope. Explicitly mark mounting dimensions that still need confirmation. If the actual actuator has already been selected, use dimensions from its drawing. A provisional motor model does not establish mounting compatibility. A physical passive assembly can use separate shafts and bushings without applying power.

Tools and organisation

Choose one CAD environment: FreeCAD or Fusion if you already know it. Do not learn both simultaneously. Prepare callipers or a ruler, your existing FDM printer, available material, suitable fasteners and a simple fixture for small static loads. Use a validated profile for your printer; the printer model does not automatically determine hole accuracy.

Create cad/native, cad/export, drawings, bom, calculations, coupons and tests. For every print, record the CAD revision, material, orientation, profile, mass and date. M02 covered forces, torques and units; M03 covered ways to describe measurement variability; M04 covered cable-routing requirements. Full kinematics, dynamic walking and actuator operation are not required here.

75 hours and a design strategy

Four blocks from the original programme

A, hours 1-20: 10 h theory + 10 h practice. Sketches, constraints, dimensions and parameters; extrusions, revolutions, patterns, holes and fillets; parts, assemblies, revolute joints and intersections. Deliverable: a link, a bracket and a joint assembly with three labelled axes.

B, hours 21-40: 10+10 h. Fasteners, inserts, bearings, shafts and couplings; fits, datums, tolerance stacks and GD&T fundamentals; cables and maintenance access. Deliverable: drawings, an exploded view, a bill of materials (BOM) and specimens for checking fits.

C, hours 41-60: 10+10 h. Free-body diagrams, equilibrium, load transfer, stress and strain, shear, beam deflection, factor of safety and load uncertainty. Deliverable: a hand calculation for an unfavourable configuration and a comparison of two link versions.

D, hours 61-75: 5+10 h. FEA: restraints, mesh and convergence; FDM anisotropy, orientation and layers; PLA, PETG and TPU; printing and a static test of one assembled leg. Deliverable: a comparison of hand calculations, FEA and measurements, with limitations stated. Total: 35 h of study and analysis and 40 h of practice.

Study rhythm and production

Study the theory in two-hour sessions; split the final 5 h into 2+2+1. Divide each ten-hour practical block into five two-hour sessions. Print small test specimens first, then the critical assembly, and print the entire leg only after checking the fits. Waiting for a print to finish does not count towards study hours; prepare drawings, calculations or the log while the printer runs.

Dates come from the overall plan. Sessions from an adjacent module may occur at the M05 boundary; rest days, holidays and integration stages remain in place. Allocate work hours sequentially across available sessions, without counting hours twice or adding hours. Plan printing in advance so that verification does not depend on the final night of the month.

Priorities when time is limited

First ensure that geometry rebuilds reliably and that connections are justified, then address strength and maintenance access, and finally appearance. If you fall behind, omit photorealistic images, elaborate enclosure styling and a second CAD environment. Retain the three-joint assembly, critical drawings, BOM, hand calculation, a simple FEA check and physical specimens.

Before testing, record the calculated load range and the fixture's permissible loads; a previous result does not automatically carry over after a change in material, layer direction or geometry. No factor of safety in this plan is a universal approval: select it for a specific scenario and support it with data. Conduct a static test on a restrained specimen, without an operating actuator.

Sketches and parameters: build an adjustable link

What to study in CAD

Study the sketch plane, origin, construction geometry and constraints: coincidence, horizontal and vertical alignment, parallelism, perpendicularity, tangency, symmetry and dimensions. Identify the degrees of freedom that remain. A fully constrained sketch should express design intent; fixing every point indiscriminately conceals relationships and makes changes difficult.

Separate driving dimensions from reference dimensions. Use the names `L_link`, `wall`, `shaft_d`, `bearing_d`, `bolt_pitch` and `cable_clearance`. Store units and comments. Practise extrusion (extrude/pad), revolution (revolve), pattern, hole and fillet operations on features of the intended part. Build the main volumes and holes first, then secondary fillets, so the cause of errors is easier to locate when the shape changes.

Task 1. Parametric link

Inputs: the distance between axes, the envelope of the future mount and the selected printing process. Sketch a link with two joint centres, a symmetrical outline and parametric thickness. Extrude the body, add holes and one lightening cavity. Create three length variants by changing only the parameter, rather than stretching faces manually.

Save the model in its native CAD format and a table of variants. Check that centre distances match the specification, the sketch is fully constrained and geometry rebuilds without errors after a length change. Edge case: the minimum length makes holes intersect the cavity. Provide a way to detect this error in the model or checklist, and explicitly state the permitted parameter range.

Task 2. Bracket and a revolved part

Model a U-shaped bracket and a spacer bushing. Use a revolved profile for the bushing and a pattern for repeated fastener holes. Identify mating surfaces, where a flat seat is needed under a screw head, and which tool must reach the fastener. Do not make a wall thin simply because CAD can construct it.

Save both parts in native CAD format, a bracket section view and the results of checking the main dimensions. Change wall thickness and shaft diameter, then verify that symmetry is retained. Deliberate error: a sketch references a face that disappears after a fillet change. Move the reference to a stable datum plane or axis and explain why the revised model tolerates edits better.

Three joints, assembly and maintenance

A component differs from a body

Understand the difference between a part, a component, an instance and an assembly. Two identical bushings in an assembly should reference one model unless one needs a separate revision. Define a fixed base and three revolute joints. Each joint has an axis, a zero position, a positive angular direction and geometric motion limits.

Do not infer mechanical feasibility from a single chosen configuration. Collisions can appear during combined rotations, and the motor housing and screw heads occupy space. Add simple envelopes for the tool, cable and connector as separate reference components; include them in access checks, but exclude them from the printed part's mass.

Task 3. Passive 3-DOF leg

Assemble the base bracket, three joint units and links. Name the joints J1/J2/J3 and label their local axes and initial angles. Check each joint's motion separately, then combinations of extreme positions with intermediate points. Initially, geometry determines the learning model's ranges; the real actuator's limits must be reconciled later.

Save the assembly, a configuration table and screenshots of intersections before and after correction. Check for exactly three intended degrees of freedom, a fixed base and no unintended motion in rigid groups. A discrete set of poses does not prove there are no collisions between them; state the checking increment and investigate suspicious intervals in greater detail.

Task 4. Cable routing and replacement of one joint

Route a provisional cable from the base to the distal joint, allowing extra length for motion, securing it and providing connector access. Check extreme poses: the cable must not become taut, enter a joint gap or rub against a sharp edge. Take the minimum bend radius from the actual cable; for an envelope sketch, mark it as an assumption requiring confirmation.

Describe disassembly in the reverse order of assembly: which screw is removed first, and whether the shaft can be withdrawn and the bushing replaced without dismantling the whole link. Save five maintenance steps and an exploded view. Edge case: the screw head is accessible, but a hex key cannot be inserted along its axis. Add the tool envelope and modify the mounting. For the future hand, this exercise will recur within the smaller space of a finger.

Fits, datums, fasteners and tolerance stacks

Basic machine elements

Study the functions of screws, nuts, washers, threaded inserts in plastic, bearings, bushings, shafts and couplings. Separate their functions: a screw clamps, a shaft defines rotation, and a bearing transfers load while allowing specified motion. The thread of an ordinary screw is generally unsuitable as a precision rotational bearing surface. Select fits to suit the assembly method and actual part tolerances.

A datum is a surface or axis from which important dimensions are specified. GD&T fundamentals explain why axis position and surface perpendicularity differ from a single linear tolerance. Choose one clear set of datums A/B/C, identify the controlled axis and explain how to measure it. Studying the complete geometric tolerancing standard is outside the month's scope.

Task 5. Specimen for checking holes and inserts

Build a small specimen containing a series of fit holes with sizes close to the nominal size of the selected shaft or insert. Choose the increment relative to the expected accuracy of the printer and measuring tool; for example, 0.1 mm may serve as an exercise increment, rather than a promised FDM accuracy. Print the specimen in two orientations with the same material and profile.

Measure the holes using an available method, then check the fit of the actual mating part. For small internal diameters, assess the limitations of calliper measurements and use a known shaft as a functional gauge. Save nominal/measured/fit_result. Edge cases: a hole becomes oval because of layer direction, an insert splits the wall, or a part fits only after undocumented reworking. Record any machining or finishing separately.

Task 6. Axial clearance and drawing

Build a tolerance stack for the assembly: bracket width, two walls, bushings, the rotating link and washers. Calculate minimum and maximum axial clearance for unfavourable tolerance combinations. Do not add standard deviations as though they were guaranteed dimensional limits: the probabilistic estimate in M03 and a worst-case tolerance check answer different questions.

Prepare a drawing of the critical link with views, a section, datums, hole dimensions, material, print orientation and revision. Acceptance check: another person understands what to manufacture and measure. Deliberate error: the dimension chain permits negative clearance and binding after tightening. Correct the spacer bushing or geometry; simply loosening the screw without a controlled fastening arrangement is not a solution.

Statics and strength: how loads are transferred

Essential mechanics

Obtain a free body by mentally separating a part and replacing constraints with reactions. In a static case, the sums of forces and moments are zero. Show how the load passes from the foot through links, shafts and the bracket to the body. Stress is internal force per unit area, and strain is relative dimensional change; modulus E relates them in a simple linear elastic model.

Study tension, compression, shear, bending and deflection. For an ideal cantilever beam with end force F , $M=F \cdot L$, $\sigma_{\max}=M \cdot c/I$ and $\delta=F \cdot L^3/(3EI)$. Here c is the distance to the outermost fibre, and I is the second moment of area of the section, not a mass moment of inertia. These equations apply for small deflections, the stated supports and an appropriate material model.

In a simple case, factor of safety n is the ratio of a selected material limit to calculated stress. First state the failure mode and the source of the limiting value. For printed plastic, parameters must correspond to layer direction, temperature and loading conditions; an unknown value cannot be replaced by an arbitrarily generous safety factor.

Task 7. An unfavourable leg pose

Choose an assumed future body mass and a load-distribution scenario, marking them as design assumptions. Do not automatically divide the weight by the total number of legs: some supports may carry no load. For three different poses, calculate the force lever arm about each joint and the base-mount reaction. Include the links' own weight if it is already known from the model or weighing.

Save three free-body diagrams, a force and torque table and the selected design load case. Check force and moment balance, dimensions and sensitivity to a load change. This provides a static reference; foot-strike impact and dynamic accelerations need a separate model. Reflect their uncertainty in the selected scenario and safety factor, rather than concealing it behind a claim of reliability.

Task 8. Two link versions

Simplify the critical part of the link to a beam with a known cross-section. Compare versions A and B: change section height or add a rib while retaining a clear load path. Calculate expected deflection, stress and mass; for a rectangle, $I=b \cdot h^3/12$, so the orientation of its height relative to bending matters.

Prepare a calculation note and two CAD model variants. Verify the prediction: doubling the force doubles deflection in a linear model, while changing height has a nonlinear effect. Edge case: a thin wall beside a hole is not represented by the beam idealisation. Mark this region for local FEA and inspection of the tested specimen. A material's catalogue strength is not yet the allowable strength of the printed assembly.

FEA: formulation, mesh and independent verification

Avoiding unjustified confidence

FEA divides geometry into finite elements and approximately solves the equations of the chosen model. Before running it, record the material, constitutive law, loads and restraints. Fully clamping a large surface is usually stiffer than a real bolted or bushed connection; compare one restraint arrangement with a physically plausible alternative.

For the first exercise, use one simple part and a linear elastic model. Confirm the units of E , loads and dimensions. An isotropic approximation of FDM is a limited model suitable for comparing trends; it does not automatically describe interlayer failure or TPU nonlinearity. Do not reach a final conclusion about a plastic part's failure solely from a coloured equivalent-stress map.

Task 9. A beam, then the actual link

First perform FEA on the same cantilever as in task 8, with the same L , section, E , force and support. Compare deflection at a selected point with the hand calculation. If the discrepancy is large, check units, boundary conditions and geometry before complicating the model. Then apply the approach to the actual link and name the metrics in advance: point deflection, support reaction and stress away from an artificial singularity.

Build three mesh refinement levels while retaining the same problem formulation. For each, record element count, mesh size, solution time and metrics. An exercise target is a change of less than 5% in the selected deflection between the final two meshes; this is a convergence criterion for this experiment, not a universal accuracy or permission to operate.

What to save

Save the original analysis model, material parameters, a list of loaded and restrained surfaces, mesh characteristics and the comparison with the hand calculation. Show the exaggerated deformed shape with its scale labelled; without a label, a viewer may mistake visual magnification for actual deflection.

Edge cases: an unrestrained rigid body, incorrect force units, a load applied to a single vertex and excessive local stress at an ideal sharp edge. A stress maximum that grows without converging needs investigation. Do not simply refine the mesh and declare the latest peak a reliable part stress. For M05, recognising the limitation and checking overall deflection and support reactions is sufficient.

Printing, materials and testing the passive leg

PLA, PETG and TPU according to part function

Compare materials in terms of stiffness, temperature response, creep, impact behaviour, layer adhesion and ease of processing. PLA is useful for initial dimensional prototypes; PETG is often used for tougher parts; TPU suits a flexible foot or pad. These are directions for selection, not guaranteed properties of a particular spool. Consult the manufacturer's data and test your own specimens.

FDM orientation affects interlayer bonding, holes, overhangs and the quality of bearing surfaces. Record layer direction relative to expected tension and bending. Infill density is only one parameter: wall thickness, perimeter count, top and bottom layers, and load transfer may have a stronger influence. Do not compare two materials while simultaneously changing the entire geometry and profile.

Task 10. Specimens and a static check

Prepare two identical beam specimens in different orientations. Measure actual dimensions and mass. Place a specimen in a simple fixture, apply a small known force using a weight or force gauge, and measure deflection at one point. Choose the load range from the preliminary calculation and the fixture's and instrument's capabilities; increase load in steps recorded in advance.

At each level, save load, deflection, hold time and residual deformation after unloading. The fixture must retain the part and weight, and the possible drop area must be clear. Stop testing if the fixture shifts, cracking is heard, a crack appears or deflection becomes unexpectedly large. Destroying the finished leg is not required to pass: a measured range and a description of any first signs of damage or unacceptable deformation are sufficient.

From a test specimen to an assembled leg

Choose orientation and profile from the specimen tests, print the critical links and assemble the passive leg. For the static experiment, use a fixture to lock the joints in one calculated pose; do not rely on the resistance of an unpowered motor. Apply a known, limited foot load and measure total deflection, play before loading and recovery after unloading.

Compare the direction and order of magnitude of deflection with the hand calculation and FEA, explaining the contribution of joints and mountings. Prepare CAD revision B to address one observed defect: a fit, access, a rib or cable clearance. Save photographs, the raw measurement table, a force-deflection plot, observations and the decision on the next revision. A successful static test does not establish walking fatigue life or impact strength.

Work sequence: hours 1-40

Block A: a model that can be changed

Hours 1-2. Choose CAD, create the project structure and a three-axis diagram. **Hours 3-4.** Study geometric and dimensional constraints and fully constrain a link sketch. **Hours 5-6.** Study parameters, expressions and units. **Hours 7-8.** Practise extrusion, revolution, patterns, holes and fillets (extrude/revolve/pattern/hole/fillet) on your own parts. **Hours 9-10.** Study components, instances, assemblies and revolute joints.

Hours 11-20 - practice. 2 h: task 1 and three link lengths. 2 h: bracket and bushing. 2 h: 3-DOF assembly. 2 h: extreme poses and intersections. 2 h: cable, tool and saving revision A. Prepare a short dimensional-specimen print at the end if helpful, so it does not delay the next practical session.

Checkpoint at hour 20: change one link's length and wall thickness. The assembly must rebuild, the axes must remain connected, and the unconfirmed motor envelope must retain its provisional status. If rebuilding fails, correct model dependencies first: adding decorative parts only increases the work needed for revision.

Block B: manufacture and maintenance

Hours 21-22. Study fasteners, inserts and the functions of connections. **Hours 23-24.** Study shafts, bearings, bushings, couplings and fit requirements. **Hours 25-26.** Cover datums and tolerance stacks. **Hours 27-28.** Study how to prepare a clear drawing and the basics of geometric tolerances. **Hours 29-30.** Check cables, tool access and the repair sequence.

Hours 31-40 - practice. 2 h: fit specimens and print preparation. 2 h: measurement and functional trial fitting. 2 h: axial tolerance stack and clearance correction. 2 h: drawings and exploded view. 2 h: BOM, assembly sequence and revision control. If specimen printing takes longer, prepare drawings while it runs; do not change fit criteria after seeing the result.

In the BOM, include part_id, revision, quantity, material or product type, dimensional source and method of obtaining the part. Separate printed parts, standard fasteners and provisional components. Totals by item must match the assembly. For every important item, make clear whether it can be manufactured now or requires selection of the actual actuator.

Work sequence: hours 41-75

Block C: calculate before loading

Hours 41-42. Draw free bodies of the links and their reactions. **Hours 43-44.** Calculate moments for three leg poses. **Hours 45-46.** Study stress, strain, shear and modulus E. **Hours 47-48.** Study a cantilever beam and the influence of cross-section. **Hours 49-50.** Describe uncertainties in load and material properties and justify the design safety factor.

Hours 51-60 - practice. 2 h: leg design load case. 2 h: simplified beam and unit checks. 2 h: versions A and B, comparing mass and deflection. 2 h: prototype test fixture and measurement plan. 2 h: preparation for printing critical parts and the calculation package. State the limits of available fixtures before choosing loads.

At hour 60, explain where the most heavily loaded region lies and why. If the answer rests only on a sense that a region looks thin, return to force lever arms and the load path. If geometry is too complex for a hand calculation, choose a clear idealisation and explicitly identify what it omits.

Block D: compare calculation, simulation and measurement

Hours 61-62. Cover FEA boundary conditions, units and material selection. **Hours 63-64.** Study mesh, convergence and limitations of local peaks. **Hour 65.** Compare PLA, PETG and TPU, layer orientation and the static verification plan.

Hours 66-75 - practice. 2 h: FEA of a simple beam and a hand-calculation comparison. 2 h: actual link and three meshes. 2 h: measured specimens, fits and joint locking. 2 h: static experiment on one assembled leg, table and photographs. 2 h: revision B, final drawings, BOM and presentation. Prepare final parts and specimens before these sessions.

If the available FEA environment requires lengthy setup, retain a minimal analysis of one simple part in an available solver and do not move to a full assembly contact model. Measuring one leg and making a justified comparison with a simple model is more useful than an unfinished complex simulation. The three meshes serve the chosen metric, rather than a target number of analyses.

Final independent check: double the assumed load or rotate a rectangular section by 90°. Predict the change in deflection and the most critical region before running the analysis. Then verify the result by hand and with FEA; increase the physical load only if the new range is justified and stays within the prepared test limits.

Acceptance checks: model, documentation and observations

Six required checks of errors and limitations

1. Rebuilding. Changing length or thickness does not break the main geometry. **2. Collision.** An extreme combined pose has been checked with fasteners and tools included. **3. Fit.** An unfavourable tolerance stack does not conceal binding; retain an unsuitable specimen with the recorded observation.

4. FEA units and supports. Incorrect scale or an unrestrained body is detected through independent verification. **5. Mesh.** A growing local peak is not automatically accepted as a reliable physical result; the selected deflection has been checked for convergence. **6. Physical specimen.** Residual deformation, play or a crack is recorded and influences the next revision, rather than disappearing from the report.

What to prepare

Save the leg assembly in native CAD format, a STEP export and print files with units and revision; critical-part drawings; an exploded view; BOM; parameter table; force, moment and beam calculations; FEA file and settings; mesh table; print profiles; specimens and actual measurements. Photographs must make it possible to identify the physical part by its part_id and layer direction.

Check that all driving sketches are constrained, the assembly has three intended degrees of freedom, the assembly and maintenance sequence is known, and calculation assumptions are not presented as properties of the finished robot. For every deflection or load value, identify its source: hand calculation, simulation or experiment. Compare trends and scale under identical conditions and explain discrepancies.

Self-check questions and handover

How does a geometric constraint differ from a dimension? Why is a part instance preferable to a copied independent body? Where are the datums for the critical hole? What happens to clearance during tightening? Why does support on a threaded surface fail to provide precise rotation? How does the foot load reach the body? Why does section height enter I to the third power?

What does a factor of safety mean when printed strength is unknown? Why does isotropic FEA not establish interlayer strength? How do you distinguish a singularity from a real stress concentration? What did the specimen reveal that CAD did not? Which conclusions cannot be carried over from a static test to walking?

Pass mounting dimensions, envelopes, cable route, mass, maintenance constraints and the test fixture to M06-M07. Once the actual actuator is selected, its mounting must be checked and its loads recalculated. For the future hand, retain the parametric bushing, bracket and fit-checking method; the complete hand and tactile elements will appear in the relevant module and a separate optional block.

Assigned reading: a focused reading route

Read only the specified sections before the corresponding experiment. Most literature is in English; keep explanations, the glossary and the report in English for this edition. Reading and preparation are included in the existing 75 hours; you do not need to work through entire books.

Core reading

1. David Roylance, Mechanics of Materials, open MIT modules. Introduction to Elastic Response; Stresses in Beams; Beam Displacements. For tasks 7-8, derive the relationship between load, stress and deflection. Then read selected parts of Finite Element Analysis for task 9: a mesh approximates a model, so comparison with the beam is compulsory.

2. Alexander Slocum, FUNdaMENTALS of Design. Chapters 4, 8-10: Linkages; Structures; Structural Conn. & Interfaces; Bearings. For tasks 3-5, study only diagrams relevant to your joint, load transfer and part connections. Record which motions the support allows and which reactions it transmits.

3. Yorik van Havre, A FreeCAD Manual. Modeling for product design; Preparing models for 3D printing; Creating FEM analyses. For tasks 1 and 9 and specimen printing, study the parametric model, mesh export and analysis setup. This is an introductory book; check current commands against the help for your version. If using Fusion, read only the general principles of models and meshes.

Three references for the chosen tools

CAD commands. FreeCAD: Sketcher: Constraints, PartDesign: Body, Tools, TechDraw: Views, Dimensions. Alternative: Fusion Parameters and Joints, Limits.

FEA. FreeCAD FEM: Workflow; SimScale Static: materials, boundary conditions, mesh as an alternative reference. In task 9, record the actual restraints and check mesh refinement.

Print material. Prusa Filament Material Guide: PLA, PETG, FLEX. This compares materials; it is not a ready-made profile for your printer. Establish temperature and operating settings from filament manufacturer data and specimen test results.

Reading deliverables. Prepare one load-transfer sketch, one beam calculation, a table of fit dimensions and a justification of print orientation. Connect these results to the decisions in your CAD revision.

Glossary: terms and definitions

Parametric model. Geometry defined by dimensions, relationships and a history of operations; changing a driving parameter rebuilds dependent features.

Sketch constraint. A geometric or dimensional condition that reduces the independent motions of sketch elements, such as coincident points or parallel lines.

Degree of freedom. An independent coordinate needed to specify an admissible mechanism configuration; the number is determined by joints and imposed constraints.

Tolerance. The permitted variation in a specified dimension or geometric parameter, selected according to function and manufacturing method.

Fit. The nature of a connection between mating parts, determined by the combination of their limiting dimensions; a fit may be clearance, interference or transition.

Backlash. Free relative motion when the load direction reverses; it arises from clearances and affects motion transmission accuracy.

Tolerance stack. The relationship between several dimensions that jointly determine a final clearance or position; each deviation's contribution depends on assembly geometry.

Load path. The sequence of components and connections through which a load passes from its application point to the structure's supports.

Stress. Internal force per unit cross-sectional area, measured in pascals. Normal and shear components are distinguished.

Strain. Relative change in material geometry; for uniaxial tension, $\epsilon = \Delta L / L$, a dimensionless quantity.

Young's modulus. The coefficient relating normal stress and strain in the linear elastic region, measured in pascals.

Stiffness. The ratio of load to the corresponding displacement in a linear model; it depends on material, shape and support conditions.

Strength. A material's or part's ability to sustain load without failure through the mechanism considered; it differs from resistance to elastic deflection.

Second moment of area. A geometric integral describing the distribution of area about an axis; measured in m^4 , it influences flexural rigidity $E \cdot I$.

Factor of safety. The ratio of a selected limiting value to an operating value for a specified failure mechanism; it cannot correct an incorrect load model.

Anisotropy. The dependence of material properties on direction; a printed part's strength between layers may differ from its strength along them.

Finite element analysis (FEA). Numerical analysis of a model divided into finite elements; the result depends on material, loads, constraints, mesh and problem formulation.

Mesh convergence. Stabilisation of a selected result as the mesh is progressively refined; it does not establish the correctness of the analysis's physical assumptions.

Equipment, installation and readiness checks

Complete preparation within the first study blocks in place of a repeated general overview. The total remains 75 hours; waiting for delivery is excluded. The statuses below identify actions for you to take, rather than purchases or installations already completed.

USE EXISTING EQUIPMENT

MacBook Pro M4, **your existing 3D printer with a profile for its exact model**, hand tools and a measurement log. There is no need to buy another printer. M05 ends with a passive leg and specimens; the future actuator envelope may still be explicitly provisional.

BUY BEFORE MEASUREMENT AND PRINTING

1 pair of 150 mm callipers with external and internal jaws, a depth gauge and specified measurement error appropriate to the part tolerances. Digital display resolution is not accuracy. Check zero and repeatability on one specimen before inspecting fits.

Material: if supplies are unavailable, 1 spool of PLA for specimens and geometry; PETG if selected; a small amount of TPU only for a flexible part or pad. Filament diameter and settings must suit the printer; calculate the required mass with the slicer, allowing a reserve for repeat specimens.

Fasteners and supports: one set for one passive leg and specimens, according to the released BOM: screws, nuts, washers, threaded inserts, shafts, bushings or bearings. Specify exact diameters and quantities from your CAD model, ensuring that tools can reach the fasteners. Do not order final actuators or a set of parts for the entire robot here.

INSTALL NOW - ONE CAD ENVIRONMENT

macOS ARM: [FreeCAD, official macOS arm64 release](#) **or** Fusion if already familiar. FreeCAD requires Sketcher, PartDesign, assembly, TechDraw and FEM; analysis needs CalculiX and Gmsh. Check whether these are included, configure their FEM paths and add missing components following the instructions for your version. [FEM Workflow](#) describes the workflow; included components depend on the chosen version's installer.

Install or update only the appropriate slicer and profile for the printer's **exact model**, following its manufacturer's instructions. If choosing Fusion, check access to static FEA beforehand; if unavailable, use a separate simple beam in FreeCAD FEM instead of buying an unnecessary package. Save the CAD, solver and slicer versions and the profile in the project.

INSTALLATION AND READINESS CHECKS

1. Fully constrain a rectangular sketch, extrude it and change a dimension. 2. Export STEP and STL, reopen them and check scale. 3. Create a PDF drawing with a dimension. 4. Analyse a static beam with a known force and compare deflection with the hand formula. 5. Check dimensions and print paths in the slicer, then manufacture one specimen; measure it before printing the whole leg.

Act on the checks: if the specimen does not achieve the required fit or sustain load across the layers, change the print profile or CAD model and produce another specimen. Order subsequent parts from the verified model revision; this result does not automatically call for a different printer or a bulk parts order.

Motion fundamentals: leg constraints and equilibrium

Required, within the 75 hours. Use 1 h from hours 41-42 and 2 h of the hours 51-60 practical work on the leg design load case. Use the existing task 3 assembly and task 7 diagrams. No new equipment or motor experiment is required. Before starting, check yourself: identify the three joint axes and convert mass 0.20 kg into weight 1.962 N at $g=9.81 \text{ m/s}^2$.

Freedom of motion requires correctly specified constraints

An open chain of three independent revolute joints with a fixed base has three degrees of freedom. Its configuration is $q=(q_1, q_2, q_3)$. Constraining a part sketch does not immobilise the assembly. If joints are locked by a fixture in a static test, additional constraints and their reactions are introduced. This is a different mechanical problem from a free leg; friction in an unpowered actuator is no substitute for a specified constraint.

Free-body model of one link

For an illustrative special case of task 7, use one horizontal rigid link with $L=0.12 \text{ m}$, $m=0.20 \text{ kg}$ and its centre of mass at $L/2$. The fixed coordinate frame originates at the left joint; x points right, y up, and positive moments are anticlockwise. Friction and acceleration are absent. The environment applies an upward force $f_{\text{ext}}=(0;2) \text{ N}$ at the right end; weight acts downward. These are exercise values, not a test load for a printed specimen.

Mentally isolate the link. Replace the base with reactions R_x, R_y and the fixture's holding torque τ_{hold} . The environmental force produces moment $+L \cdot 2 = +0.240 \text{ N}\cdot\text{m}$; weight produces moment $-(L/2)mg = -0.11772 \text{ N}\cdot\text{m}$. The sum of moments without the fixture is positive, so the free link will not remain horizontal.

Equilibrium requires $\tau_{\text{hold}} + 0.240 - 0.11772 = 0$, giving $\tau_{\text{hold}} = -0.12228 \text{ N}\cdot\text{m}$. Force balance gives $R_x = 0$ and $R_y + 2 - 1.962 = 0$: $R_y = -0.038 \text{ N}$. Here a negative reaction is admissible for a bilateral attachment to the rig; unilateral floor contact cannot realise every calculated force.

Why the Jacobian also appears in statics

For this link at $q=0$, $J=(0, L)^T$. Work done by the external force during a small rotation is $f_{\text{ext}}^T \delta p = f_{\text{ext}}^T J \delta q$, so the corresponding generalised torque is $J^T f_{\text{ext}} = +0.240 \text{ N}\cdot\text{m}$. Holding torque balances it together with the moment of weight. In M05 this is a fixture reaction; in an actuated-joint model, an actuator may fulfil this role. Here f_{ext} is the environment's force on the robot; its sign cannot change while its definition remains the same.

Two deliverables for CAD and the test fixture

1. Mark on the assembly which motions each joint permits and which the fixture prevents. Save two diagrams: the free 3-DOF leg and the restrained test pose. For task 7, identify the physical component that transmits each holding torque.
2. Repeat force and moment balance for the three poses already selected, expressing lever arms and forces in one coordinate frame. Check the moment using both $r \times f$ and $J^T f$ for a simple link. Save a reaction table with units and a load-transfer sketch. A geometrically admissible pose does not establish the structure's ability to sustain load; deflection, strength and dynamics require separate checks.

Assigned reading: Lynch, Park, Modern Robotics: [§2.2, Degrees of Freedom of a Robot](#); [§5.2, Statics of Open Chains](#). Compare the book's sign for force acting on the environment with the f_{ext} convention used here.

Why a robot needs a separate controller

M6 • 25 Feb 2027 - 22 Mar 2027 • 75 hours

Outcome of the month

Create an ESP32-S3 controller that polls an IMU and encoder, transmits consistent measurement snapshots, limits the control signal and enters a predefined, tested state on a fault. This forms the basis of the electronics for a leg or finger: the computer may delay a message, while the local controller must determine whether measurements are fresh and operation may continue. Deliver firmware, a wiring diagram, bus traces, cycle-timing measurements and a record of injected faults. Completing the module requires demonstrated controller operation on a physical test rig.

What to bring from previous months

Use the M1 telemetry format and error handling, circuit and measurement fundamentals from the electrical modules, and the M5 leg CAD model for sensor placement and cable routing. Entry check: explain voltage relative to common ground, an input versus a power output, the purpose of a pull-up and encoder-angle units. Find the power circuit and pins reserved for USB, memory and boot configuration in your exact board's documentation. If this is still difficult, include the review in the first 2 hours, before connecting external circuits.

The minimum physical setup depends on available components: ESP32-S3, one compatible IMU, an angle source or encoder, a button, a protected low-power output indicator and measuring equipment. Check exact models first; select components according to their specifications. Temporarily replace a missing sensor with a packet generator that keeps the same interface. This demonstrates program logic, but the hardware item remains open in the final report until measured on a real device.

Scope and connection to robots

M6 does not require moving a loaded leg or assembling the entire robot. Test the enable output on a low-power load; the motor rig and its limits belong to M7. The same mechanisms will detect missing finger feedback in a hand, or interrupted angle and IMU updates on a walking platform. Software inhibition of a command and independent removal of energy are separate checks. A stopping circuit must account for the fact that removing power from a real vertical axis can cause it to fall; here, test its electrical logic on the rig.

Preparing verification evidence for G1 is part of this work. The G1 integration review itself has a separate 35 hours in the overall plan and is excluded from these 75 hours. Take all hardware limits from the selected components' documentation and the load calculation; the exercise frequencies below define a measurement scenario, rather than a safe operating regime for the future robot.

Organising 75 hours and the test rig

Four work blocks

Block A, hours 1-20: 10 hours of GPIO, PWM, ADC, timer and memory theory; 10 hours of wiring, signal measurement and interrupt practice. Block B, 21-40: 10 hours of protocols and 10 hours of IMU, encoder and robust communication work. Block C, 41-60: 10 hours of task scheduling and 10 hours of cycle-time variation (jitter) measurement, queue checks and watchdog testing. Block D, 61-75: 5 hours of fault analysis and 10 hours of integration tests. This gives exactly 35 hours of study and analysis plus 40 hours of practice.

Split each 10 hours of theory into five 2-hour sessions: 35 minutes reading the relevant section, 35 minutes drawing or calculating, 40 minutes on a small verification example and 10 minutes of notes. Schedule ten-hour practical blocks on Sundays; divide them into stages with breaks and count the 10 hours as active work. Split the final 5 analysis hours into 2+2+1. Calendar boundaries may overlap adjacent modules; retain rest days, holidays and G1 from the overall plan. Complete numbered steps during available study hours without counting any hours twice.

Deliverable structure

Create firmware/ for ESP-IDF, host/ for the Python decoder, config/ for pin assignments and limits, tests/ for frames and state transitions, evidence/ for traces, and plots/ for graphs. In README, state board model, revision, SDK version, build and flashing commands, external power wiring and each sensor type. Keep raw measurements separate from processed plots, so errors can be traced to the device or script. For every run, record run_id, firmware hash, configuration, start time and workload type.

Change one thing at a time

In each hardware experiment, change one parameter: bus frequency, polling period, queue depth or logging load. Before starting, record the expected effect and termination condition. Afterwards, record actual frame count, errors and missed deadlines. If a trace is unclear, return to the last reproducible configuration. Do not change wiring, firmware and communication speed simultaneously and then claim to have identified the cause. Use separate commits for the working baseline and deliberate fault injection.

Take the first wiring photograph with ground, power, data lines and enable output labelled. In the log, distinguish “physically measured”, “verified by a software test” and “awaiting hardware verification”. This allows work to continue while components are delayed, followed by the remaining hardware checks.

GPIO, PWM and ADC: signals in a real robot

What to study

Study GPIO modes: input, output, open drain, pull-up/pull-down and active level. For PWM, relate frequency, period and duty cycle: $\text{duty} = \text{high-level time} / \text{period}$. For ADC, distinguish raw code, pin voltage and the physical quantity measured by the sensor. Resolution alone does not guarantee accuracy: know the permissible input range, calibrate the channel and assess noise. Read the board pin table and GPIO restrictions in the ESP-IDF GPIO reference; check PWM parameters in the ESP-IDF LEDC reference. A motor or coil is not an ordinary GPIO load.

Task 1. Button-controlled motion enable

The aim is to verify operator enable for a local joint controller. Initial conditions: a low-power indicator replaces the actuator, and the active-level circuit is explicitly known. 1. Create an input/state/output table. 2. Configure a safe electrical state before application startup. 3. Implement time-based debouncing while retaining raw events for analysis. 4. Perform 30 presses and releases, including short presses and holds. Save the diagram, log and state-machine test. Check for one confirmed action per press and no accidental enable at startup. Fault: a disconnected button wire must not be interpreted as a new start command.

Task 2. Check the control pulse

The aim is to check whether the software command matches the physical signal. Use only a protected test output. 1. Select frequency and resolution within peripheral limits. 2. Apply several duty-cycle values within the API's permitted range. 3. Capture at least 100 periods per mode with an analyser or oscilloscope. 4. Calculate mean, minimum and maximum period and duty deviation. Save the trace and a table of commands and measured values. Separately test zero command and an invalid parameter; the driver must report failure rather than silently retaining the previous active output.

Task 3. Analogue diagnostic channel

This task prepares for supply-voltage or current measurement, initially using a known permissible test signal. Record the channel transfer function and units. Collect a series with constant input, calculate mean and standard deviation, then change the signal to at least three points across the range and compare with independent measurements. Save conversion coefficients, calibration residuals and a saturation flag. For a signal outside the measurable range, the program marks data invalid rather than substituting a plausible value. Never apply an ADC voltage outside your exact board's limits.

Experiment references: [GPIO](#), [PWM](#). Use the configuration and API error sections for your pinned ESP-IDF version.

Timers, memory and interrupt handlers

Essential theory for reliable firmware

Distinguish a hardware timer, the system tick and elapsed-time measurement. Schedule periodic polling from its expected time, otherwise execution duration is added to every period. Learn which operations are permitted in an ISR, how to pass an event to a task, and why logging or waiting for a lock in a handler increases latency. Check the available ESP-IDF APIs in the documentation for the chosen version. A pointer to a local array cannot be used after that array's lifetime ends; volatile provides neither atomicity nor synchronisation between cores and tasks.

Task 4. Timestamp an encoder event

The aim is to associate joint angle with measurement time, rather than USB packet reception time. Begin with a pulse generator or available encoder. 1. Define a sequence with a known edge count. 2. In the ISR, save only the minimum: a count or timestamp and a task notification. 3. Process and transmit data outside the ISR. 4. Compare received events with the original trace. Deliver an interval plot, lost-event count and an explanation of counter overflow. Increase frequency until an error is detected, within the electrical rig's capabilities; the resulting limit applies to this configuration.

Task 5. Who owns the measurement snapshot?

The aim is to prevent memory overwrites from making angle and IMU readings refer to different times. Create a Snapshot structure with seq, sample time, measurement fields and validity bits. 1. First reproduce the error in a computer test: the producer changes a shared buffer and the consumer reads it later. 2. Pass the snapshot by value through a queue, or use buffers with explicit ownership. 3. Populate fields with related check values and verify that slowing the consumer does not produce mixed snapshots. Save the integrity test and data-lifetime description. Define queue-overflow behaviour in advance and add a loss counter.

Stack, heap and limited resources

Fix queue sizes and estimate memory per snapshot. Measure the minimum remaining task stack with the SDK's available tools after the highest-load test, rather than just after startup. Check memory-allocation failure and driver reinitialisation paths. Do not add dynamic allocation to a frequent loop without a reason; begin with bounded structures. State units and clock type for every timeout. Test timer-counter overflow using artificial values in a computer test rather than waiting for it on the board. The report must distinguish the largest measured latency from a proven real-time guarantee.

Experiment references: [timer](#), [encoder](#). Use the configuration and API error sections for your pinned ESP-IDF version.

Buses and packets: IMU, encoder and computer

Interfaces to study

UART: baud rate, character format, message boundaries, complete and partial frames. I2C: address, register operations, ACK/NACK, pull-ups and timeout. SPI: CS, CPOL/CPHA, bit order and register reads. Study CAN and RS-485 in terms of transceiver role, topology and termination; do not confuse CAN arbitration with shared-channel management in RS-485. Implementing all five physical buses within the month is unnecessary: primary computer communication and one real sensor interface are required; reinforce the others by interpreting traces and drawing a small circuit.

Task 6. Identify the sensor from its trace

The aim is to confirm that data comes from the intended IMU and is decoded correctly. Inputs: the installed sensor's datasheet, identification register and measurement format. 1. Check power and logic levels. 2. Read the identifier and range settings. 3. Capture a transaction with the analyser and manually label address, direction, data and acknowledgement. 4. Convert bytes to a number, accounting for signedness and byte order. Save an annotated trace and a calculation converting the raw value to physical units. Fault: an incorrect address or broken line must produce a timeout and missing-data status, rather than an endless wait.

Task 7. A telemetry frame that can resynchronise

Design a bounded-length frame: start marker or boundary encoding, version, length, seq, timestamp, payload and checksum. Choose a CRC algorithm from a documented implementation and state its parameters; a simple sum has different error-detection properties. 1. Write encoding and decoding code on the computer. 2. Feed the stream one byte at a time and in random chunks. 3. Add a missing byte, junk before a frame, incorrect length and an altered checksum. 4. Connect the same decoder to the board. Acceptance: no damaged frame in the fixed test set is accepted, and the next intact frame is recovered without restarting.

Throughput and units

Estimate traffic before flashing firmware: frame bytes $\times$ frames per second plus the chosen transport's overhead. Compare the calculation with the actual trace and retain capacity for diagnostics. For each field, state SI unit, sign, range and invalid flag; do not mix rad/s and deg/s. For the encoder, separately state counts per revolution, accounting for decoding mode and gearbox. An open computer connection does not make the last value fresh: assess measurement freshness from sample time and sequence number.

Experiment references: [UART](#), [I2C](#), [SPI](#). Use the configuration and API error sections for your pinned ESP-IDF version.

FreeRTOS: measuring timing behaviour

Separate responsibilities

Divide the work into four functions: sensor reading, state and enable checks, telemetry transmission and service diagnostics. Assign priorities according to the consequences of delay, rather than code size. First achieve predictable execution with a small number of tasks; introduce core pinning only after measurement. Read the differences in ESP-IDF FreeRTOS and its critical-section rules in ESP-IDF FreeRTOS: Critical Sections. A queue transfers data, a mutex protects a shared resource, and a notification signals an event; choose the mechanism to suit communication and synchronisation requirements.

Task 8. Measure period, latency and deadline

Choose an exercise period T , for example 10 ms for the first low-power rig, then justify it using sensor frequency and permitted communication time. The planned start of cycle k is $t_0 + kT$. Record actual start s_k and finish e_k : start jitter = $s_k - (t_0 + kT)$, execution time = $e_k - s_k$. A deadline is missed if completion occurs after the selected deadline. 1. Capture a baseline run. 2. Repeat with intensive logging and a slowed receiver. 3. Compare p50, p95, p99, maximum and missed-deadline count over equal numbers of cycles. Save raw timestamps, configuration and plots. An average must not hide a rare, large outlier.

Task 9. Queue overflow without losing control

The aim is to keep updating joint state while USB communication is slow. Create a fixed-length queue and two modes: full data recording for a rig experiment, or retaining only the latest state for monitoring. Each mode needs its own explicitly described loss policy. Slow the consumer enough to guarantee the queue fills. Check maximum occupancy, dropped_count, seq ordering and time to detect stale data. Firmware must not wait indefinitely for queue space in a task responsible for inhibiting the output. Save the observed limit together with test duration.

Task 10. Test the watchdog

Study the interrupt watchdog and task watchdog in ESP-IDF Watchdogs. On the low-power rig, deliberately block a selected task in a separate test build and record the reset cause. Measure the enable output using an independent channel before reset and after boot. Acceptance: restarting must not restore active enable from a saved command; a new explicit start procedure is required after successful checks. Watchdog servicing must reflect the health of critical functions, rather than run unconditionally from an unrelated timer. Separately stop the computer process: this is a communication failure, not necessarily a failure of the MCU itself.

Integrated rig and fault state machine

controller-io v1 specification

Rig inputs: IMU, encoder angle or count, operator command and electrical-limit sensors where available. Outputs: telemetry snapshot, diagnostic events and test-channel enable. Initial mode BOOT validates configuration and moves to DISARMED. Only an explicit start command with all conditions satisfied enters ARMED. Loss of valid measurements, violation of a monitored limit or E-stop enters FAULT. Removing the cause does not by itself enter ARMED. Recovery proceeds through a confirmed fault reset and DISARMED.

Task 11. Transition table and its code checks

Inputs are the lists of events, states and physical outputs. 1. For every state/event pair, specify the next state, reason code and channel enable. 2. Implement a pure transition function that can be tested on the computer. 3. Issue commands out of order: start during a fault, repeated reset, connection restored without confirmation. 4. Compare actual telemetry with the table. Save the table, tests and logs for at least five event sequences. Check that no path leads from BOOT or FAULT to ARMED without the required conditions and a new operator action.

Task 12. Independent stop

Draw the energy path from source to test load separately from the software command path. Identify a component capable of stopping the energy supply independently of the normal MCU loop; select it from the load and circuit documentation. First test its function on a low-power equivalent. Then hold the program in a deliberate test hang and activate the physical stop. Deliver the circuit, output measurement and recovery procedure. Measure time from the physical event to cessation of the test signal and compare it with the rig requirement set in advance. This test does not assign an industrial safety rating to a homemade system.

Configuration and reset during supply loss

Store configuration with a schema version and range validation. Do not save ARMED as a state that restores automatically. For NVS, study initialisation, writing and errors in ESP-IDF NVS. Test a corrupt or unknown format on a data copy: loading invalid limits must inhibit enable. Test power reset by controllably disconnecting an appropriate source with a low-power load, rather than by short-circuiting it. After power returns, the log shows the restart cause, configuration is checked again and motion enable remains absent. Actual power-stage tolerance of voltage dips is tested later with the specific M7 actuator.

Test matrix and verification criteria

Input-data failures

Test F1: disconnect the IMU. Record the last valid snapshot time, timeout and removal of enable. Choose the freshness threshold from the expected update period and the rig's acceptable delay. Test F2: corrupt frame and false length. The decoder rejects the packet without reading beyond the buffer, increments a counter and recovers the next valid frame. Test F3: a stuck encoder value while packet sequence numbers continue. Demonstrate the difference between fresh data and a physically plausible measurement; without a motion model, a constant value alone does not prove a fault.

Execution and recovery failures

Test F4: the telemetry receiver stops reading. Logging drops data according to the selected policy while freshness checks and channel shutdown continue; measure response latency. Test F5: the monitored task is blocked. The watchdog triggers in the test build, the reset cause is included in the report, and the controller remains DISARMED after boot. Test F6: E-stop while the MCU is hung. The independent circuit switches off the test output; releasing the button does not restart it. For F5/F6, capture the physical output, because "FAULT" in the console does not prove the signal has stopped.

Configuration and timing boundaries

Test F7: configuration contains a negative period, unknown version or nonnumeric coefficient. Each cause has clear diagnostics; reject the entire configuration or apply it using predefined atomic-update rules. Test F8: power restart while ARMED with a low-power load. A new enable is required after startup. Test F9: artificial counter overflow and two frames with the same seq. Statistical counters must not report negative age, and duplicates must not conceal missing new measurements. Test F10: ADC measurement reaches the range boundary; the program reports saturation rather than inferring an exact current value.

How to record each result

For every scenario, record circuit and firmware versions, initial state, fault-injection method, expected transition, measured times, filenames and outcome: pass, fail or "not physically verified". Repeat critical transitions several times and retain their variability; one successful reset does not characterise reliability. For each timing metric, state its start, end and measurement clock. Do not subtract PC and MCU timestamps without synchronisation or an offset estimate. If an instrument is unavailable, the computer test remains useful, but leave the physical-latency item open.

Work sequence: hours 1-40

Hours 1-10: read and design

1-2: describe rig components and parameters; check pin assignments, levels and unused pins. Draw the power path and initial output state. 3-4: study GPIO and PWM signals, calculate period and duty cycle, and select measurable modes. 5-6: cover ADC and unit conversion; tabulate raw and converted values. 7-8: study timers and ISR; draw an event-timestamp-task-packet timing diagram. 9-10: understand snapshot memory and queues; write a shared-buffer overwrite scenario. Finish the block with an interface design and a list of expected measurements.

Hours 11-20: first practical block

11-12: assemble and test the button with protected indicator; save photographs and the task 1 state table. 13-14: perform task 2 PWM measurements, including invalid configuration. 15-16: collect task 3 analogue series and plot calibration. 17-18: measure task 4 edges and move processing out of the ISR. 19-20: correct snapshot ownership in task 5, add a bounded queue and save the working baseline. Allocate the final 15 minutes of every two-hour step to documenting results; those minutes are included in the stated time.

Hours 21-30: data and protocols

21-22: study UART and frame boundaries; estimate required throughput. 23-24: study I2C and the specific IMU's registers; manually decode an example transaction. 25-26: study SPI and compare clocking modes with the sensor datasheet; do not connect a new sensor before checking levels. 27-28: create a comparative diagram of UART, I2C, SPI, CAN and RS-485: lines, addressing, shared-medium access, transceiver and termination. 29-30: design framing, CRC and resynchronisation rules; prepare a fixed set of damaged messages and expected responses.

Hours 31-40: the complete sensor path

31-32: read IMU identification and measurements; verify axes and sign. 33-34: capture and annotate task 6 traces; compare raw values and SI conversions. 35-36: implement task 7 frame reception with partial reads. 37-38: inject corruption, disconnect and reconnect the sensor; remove indefinite waits. 39-40: perform an extended run in the chosen mode and compare frame count and traffic volume with calculations. Save the second baseline as baseline-2 and describe its limitations. If new hardware is unavailable, use a test-data generator and label these steps as software work, rather than a completed physical sensor path.

Work sequence: hours 41-75

Hours 41-50: timing, resources and faults

41-42: read the FreeRTOS task model and ESP-IDF differences; draw dependencies and selected priorities. 43-44: allocate available time within a cycle and define task 8 metrics; choose frequency from the sensor and available communication. 45-46: compare queues, notifications and mutexes for your data; examine slow logging. 47-48: study watchdogs and reset causes; define testable boot behaviour. 49-50: study configuration storage and separate recovery after power loss into distinct states. Do not spend these hours optimising code before obtaining measurements.

Hours 51-60: measurement practical

51-52: implement periodic scheduling and timestamp recording. 53-54: perform the task 8 baseline run and repeat under load; produce distribution plots. 55-56: overflow the queue in task 9 and compare the selected policies. 57-58: inject the task 10 hang in a separate build and observe the physical output during reset. 59-60: check restart, memory and configuration; fix one identified cause of deadline misses and repeat only the affected experiment. Finish the block by stating measured limits, test durations and what remains unproven.

Hours 61-65: five hours of analysis

61-62: analyse rig hazards. For each, identify the energy source, failure event, possible consequence, preventive measure and an observable check of that measure's operation. 63-64: complete the task 11 state-machine table and task 12 independent-path diagram; verify that hardware and software stops do not depend on the same hung task. 65: prepare the acceptance sequence, initial states and record forms. This 2+2+1 split retains the overall 35/40 balance and adds no hours to the programme.

Hours 66-75: final rig block

66-67: integrate the state machine, measurement-freshness checks and configuration validation. 68-69: perform F1-F3 and F7; compare messages with the transition table. 70-71: perform F4-F5 and check latency under load. 72-73: independently verify F6/F8 on a low-power load and test recovery; check completeness of timestamps and raw traces. 74-75: reproduce the project from a clean build, make a short demonstration video and assemble the G1 package. Explicitly state any hardware check not completed. You may omit a second transport prototype, an extended PC interface and extra plots; the fault design, primary sensor path and prevention of automatic restart are compulsory.

Acceptance, self-check and handover to M7

Completion conditions

First: firmware builds from the instructions and starts without active output enable. Second: one real IMU and angle-measurement source have documented formats, units and timestamps; temporary substitutes are explicitly marked. Third: physical traces of the primary communication path and damaged-frame checks are available. Fourth: cycle period, jitter, execution time and missed deadlines have been measured under a described workload. Fifth: watchdog, lost communication and E-stop do not cause automatic restart after recovery. Sixth: the independent stopping path has been verified from the physical output, and the limits configuration cites the source of every value.

Set numerical requirements for frequency, timeouts and time to remove the test signal before the experiment, based on the rig's purpose. Place requirement, measurement, instrument, conditions and conclusion side by side in the final table. Where evidence is missing, "not completed" is more useful than an average without its source. Before G1, prepare the revision-labelled circuit, photographs, repository, tests, raw measurements, plots, failed-case log and list of unverified hardware assumptions.

Questions for oral review

1. Why does volatile not resolve a race between ISR and task? 2. How does measurement age differ from packet age? 3. Where should a timeout occur if the IMU does not respond? 4. Why does CRC not establish the physical plausibility of an angle? 5. What happens to the output while the bootloader runs? 6. Why must watchdog servicing depend on critical tasks? 7. How can p99 look good despite a dangerous single outlier? 8. What exactly does the physical stop switch off during a program hang? 9. Which data may be restored from NVS? 10. Why does power restoration not mean permission to resume motion?

Answer each question using your own circuit, test or trace. If your answer uses only library terminology, create a physical example: a missing finger-angle measurement, a stale leg command or USB overloaded by logging. Connecting a cause to its observable effect is this month's central skill.

What the next module needs

Pass the working controller, measurement and command interfaces, timeout list, reset log, fault state machine and independent-stop circuit to M7. Adding a real actuator requires new current, thermal and mechanical limits; low-power rig values do not transfer automatically. The completed interface will support actuator-latency measurement and reproducible identification experiments. Save release controller-io-v1 and the list of required hardware checks separately from subsequent control changes.

Assigned reading: a focused reading route

Read only the specified sections before the corresponding experiment. Most literature is in English; keep explanations, the glossary and the report in English for this edition. Reading and preparation are included in the existing 75 hours; you do not need to work through entire books.

Core reading and limits on transfer

1. Jonathan W. Valvano, **Embedded Systems: Introduction to ARM Cortex-M**

Microcontrollers. Chapters 4, 6, 9-11: input/output, pointers and data structures, interrupts and real time, analogue interfaces, communication. For tasks 1-7, read the principles; do not apply Cortex-M register descriptions to ESP32-S3 or buy a TI development board for the book.

2. Richard Barry / FreeRTOS, Mastering the FreeRTOS Real Time Kernel. Current online book: chapters 4, 5, 7: Task Management, Queue Management, Interrupt Management. Tasks 8-9 need task states, queues and deferred ISR event processing. Older PDFs use different numbering; follow these chapter titles.

3. Edward A. Lee, Sanjit A. Seshia, Introduction to Embedded Systems, 2nd ed. (2017). Chapter 3 Discrete Dynamics; chapter 11 Multitasking. Read selectively on finite-state machines and concurrent tasks. For task 10, draw the fault-machine transitions; for the queue, explain the race and data ownership. Full formal verification of the controller is outside this module.

Documentation instead of API guesses

ESP-IDF FreeRTOS. Tasks, SMP Scheduler, Critical Sections. After the book, compare the specific ESP-IDF implementation: stack-size units, cores and locks. Select the reference version matching the project's pinned version.

Failure and recovery. Watchdogs: IWDT, TWDT, JTAG and NVS: Read, Write and Erase; Error Handling. For task 10, check hangs, restarts and corrupted configuration; ARMED must not restore automatically.

Buses and peripherals. Links for GPIO, LEDC, timer, UART, I2C, SPI and PCNT appear on the relevant practical pages. Add your IMU's datasheet: registers, units, byte order and access conditions.

Reading deliverables. Prepare a pin diagram, a task-period table, a buffer-ownership diagram and a fault state machine. Every document should explain an actual trace or reproducible failure.

Glossary: terms and definitions

Microcontroller (MCU). A chip containing processor, memory and peripherals for direct device control; ESP32-S3 uses Xtensa cores.

GPIO. A programmable general-purpose digital pin; the controller and board circuit determine its mode, permissible voltage and functions.

Interrupt. An event that invokes a handler outside normal task execution; service latency depends on priorities and blocking.

ISR. An interrupt service routine with bounded execution time and restricted permitted operations; it usually delegates lengthy processing to a task.

Timer. A hardware or software mechanism for measuring time; a hardware counter and software handler have different sources of delay.

Jitter. Variation in start times or operation duration relative to a selected reference; measured in units of time, such as microseconds.

Deadline. The time by which a computation or action must finish; a miss is defined by system requirements, rather than average performance.

RTOS task. An independent execution context with a stack and state; the scheduler selects a ready task according to priorities.

Queue. A bounded buffer for passing messages between contexts; overflow and waiting require an explicitly selected policy.

Data race. Unsynchronised concurrent access to shared memory involving a write; the result may depend on execution order.

Critical section. Code accessing shared state protected by a coordinated synchronisation mechanism; its duration affects system latency.

DMA. Transfer between peripherals and memory without per-item processor copying; buffer ownership and transfer completion must be coordinated.

UART. An asynchronous serial interface with agreed baud rate and character format; the protocol specifies message boundaries separately.

I2C. An addressed synchronous bus with SDA data and SCL clock lines; its electrical implementation requires checks of pull-ups, capacitance and levels.

SPI. A synchronous serial interface with clock and device selection; mode, bit order and transaction boundaries must match the sensor.

CRC. A check code for detecting specified transmission errors; it provides neither sender authenticity nor protection against intentional substitution.

Data age. Time from acquisition of a physical measurement to its use, including transmission, queues and processing.

Watchdog. A mechanism detecting the absence of expected evidence of operation; a controller restart does not guarantee a safe power-stage state.

Finite-state machine. A model with a finite set of states and defined event-driven transitions, useful for enabling operation and handling faults.

NVS. Nonvolatile value storage with an access API; configuration needs a version and validity checks after loading.

Equipment, installation and readiness checks

Complete preparation within the first study blocks in place of a repeated general overview. The total remains 75 hours; waiting for delivery is excluded. The statuses below identify actions for you to take, rather than purchases or installations already completed.

USE EXISTING EQUIPMENT AND BUY BEFORE THE PRACTICAL

Reuse the Mac M4, multimeter, supply, breadboard, wiring and M04 low-power load. **Buy: 2 boards** [ESP32-S3-DevKitC-1](#) of one verified revision: the first is the controller, the second a pulse generator and packet-exchange device. Check Flash, PSRAM, circuit and pins reserved for USB and boot configuration.

1 IMU breakout module: access to raw three-axis accelerometer and gyroscope data, documented registers, I2C and/or SPI, an accessible interrupt line, and supply and logic levels compatible with 3.3 V. Check driver support for ESP32-S3, the chosen interface and ESP-IDF version. If you already have such a sensor, do not buy another. A sensor providing only processed angles does not replace access to raw channels.

1 logic analyser: at least 8 digital channels, inputs suitable for the chosen levels, UART/I2C/SPI decoders and a sample rate with margin for the bus being studied. An example is [Saleae Logic 8](#); check Apple Silicon software before ordering. **1 USB set:** two data cables with connectors matching the boards, an adapter or hub if needed, probes with common ground and connectors.

INSTALL NOW - macOS ARM

Start with [PlatformIO IDE for VS Code](#): build, upload and monitor a simple project. Then use [ESP-IDF, official Get Started](#) for target esp32s3 with compatible CMake, Ninja and compiler tools. Pin versions; the PlatformIO espressif32 platform version and ESP-IDF framework version are different fields and their compatibility requires a separate check.

Add [KiCad for macOS](#) for the wiring diagram and electrical checks, and software for your exact analyser, for example [Logic 2 for Saleae](#). A sigrok-compatible instrument may need [PulseView](#); check ARM and USB support before purchase. Logic 2 does not support arbitrary analysers. Use Ubuntu 24.04 ARM64 in UTM only after separately checking USB; the main build and measurement route is native Mac.

INSTALLATION CHECK

On both boards, perform build→flash→monitor, hardware reset and firmware-version reading. Then test UART loopback or communication between boards, measure PWM, perform one I2C or SPI transaction with the IMU and export the trace. Verify raw-count units and monotonic time. Open the circuit in KiCad and run ERC; save environment versions and the pin-assignment diagram.

Optional, for later: STM32 only for a separate comparison; the motor with encoder is purchased in M07. In M06 an existing encoder can be the angle source, and the second board can generate quadrature pulses. The generator tests the interface; required real sensor measurements remain separate acceptance items.

Motion fundamentals: angle, velocity and time

Required: connect measurements with motion

The leg controller receives counts, but motion is described by angle $q(t)$ and its change over time. Before the experiment, explain: $90^\circ = \pi/2$ rad; positive rotation is defined by the oriented joint axis, and an identical angle in two packets does not prove a stop. In M6, use the available sensor or a generator on the second ESP32-S3, without a new motor.

Measurement model and worked example

Let c_k be an unwrapped counter without overflow, C the counts per revolution of the measured shaft after the selected decoding, and t_k the sample time in seconds from one MCU clock. Relative angle $q_k = 2\pi(c_k - c_0)/C$, in rad. If the encoder is on the motor, converting to link angle separately requires a known gear ratio and sign. There is no reduction here.

Exercise data: $C=4000$, $c=(100;104;110)$, $t=(0;0.010;0.025)$ s. This gives $q=(0;0.006283;0.015708)$ rad. Interval velocity $v_k = (q_k - q_{k-1})/(t_k - t_{k-1})$ is 0.628319 rad/s in both intervals. Dividing the second increment by the nominal 0.010 s gives a false 0.942478 rad/s: interval variation creates the appearance of acceleration.

Finite-difference velocity is the interval average, rather than the exact instantaneous derivative. One count corresponds to 0.001571 rad; over short windows, quantisation is visible in velocity. Smoothing reduces noise but introduces delay. USB reception time is no substitute for physical sampling time: a late, intact packet may contain a stale angle.

One end-to-end experiment for tasks 4, 5 and 8

First reproduce the table on the computer. Then transmit a known A/B sequence from the second board and compare received edges with the logic analyser. Save counts, sample_time, receive_time, seq, q and interval v. Plot velocity calculated from actual and nominal intervals. Adding a constant to every c must not change relative angles.

Repeat with delayed consumer reads and identical timestamps on two frames. Reject a zero or negative interval before dividing; a stale snapshot does not become fresh because of a new packet. For example, at constant $v=0.628319$ rad/s, age 0.05 s means an angle change of 0.031416 rad. This indicates the error scale, rather than specifying a robot timeout.

Included in the 75 hours: 1 hour from 7-8 for the time model; 1 hour from 17-18 for task 4; 1 hour from 53-54 for task 8 plots. Use their existing traces and report. The generator validates the signal path; the real angle sensor requires a separate check.

Read selectively: [Lee/Seshia, Introduction to Embedded Systems, ch. 3 Discrete Dynamics](#) for state and samples; [ESP-IDF PCNT: Introduction, Channel Actions](#) for A/B counting semantics. Check the API against the pinned version.

Measure an actuator on one joint first

M7 • 04 Apr 2027 - 30 Apr 2027 • 75 hours

Main deliverable

Over the month, produce a single-actuator test rig and a model relating torque, speed, current and temperature. The result answers a specific question: can the selected actuator perform the work of a leg or finger joint over the required motion and load range? A catalogue torque at stall does not mean the actuator can hold the robot's body continuously. Define requirements, measure an available sample, separate known parameters from estimates and validate the model on new data. Decide whether to buy a complete actuator set only after testing one sample.

What must already be ready

From M6, bring the controller, synchronised telemetry, error counters, timestamps, command limits, physical stop and verified prevention of automatic restart. From M5, bring link geometry, mountings and approximate mass. Identify where angle is measured: at the rotor, after the gearbox or by a separate output-link sensor. This determines whether gearbox backlash is observable. Entry check: convert degrees to radians, revolutions per second to rad/s and grams to kilograms, and explain force versus torque. Calculate the static torque of a known mass about an axis.

Select equipment around the available actuator and its documentation. You need a securely mounted sample, compatible driver or the smart servo's built-in control interface, angle/current/temperature measurement, a current-limited supply and a guarded fixture. If these are unavailable, first complete requirements and a digital model; clearly separate calculated characteristics from measurements. A bracket may be made on the existing 3D printer after checking geometry, material and load. A printed part does not automatically have the strength of an industrial mounting.

Scope of the month

Study the differences between DC, BLDC and smart servo architectures, but experiment with one type. You do not need to develop your own FOC driver, rewind a motor or buy 12/18 actuators. A smart servo's internal loops may be closed to the user; measure its external behaviour and explicitly identify inaccessible internal signals. Low speed, holding and backlash matter for a hand; loaded torque, repeatability and heating matter for a leg. The method is shared, while requirements differ.

The final report must show the operating region supported by measurements and the modes for which data remains insufficient. This plan specifies no universal current, temperature, voltage or mass limits: derive them from the motor, driver, gearbox, supply and mounting documentation.

How to allocate 75 hours

Balance study and experiments

Hours 1-20: actuator comparison, requirements and preliminary model; 10 hours theory and 10 hours practice. Hours 21-40: measurement fixture, encoder, current, temperature and rig checks; 10+10. Hours 41-60: control loops, limits and repeatable response; 10+10. Hours 61-75: identification, validation on new data and model boundaries; 5+10. Total: 35 hours of reading, calculation and analysis, and 40 hours of practice. Count evaluation of raw measurements as practice, and equation derivation and experiment selection as analysis.

Divide each ten-hour theory block into five 2-hour sessions. In each session, study one mechanism, calculate its effect for your joint, then prepare an experiment. Schedule ten-hour practical blocks on Sundays, divided into stages with breaks; 10 hours means active work. The final 5 theory hours are split 2+2+1. Dates on the first page are the periods in the original overall plan. Shared boundary dates, rest, holidays and integration reviews are unchanged; place steps in available study hours without double counting.

A separate record for each experiment

State motor model and revision, gearbox, firmware, controller gains, supply voltage, ambient temperature, mounting, current sensor and temperature measurement location. Record command and measured angle, speed, current, voltage and temperature on one time axis, or document synchronisation. The commands file contains what was actually sent after limiting, rather than only the original desired profile. Save stop causes and missing-data intervals so fault periods remain visible in plots.

Repository organisation

Create requirements/ for joint calculations, fixture/ for CAD and assembly, acquisition/ for collection code, experiments/ for configurations, data/raw/ for raw runs, models/ for parameters and reports/ for plots and limitations. Split data into training (train) and validation sets by whole runs before fitting. Randomly splitting one run into nearly identical neighbouring training and validation points is not independent validation. Begin with a simple model and retain it as the baseline before adding nonlinearities.

If time is short, reduce the number of brands compared, omit an additional actuator type and simplify the interface. Requirements, measurement path, motion limits, thermal observation and validation on an independent run remain compulsory. Keep missing hardware measurements as open items rather than replacing them with arbitrary numbers.

Joint requirements and architecture selection

What to study

Compare brushed DC, BLDC with a separate driver, and smart servos with integrated control. Consider motor, gearbox, sensor and interface separately: a convenient bus does not guarantee suitable torque, and a powerful motor does not solve feedback problems. Study the torque-speed curve, efficiency, rotor inertia, permissible gearbox load, encoder resolution, backlash and transmission compliance. Distinguish continuous and short-duration operation, and electrical, thermal and mechanical limits.

Task 1. Load profiles for a leg and finger

Start with the M5 mass and geometry; express unknowns as ranges. For a link with a horizontal lever arm, use $\tau = mgr$, with m in kg, g in m/s^2 , r the perpendicular lever arm in metres and τ in $N \cdot m$. 1. Draw a free body with force direction. 2. Calculate three link positions and two payload cases. 3. For the leg, add a support scenario with load distribution explicitly assumed; do not divide robot mass by leg count without describing the support phase. 4. For the finger, use desired fingertip force and lever arm. Save the requirements table and sensitivity to mass and length. Check that doubling the lever arm at constant force doubles static torque. Edge case: zero force lever arm.

Task 2. Compare actuators under relevant operating conditions

Select three architectures or available candidates with primary documentation. Build a comparison table: torque in the required mode, loaded speed, mass, reduction, sensor, backlash, current control, diagnostics, supply, interface and unknown characteristics. 1. Reject any option that fails a hard requirement. 2. Separately assess uncertainty from missing data. 3. Choose one sample for the rig and describe an experiment that could disprove that choice. Prepare a decision justifying each criterion; an arbitrary weighted score is no substitute. Check that every important conclusion has a source, calculation or future test, and that stall torque has not been used as continuous torque.

Reduction and inertia

Fix the definition $N = \text{motor speed/output-shaft speed}$. In an ideal transmission, output speed is divided by N , torque is multiplied by N , and load inertia reflected to the motor is divided by N^2 . Actual efficiency and backlash impose additional limitations. For the chosen geometry, calculate how increasing the ratio changes speed and reflected inertia. These calculations are hypotheses for measurement; gear tooth counts alone cannot establish actual holding accuracy.

Measurement rig and calibration

Task 3. Fixture for one actuator

The aim is repeatable motion without movement of the motor housing. Inputs: mounting drawing, load direction and the sample's permissible operating characteristics. 1. Draw the load path from shaft through mounting to base. 2. Position travel stops, motion-area guarding, access to the stop control and cable strain relief. 3. Check alignment and free travel without powered motion. 4. Perform the first powered run at the minimum justified command, increasing it in predefined steps. Deliver CAD, photographs, a fastener list and the verification record. Edge case: the cable must not become taut and alter measured torque at the travel limits.

Create one table of permissible values for the supply, driver, wires, connectors and fuse. The weakest component determines the limit, considering operating mode, cooling and documentation. A fuse and a software current limit serve different purposes. Separately establish behaviour during braking and energy return to the supply: a supply capable of delivering energy may be unable to absorb it during braking. Do not command sharp acceleration or braking until this behaviour is understood.

Task 4. Encoder angle and speed

Check counts per revolution, quadrature decoding mode, gear ratio and sign. 1. Apply a known manual displacement with the actuator disabled, if its construction permits back-driving. 2. Compare counts with output-link angle. 3. Estimate velocity from angle differences over a known interval and, alternatively, from time between edges if pulses are available. 4. Compare noise and delay at low and higher speeds. Deliver the conversion factor and plots of both estimates. Check that a full revolution or available reference angle produces the expected count. Edge cases: overflow, reversal, zero speed and a missed edge.

Task 5. Current and temperature

Establish what the sensor measures: supply current, winding current or current in the control coordinate frame. These quantities are not automatically interchangeable as I in $\tau = K_t I$. Measure zero-current offset, compare known points with a reference instrument and retain uncertainty. Record temperature with the sensor location and settling time: housing and windings heat differently. Deliver calibration coefficients and a description of limitations. Fault: a missing or saturated instrument must not permit continuation of an experiment that depends on its readings.

Position control and bounded commands

Control-loop architecture

Determine who closes the current, velocity and position loops: your code, the driver or smart-servo firmware. Use SimpleFOC and ODrive resources to draw the cascade and label the input and output units of each block. An external controller must not duplicate an unknown internal loop without understanding its mode. PID relates error to control action; feedforward predicts the required action. Command limiting introduces nonlinearity: the integrator can continue accumulating error when the actuator can no longer meet the request.

Task 6. Repeatable position response

Initial conditions: a checked rig, known limits and a small motion profile away from stops. 1. Begin with an available simple mode and a reproducible initial state. 2. Apply a smoothly changing command with specified limits; save position, speed and actual control action. 3. Change one gain or profile parameter while keeping load constant. 4. Repeat the best configuration several times. Deliver reference/measurement plots, steady-state error, overshoot and settling time. Define the settling band beforehand in angle units; measure the time until the response stays within it. Edge case: reversal and return to zero.

Task 7. Saturation and anti-windup

Begin in simulation of the simple joint. 1. Limit available control action. 2. Apply a command that cannot be achieved quickly. 3. Compare an ordinary integrator with the chosen anti-windup method: integrator limiting or back-calculation. 4. Restore an achievable reference and measure recovery time. Save two plots of control action and integrator state under identical conditions. Check that you can explain why controller gains cannot remove a physical limit. Confirm on hardware only with a smooth motion profile inside the operating region; deliberately jamming the shaft to demonstrate saturation is unnecessary.

Task 8. Motion profile and stopping

Set position, speed and acceleration limits from rig requirements and documentation. Generate a bounded trajectory between two angles and compare it with an abrupt step in simulation. Use only the permissible profile on the rig. Measure maximum speed, current, error and behaviour near the software limit. Reject or limit an out-of-range command under an explicit rule and log it; a mechanical stop is not a routine way to end every motion. Communication loss and restart use the tested M6 state machine, while the mechanism's subsequent state depends on its load and construction.

Identification: friction, delay and backlash

A testable model

Start at the output joint: $J\ddot{q} + b\dot{q} + \tau_c \cdot \text{sign}(\dot{q}) + \tau_g(q) = \tau_a$. Here J is inertia about the axis in $\text{kg}\cdot\text{m}^2$, b is viscous resistance in $\text{N}\cdot\text{m}\cdot\text{s}/\text{rad}$, τ_c approximates dry friction, τ_g is gravitational torque and τ_a is actuator torque. Near zero speed, the sign model is inadequate: rest, breakaway and backlash require separate observations. If torque is unavailable, use the stock actuator command as input and identify the closed-loop command-to-angle system, explicitly stating that the model includes the actuator and its internal controller.

Task 9. Backlash and dead zone

The aim is to understand why a finger ignores a small command and why a leg reverses with a delay. 1. Reproduce slow motion in both directions over one range. 2. Record input command and angles before and after the transmission if both sensors are available. 3. Plot output against input in both directions and identify the region where input changes produce no measurable output. 4. Repeat with another permissible load. Save dead-zone and backlash estimates with sensor resolution. If only the motor encoder is available, output-link mechanical backlash cannot be claimed as directly measured; a separate observation method is required.

Task 10. Steps and frequency excitation

Prepare small bounded steps and a smooth chirp within the region authorised beforehand; chirp means excitation frequency changes over time. First calculate maximum profile speed and acceleration to ensure they remain within limits. 1. Record raw responses. 2. Identify delay and characteristic response time. 3. Fit a minimal model to the first group of runs. 4. Validate it with another profile and initial position. Save parameters, applicability limits, raw data and residuals. Do not increase frequency into dangerous oscillation merely to extend the measured bandwidth.

Why not fit everything at once?

Friction, delay, an inaccurate torque constant and a velocity filter can explain similar errors. Check units and measurement delay first, then a simple linear model; add one mechanism only when its effect has independent evidence. Save parameters with units, estimation method and a confidence range or observed spread. Lower error on the training record is not success if a new record is predicted worse. Show one case the model reproduces and one where it predictably loses accuracy.

Torque, speed, heating and the operating region

Task 11. A verifiable torque characteristic

If an independent force sensor and known lever arm are available, estimate torque as $\tau = F \cdot r_{\text{perp}}$. Begin with a secured rig, small justified loads and known geometric accuracy. 1. Check sensor zero. 2. Measure several safe operating conditions with different loads and speeds. 3. Repeat in both directions. 4. Compare torque with measured current, accounting for current type and transmission. Deliver characteristic points with variability and operating limits. If torque is not independently measured, give a calculated estimate and a method for later verification; do not call a smart servo's internal "load" indicator an accurate torque sensor without manufacturer confirmation.

Task 12. Thermal experiment with controlled duration

Choose a repeated profile resembling leg work or finger holding, within documented limits. 1. Begin from a known thermal state. 2. Record ambient and available-sensor temperatures, current, operating mode and fraction of time under load. 3. Stop at the preset duration or an earlier documentation-based stop condition. 4. Record cooling after the experiment. Save temperature rise above ambient and load duty cycle. A short record showing continued heating does not establish an acceptable steady-state thermal regime.

The simplest thermal model relates temperature rate of change to dissipated power and heat removal. In the first version, estimating characteristic heating and cooling times and explaining applicability limits is sufficient. The I^2R loss approximation requires the appropriate current and resistance; supply current and unknown internal commutation may not provide this information. Do not infer unmeasured winding temperature from an external sensor without a justified thermal-coupling model.

A combined operating-range map

Plot measured points in the speed-torque plane, marking current, temperature, duration and limiting factor. Display catalogue limits separately from your measurements. For a leg, state the supported profile and tested duration; for a finger, separately assess holding and repeated small motions. Check repeatability after cooling. An actuator with a different gearbox, voltage or cooling arrangement is a different configuration requiring verification. The final map informs robot design; it does not guarantee every future operating mode.

Fault tests and numerical criteria

Measurement path

F1: encoder sign reversed. Detect the error in a software test and an initial small motion before continuing closed-loop control; record expected and actual direction. F2: incorrect counts per revolution. Comparison with an independent angle must reveal the scale error. F3: current or temperature data disappears. Mark the run invalid and stop it if the relevant parameter is a safety condition for the experiment. Do not reuse the last available value as fresh protective information. F4: timestamps jump or frames are lost. Do not treat angle differentiation across a corrupted interval as reliable velocity.

Control and mechanics

F5: target angle beyond the software limit. Check the chosen command rejection or limiting rule; log requested and applied commands. F6: control action saturates in simulation. After an achievable reference returns, anti-windup restores normal operation according to a defined metric; retain the saturation interval in the report. F7: communication loss during permissible motion. Test the M6 stopping procedure and prevention of automatic restart. F8: small reversal. Compare the expected dead zone with instrument resolution; the program must not infer absence of mechanical motion from a single unchanged count.

Validate the model on another record

F9: a model fitted to one profile receives another profile, load or amplitude within its declared region. Compare angle RMSE, maximum absolute error and transition-time error. $RMSE = \sqrt{\text{mean}((q_{\text{model}} - q_{\text{measured}})^2)}$; its unit matches the angle unit. F10: load or temperature goes beyond the identification region in simulation or existing data. The report must explicitly label extrapolation rather than conceal it in an overall mean error. F11: power restoration. Motion must not resume solely from a saved reference.

Set thresholds before the experiment

Choose angle tolerance from the joint's future function and measurement capability; do not specify "0.1 degree" without reference to the encoder and backlash. Define a settling band and required time within it before measuring settling time. Compare model error with a simple baseline, such as a first-order system with constant delay; a more complex model must improve independent validation. For each experiment, retain maximum measured current and temperature with duration, sample rate and measurement location. Preserve a failed run with its stop cause and do not assess it as a fully completed run.

First 40 hours: requirements and rig

Hours 1-10: determine what must be measured

1-2: break leg and finger requirements into position, speed, torque and operating duration. 3-4: study DC, BLDC and smart servo architectures; draw a motor-gearbox-sensor-controller diagram. 5-6: calculate task 1 static torques and their geometric dependence. 7-8: study the torque-speed characteristic, reflected inertia and efficiency; plot simple expected trends. 9-10: read documentation for available candidates and list unknown parameters. Finish by selecting the question the rig experiment must answer.

Hours 11-20: document requirements and prepare the experiment

11-12: complete task 1 load scenarios and unit table. 13-14: build the task 2 comparison table with sources and characteristic conditions. 15-16: create a simple joint simulation with bounded control action; save parameters and hypotheses. 17-18: design the task 3 fixture geometry and check load transfer, stop access and cables. 19-20: prepare test commands, acquisition configuration and the first experiment procedure. If the sample is not yet known, remain at interface level and explicitly mark parameters TBD; do not copy another motor's limits.

Hours 21-30: measurement and protection

21-22: study the supply, driver, wiring, fuse and braking behaviour from the exact components' documentation. 23-24: study the encoder and velocity-estimation methods; calculate the smallest observable angle change. 25-26: establish current measurement location and type, sensor offset and range. 27-28: study thermal modes, temperature-sensor location and housing/winding temperature differences. 29-30: prepare calibration procedures, initial states and stop conditions. Finish this block with a usable measurement procedure, rather than notes alone.

Hours 31-40: first physical dataset

31-32: assemble and check the fixture without powered motion, then perform the first smooth-motion run. 33-34: verify task 4 angle scale and sign. 35-36: compare velocity estimates and retain stationary noise measurements. 37-38: calibrate current and temperature channels for task 5. 39-40: repeat a short profile, check synchronisation between fields, stopping and recreation of the plot from raw data. Saving results is included in every step. Without a real sample, complete equivalent software tests and leave physical acceptance open until the next available session.

Hours 41-75: control and modelling

Hours 41-50: theory through your joint

41-42: draw the actual inner and outer loops; label inputs and units. 43-44: study P, I, D and velocity measurement; explain the effect of each gain. 45-46: study action limits, integral accumulation and anti-windup; prepare a simulation comparison. 47-48: calculate a profile with speed and acceleration limits; distinguish a software boundary from a physical stop. 49-50: define response metrics, repeatability, settling band and communication-fault behaviour. Do not attempt complex tuning until feedback sign and angle scale are verified.

Hours 51-60: control practical

51-52: run the smooth task 6 profile; save the baseline response and several repeats. 53-54: change one control parameter and compare under identical conditions. 55-56: complete task 7 simulation of saturation and recovery. 57-58: verify the task 8 bounded profile on the rig, including a request outside the software range. 59-60: perform task 9 slow reversals; assess dead zone and whether backlash can be measured. Freeze the working configuration at the end; identification runs must not mix different controller gains without labelling them.

Hours 61-65: prepare identification

61-62: derive the simple joint model and choose a measurable input: torque, current with a justified conversion constant or an external smart-servo command. 63-64: divide future runs into fitting and validation sets; choose task 10 excitation and task 11 torque-measurement conditions; calculate permissible profile limits. 65: finalise the task 12 thermal procedure, report format and sample-suitability criterion. Split these 5 hours 2+2+1; all following 10 hours are practice, keeping the total at 75.

Hours 66-75: experiments and independent validation

66-67: record bounded steps and an available frequency profile; estimate delay and initial parameters. 68-69: obtain available torque measurements or an explicitly labelled calculated estimate; verify directions and repeatability. 70-71: run the thermal profile for the predefined duration and also record cooling. 72-73: fit the model and validate on another run; compare residuals with the baseline and explain one systematic error. 74-75: assemble the operating region, model limitations and decision on the suitability of one sample. If time is insufficient to reach steady temperature, state that in the report; a short experiment's results cannot be transferred to continuous operation.

Presenting the result and moving to mechanics

What the final package must contain

Requirements for the chosen joint with load scenarios; architecture comparison and sample justification; CAD and photographs of the guarded rig; supply, stop and limit descriptions with sources; angle, current and temperature calibration; raw synchronised recordings; controller parameters; a torque-speed-current-heating model within available measurements; independent validation and a fault list. For every model parameter, state units, estimation method, operating range and confidence level. Unmeasured torque or winding temperature remains unknown even if software can plot it.

Completion criteria from the original plan: selection rests on measurements, electrical and thermal parameters stay within justified limits, and a set of 12/18 actuators is not selected from one catalogue number. A negative decision is valid: the sample may be unsuitable because of heating, backlash, speed, mass or diagnostics. This is a useful completed result if evidence supports the reasons and requirements for the next sample are stated. Publishing the report here means preparing a reproducible package; no external sharing or posting is required.

Self-check questions

1. Why is supply current not always the current that determines torque? 2. What changes when the reduction ratio doubles? 3. How does backlash differ from elastic deformation? 4. Where is the encoder, and which error can it not observe? 5. Why does a good unloaded response not resolve leg-support requirements? 6. What does the integrator do during saturation? 7. Why can the same PID produce a different response after heating? 8. What exactly constitutes independent model validation? 9. How can measurement delay be distinguished from actuator delay? 10. Why does housing temperature not guarantee a known winding temperature? Each answer needs your own plot, calculation or explicit data limitation.

Handover to M8 and M9

Pass axis geometry, permissible angle ranges, sign, zero, resolution and measured dead zone to M8. Kinematics must generate achievable references within these limits, but does not itself establish sufficient torque. Pass the identified joint, masses, inertia estimates, friction, delay, input interface and independent recordings to M9. Keep motor and link parameters separate to avoid counting the gearbox or inertia twice. When mechanics, load or cooling changes, flag the need for new validation because the previous operating region may change.

Assigned reading: a focused reading route

Books and notes may be in English; keep the plan, definitions and your explanations in English for this edition. Read only the specified sections within the existing 75 hours.

Core reading and one reference route

- 1. Core:** K. Lynch, F. Park, Modern Robotics, §8.9. Actuation, Gearing, and Friction: motor, gearbox and reflected inertia. Read for tasks 1-2 and 9 and explain why motor and output-joint parameters differ.
- 2. Core:** K. J. Åström, R. Murray, Feedback Systems, 2nd edition. Chapter 11 PID Control: Basic Control Functions and Integrator Windup. Read for tasks 6-8; draw your loops and compare saturation with anti-windup. Other editions have different numbering.
- 3. Core, selectively:** R. Tedrake, Underactuated Robotics: System Identification. Sections on least squares and dynamics identification. For tasks 9-10, list measurable inputs, unknown parameters and a separate validation recording; skip complex models.

Documentation for the practical work

ESP-IDF: Pulse Counter - Channel Actions and Glitch Filter for the encoder; SimpleFOC: Current Sensing - motor versus supply current for task 5.

ODrive: Controller - cascade diagram and Trajectory Control for tasks 6-8. This is an architectural example; obtain your driver's limits only from its own datasheet.

Complete the reading with an action

After each section, perform one calculation or short experiment. Save the actuator, driver and instrument datasheet revisions. For tasks 10-12, use the familiar SciPy `least_squares`: check parameter physical meaning and residuals on new data. Buying printed books is optional.

Glossary: terms and definitions

Torque. The turning effect of force: $\tau = r \times F$; torque about a specified axis is the projection of τ onto that axis, in N·m.

Gear ratio. The ratio of input to output speed; here $N = \omega_{\text{motor}} / \omega_{\text{joint}}$, with rotation direction specified separately.

Efficiency. The ratio of useful output power to input power under stated conditions; it depends on speed and load.

Torque constant. Coefficient K_t relating motor electromagnetic torque to current under the adopted model, in N·m/A.

Back EMF. Voltage generated by motor rotation that opposes the supply; proportional to speed in a simple model.

Stall current. Current in a stationary motor at a specified voltage; this is a limiting condition, not an allowable continuous current.

Continuous torque. Torque permissible continuously under specified cooling, speed and temperature conditions; the manufacturer's conditions must be confirmed.

Reflected inertia. Inertia referred through a gearbox to a chosen shaft; rotor inertia at the output scales as N^2 .

Backlash. The relative angular travel between mating components over which a reversal is not immediately transmitted to the load.

Static friction. Contact resistance before motion starts; it takes the required value only up to the maximum holding limit.

Viscous friction. An approximation of motion resistance as torque proportional to angular velocity and opposing it.

Quadrature encoder. A sensor producing two phase-shifted A/B signals from which angle increments and rotation direction can be determined.

Counts per revolution. The number of counted signal transitions per revolution of the specified shaft; it depends on decoding mode and gearbox.

Feedback loop. A path using measured output to change a command in order to reduce a specified control error.

Saturation. Limiting actual control action at a physical or software boundary, even when the calculated command is larger.

Anti-windup. A rule correcting integral state during output limiting, reducing excessive error accumulation and recovery delay.

System identification. Determining model parameters or structure from recorded inputs and outputs under explicit assumptions.

Residual. The difference between measured output and model prediction for the same inputs, time and initial state.

Thermal time constant. A characteristic temperature-change time in the adopted thermal model; it depends on heat capacity and heat transfer.

Check the relationships: $P_{\text{mech}} = \tau \omega$; for an ideal gearbox, $\omega_{\text{out}} = \omega_{\text{in}} / N$ and torque increases by a factor of N . Losses and heating require separate treatment.

Equipment and software for M7

USE EXISTING EQUIPMENT

MacBook Pro M4; tools and the M4 CC/CV supply; one ESP32-S3 from the two M6 boards, IMU, logic analyser and USB cables. The 0-30 V, 5-10 A rating specifies the supply's capabilities: set experiment voltage and current according to the chosen load's limits.

BUY BEFORE RIG WORK: hours 21-40

1 motor: a 12 V brushed DC gearmotor with an A/B quadrature encoder and documented gear ratio and counts. **1 driver:** a bidirectional H-bridge with current limiting, protection and 3.3 V-compatible inputs; select current rating and cooling for the motor and chosen experiments.

1 current-measurement path in the driver, or a separate sensor or shunt with amplifier; **1 contact temperature sensor** with logged readings. **1 fixture set:** rigid mounting, hub or lever, stops, guarding, power switch, fuse and correctly sized wires. Do not pass power-stage current through a solderless breadboard.

Example datasheet study: [Pololu 37D with encoder](#). Check encoder-output voltage; add level conversion before ESP32-S3 if needed. [TI DRV8876](#) illustrates protection and current-sensing requirements; this does not promise compatibility with that motor.

ADDITIONAL ITEMS, IF JUSTIFIED BY CALCULATION

For independent torque measurement in task 11: 1 calibrated force gauge with a suitable range plus a measured lever arm. A separate smart actuator for the future leg: only 1 candidate after calculating requirements. The rig's DC motor does not automatically become the final leg actuator; do not yet order a leg set or 18 actuators.

SOFTWARE AND READINESS CHECK

No new mandatory packages: macOS ARM, pinned ESP-IDF and Python, NumPy, SciPy, Matplotlib and pytest from M6. Build firmware, record one complete revolution in both directions, compare current with a reference, and check temperature, limiting and stopping before loading. Test USB with the actual connection; a VM is not assumed verified.

Preparation is included in the existing 75 hours; arrange equipment delivery in advance and exclude delivery time from study hours.

Motion fundamentals: what a gearbox provides

Required: connect joint requirements with the motor

Angle specifies the desired pose, but holding and acceleration require torque. For a leg, foot load and link weights matter; for a finger, the object's force on a phalanx matters. Before calculating, draw the axis and lever arm: force in N and torque in N·m differ. If the force's line of action passes through the axis, its torque about that axis is zero.

A minimal transmission model

For one revolute joint, q is output angle and $v=\dot{q}$ is velocity in rad/s. Choose shaft signs so that $N=\omega_m/v>0$. Then $v=\omega_m/N$. With power transmitted from motor to load and gearbox efficiency η , output torque is approximately $\tau_{out}=\eta N\tau_m$. Hence $\tau_{out}\cdot v=\eta\tau_m\cdot\omega_m$: a gearbox changes torque and speed but does not create power.

At an illustrative operating point, let $N=20$, $\eta=0.75$, $K_t=0.04$ N·m/A, winding current $i=1$ A and $\omega_m=100$ rad/s. Neglecting losses within the motor itself, $\tau_m=K_t\cdot i=0.04$ N·m, $v=5$ rad/s and $\tau_{out}=0.6$ N·m. Mechanical power is 4 W at the motor shaft and 3 W after the gearbox. Supply consumption requires separate measurement of the electrical path.

Separately draw a horizontal link: x right, y up, axis $+z$ and positive torque anticlockwise. A mass $m=0.5$ kg at lever arm $r=0.1$ m produces gravitational torque $-mgr=-0.4905$ N·m at $g=9.81$ m/s². Rest requires compensation of $+0.4905$ N·m, excluding link weight. The difference $0.6-0.4905=0.1095$ N·m does not by itself establish actuator suitability: operating speed, heating and continuous holding differ.

How inertia affects rapid motion

Acceleration also requires torque $I_{joint}\cdot\ddot{q}$, with I_{joint} in kg·m². Rotor inertia reflected to the output is N^2I_{motor} ; load inertia reflected to the motor is divided by N^2 . High reduction therefore helps produce torque but changes dynamics and sensitivity to contact. Constant η is an approximation for the specified motoring condition; do not transfer it to rest or reverse energy flow without verification.

Checks within existing tasks 1 and 11

Build a calculation table and power balance. In the same exercise model, change N to 40: velocity becomes 2.5 rad/s, torque 1.2 N·m, output power remains 3 W and reflected rotor inertia increases fourfold. Then insert your confirmed M7 parameters and label unknowns separately. Check gravitational torque using $r\times F$ and the existing task 11 experiment; without an independent instrument, retain the result as a calculation.

Included in the 75 hours: 1 hour from 7-8 for transmission and inertia; 1 hour from 11-12 for the load table; 1 hour from 68-69 for checking the existing run. This refines deliverables already prepared. Exercise values are not specifications for the purchased motor and must not be sent as commands.

Reference: Lynch/Park, Modern Robotics, §8.9.1 DC Motors and Gearing; §8.9.2 Apparent Inertia. Read only the relationships between torque, speed, power and reflected inertia; FOC development is not added here.

Geometry that turns a target into joint angles

M8 • 02 May 2027 - 03 Jun 2027 • 75 hours

Outcome of the month

Create a verified kinematics library for a planar two-link arm and a leg with three degrees of freedom. It will answer four questions: where the mechanism's endpoint lies for given angles, which angles reach a target point, how joint velocities relate to endpoint velocity, and how an external force relates to joint torques. This directly prepares for moving a walking robot's foot and a fingertip. Deliver code, a geometry specification, visualisations, automated checks and clear explanations of inverse-kinematics failure.

What you should already understand

Prerequisites from earlier months are vectors, matrices, trigonometry, derivatives, Python, NumPy and tests. Bring the CAD leg's dimensions and axes from M5 and the available joint's sign, zero offsets and angle ranges from M7. Before starting, manually evaluate $\sin$ and $\cos$ at 0 , $\pi/2$ and π ; multiply a rotation matrix by a basis vector; convert millimetres to metres. Even if a numerical library performs everything, explain the result's direction on a drawing. The main errors this month usually arise from inconsistent conventions rather than difficult equations.

The first reference is a planar 2R mechanism: two revolute joints, lengths l_1 and l_2 , and motion in the x-y plane. The second is an exercise yaw-pitch-pitch leg: base rotation about z and two bends in a radial vertical plane. If the M5 CAD model uses a different first joint axis, such as abduction, write a separate transform chain from CAD; do not insert its dimensions into another mechanism's formula. Reference models verify the mathematics, while adaptation to the actual leg must retain its real axes and offsets.

What M8 does not require

Kinematics describes geometry, but does not establish sufficient torque, body stability or collision freedom for every part. First calculate and visualise commands; permit transmission to a real actuator only through M6-M7 limits and after a separate rig check. A complete gait planner, force control and dynamic balance belong to later modules. For a hand, 2R can model two phalanges, but tendon transmission and object contact require additional models.

At the end, check forward/inverse kinematics (FK/IK) consistency, verify the Jacobian numerically and ensure predictable failure for unreachable targets. Declare numerical-test tolerances separately from actual mechanical accuracy: a small computational residual does not remove backlash, encoder-zero error or incorrect dimensions.

Project structure and 75 hours

Allocation across four blocks

Block A, 1-20: coordinate frames, rotations and transforms, 10 hours theory and 10 practice. Block B, 21-40: link models and forward kinematics for 2R and 3-DOF, 10+10. Block C, 41-60: analytical and numerical inverse kinematics, 10+10. Block D, 61-75: Jacobians, force, singularities and acceptance checks, 5+10. Total: 35 hours of study and analysis, 40 hours of implementation and experiments. Theory includes your own derivations on paper; practice includes code, plots, tests and investigation of specific errors.

Divide ten-hour theory blocks into 2-hour sessions: one mechanism, one hand-worked example and one software check. Schedule ten-hour practical blocks on Sundays and divide them into small stages; breaks add to calendar time. Split the last 5 theory hours 2+2+1. Calendar boundaries from the overall plan may overlap adjacent modules; retain rest days, holidays and integration reviews. Allocate the hours below to available sessions without counting them again in the neighbouring month.

Library organisation

Create geometry/ for transforms, models/ for 2R and the leg, solvers/ for IK, validation/ for reference poses, visualizations/ for drawings and reports/ for results. Begin with a clear Python/NumPy implementation. Add a C++ core only for a small, already verified interface if time allows; a Rust port is optional. For every array, state shape, units and coordinate frame. Do not give two different types of point the same name p without specifying their origin and the basis in which they are expressed.

Conventions document

In conventions.md, record right-handed frames, body and joint axis directions, positive angle, length and angle units, quaternion ordering, the meaning of T_{AB} , multiplication order and chosen IK branch. Recommended convention: T_{AB} transforms coordinates from B to A, so $p_A = T_{AB} \cdot p_B$ for a homogeneous point. Check every CAD export and library adapter against this document. Do not fix an inverted leg by inserting an arbitrary minus sign before finding the cause.

For every experiment, save mechanism parameters, inputs, expected result and random-configuration generator seed. Use error statuses UNREACHABLE, LIMIT_VIOLATION, SINGULAR, NO_CONVERGENCE and INVALID_INPUT. Never return the final numerical approximation as a successful command without verification: residual and status are part of the API result.

Coordinate frames and composition of motion

What to study about transforms

Rotation R specifies basis orientation, and translation p locates its origin. A homogeneous matrix T combines them: the upper block contains R and p , and the bottom row is $[0,0,0,1]$. For a pure rotation, $R^T R = I$ and $\det(R) = +1$. A matrix with $\det = -1$ is a reflection, not an admissible rigid-body rotation. Under the chosen convention, $T_{AC} = T_{AB} \cdot T_{BC}$. Transform order matters: rotating a local vector and then translating the result is generally different from reversing these operations. Modern Robotics §3.3.1 establishes notation; follow frame indices rather than the words “left” and “right”.

Task 1. Foot coordinates in different bases

Inputs: a drawing of body, hip and foot with simple known offsets. 1. Define $T_{\text{body_hip}}$ and foot coordinates in hip. 2. Calculate body-frame coordinates by hand. 3. Verify with the matrix in code. 4. Apply the inverse transform and recover the original point. Deliver an axis drawing and tests for several poses, including a $\pi/2$ rotation. Check that a rigid transform preserves distance between two points. Edge case: incorrect multiplication order must fail a test with nonzero translation and rotation, even if the case without rotation appears correct.

Task 2. Translation of a point and a direction

The aim is to avoid adding hip position to a force or velocity vector. Take a contact point and force direction in one frame. 1. Transform the point including translation. 2. Transform the free vector using rotation only. 3. Compare results when base position changes without a change in orientation. 4. Add separate `transform_point` and `rotate_vector` functions, or types that prevent confusion. Save the test and illustration. Check that shifting the coordinate origin changes point coordinates but not a free vector's components when the basis is unchanged.

Active and passive interpretations

A matrix may rotate a physical object or describe that same object in another coordinate frame; either use needs consistent frame indices and inverse-matrix definitions. Draw two diagrams: body rotation relative to the world, and a stationary world point expressed in the rotated body's frame. Label what physically moves. For a leg, this explains why holding a foot fixed in the world while tilting the body changes the local target point. Whole-robot IK is not yet needed; demonstrating the correct transformation of one point is sufficient.

Rotations, quaternions and the mechanism tree

Essential SO(2) and SO(3)

SO(2) describes planar rotations and SO(3) spatial rotations. Learn basic rotation matrices and composition, then axis-angle orientation and a normalised quaternion. Euler angles are readable but depend on axis order and have degenerate configurations. Quaternions q and $-q$ describe the same orientation; comparing components as ordinary points is insufficient. Record the chosen order, for example $[x,y,z,w]$ in your adapter, and check every external library's requirements, including SciPy Rotation.

Task 3. Verify orientation representations

Use identity orientation, a $\pi/2$ rotation about each axis and a compound rotation. 1. Write expected axis directions by hand. 2. Convert matrix to quaternion and back. 3. Check orthogonality, determinant and angular error. 4. Supply q and $-q$ and confirm the physical result is identical. Deliver reference orientations and an adapter test. Edge cases: a zero quaternion, NaN and a strongly non-unit input. The API must reject or normalise it according to an explicit policy, without concealing invalidity.

Task 4. Coordinate-frame tree of the CAD leg

For each M5 part, specify parent, joint origin, axis, joint type, zero position and fixed offset. 1. Draw the tree from body to foot. 2. Separate fixed mounting transforms from rotations q . 3. Assemble the chain and visualise every local axis. 4. Check the zero pose and move each joint individually. Save geometry in machine-readable form and images of at least four poses. Check that one joint moves only its descendants; the body and parent links stay fixed. Error: an axis exported in world coordinates must not be treated as local without transformation.

Two ways to describe one chain

Understand what a Denavit-Hartenberg table specifies and why standard and modified versions use different conventions. An explicit transform chain matching CAD is convenient for the first implementation. Then study the product of exponentials: fixed screw axes and the home configuration define FK through a product of exponentials. Derive one planar revolute example and compare with explicit matrices. Do not rewrite the entire project in three notations; the goal is to read other models and recognise differences in conventions.

Forward kinematics of the arm and leg

Task 5. Planar 2R arm

For links l_1, l_2 and relative angles q_1, q_2 in the x-y plane, derive $x = l_1 \cdot \cos(q_1) + l_2 \cdot \cos(q_1 + q_2)$, $y = l_1 \cdot \sin(q_1) + l_2 \cdot \sin(q_1 + q_2)$. The second link's world angle is $q_1 + q_2$, not q_2 . 1. Derive the formula from a drawing. 2. Implement analytical FK and an independent matrix chain. 3. Compare straight, bent and folded poses. 4. Build a grid of permissible angles. Deliver two implementations and a reachable-point map. Check that at $q_1 = q_2 = 0$ the endpoint is $(l_1 + l_2, 0)$; with equal lengths and $q_2 = \pi$, it reaches the base. Handle zero and negative lengths according to geometry-validation rules.

Task 6. Exercise leg with three axes

For yaw-pitch-pitch, define q_1 about z and q_2, q_3 as angles in the vertical plane, with positive pitch raising the link. Let l_0 be a fixed radial offset and $r = l_0 + l_1 \cdot \cos(q_2) + l_2 \cdot \cos(q_2 + q_3)$. Then $x = r \cdot \cos(q_1)$, $y = r \cdot \sin(q_1)$, $z = l_1 \cdot \sin(q_2) + l_2 \cdot \sin(q_2 + q_3)$. 1. Draw this exact mechanism. 2. Build an independent transform chain. 3. Compare both forms at random permissible angles. 4. Add $T_{\text{body_hip}}$ and find foot coordinates in the body frame. Save a 3D view of the leg with axes and tests. If CAD uses another mechanism, use its own chain and a separate reference model.

Comparison with CAD and a library

Choose at least five numerically specified poses and measure endpoint position in CAD or the saved geometric representation. Compare values with your implementation and Robotics Toolbox or an independent matrix implementation. State what is compared in the table: coordinates, units and basis. A large error may arise from millimetres instead of metres, another joint zero, axis ordering or a different point on the part. Find the cause one transform at a time from root to endpoint, rather than compensating for the final error with an arbitrary offset.

Accuracy checks

For a purely numerical reference, choose tolerance in advance relative to mechanism length and machine precision; for example, around 10^{-6} m may be a reasonable starting target for a metre-scale model in double precision, but it is not the robot's mechanical accuracy. Check geometry scaling and errors at boundary angles. Save the maximum norm of the difference between implementations and the poses where it occurs. Random checks supplement, rather than replace, hand-worked reference poses.

Inverse kinematics and solution branches

Analytical solution for the planar arm

For target (x,y) , define $c2=(x^2+y^2-l_1^2-l_2^2)/(2l_1l_2)$. If $|c2|$ is substantially greater than 1, the target is unreachable; a small round-off excess may be clipped only within the declared numerical tolerance. For both branches, obtain $q2=\text{atan2}(\pm\sqrt{1-c2^2},c2)$, then $q1=\text{atan2}(y,x)-\text{atan2}(l_2\sin(q2),l_1+l_2\cos(q2))$. Geometrically reachable points lie between circles of radii $|l_1-l_2|$ and l_1+l_2 , but joint limits further reduce the workspace. Degenerate lengths and folded configurations need explicit handling rules, rather than division by zero.

Task 7. Both branches and joint limits

1. Select a target with two permissible solutions. 2. Calculate both branches and check FK for each. 3. Add angle ranges from geometry and M7; remove solutions that violate them. 4. Choose a solution close to the previous q while respecting the permitted range, rather than merely the shortest angular difference around a circle. Deliver an API returning permissible solutions and a failure reason. Check that $\text{FK}(\text{IK}(p))$ is close to p for all accepted points; $\text{IK}(\text{FK}(q))$ need not return the original branch. Edge case: a target on the outer boundary where the two branches coincide.

Task 8. Foot targets for the 3-DOF leg

For the exercise yaw-pitch-pitch leg, first calculate $q1=\text{atan2}(y,x)$, then solve the planar problem at $(\sqrt{x^2+y^2}-l_0,z)$. This branch assumes a selected radial-projection direction; the model must explicitly bound additional configurations and ranges. 1. Transform the target from body to hip coordinates. 2. Calculate solutions and check limits. 3. Verify them with full FK. 4. Follow a short foot-lift-and-lower trajectory while retaining the selected branch. Save animation, angles and maximum geometric residual. At $x=y=0$, yaw is undefined: return a special status or retain a permissible previous yaw according to the interface rules.

A failure must not become a command

Distinguish no geometric solution, range violation and numerical failure. Return status, $q_candidates$, residual, chosen branch and a diagnostic message. Do not silently replace an unreachable point by the nearest one: projection must be a separate explicitly requested mode returning the target actually reached. For a robot, the difference matters: an unnoticed change in support point changes body position, and a fingertip may fail to reach the object.

Numerical IK and library design

Task 9. Iterative approximation

The aim is to solve a kinematic problem whose analytical formula is inconvenient and learn to diagnose convergence. First use the familiar 2R model as a reference. Error $e = p_{\text{target}} - FK(q)$. Linearisation gives $e \approx J \cdot \Delta q$; solve the system or use a pseudoinverse instead of explicitly inverting a matrix. For damped least squares, $\Delta q = J^T \cdot (J \cdot J^T + \lambda^2 I)^{-1} e$, where λ controls attenuation of large steps near degeneracy. Implement with solve, a step limit and a finite iteration count. Do not present this method as a guarantee of a global solution.

1. Begin with a central reachable target and several initial q values. 2. Record error norm and step size at each iteration. 3. Add joint limits and a termination condition for lack of improvement. 4. Compare with analytical IK on the same targets. Save convergence plots and a table of successful and failed runs. Return SUCCESS only after checking final FK and limits. Edge cases: unreachable target, poor initial seed, nearly straight arm and maximum iteration count. Do not return the last approximation with a large residual as an executable command.

Unified robot-kinematics v1 interface

The model needs `forward(q)`, `inverse(target,seed,limits)`, `jacobian(q)`, `validate(q)` and a geometry description. Inputs must not be changed silently. `forward` returns position and, when required, orientation with its basis specified. `inverse` returns a structured result rather than an unexplained NaN array. Specify the input coordinate frame explicitly. Use one reference dataset for the analytical implementation, numerical method and library comparison. A C++ wrapper may follow these checks; porting to three languages is not the month's goal.

A small integration scenario

Place the body in several known poses and define one stationary world support point. Transform that point to the hip frame, solve IK and recover world foot coordinates. Then repeat for the 2R finger: a small target-point contour in its working plane. Plot target, reached point, angles, status and distance to limits together. This checks geometric consistency across components. Velocities and torques require separate calculations; a smooth-looking animation does not establish physical feasibility of the profile.

Jacobians, forces and singular configurations

Task 10. Calculate foot velocity from joint velocities

For position FK, matrix J contains partial derivatives of coordinates with respect to q ; each column describes endpoint motion from a small motion of one joint. Linear velocity $v=J(q)\cdot\dot{q}$. Derive J for 2R by hand, then obtain it for the leg analytically or using rotation axes and the vector to the foot expressed in one coordinate frame. Begin only with the 2×2 or 3×3 position Jacobian; a spatial velocity (twist) has 6 components and its Jacobian is $6\times n$. Specify linear/angular component ordering separately.

Central-difference check: column i is approximately $[FK(q+h\cdot e_i)-FK(q-h\cdot e_i)]/(2h)$. Compare several h values, for example 10^{-4} , 10^{-5} and 10^{-6} rad, at random and boundary poses. Save maximum error norm and a plot against step size. An excessively small h amplifies round-off, while an excessively large h increases linear-approximation error. Then specify $\dot{q}$ and compare predicted endpoint velocity with a small actual FK step. Edge case: derivative expressed in the wrong basis with a nonzero base rotation.

Task 11. Foot force and joint torque

Fix the meaning of F : external force applied at the mechanism's endpoint, expressed in the same basis as J . The corresponding generalised torque is $\tau=J^TF$; to balance this load, the actuator generally provides the opposite contribution while accounting for other forces. 1. Check one joint using $r\times F$. 2. Compare with J^TF for the arm. 3. Repeat for the leg and several force directions. 4. Check virtual work, $\tau^T\delta q\approx F^T\delta p$. Deliver a static-load calculation and sign test. The formula itself excludes link mass, acceleration and friction; M9 adds these.

Task 12. Map proximity to singularity

A singularity is a loss of Jacobian rank relative to its normal maximum. For 2R, compare bent and fully straight poses. Over a grid, calculate the smallest singular value and available velocity directions; state normalisation and units explicitly. Request motion along the direction lost in the straight pose. Save the map and explain why a small target displacement may require a large Δq . Add status checks and step limits to IK. Proximity to singularity does not itself mean collision, but requires special command-handling rules.

Work sequence: hours 1-40

Hours 1-10: conventions and manual checks

1-2: review vectors, radians and matrix rotations; create conventions.md. 3-4: study homogeneous transforms and work a body/hip example by hand. 5-6: understand active and passive transform meanings and the point/vector distinction. 7-8: study $SO(2)$, $SO(3)$, axis-angle and quaternion representations; fix component ordering. 9-10: draw the CAD leg tree and initial axis positions. At every stage, calculate at least one reference without a ready-made robotics library; adding one later provides an independent comparison.

Hours 11-20: geometry practical

11-12: implement transforms and task 1's coordinate round trip. 13-14: perform task 2, separate points and directions and add a negative test. 15-16: verify task 3 orientation representations and invalid-input handling. 17-18: build the task 4 tree and visualise local axes. 19-20: compare four known poses with a drawing or CAD, save references and correct conventions before continuing. Do not proceed to IK with even one unknown axis direction; its error becomes harder to diagnose there.

Hours 21-30: derive forward kinematics

21-22: derive 2R equations geometrically and check zero angles. 23-24: derive the 2R matrix chain and compare with the formula. 25-26: study yaw-pitch-roll and its difference from the actual CAD leg. 27-28: explore DH and the product of exponentials; reproduce one revolute example. 29-30: define reference poses, tolerance scales and comparison rules for an external library. Separately list fixed offsets that must not be confused with link lengths.

Hours 31-40: two verified models

31-32: implement task 5 and the planar arm workspace grid. 33-34: implement the task 6 exercise-leg FK and body transform. 35-36: adapt the chain to actual M5 geometry if axes differ, retaining separate reference-model tests. 37-38: compare five poses with CAD and one independent implementation. 39-40: run random tests, check units and export, and freeze baseline-fk. Finish every step by saving verification results; correcting a discovered geometric error belongs within this block and takes priority over animation styling.

Work sequence: hours 41-75

Hours 41-50: the inverse problem

41-42: derive analytical 2R IK from the triangle, including both branches. 43-44: study reachability, joint limits and the convention for selecting a nearby solution. 45-46: reduce the exercise leg to a planar problem; handle yaw and offset IO separately. 47-48: study linearisation, numerical IK and the initial guess using Modern Robotics §6.2. 49-50: prepare the result format, validation rules, stopping criteria and unreachable/boundary targets. By now, define how the program will fail before writing its main iteration loop.

Hours 51-60: targets become angles

51-52: perform task 7 for both branches; check FK/IK round trips back to the target. 53-54: perform task 8 and save the foot trajectory and body-frame position. 55-56: implement task 9 numerical IK for 2R and compare different seeds. 57-58: add limits, step bounds and failure diagnostics; test the leg's actual chain. 59-60: assemble the unified API and world-point holding scenario while the body moves. Commands remain geometric data at this stage; SUCCESS does not automatically authorise physical motion.

Hours 61-65: velocity, force and singularity

61-62: derive the arm Jacobian and geometric column for a revolute joint; study numerical derivatives. 63-64: study $\tau = J^T F$, virtual work and rank loss; select verification metrics. 65: prepare the test-pose map and final scenarios, including nonnumeric inputs and inconsistent coordinate frames. Split these five hours 2+2+1. The month's aim is understandable calculations for small mechanisms, rather than a deep proof of every Lie-group property.

Hours 66-75: verify the library

66-67: perform task 10 and compare Jacobians with finite differences for several h values. 68-69: check velocities and task 11 virtual work and external-force signs. 70-71: build the task 12 map and investigate IK near singularity. 72-73: perform every fault-checking scenario and compare an independent library with your code. 74-75: assemble README, plots, reference data and a short demonstration of both mechanisms. If time is short, omit the second language and decorative visualisation; both reference models, FK/IK consistency, Jacobian verification and correct failure behaviour are compulsory.

Acceptance, negative tests and self-check

Required scenarios

F1: rotation and translation applied in reverse order. The task 1 reference pose detects it. F2: millimetres treated as metres. Scale checking and a known length reveal the discrepancy; relative error between functions sharing the same mistake must not produce a false PASS. F3: target beyond the outer radius or inside the unreachable inner circle of 2R. Return UNREACHABLE. F4: geometrically reachable point requires a forbidden angle. Return LIMIT_VIOLATION. F5: fully straight arm. The Jacobian loses rank and IK must not return an unbounded jump as a permissible command.

F6: two neighbouring requests each have two solution branches. The continuity policy retains a permissible branch or reports the need to switch. F7: poor seed for numerical IK. At the iteration limit, return NO_CONVERGENCE with residual. F8: NaN, zero length or zero quaternion. Reject input according to interface rules. F9: q and $-q$. The orientation check treats them as equivalent. F10: force expressed in a different basis. The virtual-work test detects inconsistency. F11: incorrect encoder-zero sign. Testing the joint adapter in a known calibration configuration reveals the error.

Completion conditions

Both models have documented geometry and automated tests. For accepted IK solutions, the norm of $FK(q)$ -target lies within the declared numerical tolerance and joint angles remain in range. State maximum Jacobian error relative to numerical differentiation with the method, steps and pose set. Include hand-worked references, random checks and independent comparison; two functions with a shared error do not sufficiently validate each other. Unreachable and singular requests return clear status, and reproducing results does not require manual file corrections.

Review questions and handover to M9

Explain what T_{AB} means; why a point and force transform differently; why 2R q_2 is relative; why IK need not recover the same branch; how numerical convergence is defined; why rank matters more than one large J component; what sign the torque balancing an external force must have; and how geometry error differs from backlash. Show answers in your drawings and plots. Pass lengths, local axes, zero offsets, FK, Jacobians and permissible configurations to M9. Masses, centres of mass, inertias and contacts will be added separately; do not conceal their absence behind a successful kinematic animation.

Assigned reading: a focused reading route

Books and notes may be in English; keep the plan, definitions and your explanations in English for this edition. Read only the specified sections within the existing 75 hours.

Core reading and one reference route

1. Core: K. Lynch, F. Park, Modern Robotics. §3.3.1 homogeneous transforms; §4.1.1 product of exponentials; §5.1-5.3 Jacobian, statics and singularities; §6.2 numerical IK. This is the route for tasks 1-12: read the relevant section before deriving equations yourself.

2. Reference: S. LaValle, Planning Algorithms. §3.2 Rigid-Body Transformations and §3.3 Transforming Kinematic Chains. For tasks 1-4, compare the geometric explanation of transforms and chains; one 2D chain and the transition to 3D are sufficient.

3. Core, selectively: R. Tedrake, Robotic Manipulation: Basic Pick and Place. Named sections Forward kinematics and Differential inverse kinematics. For tasks 9-10, connect endpoint coordinates with joint velocities; subsequent motion planning is outside this module.

Documentation for the practical work

SciPy Rotation - apply, inv, from_quat and as_quat; check component ordering and q/-q equivalence in task 3.

SciPy least_squares - bounds, jac, tolerances and status; an independent numerical-ik check.

Robotics Toolbox: DH models - optional reference for fkine, jacob0 and DH conventions.

Complete the reading with an action

Finish reading by drawing axes, deriving a formula and checking one known pose. Do not transfer CAD leg dimensions to yaw-pitch-roll without checking axes and zeros. Separately explain why successful IK, reachability and permissible real motion are different checks. You do not need to read all books in full this month.

Glossary: terms and definitions

Coordinate frame. An origin and ordered axes relative to which point coordinates or physical-vector components are expressed.

Pose. A combined description of body position and orientation relative to an explicitly named coordinate frame.

Rotation matrix. A matrix R preserving lengths and basis handedness; for spatial rotation, $R^T R = I$ and $\det(R) = 1$.

Homogeneous transform. A 4×4 matrix combining rotation and translation of a point between two specified coordinate frames.

Composition. Sequential application of transforms; frame indices determine matrix multiplication order, which generally affects the result.

Degree of freedom. An independent parameter needed to specify a mechanism's configuration under its geometric constraints.

Configuration. A set of independent mechanism coordinates; for the open chain considered here, the joint-angle vector q .

Kinematic chain. Rigid links connected by joints whose relative transforms determine the position of each successive link.

Home configuration. A predefined configuration at zero joint coordinates, serving as the geometric reference.

Joint axis. An oriented geometric line of rotation or direction of translation, expressed in a specified coordinate frame.

Forward kinematics. Calculating a selected link's pose from joint configuration and fixed mechanism geometry.

Inverse kinematics. Finding configurations that achieve a specified pose or point, accounting for joint limits and solution accuracy.

IK branch. One of several distinct configurations or continuous configuration families that achieve the same target pose.

Workspace. The set of reachable points or poses of a selected mechanism endpoint under specified geometry and joint limits.

Quaternion. A four-component, unit-norm orientation representation; quaternions of opposite sign describe the same rotation.

Jacobian. A matrix relating joint velocities locally to selected endpoint coordinates; its dimensions and units depend on the task.

Singularity. A configuration with loss of Jacobian rank in which some endpoint-motion directions become locally unavailable.

Pseudoinverse. A generalisation of matrix inversion for linear least-squares problems that selects a minimum-norm solution.

Damped least squares IK. A regularised local solution that suppresses excessive joint-coordinate changes near degeneracy at the cost of residual error.

Check the relationships: $\dot{p} = J(q)\dot{q}$; with consistent signs, the generalised force from an external action is $J^T F$. Do not confuse it with a holding command.

Equipment and environment for kinematics

NO NEW PURCHASES REQUIRED

Use the existing MacBook Pro M4, CAD and measured M5 dimensions, and M7 calibrations and one rig joint. The existing 3D printer is needed only for fixture checks if necessary; select a profile for its exact model. A complete leg and additional actuators are unnecessary for this mathematics.

PREPARE FOR HOURS 1-20

Save a geometry description: parent/child, joint origin, unit axis, joint type, angle sign, zero, limits, foot point and units. Specify that axis is expressed in the joint frame. Explicitly convert CAD exports from millimetres to metres; use the same conventions when creating URDF later.

Keep the reference 2R, reference yaw-pitch-roll and actual CAD chain separate. Changing lengths does not turn one kinematic architecture into another. For physical illustration, reuse the M7 rig only after checking commands in the model.

INSTALL NOW

No new mandatory packages. On macOS ARM, use pinned Python, NumPy, SciPy, Matplotlib and pytest from M2-M7; SciPy already includes Rotation and least_squares. The existing editor and project JSON/YAML suffice for geometry descriptions. ROS, RViz and Gazebo enter later in the programme.

OPTIONAL, IF NEEDED

If an additional library FK reference is useful, add robotics-toolbox-python in a separate environment using the [official Introduction instructions](#). First check dependencies on macOS ARM or the chosen Ubuntu 24.04 ARM64. The main route uses your own matrix FK and SciPy; a MATLAB licence is unnecessary.

INSTALLATION AND INPUT CHECKS

From a clean directory, import the libraries, print versions, perform a $\pi/2$ rotation and its inverse, and compare q with $-q$. Then check FK at the 2R zero pose, FK/IK round trips back to the target, joint limits and the Jacobian using central differences. Save a three-pose plot and a machine-readable pytest report.

For independent comparison in task 5, use Robotics Toolbox or a separate implementation based on saved transforms; for task 9, use SciPy least_squares. Preparation and checks replace duplicated setup within the 75 hours.

Motion fundamentals: from a point to joints

Required worked example: FK → IK → velocity → force

Use two rigid links in the planar part of a leg with a fixed base. x points right, y up and $+z$ towards the viewer; angles are anticlockwise. Measure q_1 from $+x$ and q_2 relative to the first link. Check $200\text{ mm}=0.2\text{ m}$. Here $v=\dot{q}$, in rad/s; endpoint velocity $\dot{p}$, in m/s, is a different quantity.

For $l_1=l_2=0.2\text{ m}$, add link vectors: $x=0.2\cos q_1+0.2\cos(q_1+q_2)$, $y=0.2\sin q_1+0.2\sin(q_1+q_2)$. At $q=(0;\pi/2)$, endpoint $p=(0.2;0.2)\text{ m}$. FK applies known rotations in sequence; IK solves the inverse geometric problem. The cosine rule gives $\cos q_2=(x^2+y^2-l_1^2-l_2^2)/(2l_1l_2)=0$. The branches are $(0;\pi/2)$ and $(\pi/2;-\pi/2)$; both reach the point, but joint limits may exclude one.

The Jacobian: local motion and its limits

Rotation of joint i moves the endpoint tangentially. Thus a position-Jacobian column is $z_i \times (p-o_i)$: axis z_i and the vector from joint o_i to the endpoint are expressed in one basis. At our pose, $J=[[-0.2; -0.2]; [0.2; 0]]\text{ m/rad}$. With $v=(0.5;-0.25)\text{ rad/s}$, $\dot{p}=Jv=(-0.05;0.10)\text{ m/s}$. This is an instantaneous relation; a finite path requires recalculating J as q changes.

At the straight pose $q=(0;0)$, both tangents are vertical, the x row of J is zero and rank is 1. Instantaneous velocity along x is unavailable, although a bent chain can change x . This is a singularity: a small requested point displacement does not guarantee small joint velocities. IK step limiting and damping do not create the missing direction.

Force uses the same geometry; dynamics comes later

In a separate static experiment, apply $f_{\text{ext}}=(0;-5)\text{ N}$ at the endpoint: this is the environment's force on the robot. With link masses and friction excluded, $\tau_{\text{ext}}=J^T f_{\text{ext}}=(-1;0)\text{ N}\cdot\text{m}$, and holding requires $\tau_{\text{act}}=-\tau_{\text{ext}}=(1;0)\text{ N}\cdot\text{m}$. Torque at the second joint is zero because the force's line of action passes through its axis. Link weight and acceleration change the requirement according to the M9 model; successful IK does not establish motion feasibility.

One check connects tasks 5, 7 and 10-12

Save a drawing of both branches, a p/J value table and tests. Compare J columns with central differences of FK at $h=10^{-4}$ and 10^{-6} rad , then repeat at the straight pose. Independently calculate each torque using $r \times f_{\text{ext}}$. Check power at the stated v : $f_{\text{ext}}^T \dot{p} = \tau_{\text{ext}}^T v = -0.5\text{ W}$. This explains the transpose of J and the external-action sign; it is a calculation, not a torque command to a position-controlled servo.

Included in the 75 hours: 1 hour from 21-22 for this reference FK/IK example; 1 hour from 63-64 for force and singularity; 2 hours from 66-69 for existing Jacobian and virtual-work tests. Retain all other tasks; no new laboratory sessions are added.

Read selectively: Lynch/Park, [Modern Robotics, chapters 4 and 6: introductory FK and Analytic Inverse Kinematics](#); [§5.2 Statics of Open Chains](#). Study the $6 \times n$ spatial Jacobian and 6-component twist after the position Jacobian J .

From link position to the physics of motion

M9 • 03 Jun 2027 - 06 Jul 2027 • 75 hours

What you will produce by the end of the month

The deliverable for this month is a joint model using the parameters identified in M7, together with a dynamics simulation of a two-link mechanism or leg. Check dimensions, the energy balance and agreement between calculations and measurements. The kinematics from M8 finds the angles that place a link endpoint at a specified point; dynamics relates motion to the required torque and actuator limits. For a leg, it describes the effects of link masses, body motion and foot contact; for a finger, it describes phalange acceleration, load holding and the reaction when touching an object. Begin with one link, then add a second link and contact.

Inputs

Bring the geometry, coordinate frames, FK and Jacobians from M8. From M7, bring angle, command and current measurements and the available actuator model, including friction, delay and the limits of identification. From M5, bring CAD-part masses and centres of mass, provided they were determined correctly. Label each quantity as measured, calculated in CAD, taken from manufacturer documentation or specified for an illustrative example. Inertia calculated in CAD using a default density is not the measured inertia of a printed part. Account for fasteners, wiring and actual infill; if data are unavailable, state the uncertainty of the estimate.

Initial self-check: distinguish kg from N, N from N·m, velocity from acceleration, and inertia about the centre of mass from inertia about the joint axis. Check the work done by a force through a small displacement and calculate the energy of a load at a known height. Make sure you know the axis from which q is measured and the direction of gravity. An incorrect potential-energy sign can produce smooth simulated motion that contradicts physics.

Scope and evidence

M9 does not require the dynamics of a complete multi-legged platform with gait control. The basic model has a fixed base, two links and explicitly selected contacts. Increase complexity only after checking simple limiting cases. Simulation can confirm the correctness of the equations and numerical method, but agreement with a real actuator requires comparison against a separate measured run. If M7 did not produce physical measurements, complete the computational work and mark the comparison with the real actuator as unfinished.

Prepare a working integrator and a description of the model's limits of applicability. State the angle, velocity and load ranges and contact modes that you have checked. Separately describe the conditions under which backlash, saturation, compliance, heating or an unknown internal actuator controller become significant. Subsequent modules will use this information to choose control laws that account for model accuracy.

75 hours: from simple models to complex ones

Four blocks

Hours 1-20: Newton's and Euler's equations, mass, centre of mass, inertia, forces and energy; 10 hours of theory and 10 hours of practice. Hours 21-40: Lagrange's equations, equation structure and two-link dynamics; 10+10. Hours 41-60: normal contact, friction, impact and static support; 10+10. Hours 61-75: use of the M7 identification results, model validation, sensitivity and error assessment; 5+10. This totals 35 hours of study and analysis and 40 hours of implementation and experiments. The allocation includes documenting results rather than leaving the report to uncounted time.

Divide each of the first three 10-hour theory blocks into 2-hour sessions: reading a section, deriving equations by hand, checking units and preparing an experiment. Schedule the 10-hour practical blocks on Sundays, allowing breaks between stages. Split the final 5 hours of theory as 2+2+1. The module dates establish its place in the overall calendar; retain the boundary dates, rest days, holidays and separate integration checks. Hour numbers express the workload: do not count the same time twice when placing sessions in the calendar.

The dynamics-lab project structure

parameters/ contains masses, lengths, centres of mass, inertias and their provenance. models/ contains the single-link, 2R-mechanism and contact models. experiments/ stores inputs, initial states and integration settings. data/measured/ preserves the M7 data unchanged; data/simulated/ stores computational results. validation/ contains comparison results and runs for independent validation; reports/ contains energy balances and model limits. Each simulation records the equation version, solver step size or tolerances, duration, input signals and termination reason.

When to increase complexity

First, check the energy of one link without friction or external input. Second, add a known torque and compare its work with the energy change. Third, introduce viscous friction and check that energy decreases. Fourth, repeat these checks for two links. Fifth, add contact and account for energy losses. Sixth, use the real actuator model from M7 and an independent measurement recording. If a basic check fails, find the cause first; do not try to remove the discrepancy by increasing damping or fitting a physical parameter.

Separate measurement uncertainty from numerical error. Changing mass explores sensitivity to a physical parameter; reducing the integrator step explores numerical error. Include both experiments in the report; one cannot replace the other. At the outset, choose several simple motions that you can explain without a computer, and revisit them after adding each new equation term.

Newton and Euler: forces, torques and inertia

How to draw a force and torque diagram

For translation of the centre of mass, the sum of external forces equals $m \cdot a_{\text{COM}}$. For planar rotation of a rigid link about a fixed axis, sum torques about that axis and use the moment of inertia about the same axis. In general spatial motion, the rotational equation includes $I \cdot \dot{\omega}$ and the gyroscopic term $\omega \times (I\omega)$, expressed in a consistent basis. Do not automatically apply the planar relation $\tau = I\alpha$ to an arbitrary rotating system. The centre of mass is the point whose motion describes the translational part of the body's motion; the mass distribution relative to the axes determines its inertia.

Task 1. Physical parameters of a link

The inputs are the M5 CAD link and available mass measurements. 1. Separate geometry from material and list the contributions of fasteners and the actuator. 2. Determine mass, COM position in the local coordinate frame and inertia about the COM. 3. Transfer the inertia to the joint axis using the parallel-axis theorem; in the planar case, $I_{\text{joint}} = I_{\text{COM}} + m \cdot d^2$. 4. Compare with a rod or point-mass approximation. Deliverable: a parameter table with units and sources. Check: shifting the axis away from the COM increases inertia by a non-negative amount. Edge case: an incorrect CAD scale changes inertia more strongly than it changes a linear dimension.

Task 2. Forces and torques on a leg or finger link

Draw one link showing its weight, joint reaction force, actuator torque and an external force at its endpoint. 1. Choose the point about which you calculate torques. 2. Write down lever arms and signs. 3. Calculate the holding torque in several known configurations. 4. Check the external-force torque using $\tau = J^T F$ from M8; calculate the gravitational contribution separately. Deliverable: three labelled drawings and a torque table. Check: if a force's line of action passes through the axis, its torque about that axis is zero. Error to avoid: a link's mass cannot be added as a force without multiplying by g .

Energy and units

A single link's kinetic energy is the sum of $0.5m \cdot v_{\text{COM}}^2$ and $0.5\omega^T I_{\text{COM}} \omega$. Potential energy in a uniform gravitational field depends on COM height. For two links, add their energies; obtain COM velocities from the geometry and Jacobians in M8. Prepare a table: length m , mass kg , time s , force N , torque $N \cdot m$, energy J , inertia $kg \cdot m^2$. Check the dimensions of every term before running a numerical calculation. Using I for both inertia and electric current in different files is acceptable only when the context is clear; separate names such as inertia and current are preferable.

One link: its equation and an energy test

Task 3. A single-joint pendulum

For clarity, measure θ from the downward vertical. With moment of inertia J about the axis, mass m and distance c to the COM, a simple model is $J\ddot{\theta} + b\dot{\theta} + m\cdot g\cdot c\cdot \sin(\theta) = \tau$. A convenient zero for potential energy is the lowest point: $V = m\cdot g\cdot c\cdot (1 - \cos\theta)$. If you use an angle q measured from the horizontal axis, the gravitational term changes; do not switch conventions without transforming the coordinates. 1. Derive the gravitational torque from a drawing. 2. Write the state $[\theta, \omega]$ and its derivative $[\omega, \alpha]$. 3. Solve the equation for α , then integrate. 4. Compare the initial acceleration direction with a hand calculation.

Deliverable: angle, velocity and energy plots, plus a configuration that reproduces the calculation. Check without friction or torque: total energy should be conserved within numerical error, which you investigate by reducing the step size or tolerances. For a small angle, compare the period with $2\pi\sqrt{J/(m\cdot g\cdot c)}$ for positive parameters. Edge cases: the lowest point with zero initial velocity remains an equilibrium; reject negative mass or zero inertia as an invalid model.

Task 4. Work and dissipation

First disable gravity and apply a constant torque to a link with known inertia J : acceleration should be τ/J . Then restore gravity and apply a bounded torque pulse. Numerically integrate power $P = \tau\cdot\omega$ and compare work with the energy change. Add viscous friction $b\cdot\omega$ with $b \geq 0$: the dissipated power $b\cdot\omega^2$ is non-negative. Save the balance $E(t) - E(0) - \text{actuator_work} + \text{losses}$, as well as the q plot. Check: with zero actuator torque and positive damping, mechanical energy must not systematically increase. Negative b is a separate artificial test that the validator must reject in a passive model.

The numerical method is part of the experiment

Compare fixed-step integration with the adaptive `solve_ivp` solver. Record duration, step size, relative and absolute tolerances, and termination status. Run an experiment with successive step-size reductions while keeping physical parameters unchanged. Assess both the trajectory and the energy-balance residual. Similar position plots do not yet establish convergence of force or acceleration. High velocities and stiff contact will change the integrator requirements; preserve this basic model without contact for subsequent comparisons.

Lagrange's equations and the structure of two-link dynamics

What to derive

The generalised coordinates $q=[q_1, q_2]$ uniquely specify the 2R configuration. Measure q_1 from the horizontal x-axis and q_2 relative to the first link; use the x-y plane with y upward. Construct the centre-of-mass positions, differentiate them with respect to time, and calculate kinetic energy K and potential energy V . The Lagrangian is $L=K-V$. For each q_i , write $d/dt(\partial L/\partial \dot{q}_i) - \partial L/\partial q_i = \tau_i$, adding external forces and losses separately. A symbolic algebra system can help with derivatives, but you must be able to explain every energy expression and the choice of signs.

Task 5. Mass matrix and gravity

Use l_1, l_2 for lengths, c_1, c_2 for distances from the corresponding joints to the COMs, m_1, m_2 for masses, and I_1, I_2 for planar inertias about the COMs. Extract the structure $M(q) \cdot \ddot{q} + h(q, \dot{q}) + g(q) = \tau$ from the equations. For checking: $M_{22} = I_2 + m_2 \cdot c_2^2$, $M_{12} = M_{21} = I_2 + m_2 \cdot (c_2^2 + l_1 \cdot c_2 \cdot \cos q_2)$, $M_{11} = I_1 + I_2 + m_1 \cdot c_1^2 + m_2 \cdot (l_1^2 + c_2^2 + 2l_1 \cdot c_2 \cdot \cos q_2)$. Here h combines the velocity-dependent terms; an individual matrix C is not unique, so check the vector h first.

1. Obtain M and g from your own derivation. 2. Compare M against an independent implementation or the structure given above. 3. Check symmetry and positive definiteness in admissible configurations. 4. Compare gravitational torque with the numerical derivative of V with respect to q . Deliverable: a computational notebook with the derivation, a numerical function and tests. For the chosen coordinates, $g_2 = m_2 \cdot g \cdot c_2 \cdot \cos(q_1 + q_2)$, while g_1 additionally contains $g \cdot (m_1 \cdot c_1 + m_2 \cdot l_1) \cdot \cos q_1$. Edge case: all h terms must vanish when $\dot{q} = 0$.

Task 6. Forward and inverse dynamics

The inverse problem takes $q, \dot{q}, \ddot{q}$ and returns the required τ ; the forward problem takes $q, \dot{q}, \tau$ and solves a linear system for $\ddot{q}$. 1. Choose several states. 2. Calculate τ using inverse dynamics. 3. Feed it into forward dynamics and compare the reconstructed acceleration. 4. Repeat with endpoint forces added through $J^T F$ in a consistent basis. Save the acceleration reconstruction errors after the inverse-then-forward calculation. Use $\text{solve}(M, \text{rhs})$, not an explicit matrix inverse. This test does not replace an independent physical check: both functions may contain the same model error.

2R simulation and checks of physical behaviour

Task 7. Two links without contact

Inputs: validated M, h, g , the M8 geometry, and physically admissible masses and inertias. 1. Run free motion from a non-zero initial configuration without friction. 2. Plot angles, velocities, the endpoint path and total energy. 3. Repeat with half the step size or tighter tolerances. 4. Add damping and bounded torque, checking the work balance. Deliverable: four comparable runs and a numerical-convergence report. Edge case: with horizontal links and no holding torque, acceleration must not be zero merely because the initial velocities are zero.

Checking velocity-dependent terms

For the 2R model described above, define $a = m_2 \cdot l_1 \cdot c_2 \cdot \sin(q_2)$. Then $h_1 = -a \cdot (2\dot{q}_1\dot{q}_2 + \dot{q}_2^2)$, $h_2 = a \cdot \dot{q}_1^2$. Compare this with your own derivation and check reversal of velocity direction. Do not remove h simply because calculations agree with an experiment at low velocity: the error will change as velocities change. However, a slow-motion model may deliberately neglect some dynamics if this is an explicit approximation and its error has been checked. Record the velocity range in which the approximation remains useful.

Application to a leg and hand

Use the 2R model as a planar section of a leg or as two phalanges. For the leg, add the masses of the relevant parts and actuators mounted on the links; for the finger, specify a different length scale and fingertip load. Compare gravitational torque and inertial effects for the same normalised motion. Do not transfer planar-model equations to a complete spatial leg without a new derivation: the first axis, COM offsets and inertia tensors change the equations. It is sufficient to specify the interface through which a multibody library will later connect.

An independent numerical reference

Build the same simple mechanism in one tool, such as Drake, MuJoCo or Featherstone's reference library. First disable contact and default actuators, and align gravity, masses, COMs, inertias and initial coordinates. Compare acceleration in one pose before comparing a long trajectory; then compare energies and a short motion. Disagreement may result from built-in damping, limit stops or a different coordinate frame. In the comparison report, list the effects that are enabled and disabled. You need not install several heavyweight simulators for one plot: one independent reference is enough.

Contact, friction and body support

Task 8. Unilateral normal contact

A horizontal surface can exert an upward force on a foot, but cannot attract it towards the floor. Begin with a vertical mass and a spring-damper model of penetration $\delta \geq 0$. An illustrative model force $N = \max(0, k \cdot \delta + c \cdot \dot{\delta})$ acts only during contact; δ is positive when penetration increases. 1. Check free fall before touchdown. 2. Add contact and measure maximum penetration. 3. Check the force sign and absence of tension during lift-off. 4. Compare several k, c values and integrator step sizes. Deliverable: height, force, energy and the region of numerical stability. These parameters do not automatically represent the properties of a real foot.

Task 9. Sticking and sliding

In a simple Coulomb description, static tangential force is bounded by $|F_t| \leq \mu_s N$; during sliding it opposes velocity, with a magnitude determined by $\mu_k N$. 1. Place a body on a horizontal plane. 2. Gradually increase the prescribed horizontal force. 3. Show the transition from sticking to sliding. 4. Repeat with a different normal load. Deliverable: plots of tangential force, velocity and contact mode. Check: tangential friction must not systematically add mechanical energy. Edge case $N=0$: the contact friction force also vanishes. Smoothing near zero velocity changes the model and must be explicitly identified.

Impact and discrete time

Compare elastic and inelastic impact with a continuous compliant model. The coefficient of restitution describes the ratio of normal velocities after and before impact under specified conditions; it is not a universal material coefficient for every velocity. Do not combine a rigid impulsive impact with the same energy loss in a damper without checking for double counting. For the first implementation, explain the two approaches and implement one, then investigate sensitivity. Force, penetration and energy matter in contact; smooth animation can hide non-physical high-frequency oscillations.

Task 10. Support polygon

Take known contact points on a horizontal plane and the projection of the body COM. Construct the convex hull of active contacts and the signed distance from the projection to its boundary: positive inside and negative outside. Repeat after removing one support and shifting the COM. Deliverable: support diagrams and the static margin in metres. Check: a point on an edge has zero margin; two contacts do not form a polygon with non-zero area. This is a quasi-static geometric check under stated force and friction assumptions, not proof of the dynamic stability of a walking robot.

Validating the M7 model and assessing its accuracy

Task 11. A joint with real actuator parameters

First establish what M7 identified: mechanics under a known torque, or the response of a servo with an internal controller to a position command. 1. Import parameters with units, signs and operating ranges. 2. Check gearing and inertia: do not count the load twice. 3. Replay the measured input in simulation with the same initial state. 4. Compare $q(t)$ and the available velocities and currents on a common time axis. Deliverable: overlaid plots, a residual and an explanation of the most prominent discrepancy. Do not connect a position-command response model as an ideal torque source: these are different physical interfaces.

If torque is not directly known, validate the available command-to-angle model and state separately which mechanical properties remain unconfirmed. Current telemetry can support an explanation of saturation, but does not become an accurate torque measurement without a coefficient and a definition of the current type. Add heating, friction, backlash and delay from M7 only where their meaning is consistent with the new equation. For each effect, first show a comparison against the model without it; otherwise you cannot determine what improved the result.

Task 12. An independent recording and sensitivity

Before fitting parameters, set aside recordings of separate runs for independent validation. 1. Fit the model on the first group. 2. Freeze the parameters. 3. Predict a new run with a different admissible command waveform or load. 4. Calculate angle RMSE, maximum residual and characteristic-time error. Then vary one parameter at a time within its justified uncertainty range and compare the effects. Deliverable: a sensitivity table and independent-validation plots. If no new real data are available, perform a software test, but leave physical validation open.

Assessing model errors

Prepare a table of error source, observed effect, required accuracy and refinement method. For example: COM affects gravitational torque; J affects acceleration; delay shifts the response; friction affects low velocities; contact k affects penetration and peak force. Allocate time to the parameters that most strongly affect the result of interest. State numerical integration error and measurement noise separately. The aim is the smallest model adequate for the chosen experiment, rather than the largest number of parameters. The report must identify admissible operating conditions and specific examples where the model ceases to be reliable.

Step by step: hours 1-40

Hours 1-10: physical terminology and diagrams

1-2: check units, gravity and the coordinate frame, and assemble the parameter table from M5-M8. 3-4: study mass, COM and inertia, and transfer inertia to a joint axis for a simple body. 5-6: draw force and torque diagrams for a link and leg, and calculate static torques. 7-8: work through kinetic and potential energy and the work done by torque. 9-10: derive the pendulum equation for the chosen angle and define states and initial conditions. These ten hours do not require a complex simulation: their outputs are checkable equations and physically expected relationships.

Hours 11-20: one link as a test system

11-12: finish the parameter table for task 1 and check the provenance of CAD mass and inertia. 13-14: complete the drawings and calculations for task 2 and compare against JTF. 15-16: implement the pendulum in task 3 and plot q, ω, E . 17-18: complete the energy balance in task 4 with torque and friction. 19-20: investigate the integrator step size and compare small oscillations against the analytical period. Freeze baseline-one-link with its inputs, method and achieved accuracy. If energy grows without a source, fix the model before moving to the second link.

Hours 21-30: the two-link system

21-22: write the 2R centre-of-mass coordinates and their velocities using the M8 geometry. 23-24: construct K, V, L and derive Lagrange's equations. 25-26: extract the mass matrix, velocity-dependent terms and gravity. 27-28: study the meaning of forward and inverse dynamics and work through adding an external force. 29-30: prepare checks of M symmetry, positive energy, the derivative of V and limiting cases. If using a symbolic tool, preserve the original energy expressions and the meanings of variables, as well as the final long formula.

Hours 31-40: validated 2R dynamics

31-32: implement M, h, g for task 5 and compare with your own derivation. 33-34: check the inverse-then-forward dynamics calculation in task 6 using fixed and random admissible states. 35-36: run the contact-free simulation in task 7 and its energy checks. 37-38: add bounded torque and friction, and repeat the convergence check. 39-40: compare instantaneous acceleration and a short trajectory with one independent library. If installation is taking too long, use an available reference implementation; do not reduce unit and energy checks to accommodate a simulator interface.

Step by step: hours 41-75

Hours 41-50: contact physics

41-42: study unilateral normal force, penetration, and the difference between rigid and compliant contact. 43-44: work through the friction cone and the transition from sticking to sliding, and draw admissible forces. 45-46: study impact, the coefficient of restitution and energy losses. 47-48: assess the effects of contact stiffness, damping and integration step separately; prepare a parameter grid for comparison. 49-50: study the support polygon and static margin, and record their conditions of applicability. Relate every concept to an observable force, velocity or energy.

Hours 51-60: contact experiments

51-52: implement the vertical contact in task 8; check the absence of force before touchdown and at lift-off. 53-54: vary k, c and the numerical step, recording penetration and energy. 55-56: complete the sticking-to-sliding transition in task 9 and check the sign of frictional power. 57-58: construct several support configurations for task 10, remove a contact and move the COM. 59-60: add simple endpoint contact to the 2R mechanism or leg in the chosen simulator and explain how it differs from a geometric check. If time is limited, retain point contact, while preserving the unilaterality and sensitivity checks.

Hours 61-65: model validation plan

61-62: revisit M7 identification and align the input type, delay, gearing and load inertia with the M9 model. 63-64: select separate measured runs and metrics, estimate errors and list parameters for sensitivity analysis. 65: finalise the table of validity regions and the final report format. These 5 hours are split into 2+2+1 without adding time to the original programme. Do not revise the quality criterion after seeing validation results in a way that makes every outcome a success.

Hours 66-75: compare calculation with experiment

66-67: complete the transfer in task 11 and reproduce the measured input and initial conditions. 68-69: compare trajectories, fix unit, timing and interface errors, then freeze the parameters. 70-71: perform the independent validation in task 12 and calculate errors using the predefined metrics. 72-73: conduct sensitivity analysis and separate numerical problems from physical disagreement. 74-75: assemble supporting results, limitations, a reproducible run and a brief analysis of one failed model. If time is short, reduce contact-geometry complexity and additional tools; the identified joint, 2R or leg dynamics, energy checks and independent model validation remain mandatory.

Acceptance checks: detecting physical errors

Test scenarios

F1: no gravity, friction or external torque. Rest is maintained; with an initial velocity, the corresponding energy balance holds. F2: one link under constant torque without gravity. Acceleration agrees with τ/J . F3: positive viscous friction without actuation. Energy decreases within integration error. F4: geometry is scaled incorrectly. The unit table and comparison against the analytical period reveal the error; visual similarity of the trajectory alone is insufficient to accept the simulation. F5: the mass matrix is asymmetric or not positive definite for a physical configuration. The test rejects the model before integration.

F6: contact without penetration and during lift-off. No attractive normal force occurs. F7: tangential sliding. Frictional work does not create energy; contact force vanishes when $N=0$. F8: integration step size is reduced without changing physical parameters. The result must show convergence of the metrics of interest; otherwise conclusions about mechanics are premature. F9: the COM moves outside the horizontal static support polygon. The margin changes sign; the test does not automatically conclude that dynamic falling occurs. F10: parameters are fitted to one run, but do not reproduce a new one. Narrow the validity region or improve the model and repeat independent validation.

Evidence that the module is complete

Provide dimensional and energy tests; checks of M, h, g ; input parameters with provenance; a two-link or leg simulation; contact sensitivity analysis; transfer of the M7 identification results; and comparison of a measured trend with calculation. The measured trend may be delay, characteristic time, load dependence or decay; choose the metric in advance. The report must explicitly identify conditions under which the model fails. Without a physical recording, the corresponding item remains open even if every mathematical test passes.

Questions and handover to the next module

Explain why moment of inertia depends on the axis; where velocity-dependent terms come from; how to check the gravity sign; why to solve a system with M rather than calculate its inverse; why excessively stiff contact complicates integration; how the friction cone differs from the static support-polygon rule; why a servo model with an internal controller cannot be treated as an ideal torque actuator; and how to distinguish noise, numerical error and incorrect physics. Pass on parameters, original measurements, dynamics functions and accuracy limitations. When designing the controller next month, account for model validity limits and actuator saturation; calling a simulation function alone is insufficient.

Assigned reading: a focused reading route

Books and lecture notes may be in English; keep the plan, definitions and your own explanations in English. Read only the specified sections within the existing 75 hours.

Core reading and one reference route

1. Core: K. Lynch, F. Park, Modern Robotics, Chapter 8. §8.1.3 - the mass matrix; §8.3 - Newton-Euler; §8.5 - forward dynamics. Tasks 1-7 require force diagrams, the meaning of $M(q)$ and an independent check of the forward and inverse problems.

2. Core: R. Tedrake, Underactuated Robotics. The Simple Pendulum: Equations of Motion; Multi-Body Dynamics: Lagrangian Dynamics and Manipulator Equations; Planning and Control through Contact: physical models of contact and friction. Read for tasks 3-10, without trajectory optimisation.

3. Reference: R. Featherstone, Rigid Body Dynamics Algorithms. Chapters Modelling Rigid Body Systems and Inverse Dynamics: establish the definitions of inertia and external force used, and the directions of recursive calculations. The relevant example is sufficient for checking task 6; the complete set of algorithms is not yet required.

Documentation for specific practical work

SciPy solve_ivp - method, rtol and atol parameters, the events mechanism and the success flag for tasks 3-7. Featherstone spatial_v2 - mcl, HandC, ID and FDab as reference implementations of dynamics.

MuJoCo Computation - Contact and Numerical Integration if comparing simulators; installing MuJoCo is not required for your own numerical experiment.

After reading

After the derivation, check units, the gravity sign and the energy balance. Compare two integration step sizes, then change one physical parameter. For tasks 11-12, return to the actuator model and validation data from M7. The presence of contact alone does not imply the absence of sliding.

Key terms and definitions

Rigid body. An idealisation in which the distances between any two points of a body remain constant during motion and loading.

Centre of mass. The point whose position is the average position of all body elements, weighted by their masses.

Inertia tensor. A matrix describing mass distribution about a selected point; it determines the rotational contribution to kinetic energy and has units $\text{kg}\cdot\text{m}^2$.

Free-body diagram. A diagram of an isolated body showing all external forces and torques, including reactions from removed constraints.

Generalised coordinates. Independent configuration parameters sufficient to describe the system's position under its geometric constraints.

Lagrangian. The difference between kinetic and potential energy, used in Lagrange's equations to obtain system dynamics.

Mass matrix. The configuration-dependent matrix $M(q)$, which determines kinetic energy and relates generalised accelerations to forces.

Velocity-dependent terms. The part of the dynamics arising from coordinate motion, including Coriolis and centrifugal effects; the representation of matrix C is not unique.

Gravity vector. Generalised torques or forces corresponding to the potential-energy gradient under the chosen equation-of-motion convention.

Forward dynamics. Calculation of accelerations from current coordinates, velocities, applied forces and the chosen constraint model.

Inverse dynamics. Calculation of required generalised forces from specified coordinates, velocities and accelerations with known external inputs.

Dissipation. Irreversible conversion of mechanical energy into other forms, such as heat, through friction or damping.

Unilateral contact. A constraint that permits bodies to separate and contact to be compressed, but does not permit an attractive normal force between surfaces.

Normal force. The component of contact force along the surface normal; in a model without adhesion it points away from the surface.

Friction cone. The set of admissible contact forces in the Coulomb model: tangential force magnitude is bounded by μ times the normal force.

Sticking. A contact mode with zero relative tangential velocity, possible while the required force remains within the friction limit.

Impulse. The time integral of force, which changes momentum; it approximates a short impact without detailed calculation of peak force.

Support polygon. The convex hull of support points on a plane; the centre-of-mass projection provides only a quasi-static criterion.

Check the relationships: $M(q)\ddot{q} + c(q, \dot{q}) + g(q) = \tau + J^T f$. All forces must have consistent signs, bases and points of application.

Preparation for dynamics experiments

NEW PURCHASES: NONE REQUIRED

Use the existing MacBook Pro M4, CAD link models from M5, IMU and controller from M6, and the secured actuator and current and temperature measurement equipment from M7. Investigate 2R dynamics and contact numerically. The 3D printer is needed only if the fixtures change; choose a printing profile for the exact printer model.

PREPARE BEFORE HOURS 1-20

Create a parameter table for each link: mass in kg, centre-of-mass position in m, inertia in $\text{kg}\cdot\text{m}^2$, coordinate frame and value source. Distinguish inertia about the centre of mass from inertia about the joint axis. Mark estimated parameters and their uncertainty ranges.

Before repeating a physical experiment, check mounting, stopping, the M7 limits, angle direction and consistency of measurement timestamps. Use loads and levers from the existing fixtures; increase the load only within the previously justified operating range.

INSTALL NOW

No new packages are mandatory. On macOS ARM, use the Python, NumPy, SciPy, Matplotlib and pytest installations from M8; use SymPy if you already selected it for your own symbolic derivation. [SciPy solve_ivp](#) integrates your own equation system on the CPU.

OPTIONAL: INDEPENDENT COMPARISON

An external engine is needed only for the chosen independent comparison. This month, you can compare equations and values with [the author's spatial_v2 library](#) without installing MATLAB/Simulink. MuJoCo and other engines are not new mandatory purchases or prerequisites for completing M9.

CHECK THE ENVIRONMENT AND PHYSICAL MODEL

Reproduce pendulum motion: without friction, check energy drift as the step size decreases; with friction, check that energy decreases. For 2R, check symmetry and positive definiteness of M , and consistency between forward and inverse dynamics. In the contact model, check the absence of attraction to the surface and the distinction between sticking and sliding.

Save versions, integrator configuration, initial conditions, an energy plot and an independent validation recording. A successful simulation confirms only the model properties tested; physical robot readiness requires its own experiments. Preparation is included in the existing 75 hours.

Motion fundamentals: why torque is needed

Required: derive the equation of motion in two ways

The kinematics in M8 relates angles to endpoint position. Dynamics determines which torque produces a chosen acceleration and what happens with a limited actuator. Before deriving the equations, clarify the distinction between mass m in kg, moment of inertia I in $\text{kg}\cdot\text{m}^2$ and torque τ in $\text{N}\cdot\text{m}$. A geometrically admissible leg or finger trajectory may demand unavailable torque.

One joint: geometry, energy and equation

Example pendulum: a point mass $m=0.5$ kg at the end of a massless rod $l=0.2$ m; the pivot is fixed. In the x-y plane, y points upward and q is measured from the downward vertical towards +x. Then $p=(l \sin q; -l \cos q)$, $v=\dot{q}$ in rad/s, $I=ml^2=0.02$ $\text{kg}\cdot\text{m}^2$. There is no friction or contact; positive τ_{act} increases q .

Newton and Euler: the torque of weight about the pivot is $-mgl \sin q$, so $I\ddot{q}=\tau_{\text{act}}-mgl \sin q$.

Lagrange: $K=lv^2/2$, $U=mgl(1-\cos q)$, $L=K-U$. Substitution into $d/dt(\partial L/\partial v)-\partial L/\partial q=\tau_{\text{act}}$ gives the same equation, $I\ddot{q}+mgl \sin q=\tau_{\text{act}}$. These are two descriptions of the same mechanics; their agreement helps check the signs.

At $q=\pi/6$, an acceleration $\ddot{q}=1$ rad/s^2 requires, by inverse dynamics, $\tau_{\text{act}}=0.02\cdot 1+0.5\cdot 9.81\cdot 0.2\cdot 0.5=0.5105$ $\text{N}\cdot\text{m}$. Of this, 0.4905 $\text{N}\cdot\text{m}$ compensates for weight and 0.0200 $\text{N}\cdot\text{m}$ accelerates the mass. Forward dynamics calculates $\ddot{q}=(\tau_{\text{act}}-mgl \sin q)/I$. With $\tau_{\text{act}}=0$ at this position, $\ddot{q}=-24.525$ rad/s^2 : the link accelerates towards the downward vertical, even when starting from rest.

From a pendulum to two links

For a fixed-base chain of revolute joints: $M(q)\dot{v}+h(q,v)+g(q)=\tau_{\text{act}}+J^T f_{\text{ext}}$. Here $v=\dot{q}$; M relates accelerations to torques, h contains velocity effects, and g contains gravitational torques; f_{ext} is the force exerted by the environment on the endpoint, expressed in the same basis as the positional Jacobian J . Geometry determines centre-of-mass velocities, from which energy and M are calculated. At rest, $\tau_{\text{act}}=g-J^T f_{\text{ext}}$; motion adds inertial and velocity-dependent terms.

An actuator with a position interface does not become a source of known torque: use the validated M7 model. A moving body, distributed contacts and spatial tensors require a more detailed model; this module requires a clear pendulum calculation and the specified 2R dynamics.

Check torque, acceleration and energy calculations

In tasks 3-6, save both derivations and the numerical example. Independently estimate dU/dq using a central difference: at $q=\pi/6$, expect 0.4905 $\text{N}\cdot\text{m}$. Apply 0.5105 $\text{N}\cdot\text{m}$ to forward dynamics and recover 1 rad/s^2 . Then, without actuation or friction, compare energy $E=K+U$ at two integrator step sizes; reducing the step should reduce numerical drift. Agreement between inverse and forward dynamics does not rule out an error shared by both functions.

Included in the 75 hours: 1 hour from 9-10 for the two derivations; 1 hour from 15-16 for the pendulum; 1 hour from 33-34 for the forward and inverse dynamics check. This is a required reference example within the existing tasks. Calculations run on the CPU; no additional equipment, software or hours are required.

References: Tedrake, [The Simple Pendulum: equation of motion and the Undamped Pendulum section](#); Lynch/Park, [Modern Robotics, §8.1.1 and §8.1.4: Lagrange, Newton and Euler methods](#).

Feedback: achieving stable motion

M10 • 07 Jul 2027 - 08 Aug 2027 • 75 hours

What you will learn this month

By the end of the month, you will be able to explain why a joint reaches its target angle, why increasing gain produces oscillations, and how IMU delay affects platform stability. The deliverable is a benchmark for comparing joint and body-tilt control methods. Prepare a model, a digital controller, measured transient and frequency responses, a description of limitations and fault-test results. For a future walking robot, this provides the basis for maintaining body attitude; for a hand, it supports precise finger motion without accumulating error when pressing against an object.

Inputs and scope

Use the geometry, actuator, limits and measurements from M5-M9. Encoder and IMU reading, timestamping, and command and state logging must work before you start. Review torque, angular velocity, inertia, derivatives and integrals. Be able to read a step-response plot and distinguish measurement time from delivery time. If any of these elements is not ready, use the first two hours to diagnose it and reduce optional plots.

Begin with one axis. Full balancing of a multi-legged robot, gait planning and optimal force distribution are not required here. For tilt, use a platform model with limited rotation or a secured test rig from earlier stages. An IMU alone does not provide a plant to control: explicitly identify which actuator action changes roll or pitch. If this mechanism is unavailable, test tilt control in a model, physically measure only the joint and IMU, and leave physical tilt validation open.

What counts as a result

Show the continuous model, its discrete implementation and real data separately in the report. Give units for every coefficient and a source for every limit. The minimum final demonstration includes a joint-reference step, rejection of a repeatable disturbance, operation near saturation, tilt holding, artificial delay and loss of fresh data. Fix the criteria before the experiment, then report actual values without adjusting the acceptance limits.

The purpose of the month is to make control choices, beyond learning a list of controllers. Choose the control bandwidth, justify the loop frequency and show the conditions under which the solution stops working. A good result includes one clearly explained failure mode and the reason for excluding it from the operating region. This information will support the transition to state-space methods and estimation in M11.

Organising the 75 hours

Four work blocks

Block A, hours 1-20: feedback and joint PID. Block B, 21-40: frequency responses and noise. Block C, 41-60: digital implementation and loop timing. Block D, 61-75: body tilt and the final comparison experiment. Each of the first three blocks contains 10 hours of theory and analysis and 10 hours of practice; the fourth contains 5 hours of analysis and 10 hours of practice. Total: $35 + 40 = 75$ hours. The work steps on pages 9-10 expand these same hours and require no additional time allocation.

Divide each ten-hour theory block into five two-hour sessions: 20 minutes reviewing the previous model, 45 minutes reading the relevant section, 40 minutes deriving a small example or examining a log, and 15 minutes recording the conclusion. Also divide the ten-hour Sunday practical session into five two-hour periods with breaks between them. Breaks are excluded from net study hours. Allocate the final five hours of analysis as $2 + 2 + 1$.

Start with the question, then choose the tool

The main computational environment is Python, NumPy, SciPy and python-control; implement the control loop in C++ in the existing rig firmware. Begin each session with a testable question: how does the input affect the output, which state cannot be measured, and which data reveal its changes? Before running, write down the expected sign, scale or sequence of events. Compare the prediction with the result and examine at least one discrepancy. A plot supports a conclusion only when accompanied by an explanation of the observed relationship.

Maintain one repository with model, config, experiments, tests and reports directories. Every experiment needs a unique identifier, code version, configuration, parameter source, initial state, units and raw data. The analysis script reads a saved experiment and builds the report without manually correcting measurements. The final joint-and-attitude-control benchmark project combines comparisons of joint and body controllers; the individual exercises gradually form its test suite.

Time accounting and reducing scope

Calendar boundaries may coincide with an adjacent module. Keep rest days, holiday intervals and integration checks as specified in the overall calendar; place numbered steps in available time without counting hours twice. Do not add hours to this module. Assess equipment and library readiness at the outset rather than spending the practical session on an unplanned platform change.

If you fall behind, reduce the number of sinusoidal test frequencies and gain-scheduling variants; thoroughly check one body axis. Retain the joint model, a saturation experiment with anti-windup, frequency-response measurement, firmware timing assessment and the fault test for stale IMU data. You may investigate the second axis in a model if you state this in the report. Preserve the comparison of joint and tilt control; physical validation remains unfinished until there are experimental results from the rig.

Feedback, modelling and PID tuning

Study the relationship between torque and position

Examine the forward path command $\rightarrow$ actuator $\rightarrow$ mechanics $\rightarrow$ angle and the measurement feedback path. For the example joint, use $J\ddot{q} + b\dot{q} = \tau$: J in $\text{kg}\cdot\text{m}^2$, b in $\text{N}\cdot\text{m}\cdot\text{s}/\text{rad}$, τ in $\text{N}\cdot\text{m}$ and q in rad . Under torque control, the transfer function from τ to q is $1/(J\cdot s^2 + b\cdot s)$. If the actuator accepts only position, identify the actuator together with its internal closed loop as a single plant; a position command cannot replace torque in the model. Explain where the sensor, delay, saturation and disturbance appear in the diagram.

Poles determine natural modes; zeros change response shape. Stable poles of a continuous linear model have negative real parts; an integrator at zero needs separate reasoning, and simulation does not replace analysis. Study negative feedback $T = PC/(1 + PC)$, error $e = r - y$ and PID: proportional action responds to current error, integral action to accumulated error, and derivative action to rate of change. In a practical implementation, take the derivative of the measurement and filter it to reduce command kick when the reference changes.

Task 1. A joint model with three proportional-control responses

Inputs: estimated J and b or an explicitly labelled illustrative parameter set, a rest position and a bounded reference step. 1) Build the continuous model. 2) Choose three P values producing slow, acceptable and oscillatory responses. 3) Record angle, error and torque on the same time grid. 4) Explain changes in poles and curve shape. Deliverable: a computational notebook and a table of rise, peak and settling times. Check: the first motion has the correct sign relative to the command; doubling inertia must change the response. Error to investigate: reversed feedback sign, tested first in the model only.

Task 2. Integral action near the actuator limit

Add a constant external disturbance and the command limit from the rig specification. Compare P , PI and PID , then PID with anti-windup: for example, stop integrating when the command is already limited and the error demands more action in the same direction. Record the unsaturated command, limited command and integral state separately. Return the reference from an unreachable region to the admissible region and compare recovery times. Deliverable: a test and three plots. Check: the command never exceeds the declared limit; after returning, the integral must not hold the actuator in saturation beyond the expected recovery transient. Add feedforward as a separate calculation compensating for a known load, retaining feedback for the remaining error.

Frequency domain: bandwidth, margins and measurement

Study the meaning of a Bode plot

For sinusoidal excitation, frequency determines whether the joint can keep up with the command. Study magnitude and phase, logarithmic scales, $20 \cdot \log_{10}$ of the amplitude ratio, and the conversion $\omega = 2\pi f$. Closed-loop bandwidth and open-loop crossover frequency are different quantities. Gain and phase margins are calculated from the loop transfer $L = PC$ under the chosen negative-feedback convention. A closed-loop response plot alone cannot establish a stability margin without knowing the loop structure.

Interpret $S = 1/(1 + L)$ and $T = L/(1 + L)$ as descriptions of disturbance rejection and signal and noise transmission at the chosen point in the diagram. State where the disturbance enters before comparing curves. Higher gain improves regulation over some low frequencies, but can amplify measurement noise and bring unwanted dynamics closer. A sensor filter changes phase as well as noise; include its delay in the loop analysis.

Task 3. Model margins and frequency response

Take the model from task 1 and the working PID. 1) Construct L and T and calculate margins with margin. 2) Check phase-argument units: phase data supplied to margin are in degrees, whereas `frequency_response` returns radians. 3) Introduce a known delay and rebuild the response. 4) Calculate step responses for the original and degraded cases. Deliverable: one sheet containing the Bode plot, poles and transient response. Check: a conclusion of reduced stability must agree with the time-domain experiment. If there is no crossover, explain the meaning of an infinite or undefined margin without treating it as a guarantee.

Task 4. A sinusoidal experiment on the rig

At small amplitude, apply 5-8 frequencies within the admissible range. At each frequency, wait for the transient to end, record at least several complete periods, and estimate amplitude and phase by fitting a sinusoid. Compare the reference-to-angle frequency response with model T . Deliverable: measured points with a range of variability and a configuration table. Check: repeat at half amplitude; a strong change in the ratio indicates nonlinearity or insufficient signal-to-noise ratio. Stop the experiment at the predefined limits if saturation, noticeable heating or excessive travel occurs. If margins are unconfirmed, describe tolerance to delay and load changes using the experimental series.

Digital control: discretisation and real time

Study system behaviour between samples

A digital controller reads a sensor at intervals T_s , calculates a command and holds it until the next update. Study the zero-order hold, conversion from continuous to discrete systems, and the ZOH and Tustin methods. A discrete linear model requires poles inside the unit circle for stability. Euler's method is useful for understanding, but an excessively large step creates numerical instability itself. Compare `cont2discrete` with a manual update of a simple first-order model.

Describe the loop period, computation time, measurement delay and data age separately. Jitter is variation in interval duration; quantisation is the finite resolution of a measurement or command. "A frequency of 200 Hz" without the distribution of actual intervals does not characterise operation. The Nyquist condition relates sampling frequency to recoverable signal bandwidth, but is not sufficient for choosing a control frequency: phase margins, filters, computation and the actuator need additional headroom, supported by modelling and experiment.

Task 5. One algorithm at several frequencies

Take one controller designed for the continuous model and discretise it for three T_s values selected relative to the desired bandwidth. Use the same physical plant, duration and reference amplitude. Introduce delays of 0, 1 and several samples. Deliverable: a $T_s \times$ delay experiment table, poles, RMSE and maximum command. Check: T_s is stored in the configuration, and integral and derivative actions scale with time. Deliberately retain old coefficients after changing T_s , show the error and add a test that detects it. Do not claim that example values apply to every actuator.

Task 6. Measure the embedded loop

In the existing firmware, timestamp the start of reading, end of calculation and command transmission using a monotonic clock; where possible, check the period with GPIO and a logic analyser. Run a quiet mode, intensive logging and an artificial delay in one handler. Save distributions of computation time, intervals and measurement age: median, p95, p99 and maximum, together with sample counts. Check: logging does not perform a blocking write in the critical section; a stale measurement does not become fresh through late delivery. Deliverable: a timing-budget report. Explicitly state the sensor polling, controller and logging frequencies; mark a repeated previous value rather than counting it as a new measurement.

Roll and pitch: controlling body tilt

Study the two-loop structure

The outer tilt controller receives roll and pitch and generates a bounded reference for the actuator's inner loop. Check one axis first, then the other. The inner loop must track the outer-loop reference within the chosen range; tuning two fast loops independently without analysing their interaction can produce oscillations. Derive the relationship positive command change $\rightarrow$ tilt change from the geometry and verify it with a small test input. For a stationary model, distinguish tilt from linear acceleration, which distorts the accelerometer-based estimate.

Gain scheduling is specified in advance, for example as a function of position or load. It requires checks of boundaries and transitions, rather than arbitrary increases in P for large error. Limit the target angle, reference rate of change, inner-loop command and admissible IMU data age separately. Recovery after a fault must be an explicit state; the return of packets need not automatically resume motion.

Task 7. Holding a tilt angle

Initial conditions: a model or secured platform with a known axis, an IMU and a working inner controller. 1) Check zero and axis directions. 2) Command a small roll with zero pitch. 3) Repeat for pitch. 4) Introduce a repeatable disturbance pulse. 5) Compare controller-off and controller-on results under the same input. Deliverable: curves for both axes and commands. Check: calculate RMSE over the same window after the disturbance, peak error as the maximum absolute error, and settling time from entry into and continued residence within a predefined band. For a reference near zero, specify an absolute band in radians rather than a percentage of zero.

Task 8. Surface tilt, limits and recovery

Save two base orientations and two admissible loads; run every combination with fixed gains, then with two gain sets selected by operating point. Introduce a transition between gain sets and an IMU dropout. Deliverable: a table of stable and unconfirmed operating conditions, detection time and state sequence. Check: output continuity when gains change and correct handling of the integral; during a dropout, new motion commands stop within the time allocated for software fault detection. The physical stopping method depends on the mechanics: zero torque does not always hold a platform. Use the predefined behaviour of the secured rig and describe separately what a standing robot would require.

For the hand, repeat only the reasoning: object contact obstructs the commanded finger motion, the internal actuator reaches its limit and the integral accumulates error. Describe which implemented limits help and why a position PID alone is insufficient to control contact force.

Specification for the final benchmark

Defining a reproducible experiment

One scenario specifies the plant identifier, model version, initial angle, initial velocity, step or sinusoidal reference, disturbance, limits, T_s , filter and duration. The output log must contain measurement and control times, reference, measured joint angle or body tilt, estimated velocity, error, P, I and D terms, commands before and after limiting, saturation and deadline-miss flags, and state-machine state. Record the command in the units actually used and separate any conversion to torque.

For the joint, compare at least P, PID and PID with anti-windup. For the body, compare disabled stabilisation with the working outer loop, then one variant with increased delay. Every variant receives the same reference sequence and initial state. Save the scenario and configurations before comparison; do not choose different convenient segments for each algorithm. On the real rig, repeat the key scenario at least three times to distinguish a trend from a single successful run.

Calculating and interpreting metrics

$RMSE = \sqrt{\text{mean}(e^2)}$ has the units of the measured angle. Peak error = $\max(\text{abs}(e))$; define percentage overshoot relative to a non-zero step amplitude. Calculate settling time as the time after which the signal remains inside the chosen band until the end of the window; if the window is too short, record “did not settle during the experiment”. The saturation fraction is the number of samples with a limited command divided by the number of commands. For uneven sampling, also calculate the fraction of time, since counting rows can distort the result.

Plots: reference and output; error and settling band; command and limits; data age; model Bode plot and measured points. Separate calibration and validation data: do not assess model suitability solely on the experiment used to fit J , b and delay. Use one validation experiment with a different admissible load or excitation frequency.

Deliverable-package requirements

A launch command from a clean working directory must build the baseline report and run the limit checks. The README explains the plant, wiring and mounting for physical experiments, parameters, limits and stopping sequence. Include a log of failed scenarios and the reasons for excluding them. State the basis for every conclusion: analytical linear model, nonlinear model, rig experiment or a hypothesis awaiting confirmation. This will allow M11 to compare LQR with the real PID using the same criteria.

Tests that reveal controller errors

Required fault scenarios

Scenario F1: reverse the encoder sign in the model. Expected: error grows rapidly and the software monitor detects departure from the permitted region; do not run this experiment physically. Save a short log up to stopping and demonstrate that the event is caused by the feedback sign, rather than random noise. The corrected configuration must pass the same test.

Scenario F2: an unreachable step followed by a return. Expected: the command is limited, the integral state is controlled, and recovery is faster than without anti-windup in the chosen test. Do not claim universal superiority from one curve; compare the specific recovery time and peak after the limit is removed.

Scenario F3: measurement delay and a stale packet. Separate transport delay from the loss of new measurements. Expected: the monitor uses the age of the original measurement, while frequency analysis and the time-domain model show degradation as delay increases. The last late packet must not reset the freshness timer without checking its timestamp.

Scenario F4: encoder noise and coarse quantisation. Expected: their effect on the D term and command is visible; a filter reduces some high-frequency noise, but its phase lag is represented in the model. Compare spectra and command RMS at a stationary reference, keeping this result separate from step-tracking accuracy.

Scenario F5: increased inertia or load. Choose a model change and a separate admissible rig load. Expected: metric changes that can be related to the model; the operating region is limited to measured cases. Do not mark untested masses and configurations as passed in the table.

Scenario F6: occasional long computations. Introduce one computation deadline miss every N cycles and batched logging. Expected: the event is explicitly marked, data losses are counted, and the response to lateness is checked. Compare p99 and the maximum; average duration must not hide an occasional overrun.

Analysing a failed experiment

First reproduce the failure using the saved configuration. Check units, signs and the time scale, then limiters, and only then gains. Change one factor per experiment. If the model is stable but the rig oscillates, check backlash, the actuator's internal controller, the IMU filter, communication delay and frequency mismatches. Every unexplained effect becomes a separate task for additional measurements; reducing P without understanding the cause is a temporary measure.

Hours 1-40: foundations and frequency checks

Hours 1-20: joint and feedback

Theory 1-10: five 2-hour sessions. 1-2: examine the actuator interface and units; 3-4: derive the model and poles; 5-6: study the step response; 7-8: work through PID, feedforward and saturation; 9-10: design the sign and anti-windup checks. In each session, perform a small calculation using numbers from your configuration so that theory immediately reveals missing parameters.

Practice 11-20: 11-12: build the model and log; 13-14: complete task 1; 15-16: add integral and derivative actions and limiting; 17-18: complete task 2; 19-20: transfer one validated operating condition to the secured rig and save a validation recording. This is one 10-hour practical session in five periods; mounting preparation and the stopping check are included.

Before proceeding, check that you have a model with dimensions, three operating responses, a limited command and a return-from-saturation experiment. Be able to explain why an actuator with internal position control cannot be treated as an ideal torque source.

Hours 21-40: frequency and real data

Theory 21-30: 21-22: sinusoid, magnitude and phase; 23-24: Bode plots and crossovers; 25-26: gain and phase margins; 27-28: sensitivity and noise; 29-30: plan a frequency experiment with bounded amplitude. For one frequency, calculate the amplitude ratio and phase shift by hand, then compare with the library.

Practice 31-40: 31-32: complete task 3; 33-34: configure sinusoidal-response logging; 35-36: complete task 4 on the available rig; 37-38: repeat the end frequencies and half amplitude; 39-40: overlay the points and model, and document validity limits. If data are unusable, retain fewer frequencies with good repeated recordings rather than drawing a smooth curve from noise.

Before proceeding, verify that you know exactly what was measured: closed-loop T or loop transfer L . Magnitude and phase must have the correct units, and conclusions about margins must identify their basis.

Hours 41-75: the digital loop and body tilt

Hours 41-60: discretisation and the timing budget

Theory 41-50: 41-42: ZOH and the discrete model; 43-44: poles inside the unit circle; 45-46: T_s and PID coefficients; 47-48: delay, jitter and quantisation; 49-50: separating IMU, controller and logging frequencies. Draw a timeline for one measurement from physical sampling to command output and label every delay.

Practice 51-60: 51-52: complete task 5 for three T_s values; 53-54: add delays and the incorrect-coefficient test; 55-56: add firmware timestamps; 57-58: complete task 6 under load; 59-60: plot the distributions and reproduce a computation deadline miss. Do not spend these hours optimising before measuring the dominant source of delay.

Before proceeding, check that you have a timing distribution, each component's original frequency, a stale-data test and an explanation of differences between the model and embedded implementation.

Hours 61-75: stabilisation and final validation

Analysis 61-65: 61-62: outer- and inner-loop diagram, actuation sign and limits; 63-64: RMSE, peak and settling criteria, and gain scheduling; 65: compile the final experiment table. These are the last 5 hours of theory and analysis, split as 2 + 2 + 1; do not add new chapters on multi-legged dynamics.

Practice 66-75: 66-67: complete task 7 for one axis; 68-69: repeat for the second and introduce a disturbance; 70-71: complete task 8 and the IMU-loss test; 72-73: repeat key scenarios and compile final metrics; 74-75: reproduce the package from a clean directory, write conclusions and list outstanding physical checks. If no platform is available, explicitly retain the status "tilt control validated in a model".

Before proceeding, check that the comparison of joint and body control is complete and that command limits and fault behaviour have been tested. The next module receives a frozen baseline model and PID, rather than a set of unrelated demonstrations.

Acceptance and handover to M11

Practical defence of the result

In 10-15 minutes, show the loop diagram, one stable and one rejected operating condition, a frequency response and a fault recording. Explain how the model relates to the real actuator interface. Then change only delay or inertia and predict the direction of the response change in advance. A reviewer must be able to recalculate metrics from the raw recording and reach the same conclusion about whether the criteria were met.

Required evidence of readiness: positive axis directions are checked; settings and units are fixed; the declared operating region is bounded; saturation and loss of fresh data are tested; measured timing costs are documented. Support stability with margins from a correct loop model or experimental evidence over declared delays and loads. Do not describe the latter as a rigorous proof of stability for every possible operating condition.

Self-check questions

1) How does a pole differ from a zero in the joint example? 2) Why can increasing P both speed up and degrade the response? 3) What does the integral do when the physical command is already at its limit? 4) Why does differentiating the measurement differ from differentiating the error at a reference step? 5) Which transfer is used to calculate phase margin? 6) Why does the Nyquist frequency not determine a sufficient control frequency?

7) How can you distinguish sensor noise from loop oscillations? 8) What does a filter change besides noise amplitude? 9) How do you specify settling time for zero target tilt? 10) Why does average loop time not replace p99 and the maximum? 11) How can an old packet mistakenly appear fresh? 12) Why are zero torque and safe body holding not always equivalent? Support your answers with your own plots, as well as definitions.

What to save and which claims remain unconfirmed

Pass the model, baseline PID, discretisation parameters, disturbance scenarios, noise and delay configuration, and common metrics script to M11. They form the fixed basis for comparison with LQR and an estimator. In a separate table, list unmodelled effects: backlash, dry friction, torque-speed dependence, structural compliance and incorrect tilt estimation during acceleration.

If time is limited, retain one thoroughly tested body axis and a model demonstration of the second axis with explicit status. Reduce frequency count and presentation work, but retain closed-loop response measurement, saturation and fault scenarios. The module is complete only within the checks actually performed; any deferred physical work must appear in the subsequent integration plan.

Assigned reading: a focused reading route

Books and lecture notes may be in English; keep the plan, definitions and your own explanations in English. Read only the specified sections within the existing 75 hours.

Core reading and one reference route

1. Core: K. J. Åström, R. Murray, Feedback Systems, 2nd edition. Chapters Transfer Functions, Frequency Domain Analysis and PID Control; within the last chapter, Basic Control Functions, Integrator Windup and Implementation. For tasks 1-6, derive the loop equations, examine the effect of delay and implement a discrete PID.

2. Core worked example: B. Messner, D. Tilbury and the CTMS team, Control Tutorials for MATLAB and Simulink. The page Introduction: PID Controller Design, followed by DC Motor Speed: PID Controller Design. For tasks 1-2, reproduce the P, PI and PID comparison on your own model in Python.

3. Reference: J. Doyle, B. Francis, A. Tannenbaum, Feedback Control Theory. §3.1 Basic Feedback Loop and §3.4 Performance; §4.1 Plant Uncertainty - the introductory formulation only. For tasks 3-4, explain the trade-off between error, noise and model inaccuracy.

Documentation for specific practical work

Python Control step_info - transient-response criteria; margin - phase units and loop margins. Before plotting, check units: rad and deg, Hz and $\text{rad}\cdot\text{s}^{-1}$.

SciPy cont2discrete - zoh, bilinear and euler for task 5. For the spectral analysis in F4, use the familiar `scipy.signal.welch` with explicit fs and scaling.

After reading

For each section, produce one plot and a short explanation using your own axis. Do not transfer coefficients from an illustrative example to the rig. MATLAB/Simulink are not mandatory purchases: translating the mathematical model into Python preserves the exercise's purpose. Investigate delay in the model first, then repeat an admissible experiment on the rig.

Key terms and definitions

Plant. A physical system or its model whose output changes under a control input and external disturbances.

Control error. The difference between the reference and measured output, in the same units and a consistent coordinate frame.

Proportional action (P). The part of the command equal to current error multiplied by a gain.

Integral action (I). The part of the command that depends on error accumulated over time; it helps remove constant offset under admissible operating conditions.

Derivative action (D). The part of the command responding to the rate of change of error or measurement; it usually requires measurement-noise filtering.

Feedforward. A command calculated from the reference and a model of the expected input, without waiting for measured error to appear.

Transfer function. The ratio of the Laplace transforms of output and input for a linear time-invariant system under zero initial conditions.

Pole. A singularity of a transfer function associated with a natural dynamic mode; internal stability is checked with reference to the realisation.

Frequency response. The complex ratio of output and input amplitudes of a linear system under steady-state sinusoidal excitation at a specified frequency.

Bode plot. Plots of frequency-response magnitude and phase against a logarithmic frequency axis, with explicitly stated units.

Phase margin. The additional phase lag to the critical value at a selected unity-gain frequency of the open loop.

Gain margin. The permissible gain multiplier before the critical condition at a selected open-loop phase crossover.

Bandwidth. The frequency range over which a signal is reproduced acceptably; the boundary criterion must be declared for the selected channel.

Overshoot. The amount by which the output exceeds the final step level; a relative measure is normalised by the non-zero amplitude of that step.

Settling time. The time after which error remains within the declared band until observation ends for the given scenario.

Sampling period. The interval T_s between scheduled digital-controller updates; actual intervals are also measured using timestamps.

Zero-order hold. Keeping the last command constant until the next output update.

Jitter. Variation in actual execution times or update intervals relative to the chosen digital-loop schedule.

Gain scheduling. A predefined choice of parameters based on operating point, with validated transitions between modes.

Check the relationships: for unity negative feedback, $L=PC$, $T=L/(1+L)$, $S=1/(1+L)$. Frequency $\omega=2\pi f$.

Equipment and software preparation

USE EXISTING EQUIPMENT

MacBook Pro M4, the validated M7 rig, ESP32-S3, IMU and 8-channel logic analyser from M6, and the power supply and multimeter from M4. Use the tilt geometry and actuation axis from M8. Task 7 permits a model-based option; you do not need to buy a physical two-axis platform for this module.

PURCHASES: NO MANDATORY ADDITIONS

Optional: 1 oscilloscope only if current, ripple or signal edges cannot be assessed with existing equipment. Minimum: 2 channels, triggering, recording export and probes with suitable ratings. Choose bandwidth and sample rate for the fastest signal under investigation, rather than the mechanical-motion frequency.

Before buying, check data export on macOS ARM and input types. H-bridge potentials require a suitable differential measurement path; do not connect an ordinary grounded probe across two switching terminals. Selection guide: [Tektronix: XYZs of Oscilloscopes](#).

INSTALL NOW: BEFORE HOURS 1-20

Add the **control** package (Python Control Systems Library) to the pinned macOS ARM environment, following [Introduction → Installation](#). Use the existing NumPy, SciPy, Matplotlib and pytest installations. Slycot is optional for this module's SISO experiments; retain the M6 ESP-IDF firmware environment.

INSTALLATION CHECK

Import control, print its version, create a first-order model and plot its step response. Compare the specified time constant with the plot; calculate step_info, the frequency response and margin for your loop. Obtain a discrete model through cont2discrete and compare it with the continuous model at small Ts.

Build the previous firmware on the microcontroller; measure the actual GPIO period with the logic analyser and compare it with timestamps. Save p95 and p99, the maximum, data age and a deadline-miss scenario in the log. Separately check stopping and recovery without spontaneous motion resumption.

You may repeat the calculations in a separate Ubuntu 24.04 ARM64 environment; test actual USB passthrough and virtual-machine graphics. Preparation is included in the 75 hours; order the instrument in advance if it proves necessary.

Motion fundamentals: holding and tracking

Required: why a stable position is insufficient

Holding specifies a constant angle q_d ; tracking specifies $q_d(t)$, velocity and acceleration. A leg joint must change angle on time during foot swing. Small error at the end of motion is insufficient: lag along the path changes foot position. Before calculating, explain the negative-feedback sign and the distinction between angle, angular velocity and torque.

Joint model and error equation

The base is fixed; positive q and τ are chosen along the same axis. q is in rad, and primes denote time derivatives: $\dot{q}$ in rad/s, $\ddot{q}$ in rad/s². The link moves without gravitational torque; J is inertia about the joint in kg·m², and b is viscous resistance in N·m·s/rad. For now, assume no saturation, delay or model error:

$J \ddot{q} + b \dot{q} = \tau$; $e = q_d - q$. Choose $\tau = J \ddot{q}_d + b \dot{q}_d + K_p e + K_d \dot{e}$. The first two terms provide acceleration and overcome resistance in advance; PD corrects deviation. Substituting $q = q_d - e$ into the model gives $J \ddot{e} + (b + K_d)\dot{e} + K_p e = 0$. This is error dynamics, not a separate physical joint.

Let $J = 0.02$ kg·m², $b = 0.04$ N·m·s/rad, $K_p = 2$ N·m/rad and $K_d = 0.36$ N·m·s/rad. Then $\ddot{e} + 20\dot{e} + 100e = 0$; both poles equal -10 s⁻¹. Error decays with critical damping in the ideal continuous model. This conclusion alone does not confirm the stability of a digital actuator with limits.

Calculation at a selected instant

At the selected instant, $q_d = 0.30$ rad and $q = 0.20$ rad; $\dot{q}_d = \dot{q} = 0.50$ rad/s and $\ddot{q}_d = 1$ rad/s². Thus $e = 0.10$ rad and $\dot{e} = 0$; feedforward is $0.02 \cdot 1 + 0.04 \cdot 0.50 = 0.04$ N·m, feedback is 0.20 N·m, and total $\tau = 0.24$ N·m. The model gives $\ddot{q} = (0.24 - 0.04 \cdot 0.50)/0.02 = 11$ rad/s² and $\ddot{e} = -10$ rad/s²: the joint accelerates, reducing its lag behind the reference.

If feedforward is removed and q_d moves at a constant velocity of 0.50 rad/s for a long time, steady-state $e = b \cdot \dot{q}_d / K_p = 0.01$ rad. The error creates the torque needed to overcome resistance. For a stationary target in this model, e tends to zero. Holding and tracking therefore require different test references.

Two experiments within the existing laboratory work

1. For a constant target, take $e(0) = 0.10$ rad and $\dot{e}(0) = 0$. Save an e plot and compare with $e(t) = 0.10(1 + 10t)\exp(-10t)$: at $t = 0.40$ s, expect 0.00916 rad. **2.** For a linearly changing reference, compare PD with and without feedforward; save q_d , q , τ and the individual terms. Check: error without feedforward approaches 0.01 rad; mark saturated segments separately.

Time: 1 theory hour from hours 7-8 (PID and feedforward), and 2 practice hours from 15-16 (integral and derivative actions, limits); this analysis and these plots belong to existing tasks 1-2, within the 75 hours. The numbers apply to the model. Do not send torque τ to a position servo: its internal loop is studied separately. Full multi-joint computed-torque control remains outside the mandatory minimum.

Read selectively: Lynch, Park, Modern Robotics, [§11.4, Part 1: PD and error dynamics](#); [§11.4, Part 3: model-based feedforward](#).

Robot state: estimation and admissible control

M11 • 08 Aug 2027 - 12 Sep 2027 • 75 hours

This month's deliverable

You will connect three engineering questions: what is happening inside the mechanism, what can be reconstructed from sensors, and which command is admissible to send. The deliverable compares PID, LQR, estimation with KF and EKF, and a simple QP on a model with known parameters, using identical disturbances and testing optimiser failure. For a leg, this prepares for contact-force allocation; for a finger, it prepares for control using an estimated state with bounded actuation.

Begin with the axis from M10. State $x = [q, v]$ contains angle q in rad and angular velocity v in rad/s; input u is torque in N·m; the encoder measures angle. In this model, velocity can be reconstructed from a sequence of measurements even without a separate velocity sensor. In a small separate experiment, estimate tilt from a gyroscope and accelerometer. Then solve a planar force-allocation problem with two supports. These three parts share rules for time, units and checks, but each needs its own mathematical model.

Prerequisites

Review matrix multiplication, derivatives, eigenvalues and the meaning of variance. Bring the baseline PID, joint model, command limiter and fixed test scenarios from M10. Distinguish measurements, simulation ground truth and estimates. Validate estimates against a reference: the known state in simulation or readings from an independent reference system. The output of an IMU-processing library is not, by itself, true orientation.

Limits on complexity

This month does not require a complete MPC, a general-purpose 3D-EKF or whole-robot balancing. Required topics are state space, linearisation, controllability and observability, the linear Kalman filter, a small tilt EKF, LQR and a convex quadratic programme (QP) with explicit constraints. Mathematics should explain the experiment rather than block it for weeks. You will check each step from model to solution and see the limitations of each approximation.

The final project contains code, equations, units, covariances, chosen objective weights, raw logs and automated checks. All numerical thresholds in example models are experiment parameters. For a physical actuator, limits come from its specifications, load and the previously validated rig. "The optimiser found a solution" implies neither physical feasibility nor a correct model.

Organising the 75 hours

Four work blocks

Block A, hours 1-20: state space and observability. Block B, 21-40: KF, EKF and estimation quality. Block C, 41-60: LQR and a comparison with PID under identical conditions. Block D, 61-75: force QP, faults and integration. Each of the first three blocks contains 10 hours of theory and analysis and 10 hours of practice; the fourth contains 5 hours of analysis and 10 hours of practice. Total: $35 + 40 = 75$ hours. The work steps on pages 9-10 expand these same hours and require no additional time allocation.

Divide each ten-hour theory block into five two-hour sessions: 20 minutes reviewing the previous model, 45 minutes reading the relevant section, 40 minutes deriving a small example or examining a log, and 15 minutes recording the conclusion. Also divide the ten-hour Sunday practical session into five two-hour periods with breaks between them. Breaks are excluded from net study hours. Allocate the final five hours of analysis as $2 + 2 + 1$.

Start with the question, then choose the tool

The main environment is Python, NumPy, SciPy and python-control; use FilterPy for cross-checking and CVXPY and OSQP for a small convex problem. Begin each session with a testable question: how does the input affect the output, which state cannot be measured, and which data reveal its changes? Before running, write down the expected sign, scale or sequence of events. Compare the prediction with the result and examine at least one discrepancy. A plot supports a conclusion only when accompanied by an explanation of the observed relationship.

Maintain one repository with model, config, experiments, tests and reports directories. Every experiment needs a unique identifier, code version, configuration, parameter source, initial state, units and raw data. The analysis script reads a saved experiment and builds the report without manually correcting measurements. The final state-estimation-and-control benchmark project combines comparisons of estimation and control methods; individual exercises gradually form its test suite.

Time accounting and reducing scope

Calendar boundaries may coincide with an adjacent module. Keep rest days, holiday intervals and integration checks as specified in the overall calendar; place numbered steps in available time without counting hours twice. Do not add hours to this module. Assess equipment and library readiness at the outset rather than spending the practical session on an unplanned platform change.

If you fall behind, reduce the number of LQR weight sets, support geometries and validation runs with different random-number seeds after the minimum checks. Retain the axis model with a two-dimensional state, an unobservable case, KF with a missing measurement, tilt EKF with an outlier, an equal-conditions PID/LQR comparison and a two-support QP with infeasible and timeout statuses. Full 3D orientation and MPC are unnecessary. The result must connect modelling, estimation and constrained control; all declared results can be obtained in simulation with known parameters.

State space and the operating point

Study the state and its evolution

Write $\dot{x} = Ax + Bu$, $y = Cx + Du$. For $J\ddot{q} + b\dot{q} = u$, the matrices are $A = \begin{bmatrix} 0 & 1 \\ 0 & -b/J \end{bmatrix}$, $B = \begin{bmatrix} 0 \\ 1/J \end{bmatrix}$, $C = \begin{bmatrix} 1 & 0 \end{bmatrix}$, $D = \begin{bmatrix} 0 \end{bmatrix}$. Dimensions: A 2×2 , B 2×1 , C 1×2 , D 1×1 . Rows have different physical units; matching matrix shapes do not make q and v interchangeable. In discrete time, use $x[k+1] = A_d \cdot x[k] + B_d \cdot u[k]$, obtaining A_d and B_d for the declared T_s with a held command.

For a nonlinear model $\dot{x} = f(x,u)$, choose an equilibrium $f(x_0, u_0) = 0$ and work with deviations δx , δu . Evaluate Jacobians $A = \partial f / \partial x$ and $B = \partial f / \partial u$ at that point. Check the meaning of gravity compensation in u_0 for a loaded axis: do not linearise around an accelerating state and then treat it as stationary. Small-deviation equations apply only within a limited neighbourhood.

Task 1. A one-step transition

Inputs: the M10 model, two initial states and three constant torques within the model's admissible range. 1) Derive the matrices. 2) Obtain A_d and B_d . 3) Compare one step with numerical integration on a fine grid. 4) Repeat for zero torque and non-zero velocity. Deliverable: `state_transition_tests` and an error table. Check: $q=v=u=0$ remains the zero state; for positive torque, the first velocity change has the correct sign. Error to test: omitting T_s from an update creates a scaling error that the test must detect.

Task 2. Observability and controllability

Construct the controllability matrix $[B, AB]$ and observability matrix $[C; CA]$, and calculate rank and singular values. For comparison, replace the angle sensor with a velocity-only sensor $C = [0, 1]$. For a free axis without positional stiffness, absolute angle is then unobservable. Specify two different initial angles with the same velocity and show identical measurements. Deliverable: a short demonstration using two trajectories, as well as `rank()`. Check sensitivity of numerical rank to unit scaling; poor conditioning can hinder estimation even when rank is formally full.

Then linearise a model with gravity near a chosen q_0 and compare it with the nonlinear model for small and large deviations. State the region in which the linear prediction error is acceptable under your predefined criterion.

KF and EKF: estimation with testable uncertainty

Study filter prediction and update

Prediction calculates the next estimate and covariance from the model; the update refines them using a new measurement. Q describes process-model uncertainty, R measurement uncertainty, and P estimate uncertainty. These are covariances, not “confidence coefficients between zero and one”. Their dimensions correspond to products of component units. If T_s changes, reconsider discrete Q using the chosen noise model. Scalar angle variance has units rad^2 ; angle-velocity covariance has units rad^2/s .

The measurement residual, or innovation, $v = z - h(\hat{x}^-)$ is the difference between a measurement and its prediction; S is the predicted covariance of that difference. A smooth estimate can be wrong even when its claimed uncertainty is small. Check error against simulation ground truth, changes in P and innovation statistics. For NIS, calculate $v^T S^{-1} v$ by solving a linear system without explicitly inverting the matrix. Comparison with a χ^2 distribution is meaningful under correct Gaussian assumptions and the appropriate measurement dimension. This is a diagnostic check with stated conditions of applicability.

Task 3. Angle and velocity from a noisy encoder

Generate joint motion with a fixed seed, encoder noise and a small model error. Implement prediction and update for $x=[q,v]$; compare with a library using identical data. Compare the velocity estimate with a simple finite difference. Save true and estimated q and v , the diagonals of P , the innovation and RMSE. Check: noisy data are not written to truth; when a measurement is missing, perform prediction only and let the corresponding uncertainty grow. Then set R artificially 100 times smaller than the actual variance and explain the consequences using innovations and estimation error.

Task 4. An EKF for one tilt axis

For one axis, choose state $[\theta, b_g]$: tilt in rad and gyroscope bias in rad/s . Prediction: $\theta[k+1] = \theta[k] + T_s \cdot (\omega_m - b_g)$, $b_g[k+1] = b_g[k]$. For a consistent axis, you may use accelerometer model $h=[g \cdot \sin\theta, g \cdot \cos\theta]$ in m/s^2 ; explicit signs must match the coordinate-frame convention and rest test. Derive $H=[g \cdot \cos\theta, 0; -g \cdot \sin\theta, 0]$. Check the Jacobian with finite differences. Introduce an outlier and translational acceleration; innovation gating must mark rejected measurements and preserve the prediction. Deliverable: an experiment with gyroscope drift, accelerometer correction and temporary rejection of updates; yaw cannot be determined from gravity direction alone.

LQR and LQG: the cost of error and control effort

Study the objective function

For the discrete axis, define the sum $x^T Q x + u^T R u$ and find feedback $u = -Kx$. Q is positive semidefinite and R is positive definite; do not confuse this Q with the filter's process covariance. Name them Q_cost and Q_noise in code. To compare rad, rad/s and N·m meaningfully, normalise components by selected characteristic scales q_ref , v_ref and u_ref . The weights then express the relative cost of error and actuation, rather than depending accidentally on a conversion to degrees.

LQR does not impose saturation itself. On a real actuator, a limiter changes the closed-loop dynamics, so test it separately. Tracking a non-zero reference requires deviations from an equilibrium and the corresponding u_0 ; simply inserting absolute x into $-Kx$ drives it towards zero. LQG combines a linear Kalman filter with LQR. The separation principle applies under specific linear conditions; it provides no automatic stability guarantee for delays, saturation, nonlinearities or EKF.

Task 5. LQR on an axis with true state

Use A_d and B_d from task 1. 1) Specify two sets of normalised weights: prioritising accuracy and prioritising small torque. 2) Calculate K and the poles of $A_d - B_d K$. 3) Check stabilisation at zero and a small step change of equilibrium. 4) Add the same limit on u . Deliverable: two responses and a table of RMSE, peak command and the integral $u^2 dt$ in $N^2 \cdot m^2 \cdot s$. Check: predicted poles are inside the unit circle for the linear case; under saturation, support the conclusion with a separate experiment. Failure case: weights designed for degrees are applied to radians without conversion.

Task 6. Comparing PID and LQR under identical conditions

Take the PID with frozen settings from M10. First compare it with LQR under equal access to true state or measurements; state explicitly which controller receives velocity. Then connect KF and repeat the noisy cases. Use the same plant, limits, T_s , initial conditions, disturbance and test seeds. Deliverable: a table of accuracy, control effort, computation time and saturation fraction. Check: tuning uses a separate scenario set; the final set is not used to fit weights. If LQR wins through access to true velocity, describe this as an information advantage rather than an advantage of the algorithm itself.

Separately compare an inaccurate model and estimation delay. Make the conclusion specific: which controller is better for the chosen axis and conditions, where behaviour differs and which added complexity is justified. LQR superiority is not a required outcome of the exercise.

QP: allocating forces between two supports

Variables, geometry and dimensions

Model: two point supports in the x-z plane, with coordinates relative to the body centre $r_1=(-a,-h)$, $r_2=(a,-h)$, where a,h are in metres. The unknown vector is $f=[f_{x1},f_{z1},f_{x2},f_{z2}]^T$; all four components are in newtons. Contact pushes upward: $f_{zi} \geq 0$. The prescribed force-and-moment vector, or wrench, $w=[F_x,F_z,M_y]$ has component units N, N and N·m. From $r \times f$, obtain $M_y=-h \cdot f_{x1}+a \cdot f_{z1}-h \cdot f_{x2}-a \cdot f_{z2}$. Thus A is 3×4 with third row $[-h,a,-h,-a]$; the first rows sum f_x and f_z .

To avoid directly adding squared N and N·m values, define $D=\text{diag}(1/F_{ref},1/F_{ref},1/M_{ref})$. Minimise $\|D(Af-w)\|^2 + \lambda \|f/F_{ref}\|^2$ with $\lambda > 0$. Constraints: $0 \leq f_{zi} \leq F_{max_i}$ and $-\mu f_{zi} \leq f_{xi} \leq \mu f_{zi}$; μ is dimensionless. This is a planar friction cone with linear boundaries and a convex QP. Label $a,h,F_{ref},M_{ref},F_{max}$ and μ as illustrative values or measurement-derived values. Contact forces do not yet convert automatically into real actuator commands; that will later require Jacobians and model validation.

Task 7. A feasible allocation

Take symmetric geometry and request $F_x=0$, $F_z=mg$, $M_y=0$. 1) Write A and the bounds. 2) Solve the QP. 3) Check $f_{z1}=f_{z2}$ for symmetric limits. 4) Add a small M_y and explain the load transfer. 5) Increase F_x until the friction boundary is reached. Deliverable: a computational notebook, forces at each support, a normalised residual and active constraints. Check: independently recalculate Af , every bound and the moment sign. Do not trust solver status alone: check residuals in physical units with an explicit tolerance.

Task 8. An unreachable request and solver failure

The problem with the soft objective above usually has a feasible solution $f=0$, so separately add the equality $Af=w$ and require $F_z > F_{max1}+F_{max2}$ to create a deliberately infeasible request. Then check premature termination at the iteration limit and an artificially late result. Record status, residual, iteration count and time; compare cold and warm starts on the same request sequence. Deliverable: p50, p95, p99, maximum and the fraction of unusable solutions. The fallback mode uses only a previously validated action for the current constraints; a previous solution cannot be repeated unconditionally after losing contact.

A common experiment and component compatibility

Three connected experiments

Part A - axis: model $\rightarrow$ sensor $\rightarrow$ KF $\rightarrow$ PID or LQR $\rightarrow$ limiter $\rightarrow$ model. Part B - tilt: gyroscope and accelerometer $\rightarrow$ EKF $\rightarrow$ log of estimates, errors and rejected updates. Part C - force allocation: requested wrench $\rightarrow$ QP $\rightarrow$ independent constraint check $\rightarrow$ admissible output or fallback mode. Align units, timing and scenario format. QP forces cannot be inserted into the joint model without a physically justified transformation; a full model of a balancing robot is not required this month.

The axis scenario contains J , b , T_s , noise, initial state, u limits, seed and reference trajectory. The EKF scenario contains the θ trajectory, true gyroscope bias, noise intensities and intervals of additional acceleration. The QP scenario contains support coordinates, μ , force limits, requests and time limits. Label all synthetic parameters as synthetic; measured contains only physically measured data.

Meaningful metrics

For the axis, calculate angle and velocity RMSE, peak command, the integral of squared torque and saturation fraction. For state estimation, calculate RMSE against simulation ground truth, innovation mean and temporal correlation, NIS and the fraction of rejected updates. For QP, calculate the wrench residual in physical and normalised units, maximum bound violation, solution time, iteration count, status and the number of transitions to fallback mode. Laptop timing applies only to that machine and configuration and does not guarantee the same speed on a microcontroller.

Use one set of seeds for parameter selection and a separate held-out set for final validation. Before increasing the run count, try small limiting cases: zero state, zero load, symmetric supports and a missing measurement. Save at least one case you cannot explain and make it an open task with a minimal reproducing example.

Files and reproduction

The README describes mathematical models and applicability limits; config stores units and normalising scales; tests checks transitions, Jacobians, matrix dimensions and constraints. reports contains matching plots for all compared algorithms and a separate fault page. The reproduction command must recalculate aggregates from saved logs; repeating a simulation with pinned versions and seed must give statistically and numerically consistent results. Do not require bitwise equality across different BLAS libraries, operating systems and solver versions without a separate check.

Six tests for incorrect solutions

Estimation and modelling

F1 - unobservable absolute angle. Two states differ only in q ; the sensor sees v on a free axis. Expected: identical measurements; the estimator must not confidently “recover” an unknown offset without additional information. Deliverable: two trajectories and an explanation of which sensor or constraint would change the situation.

F2 - missing encoder samples. Remove several consecutive measurements while retaining the true motion. Expected: prediction continues, updates are skipped and uncertainty changes according to the model. An incorrect variant repeats an old measurement as new and artificially reduces P ; include it as a negative test.

F3 - an IMU outlier and linear acceleration. Introduce one large outlier and a separate sustained acceleration interval. Expected: gating flags the outlier; do not hide the sustained effect by claiming that an accelerometer always measures gravity alone. Compare the number of rejected updates and accumulated drift; separately record resumption of updates.

Control and optimisation

F4 - inaccurate model and LQR saturation. Increase J , add delay and constrain u to the same limit as PID. Expected: measurable changes; the original model’s linear poles are not used as proof of stability for the saturated system. The report shows where the chosen controller’s advantage disappears.

F5 - an infeasible QP with hard equality constraints. Request a vertical force above the sum of the limits with $Af=w$. Expected: an unusable status is handled before publishing a command; the log contains a reason rather than an empty vector. Alongside it, show the soft problem with a non-zero residual to distinguish an optimal approximation from exact fulfilment.

F6 - a late or non-finite solution. Substitute NaN for the output or delay it until the next step. Expected: checks of finiteness, compatibility with current contacts and the computation deadline reject the result. Also recheck the last admissible solution against new constraints; if a support has disappeared, its old forces are no longer permitted.

Analysis and correction

For each test, save inputs, the expected event and the actual event. Check dimensions and normalisation first, then the model, and only then Q , R or solver settings. Do not reduce R merely because an estimate looks smoother, or weaken physical constraints to remove an infeasible status. The corrected version must pass the same original scenario; any criterion change requires an explanation and a new experiment identifier.

Hours 1-40: modelling and estimation

Hours 1-20: state space

Theory 1-10: 1-2: select states, inputs and outputs and list units; 3-4: derive A, B, C and D; 5-6: work through Ad, Bd and the matrix transition; 7-8: controllability and observability; 9-10: operating point and Jacobians. These are five two-hour sessions. End each with a dimensional check and one limiting case for your axis.

Practice 11-20: 11-12: transfer the M10 model; 13-14: complete task 1; 15-16: complete task 2 with two sensors; 17-18: check linearisation through finite differences and trajectories; 19-20: save a validation set of transitions. The ten-hour practical session ends with executable checks, as well as equations in your notes.

Before proceeding, verify that you can explain observable and unobservable states, have correct matrices and have defined the linear model's operating region. Move on to Q and R units after understanding the model.

Hours 21-40: Kalman filter and tilt EKF

Theory 21-30: 21-22: mean and covariance; 23-24: prediction and update; 25-26: innovation and tuning Q_{noise} and R_{noise} ; 27-28: EKF and the measurement Jacobian; 29-30: innovation gating and statistical consistency. Derive one scalar update by hand and trace how changing R changes the correction.

Practice 31-40: 31-32: implement the linear KF in task 3; 33-34: test missing measurements and incorrect R; 35-36: build the tilt model in task 4; 37-38: check the Jacobian, outlier and acceleration; 39-40: plot errors, innovation and P, and write an explanation. Use a library for cross-checking; the central requirement is to explain every input and failure mode.

Before proceeding, check that estimates are compared against an independent reference and that simulation ground truth is stored separately. Explain why a smooth estimate may be wrong and why yaw cannot be determined from gravity direction alone.

Hours 41-75: optimal control and constraints

Hours 41-60: LQR and comparison

Theory 41-50: 41-42: quadratic objective; 43-44: state and command normalisation; 45-46: discrete LQR and feedback; 47-48: tracking relative to equilibrium; 49-50: LQG and the limits of the separation principle. Describe PID and LQR objectives in the same terms so that the comparison addresses the same task.

Practice 51-60: 51-52: complete task 5 with two weight sets; 53-54: add limits and an inaccurate model; 55-56: complete task 6 without noise; 57-58: enable KF and the held-out seeds; 59-60: compile a table of the trade-off between accuracy and control effort. If the result is unexpected, check which information each controller receives before attempting to retune.

Before proceeding, verify that the baseline PID is unchanged, the comparison is reproducible and objective-function dimensions are understood. Tie any conclusion about winning or losing to its conditions, and explicitly mark saturated segments.

Hours 61-75: QP and solver failure

Analysis 61-65: 61-62: two-support geometry and matrix A; 63-64: normalised quadratic objective, friction and infeasibility; 65: solver and fallback requirements, and the final acceptance plan. The last theory block is 2 + 2 + 1 hours; learning general MPC is outside its scope.

Practice 66-75: 66-67: complete task 7; 68-69: complete task 8 with infeasible hard constraints; 70-71: compare warm and cold starts and test timeout and NaN; 72-73: run the six fault tests and compile the common report; 74-75: reproduce the package and prepare interface descriptions for ROS2. If time is limited, reduce geometry count while retaining independent checks of constraints and late results.

Before proceeding, verify that the three package components agree on timing and units, QP does not output unchecked forces and every result has provenance. M12 receives clear data and fault interfaces.

Acceptance: explain and reproduce the solution

What to demonstrate without prompting

Begin with the axis's two-dimensional state and give every matrix dimension. Show an unobservable case, then a successful KF and a poor covariance choice. Explain the EKF Jacobian and demonstrate its comparison with finite differences. Next, show an identical test for PID and LQR and a solved QP with a manual check of total force and moment. Finish with a forced solver failure and the log of transition to fallback mode.

Readiness criteria: state-transition tests pass; every sensor has a measurement model and units; covariance is not treated as an arbitrary confidence percentage; innovations and missing measurements have been investigated; tuning is separate from validation; QP constraints are checked independently; timeout, infeasible and non-finite results are handled. Justify how well innovation statistics agree with the assumed noise model. A histogram's appearance alone is insufficient.

Self-check questions

1) Why can two different states produce the same measurement? 2) How does poor conditioning differ from formal rank deficiency? 3) When is linearisation needed, and how do you check its validity region? 4) Why can a filter not keep updating indefinitely with an old number after a packet is lost? 5) How does Q_{noise} differ from Q_{cost} ? 6) What are the units of R for an accelerometer? 7) Why does increasing the filter's confidence not guarantee a smaller error? 8) Where is LQR action limited? 9) What changes when an angle is converted to degrees without normalisation? 10) Why can a QP with a soft residual penalty remain feasible for an infeasible request? 11) Why do force and moment need different cost scales? 12) When can the last successful solution no longer be used? Relate every answer to a saved negative test.

Handover to the next module

Prepare field-level definitions of StateEstimate, SensorSample, ControlRequest and ControlStatus: timestamp, coordinate frame or axis, units, value, covariance or quality status, and scenario identifier. Do not create ROS interfaces before M12, but record which component produces each field and which state is considered stale. Separately list frequency, admissible age and diagnostic requirements.

Numerical experiments do not close unfinished physical checks. As specified by the final project, this month can be completed entirely on models with known parameters; transfer to real equipment remains a separate, justified step. If reducing scope, retain one linear axis, one tilt EKF and a two-support QP. Complex three-dimensional models and rewriting all algorithms in a third language are optional.

Assigned reading: a focused reading route

Books and lecture notes may be in English; keep the plan, definitions and your own explanations in English. Read only the specified sections within the existing 75 hours.

Core reading and one reference route

- 1. Core:** S. Lall, [Stanford EE363: Linear Dynamical Systems, 2026](#). Slides 9 (discrete systems), 11-12 (controllability and observability), 18 (Infinite horizon linear quadratic regulator), 22 (The Kalman filter). For tasks 1-3 and 5, work through only the equations for your axis.
- 2. Core:** R. Labbé, [Kalman and Bayesian Filters in Python](#). Chapters 04 One Dimensional Kalman Filters, 07 Kalman Filter Math, 08 Designing Kalman Filters, 11 Extended Kalman Filters. For tasks 3-4: one manual update step, covariance selection and a Jacobian check.
- 3. Core, selected sections:** S. Boyd, L. Vandenberghe, [Convex Optimization](#). §4.4 Quadratic optimization problems and §5.5 Optimality conditions. For task 7, distinguish the objective from the constraints and establish which properties make the two-support QP convex.

Documentation for specific practical work

[Python Control dlqr](#) - outputs K, S and E, and the discrete objective; [CVXPY: Quadratic program](#) - variables, quad_form, equalities and inequalities.

[OSQP Solver settings](#) - max_iter, time_limit, eps_abs, eps_rel and warm_starting. For task 8, set a time limit, log status and check residuals independently; finding a point does not yet establish successful control.

After reading

For each topic, first write the state vector, matrix dimensions and units, then open the code. Q_cost and Q_noise have different meanings and different names in the project. After reading, implement KF prediction and update and one LQR feedback law, then manually check total force and moment from QP.

Key terms and definitions

State. A vector of variables sufficient to predict the model's subsequent motion given known future inputs.

Operating point. The state and input around which a system is studied; equilibrium additionally requires an unchanging state.

Linearisation. A local approximation of nonlinear equations using first derivatives with respect to state and input around a chosen operating point.

Controllability. The ability to move a linear model between arbitrary states in finite time using an input admissible under the problem formulation.

Observability. The ability to determine a model's initial state uniquely from its output history with known inputs.

Covariance matrix. A matrix describing the joint variability of random errors; the diagonal contains variances, and each element's units are products of component units.

Process noise. A random input representing uncertainty in modelled state evolution; its covariance is denoted Q_{noise} .

Measurement noise. The random component of sensor error in the observation equation; its covariance R_{noise} refers to the measured quantities.

Innovation. The difference between a new measurement and the predicted measurement before updating; it signals disagreement between data and prediction.

Kalman gain. The matrix mapping innovation to a state correction, determined by the measurement model and estimated covariances.

Kalman filter (KF). Recursive state estimation for a linear model through prediction and update; optimality requires stated noise assumptions.

Extended Kalman filter (EKF). An estimator for a nonlinear model using local Jacobians to propagate covariance and update from measurements.

LQR. Feedback that minimises a quadratic cost of state and actuation for a linear model under specified problem conditions.

LQG. The combination of LQR and Kalman estimation in a linear problem with Gaussian noise and a quadratic criterion.

Quadratic programming (QP). Optimisation of a quadratic objective subject to linear equalities and inequalities; convexity requires a positive semidefinite quadratic form.

Feasible solution. A variable vector satisfying every problem constraint within declared numerical tolerances; optimality is checked separately.

Constraint residual. A quantitative measure of equality or inequality violation, calculated from returned variables independently of the solver message.

Fallback control. A predefined admissible action when estimation or optimisation fails, consistent with the system's current limits.

Check the relationships: $x[k+1] = A_d \cdot x[k] + B_d \cdot u[k]$; with $u = -Kx$, the closed-loop matrix is $A_d - B_d K$. Command limiting changes the linear model.

What to install for KF, LQR and QP

NEW PURCHASES: NONE REQUIRED

Use the existing MacBook Pro M4 and models and recordings from M7-M10; the physical rig, IMU and ESP32-S3 have already been purchased. Solve the two-support force-allocation problem numerically. It requires no second physical rig, new joints, GPU or onboard computer.

USE THE PINNED ENVIRONMENT

macOS ARM: Python, NumPy, SciPy, Matplotlib, pytest and control from M10. Retain the selected Python and compatible dependency versions. For repetition on Ubuntu 24.04 ARM64, create a separate environment; do not mix binary packages from different architectures or environments.

INSTALL BEFORE TASKS 7-8

Add **cvxpy** and its available CPU solver **osqp**; verify its presence in `installed_solvers`. Instructions: [CVXPY Install](#). On macOS, use the configured Xcode Command Line Tools. Pin versions after a successful short experiment; there is no need to update the entire environment.

OPTIONAL: BEFORE TASKS 3-4

Add **filterpy** only for independent comparison with your own KF and EKF implementations. [FilterPy: Installation](#) describes release installation; [ExtendedKalmanFilter](#) describes `dim_x` and `dim_z` and the `Hjacobian` and `Hx` functions. Write the core matrix steps yourself first.

CHECK INSTALLATION AND BEHAVIOUR

1. Import packages and record versions and the solver list. 2. Perform one KF prediction-and-update cycle and compare it with a hand calculation. 3. Obtain `dlqr` for the discrete axis and check the poles of `A-BK`. 4. Solve the scalar QP $\min (x-2)^2$ with $0 \leq x \leq 1$: expect $x \approx 1$, then independently check the bounds.

5. Run the module's two-support QP and check forces, moment and friction with declared tolerances. 6. Supply incompatible constraints and a deliberately short computation deadline: the system must enter the specified fallback mode. Check both the solver call and the total command-computation deadline; an individual solver's `time_limit` does not cover the whole loop.

Save `Q_noise`, `R_noise`, `Q_cost` and `R_cost` configuration, `Ts`, constraints and the check record in the README. Installation and preparation are included in the existing 75 hours.

Motion fundamentals: state and support forces

Required: what a sensor allows you to reconstruct

Joint state $x = [q, v]$ stores angle and velocity for prediction. One angle measurement does not give velocity; angle history and a known input allow it to be reconstructed. Before calculating, explain why the same q with different v produces different leg motion, and draw the ground forces on the body through two supports.

From measurement history to velocity

A fixed-base joint without friction or gravitational torque: $q' = v$, $Jv' = u$. q is in rad, v in rad/s, J in $\text{kg}\cdot\text{m}^2$ and u in $\text{N}\cdot\text{m}$ with the same positive direction as q . For constant u over interval T_s , the exact model is **$q1 = q0 + Ts\cdot v0 + Ts^2u/(2J)$** ; **$v1 = v0 + Ts\cdot u/J$** . Time, input and parameters must be known.

Let $J = 0.02 \text{ kg}\cdot\text{m}^2$, $T_s = 0.10 \text{ s}$ and $u = 0.04 \text{ N}\cdot\text{m}$; the sensor reports $q0 = 0.20$ and $q1 = 0.24 \text{ rad}$. The acceleration contribution to angle is 0.01 rad . Therefore $v0 = (0.24 - 0.20 - 0.01)/0.10 = \mathbf{0.30 \text{ rad/s}}$, and $v1 = \mathbf{0.50 \text{ rad/s}}$. A simple angle difference would give the mean velocity 0.40 rad/s , rather than $v0$ or $v1$.

When measuring q , the two observability rows are $[1, 0]$ and $[1, T_s]$; they are independent for $T_s \neq 0$. When measuring only v , both rows are $[0, 1]$: an arbitrary constant shift in q is invisible. A Kalman filter cannot create this missing information. With noise, reconstruction becomes estimation with uncertainty, making R , Q and timestamps essential.

From desired motion to admissible forces

Model two point supports in a right-handed frame: x forward, z upward, moment M_y about $+y$. From the centre of mass to the supports, $r1 = (-a, 0, -h)$, $r2 = (a, 0, -h)$. Contact forces f act **from the ground on the robot**. If horizontal forces are zero, $F_z = fz1 + fz2$ and $M_y = a(fz1 - fz2)$. These are force and moment sums; QP seeks an admissible allocation.

For $m = 2 \text{ kg}$, $g = 9.81 \text{ m/s}^2$, $a = 0.20 \text{ m}$, $F_z = mg = 19.62 \text{ N}$ and $M_y = 0.40 \text{ N}\cdot\text{m}$, obtain $fz1 - fz2 = 2 \text{ N}$, hence **$fz1 = 10.81 \text{ N}$, $fz2 = 8.81 \text{ N}$** . At $M_y = 0$, both forces are 9.81 N . A non-zero resultant moment specifies angular acceleration of the body; this is no longer static equilibrium. With a limit of 12 N per support, a request $F_z = 30 \text{ N}$ is infeasible because the total cannot exceed 24 N .

Support forces are not yet joint commands. Their generalised external action is $J^T f$; the actuator accounts for gravity and required dynamics, and opposes external action when holding. Check only force allocation here.

Exercises and verifiable results

1. In task 2, compare initial $q = 0.20$ and 0.70 rad with the same v and input. Save two angle and velocity histories: v measurements will agree, while q differs by 0.50 rad . **2.** In task 7, compare the manual forces with the solver and record sums and moment; then, in task 8, request 30 N with a hard equality and check infeasible. For the soft problem, check the residual rather than expecting the same status.

Time: 1 theory hour each from 7-8 (observability) and 61-62 (support forces); 1 practice hour each from 15-16 (task 2) and 66-67 (task 7). These calculations belong to existing tasks within the 75 hours; floating-base dynamics comes later.

Read: Lall, EE363, [Observability and state estimation, slides 3-11](#); Lynch, Park, [Modern Robotics §5.2: the relationship between forces and joint torques](#).

ROS 2 as a testable robot software system

M12 • 12 Sep 2027 - 14 Oct 2027 • 75 hours

This month's deliverable

You will combine individual models, drivers and controllers into a reproducible multi-node system. The final telemetry-and-control stack v1.0 includes custom interfaces where needed, specified QoS, separated callbacks, coordinate transforms at the correct time, a software simulator of the hardware with lifecycle management, diagnostics and replay of a recorded fault. This is the software foundation for the legs and hand; one leg and one representative finger in separate namespaces are sufficient for now.

Use this architecture: an IMU or joint data source publishes data; an estimator outputs state; a controller produces a bounded request; the hardware layer executes authorised commands; a monitor checks freshness and state; and a recorder saves the experiment. One component owns each physical resource. Activate control after checking readiness; the presence of a topic, a message publication channel, in the graph does not establish sensor or actuator readiness.

Environment and prerequisites

The main programme route uses ROS 2 Jazzy on Ubuntu 24.04. Packages support amd64 and arm64; on an Apple Silicon MacBook, use an ARM64 Linux environment. State the virtualisation method, network settings and device access. Test DDS participant discovery, USB passthrough to the guest, graphics and tracing in a small experiment: their operation depends on the virtual environment. If the graphical interface causes problems, first run the system from the console and record data, then state graphics and physical-hardware limitations separately.

Previous months must provide stable telemetry fields, units, monotonic measurement timing, controller and state-estimation models, and negative tests. Do not rewrite the mathematics when moving to ROS. Extract validated calculations into a library and call it from a small node so the test without ROS continues to work. Use C++ for execution-time measurement and the hardware layer, and Python for scenarios and analysis; a third language is optional.

Completion criteria

The system builds and launches from its documentation; the graph and interface ownership are clear; QoS and execution choices are justified by measurements. The simulator activates and deactivates correctly, and a recorded failure can be replayed. Prepare material for the architectural review and recording replay at checkpoint G2. Its integration hours are separate from these 75 hours. A simulator running in ROS does not close the separate HIL criteria involving real hardware. State every unconfirmed property in the report.

Organising the 75 hours

Four work blocks

Block A, hours 1-20: graph, messages and launch. Block B, 21-40: DDS, QoS and callbacks. Block C, 41-60: TF2, URDF and hardware-component lifecycle. Block D, 61-75: diagnostics, recording and replay. Each of the first three blocks contains 10 hours of theory and analysis and 10 hours of practice; the fourth contains 5 hours of analysis and 10 hours of practice. Total: $35 + 40 = 75$ hours. The work steps on pages 9-10 expand these same hours and require no additional time allocation.

Divide each ten-hour theory block into five two-hour sessions: 20 minutes reviewing the previous model, 45 minutes reading the relevant section, 40 minutes deriving a small example or examining a log, and 15 minutes recording the conclusion. Also divide the ten-hour Sunday practical session into five two-hour periods with breaks between them. Breaks are excluded from net study hours. Allocate the final five hours of analysis as $2 + 2 + 1$.

Start with the question, then choose the tool

The main environment is ROS 2 Jazzy on Ubuntu 24.04, colcon, rclcpp for C++ and rclpy for Python; also use rosbag2, ros2_control, tf2 and one plotting tool. Begin each session with a testable question: how does the input affect the output, which state cannot be observed, and which data reveal its changes? Before running, write down the expected sign, scale or sequence of events. Compare the prediction with the result and examine at least one discrepancy. Accompany every plot with an explanation of the observed relationship.

Maintain one repository with model, config, experiments, tests and reports directories. Every experiment needs a unique identifier, code version, configuration, parameter source, initial state, units and raw data. The analysis script reads a saved experiment and builds the report without manually correcting measurements. The final ROS 2 telemetry-and-control stack v1.0 project combines telemetry and control; individual exercises gradually form its test suite.

Time accounting and reducing scope

Calendar boundaries may coincide with an adjacent module. Keep rest days, holiday intervals and integration checks as specified in the overall calendar; place numbered steps in available time without counting hours twice. Do not add hours to this module. Assess equipment and library readiness at the outset rather than spending the practical session on an unplanned platform change.

If you fall behind, reduce the dashboard to four plots and retain one hardware simulator and one way of composing nodes in a process. Preserve the interface-ownership table, QoS experiment, callback-delay measurements, TF at measurement time, lifecycle check and automated fault replay. A second visualisation tool and porting nodes to Rust are optional. If tracing or a physical driver is unavailable, name the specific untested criterion; architectural validation on a simulator does not replace HIL.

Graph, interfaces and launch

Study how nodes interact

A topic suits an IMU or joint-state stream; a service suits a short configuration or state request; an action suits a long-running motion with feedback and cancellation. A parameter stores a setting but does not replace a reference stream on every cycle. Study nodes, topics, services, actions and parameters, then launch, namespaces and composition of nodes within one process. Composition changes data transfer and failure consequences; the number of logical components need not equal the number of processes.

First use standard `sensor_msgs/Imu` and `JointState` messages wherever they express the intended meaning accurately. A custom message is needed for fields missing from the interface description: command identifier, failure reason, state version or associated quality flag. Study conversion of `.msg`, `.srv` and `.action` to IDL, bounded arrays and strings, and serialisation. State units and coordinate frames for every quantity, and the time source for timestamp. Version semantic changes, as well as package names.

Task 1. Interfaces for one leg and finger

Inputs: definitions from M11, one leg with known joint names and one software finger simulator. 1) Draw a graph containing source, estimator, controller and monitor. 2) Create a field-ownership table. 3) Create an interface package and two minimal publishing and checking nodes. 4) Launch `/leg` and `/hand` in separate namespaces. Deliverable: an interface document, messages and a working example. Check: angles in rad, gyroscope angular velocity in rad/s and acceleration in m/s²; map joints by name rather than arbitrary array position. Error to detect: graph inspection reveals two publishers simultaneously setting the same reference configuration.

Task 2. Controlled launch and cancellation

Create a launch file for two rig instances, YAML parameters and an action for a small simulated joint motion. Handle goal, feedback, result and cancel; after cancellation, the simulator must enter the specified stopped state. Validate parameters before activation. Save one launch command and a cancellation scenario. Check: package resources can be found from another working directory; reject an invalid limit or unknown joint name with a clear reason. Endless respawn must not hide errors, and restarting must not resume motion on its own.

Separately compare nodes in different processes with components in one container. At this stage, measure correctness of names and data; assess performance gains from intra-process communication through measurements in the next block.

DDS, QoS and callback execution

Study delivery versus freshness

RMW connects the ROS API to middleware; DDS discovery finds participants, and a domain separates graphs on the chosen network. Record RMW_IMPLEMENTATION, ROS_DOMAIN_ID and network conditions. QoS includes reliability, durability, history and depth, deadline, lifespan and liveliness. Study compatibility between offered and requested profiles: incompatible nodes can appear in the graph without exchanging messages. Reliable mode does not guarantee that a command reaches the application fresh by its deadline.

For a sensor stream, freshness may take priority over complete delivery; motion commands need an explicit sequence number, age and retry policy. A new subscriber must not accidentally replay an old stored command because of transient_local. A deadline event reports a violation of expected periodicity, but the application implements stopping. History depth N limits the message queue but does not itself specify maximum delay. Check the combination of policy and actual behaviour of the chosen RMW.

Task 3. QoS profiles and message loss

A source publishes N numbered measurement packets at a known frequency. Create three compatible profile pairs and one deliberately incompatible pair; add a slow subscriber and a late start. Record transmitted, received, duplicate and missing sequence numbers, message ages and QoS events. Deliverable: a profile → observed behaviour table. Check: count missing packets by seq within one source and session; do not treat a restart as a huge negative counter jump. Error check: a bag recording made with incompatible recorder QoS cannot be considered complete.

Task 4. Callbacks that delay control

Implement a fast telemetry callback and deliberately slow processing or a service. Compare SingleThreadedExecutor, MultiThreadedExecutor with one mutually exclusive group, and a version with separate groups. In C++, measure callback start and end, queue waiting time and publication-to-callback-start delay on a common time scale. Save p50, p95, p99 and maximum distributions and a group diagram. Check: additional threads need not speed up callbacks in one mutually exclusive group; a blocking request from a callback must not cause an unnoticed deadlock.

Repeat the correct configuration with intra-process communication and state the message-transfer method actually supported. Compare copy counts and latency only for the measured configuration: do not treat zero-copy transfer as a property of every ROS2 exchange. Isolating computation helps, but does not itself turn a general-purpose operating system into a hard real-time system.

TF2, URDF/Xacro and the hardware lifecycle

Study coordinate transforms at the correct time

Describe the tree `base_link` → `imu_link` and `base_link` → `hip` → `knee` → `foot`; use separate names for the finger. TF2 stores timestamped transforms and answers queries for a selected instant. The latest available transform may differ from the transform at measurement time. Study static and moving coordinate frames, buffering, timeouts, extrapolation and handling of a missing link. An incorrect coordinate frame or timestamp can cause a spatial error that resembles a filter error.

In URDF/Xacro, distinguish visual, collision and inertial. Mass is in kg, centre of mass in metres and inertia in $\text{kg}\cdot\text{m}^2$; check the tensor's positivity and physical meaning. A detailed mesh may be used for visual, with a simplified mesh for collision. The joint axis and limits must match the CAD model and actuator. `ros2_control` provides hardware state and command interfaces; read and write connect them to the device. A hardware component's lifecycle and an ordinary node's lifecycle are related by the architecture but belong to different objects.

Task 5. Foot position at measurement time

Use the M5 geometry and a test joint trajectory. 1) Create a leg Xacro and a separate IMU coordinate frame. 2) Publish joint states and tree transforms. 3) Transform a foot point into `base_link` at the measurement time specified by timestamp. 4) Compare the result with a hand calculation and kinematics from earlier months. Save the URDF, frame tree and coordinate test. Check: a known zero configuration gives the expected values in metres; a late measurement requests a historical transform. If a transform is missing or stale, produce a diagnosable failure rather than a zero point.

Task 6. Hardware simulator: preventing premature commands

Create a minimal `ros2_control` hardware component with position state and command interfaces for selected joints. Configure checks names and limits; activate enables commands after obtaining an initial state; deactivate puts the simulator in the specified mode. Read returns synthetic feedback; write applies limits and updates a counter. Save the package, controller manager configuration and transition test. Check: no motion is executed before activation; a read or write error triggers the expected state and event. The simulator validates interfaces and logic; check the real driver, delays and physical stopping separately.

Diagnostics, tracing and fault replay

Study tools for observing the software

A log explains an individual event; a diagnostic status describes component state; a metric shows behaviour over an interval; tracing reveals execution order and duration. Monitor IMU and joint-data freshness, hardware-layer state, controller errors, missing-message counters, command age and recording load. Every warning must contain a reason, timestamp and recovery condition. A permanent “OK” line does not provide this information.

Rosbag2 records messages and their timing information. Save parameters, code versions, random-number seeds, hardware events and external clocks separately. Reprocessing a recording does not guarantee the same thread-scheduling order. For a reproducible experiment, fix the initial state, use `use_sim_time`, `/clock` source and input-order rules. Events sharing a timestamp may require a custom seq or a deterministic handler in the test.

Task 7. Recording, dashboard and causal sequence

Record IMU data, joint states, estimates, commands, diagnostics and the required TF. Save parameters and a version manifest. In Plotjuggler or Foxglove, create one dashboard: reference and angle, IMU age, hardware-layer state and controller error. Then delay the source and trace sensor data stale → monitor detects → command blocked. Deliverable: a bag recording, dashboard configuration and a brief report. Check: the original measurement sequence number and every transition are visible; the event does not disappear when switching plots. Compare source and recording counters to assess bag completeness.

Task 8. Automated fault replay

Create a `launch_testing` test: start the system with the simulator, wait for readiness, replay a fixed bag and verify the fault event. Add a timeout and terminate all processes. Separately enable `ros2_tracing` for one callback-delay case if the Linux environment supports the chosen tracing mechanism; state environment limitations. Check the exit code, result contents and system state after the test. If a node fails to become ready or hangs during shutdown, the test must fail rather than wait indefinitely.

Define the required repeatability: matching diagnostic events and values within tolerance, or an identical sequence of processed messages. Do not call a recording deterministic merely because its file is unchanged. State the controlled sources of nondeterminism and any remaining differences in delays measured by physical clocks.

Requirements for the ROS 2 v1.0 system

Packages and responsibilities

robot_interfaces contains only messages, services, actions and unit descriptions. robot_description stores Xacro and meshes. robot_core contains calculations without ROS dependencies; robot_nodes calls them from estimator, controller and monitor nodes. robot_hardware provides the hardware simulator; robot_bringup stores launch and YAML; robot_tests stores integration scenarios; experiments stores version manifests and recordings. Packages may be combined if these responsibility boundaries and the absence of cyclic dependencies are preserved.

Choose one flow: mock_joint/imu → estimate → control_request → hardware. The monitor receives independent freshness indicators and permits or prohibits activation. The command interface has one owner; the diagnostic dashboard does not bypass it to write commands. For every graph edge, record message type, units, QoS, frequency, time source and stale-data behaviour. Namespaces allow a second hand instance to launch without changing source code.

Aligning time

Within a process, measure durations with a monotonic clock. A ROS message timestamp provides a shared time reference and supports TF; across multiple machines, declare the synchronisation method or restrict one-way latency measurement to one machine. Do not subtract clocks from different devices and call the difference transport latency without estimating their offset. Separately measure publication period, callback start, callback duration and data age when a command is calculated.

Changes to system time or /clock must not silently change freshness monitoring. Choose virtual time in advance for replay, and a defined stable time base for hardware freshness checks. When replay pauses, controller behaviour must match the selected mode. Indefinitely repeating an old physical command is unacceptable. Test replay on the simulator using a separate namespace and configuration.

Reproducibility and documentation

One command builds the workspace, another starts the simulator and a third runs checks. State dependencies, tested OS and architecture, terminal procedure and expected topics. The report contains the graph, QoS table, lifecycle diagram, callback measurements, recorded failure and a test link. Building from a clean directory checks paths to installed resources, rather than accidental absolute paths from your computer.

Dashboard snapshots provide a quick overview, but the bag, manifest and calculated metrics remain the primary evidence. The project rests on clear responsibilities: who produces data, who makes decisions and who may operate the actuator.

Six integration and fault tests

Delivery and execution

F1 - incompatible QoS. Launch a publisher and subscriber with conflicting reliability or durability requirements and demonstrate the missing expected exchange. Expected: a diagnostic explanation rather than “the sensor is broken”. Then fix the profile and repeat the same experiment with N packets, saving counters.

F2 - slow callback. Artificial processing takes longer than the source period. Expected: queue size and data age grow, or messages are lost according to the profile; measure p99 and the maximum. Explain how executor and callback-group choices affect the result. An average frequency close to the target does not confirm data freshness.

F3 - source restart. The source resets seq and temporarily disappears. Expected: a new session_id distinguishes restart from counter overflow or packet reordering; freshness monitoring detects the pause; data returning does not automatically activate the device. Record recovery state as a separate event.

Geometry, hardware layer and replay

F4 - TF from the future or past, and an incorrect coordinate frame. Supply one measurement packet outside the available buffer and another with an unknown frame_id. Expected: the failure reason is visible, and no command is generated from an unverified transform. In the corrected case, a known foot point agrees with an independent calculation within the declared tolerance.

F5 - configure, read or write error. Supply an invalid joint limit, then simulate a hardware error in the active simulator. Expected: premature activation is prohibited, the cause is diagnosed, and controllers do not silently keep writing to an invalid interface. Fix the specific sequence of controller and hardware-component actions according to the installed ros2_control version and the test.

F6 - an incomplete bag and a shutdown hang. Remove a required stream or stop one node before the scenario ends. Expected: the replay test detects the missing event or data and ends at its timeout; all processes terminate. A repeated run inherits neither old nodes nor an accidental /clock from the previous test.

Confirming a correction

Each negative test has a corrected counterpart with identical initial conditions. Record the injected error, observed event, detection delay, final state and path to the bag or trace. If the cause remains unknown, leave the status open. Do not substitute increases to every timeout for a correction: any new time limit must be justified by measurements and control requirements.

Virtual-machine special case: participant discovery may depend on network mode, while tracing depends on the kernel and access to system mechanisms. Do not hide these environment limitations under “everything checked”; still complete console and simulator checks, and prepare a separate rerun plan for any unavailable hardware check.

Hours 1-40: the graph and delivery behaviour

Hours 1-20: interfaces and launch

Theory 1-10: 1-2: graph and responsibility; 3-4: topics, services and actions; 5-6: parameters and IDL; 7-8: launch and namespaces; 9-10: node composition within a process and the motion-cancellation plan. These are five two-hour sessions; end each by recording an interface decision for the leg and its reason. Do not study every ROS package at once.

Practice 11-20: 11-12: build a minimal Jazzy workspace and check networking and architecture; 13-14: complete task 1; 15-16: connect nodes to the M11 library; 17-18: complete task 2; 19-20: launch two namespaces and check invalid-parameter and cancellation handling. Save the environment configuration and exact commands for clean reproduction.

Before proceeding, check that you have a graph, an owner for every interface, a working launch and correct cancellation of simulated motion. No hidden dependency on the current directory or an incidentally open terminal should remain.

Hours 21-40: QoS and the executor

Theory 21-30: 21-22: RMW, DDS, domains and discovery; 23-24: reliability, durability and history; 25-26: deadline, lifespan, liveliness and profile compatibility; 27-28: executors and callback groups; 29-30: definitions of measured delays and intra-process communication. Record your prediction for a slow subscriber before running the experiment.

Practice 31-40: 31-32: compare the profiles in task 3; 33-34: add a late subscriber and restart; 35-36: complete task 4; 37-38: compare groups, processes and intra-process communication; 39-40: plot distributions and choose a profile for each stream. Record both the selected option and a rejected configuration with its reason.

Before proceeding, verify that QoS is chosen for each stream's requirements, data age is measured correctly, and deadlock and queue accumulation can be reproduced. Performance claims apply to the measured OS and RMW.

Hours 41-75: geometry, simulator and recording

Hours 41-60: coordinate frames and hardware layer

Theory 41-50: 41-42: TF tree and axes; 43-44: time, buffer and extrapolation; 45-46: URDF/Xacro, masses and collision geometry; 47-48: `ros2_control` state and command interfaces; 49-50: configure, activate, deactivate and errors. End every session by checking one concrete fact: joint sign, metric dimension, timestamp or authority to write a command.

Practice 51-60: 51-52: assemble the leg description; 53-54: complete task 5; 55-56: implement the minimal simulator in task 6; 57-58: connect controller manager and check transitions; 59-60: inject TF, read and write errors and document the diagram. A complex visual model is unnecessary in this block: geometrically correct simple shapes are preferable to an unverified detailed mesh.

Before proceeding, verify a known foot point numerically, check that the hardware simulator does not execute a premature command, and confirm that errors appear in diagnostics. Do not separately claim physical readiness.

Hours 61-75: observation and fault replay

Analysis 61-65: 61-62: `rosbag2`, `/clock` and state during replay; 63-64: diagnostics, logging, tracing and `launch_testing` criteria; 65: compile the final list of supporting results. Split the last five hours as 2 + 2 + 1; preparing for G2 here means collecting results of checks already completed.

Practice 66-75: 66-67: complete task 7; 68-69: save and explain a recorded failure; 70-71: complete task 8 with timeout and process termination; 72-73: run the six negative scenarios; 74-75: clean build, replay and report. If tracing is unavailable, include callback timestamps, an explanation of the environment limitation and instructions for repetition on a suitable Linux system.

Before proceeding, verify that system v1.0 can be reproduced and explains one actual software failure. The material is ready for architectural review and recording replay at G2; the checkpoint's other criteria are accounted for separately.

Acceptance and preparation for a digital twin

Demonstrating the working stack

From a clean environment, build the packages, launch two namespaces and show IMU and joint data flow and action cancellation. Enable a slow callback and explain what happens using metrics. Finish by replaying a fault bag and checking that the automated test detected the expected event. Show RViz, counters, hardware-layer state and the absence of processes left running.

Required acceptance checks: build and launch are reproducible; units and data ownership are documented; QoS is compatible and tested; TF uses measurement time; the simulator handles a fault correctly and enters the specified state. The recording contains required streams, repeatability criteria are explicit, and automated tests terminate on both success and failure. Starting a node alone does not confirm correct command execution.

Self-check

1) When should you use an action instead of a service? 2) Why do parameters not replace a command stream? 3) How does incompatible QoS appear in the graph? 4) Why does reliable not mean fresh? 5) How does deadline differ from application-level freshness monitoring? 6) Why do multiple threads not rescue one mutually exclusive callback group?

7) How do you obtain a transform at measurement time? 8) Why should visual and collision differ? 9) Who owns the command interface? 10) What happens after a read or write error in your particular stack? 11) Which data outside the bag are needed for repetition? 12) Why can the same bag run with different delays? Support answers with configuration, a log or a test.

Handover to M13 and the G2 boundary

Pass on the leg Xacro, joint names, hardware interfaces, launch, QoS, bag template and fault catalogue. This is the framework for a digital twin in Gazebo: the model will replace the hardware simulator while preserving external interfaces and their requirements. Initial quadruped geometry in M13 will extend one validated leg; a physical quadruped is not yet required.

Architectural and replay material is ready for G2, but the overall plan also includes a one-leg HIL criterion developed in M13. This criterion remains open until an experiment with real hardware; a software simulator does not satisfy it. Reaching the G2 calendar date does not justify assigning PASS to every item.

If behind schedule, reduce the dashboard to four plots and retain one hardware simulator. Preserve QoS and execution-time measurements, TF at measurement time and a reproducible failure. If physical-interface validation or tracing is deferred, state the environment, command, expected file and specific untested criterion.

Assigned reading: a focused route through the sections

The references are mainly in English; keep the plan and definitions in English. Read selected sections before the task rather than whole books. Core reading is included in the existing 35 hours of analysis; consult references as needed.

Core reading

E. Renard, ROS 2 from Scratch (2024). [Book and contents](#). Chapter 3 Uncovering ROS 2 Core Concepts; in Chapters 5-9, only sections on custom interfaces, action mechanisms, YAML and launch configuration. For tasks 1-2, list requirements for a stream, a request and a cancellable action. Use Chapters 10-12 selectively for TF and URDF.

K. Lynch, F. Park, Modern Robotics (2017). [Book and teaching sections](#). §3.2.1 Rotation Matrices and §3.3.1 Homogeneous Transformation Matrices. For task 5, construct camera → body → world and its inverse by hand; this checks the meaning of TF rather than repeating the kinematics course.

Reference reading

M. van Steen, A. Tanenbaum, Distributed Systems, 3rd ed. (2017). [Authors' contents](#). In Chapters 3 Processes, 4 Communication and 8 Fault tolerance, select explanations of threads, communication and partial failures. For tasks 3-4 and 8, distinguish delivery, execution, timeliness and recovery; the entire textbook is unnecessary.

Three practical references

[ROS 2 Design: QoS](#) - DDS Quality of Service policies and QoS profiles; task 3, compatibility table. Check values and actual behaviour against Jazzy.

[Jazzy: Using time in tf2](#) - Background and Tasks; task 5, requesting a transform at measurement time and handling its absence.

[ros2_control: Hardware Components](#) - lifecycle and state and command interfaces; task 6, one resource owner and permission to activate.

After reading, answer for your stack: who owns the command, when does a measurement become stale, and what stops actuation after a fault? See page 14 for installation, recording and additional tools.

Terms and definitions

Node. A logical ROS component with its own interfaces; several nodes may execute inside one process.

ROS graph. The collection of nodes and their communication links discovered within a specified domain and accessible network.

Topic. A named, typed message stream; publication generally does not acknowledge command execution by a subscriber.

Service. A request-response interaction for a bounded operation; an action is more suitable for a long motion with cancellation.

Action. A protocol for a long-running task with feedback, a result and a cancellation request that the server must handle.

Parameter. A named node setting with a type and change rules; it does not replace a sequence of control commands.

Namespace. A ROS-name prefix that separates subsystem instances; it does not itself provide network isolation.

DDS and RMW. DDS implements data exchange; the RMW layer connects ROS to a specific middleware implementation.

QoS. A set of message-delivery and storage policies; offered and requested profiles must be compatible for communication.

Reliability. A best effort or reliable delivery policy; retransmission does not guarantee message freshness by the control deadline.

Durability. A policy governing availability of previously published data to late subscribers; transient local requires care for motion commands.

Deadline. The expected maximum interval between updates; a violation event reports a problem and the application implements the response.

Executor. A mechanism that selects ready callbacks and executes them on available threads according to group configuration.

Callback group. A group of callback functions governed by mutual exclusion or permission for concurrent execution.

Deadlock. A state of mutual waiting for resources or events in which participants cannot continue execution.

Timestamp. The numerical time of an event on a specified time scale; measurement time may differ from message-receipt time.

TF2. A buffered tree of coordinate transforms that supports queries for relationships between frames at a specified time.

Lifecycle. Explicit component configuration, activation, deactivation and error states, with permitted transitions between them.

Hardware interface. A contract for reading a resource's state and writing its command; control authority must belong to a defined owner.

Replay. Resupplying recorded data with known timing and configuration; the original computation order may not be reproduced.

What to buy and install this month

USE EXISTING EQUIPMENT

Use the MacBook Pro M4, UTM and Ubuntu 24.04 ARM64 installed in M1, Git, CMake, C++ and Python. A software simulator of the hardware is sufficient for M12; no new onboard computer, motor or sensor is needed. Boards and measurement tools from M4-M7 will be needed later for HIL.

INSTALL NOW

In the Ubuntu guest, install ROS 2 Jazzy Desktop with RViz, colcon, rosdep, vcstool, `ros2_control` and `ros2_controllers`, `tf2`, `robot_state_publisher`, Xacro and `rosbag2`. Start with the [official Jazzy installation guide](#). Record arm64 architecture, package versions, RMW and `ROS_DOMAIN_ID`; do not mix distributions in one shell or workspace.

First run the [ros2_control RRBot demo](#) with simulated hardware. Keep the C++ or Python controller library independent of ROS. For the interface package, keep [IDL: Type system](#) available; define array sizes, strings and units before building.

AS NEEDED

For diagnostic tasks, use [Plotjuggler](#) or an existing graphical analyser. Install [ros2_tracing](#) if Linux supports the chosen mechanism and permissions are available; otherwise save callback measurements and state the tracing limitation. Add [launch_testing](#) to launch processes with a timeout.

INSTALLATION CHECK

1) Build two minimal nodes with colcon, check talker and listener and discovery on the chosen network. 2) Open RViz with the leg TF; test UTM graphics in practice. 3) Activate and deactivate the hardware simulator without spontaneous startup. 4) Record and replay `rosbag2` with timestamp and seq, checking QoS compatibility. Test USB passthrough to the guest before M13 rather than assuming every virtual machine supports it.

Preparation and these checks replace repeated setup and reading within the current 75 hours. An installation page does not mean the software is already installed. Buying a licensed copy of the book is optional if access is available; there are no hardware purchases this month.

Motion fundamentals: the physical meaning of URDF

Required: four descriptions of one link

Which torques will the leg shown in RViz require? Kinematics specifies connections, axes and transforms; visual specifies appearance; collision specifies collision and contact geometry; inertial specifies mass, centre of mass and inertia. Before starting, transform one point between two coordinate frames and explain the distinction between kg and kg·m².

robot_state_publisher computes TF from geometry and joint positions without integrating the effects of forces. ros2_control defines command and state interfaces. The name effort does not turn a position servo into a controllable torque source.

One bar: geometry and inertia from dimensions

Base B is fixed in a right-handed frame: x forward, z upward. The joint at the origin rotates about +y. At $q = 0$, the link points along +x; positive q lowers its endpoint: **p_tip in B = (L cos q, 0, -L sin q)**. For $L = 0.20$ m, $q = 0$ gives (0.20; 0; 0) m and $q = \pi/2$ gives (0; 0; -0.20) m. Mass does not enter this calculation.

A uniform bar has length $L = 0.20$ m along the link's x-axis, width and height $d = 0.02$ m, and $m = 0.30$ kg. The link-frame origin is at the joint; its centre of mass in this frame is (0.10; 0; 0) m. Place the inertia frame at the centre of mass with parallel axes:

$$I_{xx} = m(d^2 + d^2)/12 = 0.000020 \text{ kg}\cdot\text{m}^2;$$

$$I_{yy} = I_{zz} = m(L^2 + d^2)/12 = 0.001010 \text{ kg}\cdot\text{m}^2.$$

Off-diagonal elements vanish by symmetry. In inertial, specify this tensor about the COM and the centre-of-mass origin; also offset the bar's visual and collision by $L/2$.

Inertia about the joint axis is **I_joint = I_yy + m(L/2)² = 0.004010 kg·m²**. The added term accounts for circular COM motion. Substituting I_joint for I_yy while origin is at the COM counts the axis transfer twice and distorts dynamics.

At $q = 0$, gravity (0; 0; -mg) N acts at $x = 0.10$ m. Its moment about +y is $+mgL/2 = +0.2943 \text{ N}\cdot\text{m}$; holding requires actuator torque $-0.2943 \text{ N}\cdot\text{m}$. At $q = \pi/2$, the force lever arm is zero. The sign follows from $\mathbf{r} \times \mathbf{f}$.

Doubling mass without changing shape doubles inertia and gravitational torque; TF at a given q is unchanged. Changing visual changes appearance. Enlarging collision geometry changes contact and clearances but does not establish link strength. The bar checks units; real masses and material distributions come from measurements or CAD.

Two model checks within hours 41-60

1. Select a test link in the leg description; save its URDF and p_tip from manual FK and TF at $q = 0$ and $\pi/2$. The example tolerance is 10^{-6} m for this model. **2.** Double the mass in a copy; use a script to check doubled I and torque and unchanged FK. Save mass, COM, the six I elements and coordinate frames; principal moments of inertia must be positive.

Time: 1 theory hour from 45-46 (URDF, masses, collision geometry), and 1 practice hour each from 51-52 (leg description) and 53-54 (task 5, TF). These check the existing model within the 75 hours. Contact comes in M13; no real actuator commands are sent here.

Read: [Lynch, Park, Modern Robotics §8.2, Part 1](#) - COM and inertia; [ROS urdf tutorial, 07-physics.urdf](#) - visual, collision and inertial structure. Do not copy placeholder inertias from XML: they are calculated here.

A digital twin that can be checked against measurements

M13 • 26 Oct 2027 - 21 Nov 2027 • 75 hours

Monthly outcome

Build a quadruped model in Gazebo, verify it against simple physical examples and connect the controller in stages: first SIL, then PIL and single-leg HIL with a real processor and communication channel. Deliver a digital twin with reports on fidelity, timing and HIL testing. The quadruped is an intermediate architecture on the way to a six-legged robot: physical assembly is scheduled for M22, and the transition to six legs for the conditional M23 stage.

A digital twin must reproduce a specific geometry and use parameters and measurements from the actual system. Start with the leg CAD model, actuator specifications and M7-M10 logs. Identify the sources of mass, inertia, damping, limits, delays and noise. An unmeasured parameter may be assumed if it is explicitly labelled and given a range for sensitivity checks. A calculated curve must never be presented as an experimental measurement.

Prerequisites

From M12, you need Xacro, joint names, hardware interfaces, launch files and diagnostics. From earlier stages, you need controller firmware, the communication protocol, a secured single-leg or joint test rig, and rules for limiting applied commands. Before HIL, understand every packet field and the meaning of its timestamps. The model can be developed without an actuator; checking hardware and actual delays requires a real test rig.

The main environment is ROS 2 Jazzy, Ubuntu 24.04 and Gazebo Harmonic with gz_ros2_control. ARM64 installation is supported, but verify the particular plugins, graphics, GPU-processed sensors, virtual networking and device access separately. Running a server without a GUI and rendering depth images without a window are different modes: the latter still requires a working rendering engine. Any replacement of missing depth data with ideal values must be explicitly declared.

What the evidence establishes

Check contact, noise and timing quantitatively. Successful simulation does not establish actual strength, heating, surface friction or emergency stopping. HIL validates operation only within the declared hardware and software configuration; the physical actuator and foot contact remain separate experiments when represented by models. The G2 HIL criterion remains open until the corresponding experiment in this module is completed, even though the checkpoint itself is scheduled earlier.

Organising the 75 hours

Four work blocks

Block A, hours 1-20: SDF, physics and simple reference cases. Block B, 21-40: the quadruped, sensors and tests without a graphical interface. Block C, 41-60: SIL and PIL, scenarios and timing boundaries. Block D, 61-75: single-leg HIL and comparison with measurements. Each of the first three blocks contains 10 hours of theory and analysis and 10 hours of practice; the fourth contains 5 hours of analysis and 10 hours of practice. Total: $35 + 40 = 75$ hours. The work steps on pages 9-10 allocate these same hours; do not add them again.

Divide each ten-hour theory allocation into five two-hour sessions: 20 minutes to reconstruct the previous model; 45 minutes to read the assigned section; 40 minutes to derive a small example or analyse a log; and 15 minutes to record the conclusion. Divide each block's ten-hour Sunday practical into five two-hour work periods with breaks between them. Breaks do not count towards net study hours. Split the final five analysis hours as $2 + 2 + 1$.

Start with a question, then choose the tool

The main tools are Gazebo Harmonic, ROS 2 Jazzy and `gz_ros2_control`; Python for scenarios and analysis; and the existing microcontroller and physical communication channel for HIL. Begin each session with a testable question: how should the input affect the output, which state is unobservable, and what data can establish this? Before running the experiment, record the expected sign, scale or event sequence. Afterwards, compare the prediction with the result and investigate at least one discrepancy; producing a plot alone does not complete the task.

Maintain one repository with model, config, experiments, tests and reports directories. Every experiment needs a unique identifier, code version, configuration, parameter sources, initial state, units and raw data. The analysis script must read a saved experiment and generate its report without manual corrections to measurements. The monthly deliverable is a Gazebo digital twin with fidelity and HIL reports; every exercise adds to its verification suite.

Time accounting and reducing scope

Calendar boundaries may overlap with the neighbouring module. Rest, holiday periods and integration checks remain as specified in the overall roadmap; place the numbered steps in available study intervals without counting hours twice. No additional hours are assigned to this module. Assess equipment and library readiness at the start, rather than spending the practical on an unplanned platform change.

If you fall behind, reduce visual mesh detail and the number of worlds, and omit the second physics platform. Retain analytical reference cases, contact verification, the four-leg structure, a sensor reference scene and a dynamic single-leg experiment. Separate SIL, PIL and HIL results and a table of discrepancies against measurements are mandatory. Do not add gait generation or RL yet. If the target processor or physical rig is unavailable, leave that item open and describe its verification procedure; another run on the host computer does not replace it.

Verify the physics model before adding the robot

Study: the world and the solver

Study [SDF world/model/link/joint/sensor](#), the physics step, gravity, contact and friction coefficients. Identify the selected [physics engine](#) and its available parameters; tag names do not guarantee identical interpretation across engines. Mass, inertia and collision geometry determine dynamics, while visual defines appearance. A detailed visual foot may differ from a simple contact geometry, provided the contact model is explained.

Add joint limits, damping and an actuator model. Ideal torque control, ideal position control and an actuator with an internal control loop and delay produce different results. First test a simple mass and a pendulum: their simulations can be compared with analytical calculations. Choose the physics step by considering convergence and computational cost. A smaller step will not correct an erroneous mass, negative inertia or incorrect axis direction.

Task 1. Analytical reference cases

Create a freely falling body without contact and a small-angle pendulum. For free fall, compare with $z(t)=z_0+v_0t-gt^2/2$ before impact; for the small-angle pendulum, compare its period with $2\pi\sqrt{l/g}$, provided the model is a point mass on a massless link. Repeat with steps dt and $dt/2$. Deliver error curves and a table showing how the metric changes when the step is reduced. Check units of m, s and kg, identical initial conditions and a fixed duration. Error check: if pendulum inertia does not match the chosen formula, correct the reference calculation before adjusting solver settings.

Task 2. Foot contact and an inclined plane

Use a simple block with the mass and contact geometry of a foot. Place it on a horizontal surface, then vary the inclination. Record penetration, sliding velocity, normal force and energy change. Compare the onset of sliding with the teaching approximation $\tan(\alpha)\approx\mu$ for the idealised model; explain the effects of the solver, damping and contact shape. Deliver a scenario series with tolerances chosen before the final comparison. Check that a block with zero friction cannot remain stationary on an incline without another cause; any qualitative change when the step is reduced needs an explanation.

The block deliverable is a set of simple cases with expected physical behaviour and at least one reproducible contact-model failure. Only then add dozens of links and sensors.

The quadruped model and sensors

Study: repeatable geometry

Assemble a body and four instances of the verified leg using Xacro. Joint names and axes must match M12; mirroring the left and right sides requires a separate sign check. Before exporting to simulation, verify total mass, centre of mass, joint origin/axis, permitted travel, collision geometry and transmission ratio. Connect command and state interfaces through `gz_ros2_control` following the Jazzy/Harmonic conventions. Declaring a transmission does not by itself guarantee that the plugin applies its reduction ratio.

Add an IMU, joint-state and foot-contact sensors, and a depth sensor. For each, specify its coordinate frame, units, update rate, noise, delay and ground-truth source. The simulator publishes ground truth on a separate stream for evaluation; the controller receives only permitted sensor readings. Ground-truth leakage distorts the assessment of algorithm accuracy. Check IMU tilt using the adopted gravity convention, and depth against a plane at a known distance.

Task 3. Turn one leg into four

1) Verify one leg in three known configurations. 2) Compare the link endpoint position with an independent kinematic calculation. 3) Create the remaining instances with their mirroring parameters. 4) Command the same intended movement on all legs and compare directions. Save Xacro, SDF, launch files, a table of joints and interfaces, and a geometry test. Check the mass sum against the parts list, physically meaningful inertias and that the body does not fall through the floor. If a mirrored leg moves in the wrong direction, the test must identify the specific joint.

Task 4. Sensor reference scenes without a GUI

Create a world with a horizontal platform, a wall at a known distance and a fixed robot position and orientation. Run the server without a GUI, wait for the streams and record IMU, contacts, joint states and depth. Prepare an automatic test with a timeout and a report on frequencies, noise and distance error. A disabled sensor, unavailable rendering engine or empty depth map must produce an explicit error or be recorded as a limitation. Do not fill a missing stream with zeros. Compare ground truth and readings at aligned timestamps; model delay separately from publication frequency.

For the arm, transfer only the architectural principle: parameterised links, joint limits, correct contact geometry and separate sensor-reading and ground-truth streams. A complete arm and tactile models are outside this month's scope.

SIL and PIL: where each component runs

Study: experiment boundaries

In SIL, the controller runs on the host computer and controls a mechanical model in a closed loop. In PIL, its code runs on the target processor and exchanges data with the model; this checks the build, numerical types and calculations on the selected hardware. PIL can run step by step, slower than real time. HIL in the next block also checks the physical communication channel and the declared closed-loop timing requirements. Terminology varies between tools, so every report diagram must show the actual distribution of components.

For each scenario, specify the random-number seed, versions, initial robot configuration and controller state, world parameters, physics step, noise model, injected fault and expected result. A fixed seed improves repeatability but does not guarantee bitwise equality for a multithreaded physics engine across operating systems. Distinguish simulation time (sim time) from real elapsed time (wall time). Real-time factor = sim time increment / wall time increment; this ratio does not directly describe whether an individual packet arrives on time.

Task 5. SIL with the same controller

Use the M10/M11 controller library without copying its equations into a new node. 1) Run a fixed single-leg trajectory. 2) Record error, commands, saturation and contact states. 3) Repeat three times with the same seed. 4) Introduce sensor delay and additional friction. Deliver a scenario catalogue and discrepancy report. Check that identical inputs produce results within the declared numerical tolerance and that the fault scenario survives a restart. If callback order affects the result, save seq and fix the data-input order in the test.

Task 6. PIL on the microcontroller

Build the computational core for a real target processor, such as the board from earlier modules. The host sends a snapshot of the leg state; the processor calculates a command and returns seq/status. In stepwise mode, the model continues only after the specified response. Save the protocol, firmware hash and a target-versus-host comparison of commands for identical inputs. Differences between float and double precision must remain within a tolerance justified in advance; reject NaN and overflow. A damaged packet or missing seq must not result in data being accepted as a new valid snapshot.

After comparing functionality, measure target computation time and round-trip time separately. Do not subtract readings from unsynchronised clocks: measure the round trip on one side and local computation with the processor timer. A PIL result does not yet establish timely control of a physical actuator.

Single-leg HIL and comparison with real mechanics

Study: HIL requires real hardware

The minimum HIL experiment in this programme uses a real controller with firmware and a physical USB/UART/CAN channel. They exchange state and command data with the leg model in a closed loop under timing requirements defined in advance. The controller acts on the model, whose response returns to the controller. If all computation runs on a laptop, the experiment is SIL. If the board merely receives and returns a packet without calculating control or affecting the model, the experiment tests communication but does not yet demonstrate HIL control.

Declare the period, permitted data age, late-data handling and stopping method in advance. Simulation on the host must meet the chosen timing mode; if performance is insufficient, record failure of the timing criterion. A slow stepwise run is useful for PIL but does not satisfy timely HIL. Choose the threshold from your loop's timing budget, rather than a universal number in an instruction sheet.

Task 7. Close the hardware loop for one leg

Use the verified PIL protocol and connect the firmware to the leg model through a physical channel. 1) Run a small, safe trajectory with bounded commands. 2) Record sim time, host receive/send, target compute duration and seq. 3) Introduce dropped packets, delay and a disconnected line. 4) Check the watchdog, fallback mode and resumption through an explicitly defined state. Prepare an HIL report, firmware, component diagram and fault log. Every cycle must correspond unambiguously to a state snapshot; reject expired commands. The actuator and contact are modelled here, so this experiment does not establish physical stopping of the leg.

Task 8. Connect the model to a physical test rig

On a secured real leg or joint, repeat one previously authorised small M10 test and compare position, delay, settling time and available load information with the model. Use the same command and consistent initial conditions. Estimate parameters from one set of recordings and validate on another. Deliver a simulation-versus-measurement table and a list of discrepancies. Check that you do not fit a single damping value to align every curve while concealing an incorrect delay or internal actuator-loop model. If the physical rig is unavailable, prepare the procedure and leave the corresponding validation open.

Use the command limits and mounting arrangements from the already verified mechanics. Do not connect a real actuator to a virtual environment in a way that lets two controllers set the same command simultaneously. Before the experiment, explicitly select model-only, HIL-controller or physical-bench mode.

Digital-twin specification and scenario catalogue

Project structure

Store geometry and interfaces in `robot_description`. The simulation directory contains SDF worlds, physics and sensor settings, and the bridge; scenarios contains initial states, inputs and criteria; firmware contains the target-processor build; `hil_bridge` contains the protocol and timeouts; fidelity contains model and test-rig data; reports contains comparisons and limitations. Record the Git revision, Jazzy/Harmonic versions, engine, operating system and architecture, physics step, seed and actuator model in the common manifest.

The mandatory world catalogue contains an analytical reference case, foot contact, a stationary quadruped with sensors and a single-leg test scene. A complete gait is not needed yet. The quadruped model must have correct axes, limits and sensor streams, while the dynamic experiment may use one leg with a fixed body. This reduces the number of simultaneously unknown causes of discrepancy.

Accuracy requirements by subsystem

For geometry, compare lengths and coordinates in known configurations; for mass and inertia, compare CAD data and available measurements; for the actuator, compare delay and transient response; for contact, compare sliding and penetration. Check IMU noise, frequency and delay; depth-sensor distance and pixel dropout; and communication losses, data age and missed deadlines. Each table row must give the parameter source, criterion, result, limitations and next physical experiment.

Do not assign one number such as "digital-twin accuracy 95%". Position error in millimetres and timing error in milliseconds describe different properties. Set tolerances according to the question the model will answer: interface testing, step-size selection, controller tuning or contact. A model may be suitable for watchdog testing and unsuitable for predicting foot slip.

Automating a run

The launch script must open the world, wait for components to become ready, reset the state, execute the scenario, record streams, check criteria and terminate every process. Account separately for launch failure, missing clock or sensor, and timeout. Run tests without a GUI first; check rendering availability separately for depth. Save a machine-readable pass/fail result and a plot that makes its cause understandable.

Define the HIL boundary in every report: which signals are physical, which are generated, where the control law runs and how time is measured. A photograph of the board and wiring helps reproduction, but the evidence consists of commands, responses and the closed-loop model's reaction to an active hardware controller.

Testing the model under faults

Physics and sensors

F1 - Incorrect inertia or axis. Change one entry and check a known configuration or a small input. Numerical geometry or dynamics checks must detect the discrepancy before complex motion. Give the corrected model file a new version; visual resemblance to the robot does not compensate for a failed check.

F2 - Unstable contact. Increase the step, reduce friction or change the foot contact geometry. Expect to detect penetration, sliding or energy growth; repeating with a smaller step and the original geometry should explain the source. Record the physics engine and parameters, since the same SDF can behave differently in another implementation.

F3 - Delayed IMU data and an empty depth map. The sensor supplies old timestamps or rendering fails to produce a valid map. The controller or test must detect stale or missing data; ground truth must never silently replace readings. Specify the frequency of valid messages and the fraction of missing or non-numeric values; explain the total used as the denominator.

Execution and the hardware boundary

F4 - Loss of reproducibility. Repeat one seed after changing the thread count or platform. Compare defined metrics and tolerances rather than claiming unconditional bitwise equality. If contact events differ, isolate a minimal scenario and state which conclusions are materially affected by that instability.

F5 - Data corruption or loss of physical communication. Drop a packet in the HIL stream, alter its CRC or disconnect the line using the method specified in the test. An invalid command must be rejected and data age must increase; the hardware controller and host must record a consistent cause and transition to fallback mode. Specify which side checks CRC and which side decides what to do if communication fails in one direction.

F6 - The simulator cannot keep up. Add computational load or a sensor and measure real-time factor together with data age and deadline compliance. Expect failure of the timing criterion to be recorded even with a normal average RTF. A powerful computer and $RTF \approx 1$ do not guarantee freedom from isolated long delays.

Decide how to address each failure

Distinguish a model defect, an interface defect, an environment limitation and missing physical data. Change one factor when correcting a failure. Repeat the reference scenario and preserve the original failure as the basis of a regression test. If a discrepancy remains unexplained, add it to the unresolved fidelity issues with a specific measurement that will distinguish possible causes.

"Simulation is imperfect" is too general to support an engineering decision.

Hours 1-40: model physics and sensors

Hours 1-20: simulation foundations

Theory 1-10: 1-2 - SDF structure and the selected engine; 3-4 - mass, inertia and integration; 5-6 - contact and friction; 7-8 - joint limits, damping and actuator model; 9-10 - analytical reference cases and convergence criteria. End each of the five two-hour sessions with a prediction of a specific physical quantity.

Practice 11-20: 11-12 - run a minimal Harmonic world; 13-14 - complete Task 1; 15-16 - repeat with two steps; 17-18 - complete Task 2; 19-20 - save the contact failure and reference configuration. Do not spend the ten-hour practical building a complex robot before the simple cases pass.

Progression criteria: free-fall and pendulum results agree with the chosen analytical calculation, contact has numerical criteria, and the physics step is selected from changes in results and computational cost.

Hours 21-40: model and sensors

Theory 21-30: 21-22 - Xacro and mirroring; 23-24 - inertial/collision and transmissions; 25-26 - gz_ros2_control interfaces; 27-28 - IMU, contacts, depth and ground truth; 29-30 - operation without a GUI, component readiness and time bases. Before installing additional packages, establish that a specific sensor needs them and that the selected environment supports them.

Practice 31-40: 31-32 - verify one leg; 33-34 - complete Task 3 with four legs; 35-36 - connect the sensors; 37-38 - complete Task 4 without a GUI; 39-40 - test an incorrect axis and a missing depth map, and record the manifest. If graphics are unavailable, retain working physics and IMU tests and separately mark depth verification as incomplete.

Progression criteria: the quadruped exists as a verifiable model, joint names match the ROS stack, ground truth is separated from sensors, and missing data is detected explicitly.

Hours 41-75: SIL, PIL, HIL and fidelity

Hours 41-60: the controller inside the loop

Theory 41-50: 41-42 - SIL, PIL and HIL boundaries; 43-44 - state reset and seed; 45-46 - simulation time, wall time and RTF; 47-48 - target numerical types, protocol and seq; 49-50 - timing measurements without mixing clocks. Draw the path of one state snapshot and command, identifying the component responsible for each state.

Practice 51-60: 51-52 - run SIL from Task 5; 53-54 - repeat the experiment and add delay and friction; 55-56 - build the target computational core; 57-58 - run PIL from Task 6; 59-60 - compare host and target calculations, and separately measure computation and round-trip times. Use the verified control law with its previous settings.

Progression criteria: SIL and PIL are reported separately, numerical differences are explained and exchanged data is recorded. Stepwise execution is not presented as a real-time test.

Hours 61-75: the hardware loop and model fidelity

Analysis 61-65: 61-62 - HIL requirements, watchdog and response deadline; 63-64 - comparison of simulation with measurements and subsystem tolerances; 65 - final readiness table. The last theory block consists of 2 + 2 + 1 hours. Keep prepared scenarios within the rig's already verified safe limits.

Practice 66-75: 66-67 - close the Task 7 HIL loop on a real target processor; 68-69 - test physical communication, delay and disconnection; 70-71 - complete Task 8 on an available secured rig or record that physical verification was not completed; 72-73 - prepare the fidelity report and check faults; 74-75 - reproduce the project from a clean start and prepare the handover. Another host-computer run does not replace missing HIL and does not confer the same readiness status.

Progression criteria: the report gives separate statuses for the model, PIL, HIL and physical comparison. Close the open G2 HIL criterion only with the corresponding hardware experiment; subsequent models inherit the measured limitations.

Acceptance checks, limitations and further work

Demonstrate model suitability

Begin the demonstration with an analytical test and a leg-coordinate check, then show the sensor reference world without a GUI. Next, demonstrate one SIL scenario, PIL computation on the target processor and HIL with a physical communication channel. Identify the real and modelled robot components in each experiment. Finish by comparing a physical recording with the model and listing properties that this twin cannot yet predict.

Acceptance requires geometry, inertia and limit checks; recorded contact and timing violations; separate sensor readings and ground truth; a GUI-free test with a timeout; a manifest; a host-versus-target comparison; an HIL log; and unresolved fidelity issues. If hardware is unavailable and HIL or physical comparison has not been completed, record partial readiness and the specific open item. A hil directory or a connected USB cable does not itself establish that the criterion is met.

Self-check questions

1) How does visual differ from collision and inertial? 2) How can you tell whether the physics step is small enough? 3) Why does identical SDF not guarantee identical contact in a different engine? 4) How do you check the axis of a mirrored leg? 5) How does sensor noise differ from delay? 6) Why must ground-truth state be withheld from the operational state estimator?

7) How does a server without a GUI differ from camera rendering without a window? 8) Where does the controller run in SIL, PIL and HIL? 9) Why does $RTF \approx 1$ not establish compliance with every deadline? 10) How can you measure communication delay without synchronised clocks? 11) What does HIL with a modelled actuator verify, and what does it leave unverified? 12) Why should digital-twin fidelity be described by subsystem? Answer using your own tests and hardware boundaries.

Handover and selection of physical experiments

Pass the worlds, sensor models, actuator settings, scenario catalogue and list of permitted uses to later stages. For example: the model is suitable for interface and delay testing, has limited suitability for tuning small movements and is not yet validated for impact contact. Plan separate tests of heating, backlash, foot forces, friction on the actual surface and emergency-response delay; these cannot be inferred solely from a gait simulation.

If time is short, reduce visual mesh detail, the number of worlds and use of the additional MuJoCo engine. Retain the four-leg structure, one thoroughly verified dynamic leg, sensor reference scenes and the hardware loop. Do not add a new gait algorithm, RL or a complete arm model to these 75 hours. Turn model errors into specific engineering tasks with an expected measurement, rather than a general doubt about the simulator's usefulness.

Assigned reading: a focused route through the sections

The literature is mainly in English; the programme and definitions are also in English. Read the selected sections before the relevant task, rather than entire books. Core reading is included in the existing 35 analysis hours; consult references as needed.

Core reading

E. Renard, ROS 2 from Scratch (2024). [Book and contents](#). Ch. 13 Simulating a Robot in Gazebo: How Gazebo works, Adapting the URDF for Gazebo, Spawning the robot in Gazebo. Before Tasks 3-4, understand the relationship between model, world and plugin. Use commands appropriate to Jazzy/Harmonic.

K. Lynch, F. Park, Modern Robotics (2017). [Learning resources](#). §8.1.3 Understanding the Mass Matrix, §8.2 Dynamics of a Single Rigid Body and §8.5 Forward Dynamics of Open Chains. Before Tasks 1 and 8, check inertia, the justification for the pendulum reference case and the meaning of forward dynamics.

R. Tedrake, Underactuated Robotics, MIT notes. [Multi-Body Dynamics](#). Sections The Dynamics of Contact and Compliant Contact Models. Before Task 2, establish the relationship between penetration, stiffness, friction and integration step; calculate the condition for the onset of sliding.

Implementation references

[SDFormat specification](#) - model/link/inertial, joint/axis, world/physics and sensor. In Tasks 1-4, record the schema version and check which fields the selected engine uses.

[Gazebo Harmonic: Sensors](#) - IMU sensor and Contact sensor. For Task 4, study frequency, coordinate frames, plugins, noise and separation of sensor readings from simulation ground truth.

[gz_ros2_control Jazzy](#) - Add ros2_control tag, demos and Notes on the command interface. In Tasks 3 and 5, check which quantity the interface commands: torque, velocity or position.

What to record in the laboratory log

Give a source for every parameter: CAD, specifications, measurement or an assumption with a range. After reading, identify one numerical physics test and one boundary it does not establish. HIL requires a real controller and channel; conclusions about mechanics require a separate physical experiment.

Terms and definitions

Digital twin. A model of a specific system tied to its geometry, parameters and measurements within a declared domain of applicability.

SDF. A format describing simulation worlds, bodies, joints, physics and sensors; tag semantics depend on the supported schema.

Inertial parameters. A body's mass, centre of mass and inertia tensor, expressed relative to a specified coordinate frame.

Collision geometry. Geometry used to detect collisions and contacts; it may differ from the model's detailed visual surface.

Forward dynamics. Calculation of accelerations from state, forces and constraints; an integrator then updates the state over time.

Integration step. The increment of simulation time between numerical updates; it affects error, stability and computational cost.

Step-size convergence. Stabilisation of a selected result as the step is progressively reduced; checked with identical conditions and duration.

Contact. Interaction between surfaces that come together, involving normal and possibly tangential forces; contact can allow sliding.

Normal force. The contact-force component perpendicular to the surface; in a unilateral contact model, the surface pushes but does not pull.

Friction coefficient μ . The dimensionless ratio of limiting tangential force to normal force in the adopted model; it depends on materials and contact regime.

Penetration. Calculated overlap of contacting surfaces; in a compliant model it determines force together with stiffness and damping.

Compliant contact. A model in which force depends on deformation and its rate; high stiffness places stronger demands on the integrator.

Ground-truth state. The model's true internal state, used to calculate errors; the operational state estimator must not receive it.

Seed. The initial state of a random-number generator; it fixes the random sequence, but not every effect of parallel execution.

SIL. A closed-loop software experiment in which a host-computer controller acts on a model and receives its response.

PIL. An experiment using controller code on the target processor and a plant model; coordinated stepwise execution is permitted.

HIL. A closed-loop experiment with a real controller and channel, checking explicitly declared hardware boundaries and timing requirements.

Real-time factor. The ratio of simulation-time and wall-time increments; it does not describe command latency.

Round-trip time. The time for an outward-and-return exchange measured using one clock; without further assumptions it does not give one-way delay.

Model validation. Checking predictions against independent measurements for a specified task; it does not establish applicability in every operating regime.

Equipment and setup for this month

Use existing equipment

MacBook Pro M4, Ubuntu 24.04 ARM64 in UTM and the Jazzy environment from M12; ESP32-S3, a USB/UART/CAN channel, analyser and power supply from earlier modules. HIL requires a real microcontroller (MCU) that calculates control, and a real channel. The M7 bench motor remains suitable for repeating joint measurements.

Purchase after verifying the selected actuator

A separate physical 3DOF leg requires compatible actuators from the selected family. If one actuator has already passed torque, speed, heating, feedback, protocol and shutdown checks, buy another 2, making 3 in total. If there is no verified candidate, first buy and test 1; order the next 2 only after it passes acceptance. The bench DC motor does not automatically count as that candidate.

For the approved leg CAD model and BOM, prepare 1 set of printed links, fasteners, bearings and wiring to the final geometry, plus 1 restraining fixture. Use the existing 3D printer with a profile for its exact model. Do not yet buy complete sets of 12/18 actuators or an onboard Pi/x86 computer. Single-leg readiness is established by measurements, not the G2 date.

Install now

In the same Ubuntu guest, install Gazebo Harmonic, `ros_gz/bridge`, `gz_ros2_control` and the demos. Follow the [official ROS/Gazebo pairing](#) and [jazzy plugin installation](#); use `ros-jazzy-gz-ros2-control`. Do not mix Harmonic with Gazebo Classic. Retain ESP-IDF, firmware and the protocol from M6-M12.

Check the installation and its limitations

Run the cart demo, check joint states and position changes, then verify free fall and the pendulum with dt and $dt/2$. Separately check [depth-map rendering without a GUI](#): a server without a window does not establish that depth frames exist. In UTM, check USB exchange and measure dropouts and round-trip time; if timing cannot be maintained, leave the HIL criterion open.

Simple HIL with a microcontroller and model does not require a physical leg, but it does not validate mechanics either. Purchasing three actuators serves the separate physical experiment in Task 8. Preparation fits within the 75 hours in place of repeated setup; delivery time is outside study hours. Power on new actuators only after matching the supply, limits and mounting.

Motion fundamentals: contact and model fidelity

Required: why touching the ground does not ensure support

A foot can touch the floor while sliding, and a contact that appears rigid can still allow penetration. Before moving on to the leg, check these effects with a simple body. Before calculating, resolve gravity into normal and tangential components and explain why static friction need not equal μF_n .

A minimal friction model on an incline

A block of $m = 1$ kg neither rotates nor tips. The plane is stationary at angle α ; the local t-axis points downhill and n points outwards. $g = 9.81$ m/s². $F_n \geq 0$ and F_t are the forces exerted by the plane on the block, in N; acceleration along n is zero: **$F_n = mg \cos \alpha$; $m a_t = mg \sin \alpha + F_t$** . Friction coefficients are dimensionless.

Rest requires $F_t = -mg \sin \alpha$ and $|F_t| \leq \mu_s F_n$, where μ_s is the coefficient of static friction. During downhill sliding, **$F_t = -\mu_k F_n$** , where μ_k is the coefficient of kinetic friction. The normal force does not oppose motion along the slope; the boundary of static equilibrium here is $\tan \alpha = \mu_s$.

Let $\mu_s = 0.40$ and $\mu_k = 0.30$. At $\alpha = 20^\circ$: $F_n = 9.218$ N, $mg \sin \alpha = 3.355$ N, and the friction limit is 3.687 N. Thus $F_t = -3.355$ N is admissible, and a block initially at rest remains stationary. The actual friction is smaller than its limiting value.

At $\alpha = 30^\circ$: $F_n = 8.496$ N, the downhill force is 4.905 N, and the limiting static-friction force is 3.398 N. Rest is impossible; during sliding, $F_t = -2.549$ N and **$a_t = 9.81(\sin 30^\circ - 0.30 \cos 30^\circ) = 2.356$ m/s²**. Friction dissipates mechanical energy: $F_t \cdot v_t \leq 0$. The wrong sign would cause energy to grow.

In the spring model, $F_n = k\delta$, where k is stiffness in N/m and δ is penetration in m. At rest on a 20° incline with $k = 10\,000$ N/m: $\delta = 9.218/10\,000 = \mathbf{0.000922$ m, or **0.922 mm**. This is consistent with the adopted compliance; a zero visible gap is not an accuracy criterion.

Increasing k changes the physical model and raises the oscillation frequency; reducing the step Δt changes numerical accuracy. Compare Δt and $\Delta t/2$ with identical mass, friction, stiffness and solver. If the engine uses a different contact law, first map its parameters: $\delta = F_n/k$ may not apply.

Two experiments within existing Tasks 1-2

1. Start the block from rest at 20° and 30° ; save forces, velocity and acceleration after the transient. Verify rest or sliding, the sign of F_t and the numbers above. If the engine uses a single μ , set $\mu_s = \mu_k$ and recalculate the prediction; record the settings explicitly.

2. For 30° , compare two steps over the same interval of established sliding. Save mean a_t , F_n , penetration, kinetic energy K and potential energy U . The teaching target for a_t is a difference from the analytical value and between steps of $\leq 2\%$ of 2.356 m/s² with the original μ_s and μ_k . Check that K may increase as U decreases; total mechanical energy, including the contact spring, decreases through friction.

Time: 1 theory hour from 5-6 (contact), plus 1 practice hour each from 15-16 (two steps) and 17-18 (Task 2). These calculations and checks are included in the 75 hours. The block validates the selected contact regime; the complete leg and physical HIL require their own checks.

Read: Lynch, Park, [Modern Robotics §12.2.1: Friction](#); Tedrake, [Multi-Body Dynamics → Compliant Contact Models](#).

Monthly outcome: the robot measures the visible scene

M14 • 21 Nov 2027 - 19 Dec 2027 • 75 hours

Build a reproducible RGB-D processing pipeline: use colour images and depth to determine the geometry of a floor, a step and an object on a table. For a walking robot, this will provide heights and information about unknown regions ahead of a foothold; for an arm, it provides the position of a surface and a potential grasp object. The M14 deliverable contains calibration files, reference scenes, classical and learned methods, measurements of geometric error and latency, and a failure log.

What you need to understand before starting

Review coordinate transforms, rotation matrices, the 4×4 homogeneous matrix, measurement noise, NumPy, testing and ROS 2 recording from earlier modules. Check your understanding with a small task: transform a point from the camera frame into the body frame and back; the recovered point must match within the chosen numerical tolerance. Separately explain how a pixel differs from a metre and why the RGB timestamp may differ from the depth measurement time.

Working setup and boundaries

The main option is recorded RGB-D scenes or a simulator with known geometry; your own camera is useful but optional. Create three scenes: a flat surface, a box representing a step, and an object for a future grasp. TUM data is suitable for testing loading and geometry; prepare separate "support" and "obstacle" labels for your tasks. Do not attribute semantic annotations to the original dataset if it does not contain them.

Use Ubuntu 24.04 and ROS 2 Jazzy from previous months. Use Python for experiments, OpenCV and Open3D for geometry, and PyTorch for a small learning experiment. If necessary, port only a stage whose high computational cost has been demonstrated by measurement to C++. On the Mac, continue using the existing Linux environment. For every result, identify the CPU, architecture and accelerator used.

What understanding looks like

You can show how an incorrect depth scale distorts the estimated box height, and how a low error on training images can conceal poor calibration. Distinguish a recognised "floor" from geometrically verified support. Full SLAM, step selection and a real closed-loop walking controller belong to later stages. In M14, processing outputs describe only the observed scene and data reliability.

75 hours: work sequence and time accounting

Four work blocks

Hours 1-20: camera geometry, calibration, units and time; 10 hours of analysis and 10 hours of practice. Hours 21-40: colour, edges, features, RANSAC and tracking; 10+10 hours. Hours 41-60: point clouds, filtering, normals, ICP and heights; 10+10 hours. Hours 61-75: CNNs, transfer learning, comparison and acceptance; 5 hours of analysis and 10 hours of practice. Total: 35 hours of theory and analysis and 40 hours of practice, with no hours added beyond the programme.

Divide theory in each of the first three blocks into five 2-hour sessions; divide the final 5 hours as 2+2+1. Organise practice into four Sunday blocks of 10 hours, with breaks outside study time. A practical block may be split across available intervals with the same total duration. Roadmap calendar boundaries may overlap between neighbouring modules; retain rest, holidays and separate integration checkpoints without counting hours twice.

One experiment, one testable version

At the start of a session, record a question such as "How does voxel size affect estimated step height?" Then identify the parameter to change, the fixed input dataset and the expected sign of success. At the end, save the configuration, dataset hash, plot and failure explanation. An experiment that changes calibration, image size and network weights simultaneously cannot establish the effect of one factor.

Organise the project as follows: calibration/ - K, distortion and procedure; datasets/manifest - data provenance and splits; geometry/ - depth processing; perception/ - masks; tests/scenes - reference scenes; reports/ - metrics. Store large raw recordings separately from code, preserving exact names, checksums and acquisition instructions. The entire dataset must be reprocessed with one command, without manually selecting successful frames.

Working as an experienced developer

Read until you can answer the current question, then move to measurement. For each new algorithm, work through a few points by hand, build a small test and only then process a full recording. "It did not work" is insufficient: identify the stage, input, version and symptom. If package installation exceeds its allotted time, replay a prepared recording and reduce the experiment's scale; retain calibration and unit checks.

The camera: from pixel to geometric ray

Topics to study

Study the pinhole model: $u = f_x \cdot X/Z + c_x$, $v = f_y \cdot Y/Z + c_y$. Coordinates X, Y, Z are in metres in the optical frame; u, v and f_x, f_y, c_x, c_y are in pixels. Matrix K describes the intrinsic parameters. Radial and tangential distortion alter observed pixels; the extrinsic transform relates the camera to the body. Do not confuse lens calibration with the camera's pose on the robot.

Study stereo depth using $Z = f \cdot b/d$: focal length f is in pixels, baseline b in metres and disparity d in pixels. At small disparity, the same pixel error produces a larger depth error. Compare two distances with the same error in d : a distant surface is less suitable for precise foothold selection. For RGB-D, understand scale, channel alignment and the representations of missing or invalid depth values. Hand-eye calibration relates camera and link motions: study $AX=XB$ and the eye-in-hand/eye-to-hand distinction. M14 requires a measurement scheme and understanding of the necessary extrinsics; a separate solver is not required.

Task 1. Check the camera model

Input: synthetic 3D points in front of the camera and a specified K . Project the points, recover X, Y from known Z values, then test the image centre, doubling Z and a point behind the camera. Output: a projection-and-reconstruction round-trip test and a drawing of the rays. For synthetic data, specify a relative numerical tolerance, such as 10^{-6} , as a computational requirement. $Z \leq 0$ and missing depth must cause rejection rather than produce a point at the origin.

Task 2. Calibrate for the working field of view

Use at least 15-20 varied chessboard images: include image edges, different tilts and distances. If you have no camera, use published OpenCV images; document square size as either a known value from the source or an arbitrary unit. Split images by capture pose into a parameter-fitting set (fit) and a held-out set. Find and refine internal corners, estimate K and distortion parameters, and save a list of excluded images with reasons.

Check by overlaying reprojected points, calculating RMS pixel distances and reporting each image's errors separately. On a held-out image, estimate only the board's position and orientation while keeping K and distortion fixed. Repeat using only central views and compare edge errors. Document all checks; one final number is insufficient. A small RMS does not establish correct physical scale or the correct camera-to-body transform.

Classical vision: a baseline and its limits

Sections to cover

BGR/RGB and HSV colour spaces; smoothing, gradients and Canny; binary masks, morphology and connected components; local features and matching; geometric models and outliers. Separate the signal from the decision: an image edge is not the edge of a step expressed in metres. A homography is sufficient for a textured planar surface; one homography cannot describe an arbitrary 3D scene.

Task 3. Find an object and a surface edge

Input: 30 images from three scenes, with a manual object mask and marked edge. Build an HSV mask, then an edge-based alternative; fix thresholds using the training portion. Remove small components and save the object contour and bounding rectangle. Output: masks, overlays and a detection-results table. Evaluate IoU = intersection area / union area on the same dataset, and the fraction of images with no detection.

Change lighting, add a similarly coloured object, then add a shadow on the plane. Do not adjust a threshold to fix one image after inspecting the test set. A false object detection beside the arm and a missed edge ahead of a leg have different consequences; annotate these cases in the report. Separately test an empty mask, an entirely white mask and an object cropped by the image boundary.

Task 4. Robust matching and tracking

Find features in a pair of images of a flat board, match descriptors and estimate a homography using RANSAC. Introduce a controlled fraction of deliberately incorrect pairs. Save the number of matches and model-consistent matches (inliers), the geometric error of reference points and an "insufficient data" flag. Test a degenerate, nearly collinear configuration and a repeated pattern. Do not accept a matrix simply because the function returned one.

Next, run Lucas-Kanade on selected corners in a short video: compare forward and backward motion and reject unreliable point tracks. Save the tracks and a plot of the number of points that remain tracked. Blur or switching off the light must reduce confidence in the estimate. Tracking establishes correspondence between images; without geometry and depth, it does not yet provide metric body velocity. Save original images and the RANSAC seed so that repeated runs are comparable.

Point clouds: the surface in the robot frame

What to study and check by hand

For an aligned depth map: $Z = \text{depth}/\text{scale}$, $X = (u - c_x) \cdot Z/f_x$, $Y = (v - c_y) \cdot Z/f_y$. First exclude missing and invalid values. Transform into the body frame with $p_{\text{body}} = R_{\text{body_camera}} \cdot p_{\text{camera}} + t_{\text{body_camera}}$. Record the direction of every transform and check which inverse is required. Optical axes differ from body axes, so check labelled basis vectors and a point directly in front of the camera.

Filtering includes cropping to the working region, voxel-grid downsampling and outlier removal. A normal describes the local surface direction; neighbourhood radius affects results near an edge. For a plane $n \cdot p + d = 0$, normalise n to unit length so that the residual is in metres. ICP reduces correspondence error with a suitable initial estimate, but does not guarantee correct registration of every scene.

Task 5. From depth to a plane and a step

Use a synthetic scene with a known box height, then a real recording. Convert depth to points, crop the area in front of the body and compare three voxel sizes. Use RANSAC to extract a support plane, estimate normals and build a height grid with an observation count per cell. Output: a point cloud, plane.yaml, a height map, an unknown-cell map and height error over reference regions.

Check that synthetic height is recovered within a tolerance chosen beforehand; for recordings, use a measured reference and its uncertainty. Mark empty cells unknown; do not substitute zero. Add 10% random outliers and a region of missing depth at the edge. Compare median absolute height error and the fraction of cells without an estimate. Choose the threshold in metres to reflect scene scale and sensor noise.

Task 6. Register adjacent observations

Create a pair of clouds with a known translation and rotation. Run point-to-point and then point-to-plane ICP from identical initial conditions. Report transform error, correspondence fraction, residuals and time; classify small residuals with an incorrect displacement as a failure. Repeat on a single plane, in a symmetric corridor and with a poor initial estimate. Explain how incorrect registration would shift an object for the arm and create a false support height for the leg.

Learned vision: a small comparison experiment

The minimum theory for the week

Study convolution, receptive fields, feature maps, backbone weights and the output head. Learn training and inference, train/eval modes, transfer learning, learning rate and overfitting. The aim is to design an experiment without developing a large network. Compare methods on the same task and annotations: for example, segment a specified class of object on a table or a region of flat surface.

Image classification, bounding-box detection and pixel segmentation produce different outputs. Choose a compact segmentation model with a pretrained backbone and fine-tune only a small head, or use a ready-made detector for a supported class. In the latter case, perform a separate transfer-learning experiment on a small classifier of cropped image patches; do not describe an unchanged ready-made model as fine-tuned. Limit annotation of your own scenes to 60-100 temporally separated images.

Task 7. Train without data leakage

Split data by scene or capture session rather than randomly distributing neighbouring images. Initial split: train 60%, validation 20%, test 20%. Save each file's assignment and check overlaps using hashes and timestamp proximity. Apply augmentations only to train; transform images and masks consistently for geometric changes. Do not flip a scene vertically unless those conditions are physically possible in the intended setting.

Limit head-training time, save the best version according to the validation set and fix the threshold. Prepare weights, preprocessing parameters, the training log and manifest. Check train/eval modes on the same input; during inference, results must not depend on test-image order. Explicitly define behaviour when the required class is absent or the image is corrupted.

Task 8. Quality, confidence and latency

Compare results with Task 3 on the test set using precision, recall and IoU. State the counting unit: a pixel or a detected object. For five confidence bins, tabulate mean model confidence and the actual fraction of correct results; note the small sample size. Measure classical and learned method latency over the same complete processing segment. A CNN may improve recall yet fail on a new material; reflect this in the report. Permit unknown outside the studied conditions so uncertainty does not become a robot command.

Integration laboratory: surveying terrain and a table

Specification

The replay command accepts an RGB-D manifest, calibration.yaml, frames.yaml and method parameters. For each image it outputs the measurement timestamp, frame_id, a valid-depth mask, a plane, a height grid, an object mask, diagnostic status and processing time. Add a small ROS 2 message adapter to the existing project. The computational core must process recordings without a connected robot. Preserve the original timestamp; do not replace it with processing-completion time.

Assembly sequence

1. Load an RGB/depth pair and check size, encoding and scale.
2. Match timestamps within a specified tolerance and reject unpaired images.
3. Apply the correct calibration; update K consistently when image size changes.
4. Generate points and transform them into the body frame.
5. Compute geometry and the semantic mask independently.
6. Combine results in a report while preserving reasons for unknown values.
7. Check reproduction with one command.

Task 9. Verify what the outputs mean

In scene A, the robot looks at a level floor; in B, a box is ahead; in C, the camera observes an object on a table. For each, define 5 reference regions with known roles: floor, box top, edge, a region without depth data and background. Compare height, normal and the fraction of valid measurements. Output: a region table, overlays and an explanation of where the network disagrees with geometry. Do not allow a "floor" mask to remove a geometric obstacle automatically.

For future foothold selection, propose only candidate regions: visible flat cells with estimated heights. Leg reachability, load-bearing capacity, friction and stability are not checked here. For the arm, propose the observed object surface; grasp orientation and collision freedom will be checked in M17. This establishes which conclusions the data supports.

What to include in the deliverable

Save a README with commands, environment and library versions, input data or acquisition instructions, calibration and the coordinate-frame chain, a table of all scenes, numerical metrics, at least six failure or boundary cases and usage limitations. Add a short replay video labelled with time and coordinates. Check results against raw arrays: video compression may conceal dropped frames, so video does not replace the log.

Error checks: eight mandatory scenarios

A set of controlled faults

S1 - Incorrect depth scale: multiply distances by 1000 while retaining the settings. A range check or comparison with reference geometry must detect the error; record the stage that detects it. S2 - Incorrect K after resizing: halve the image size while keeping the original K. Reference height or projection must reveal a discrepancy. Restore a consistent transform and save both results.

S3 - Time offset: pair RGB and depth with offsets of 0, 1 and 3 frames in a moving scene. Measure mask-boundary error relative to the cloud and the number of rejected pairs. S4 - Missing depth at a potential foothold: zero depth in a square beneath the future leg position; the output must contain unknown. Do not increase map completeness by interpolating across an unknown edge without labelling that interpolation separately.

S5 - Geometric degeneracy: feed ICP a single plane translated along itself. Compare position-and-orientation error with the residual; explain the lack of constraints. S6 - Changes in lighting and material: compare the classical method and network on a fixed test set. Show false detections, misses and model confidence. Either method may lose the comparison, provided experiment parameters were fixed beforehand.

S7 - Data leakage: deliberately add a copy of a training image to test; the automatic manifest check must fail before quality metrics are calculated. S8 - Overload: add delay to network inference, then inspect counts of received, processed and discarded images. Publishing an old map must preserve its age and diagnostic status rather than present it as fresh.

Units and metrics

Reprojection error is the RMS of Euclidean distances in pixels; do not mix it with mean norm/N from another example. Height error is the absolute difference in metres at reference points. Measure computation time using the end and start readings of one monotonic timer; data age requires aligned clocks. Report median, p95, maximum and image count. Measure file loading and warmed-up network inference separately.

Documenting a failure

Each scenario stores its original configuration, the single introduced change, the expected observation and the actual outcome. If a check misses S1 or S4, refine data requirements before improving the CNN. With a physical camera, separately record lighting, mounting and reference conditions: a synthetic tolerance does not automatically become an acceptable error for the real robot.

Sessions 1-40: the camera and classical method

Hours 1-10: five 2-hour sessions

1-2: understand K, projection and units; calculate the centre and one off-axis point by hand, then work on Task 1. 3-4: radial and tangential distortion and the meaning of internal board corners; prepare the capture procedure. 5-6: extrinsics, optical frame and body-frame transform; check three axes. 7-8: depth, stereo baseline, scale and uncertainty at two distances. 9-10: reprojection criterion, image splits and expected Task 2 errors.

Hours 11-20: first practical, 10 hours

11-12: prepare 15-20 images and a manifest; check provenance and usage rights. 13-14: find and refine corners, then visually check their order. 15-16: calibrate and estimate the held-out board's position and orientation; save parameters. 17-18: plot residuals by image and compare central-view and varied-view sets. 19-20: prepare recording reprocessing, a README and the first report. By the end of the block, calibration must be reproducible from saved data.

Hours 21-30: five 2-hour sessions

21-22: colour and preprocessing; explain the effect of BGR/RGB on HSV. 23-24: edges, morphology, connected components and IoU. 25-26: features, descriptors and nearest neighbours; record indicators of incorrect matches. 27-28: RANSAC and the geometric model; draw an example with outliers. 29-30: optical flow, violations of brightness constancy and a track-rejection criterion. Prepare a specific input for the next practical during every session.

Hours 31-40: second practical, 10 hours

31-32: annotate reference scenes and implement the Task 3 HSV and edge baselines. 33-34: evaluate quality, fix thresholds, change lighting and add a similar object. 35-36: complete Task 4 with features, homography and outliers. 37-38: track points through a short sequence, check forward/backward consistency and failure when images are lost. 39-40: repeat the dataset with a fixed seed, check output units and prepare failures.md.

Even if you already know a topic from software development, check it in the robotics context. Use the time saved for an additional reference viewpoint and failure explanation. Do not add a separate web interface: an error plot and an image with labelled axes are more useful at this stage.

Sessions 41-75: 3D, the network and the final package

Hours 41-50: five 2-hour sessions

41-42: XYZ reconstruction and masks for missing and invalid values; check 3 pixels by hand. 43-44: voxels, outliers and normals; compare the scales needed for a floor and a small object. 45-46: RANSAC planes, normalised residuals and height grids. 47-48: ICP, initial estimates, point-to-point/point-to-plane variants and degeneracy. 49-50: reference selection, transform error, applicability limits and acceptable processing latency.

Hours 51-60: third practical, 10 hours

51-52: Task 5, a synthetic cloud and a recording with known scale. 53-54: plane, height map and unknown; compare three voxel sizes. 55-56: Task 6 with a known displacement and two neighbouring images. 57-58: test a plane without texture or structure, an incorrect transform and areas without depth. 59-60: assemble the geometry part of Task 9, the report and a fixed interface for the next month.

Hours 61-65: theory 2+2+1

61-62: CNN features, backbone and head, train/eval modes and transfer learning; select a small model. 63-64: session-based data splits, augmentation, precision/recall/IOU and confidence calibration. 65: formulate the comparison hypothesis, training-time limit and stopping criterion. Check that the data contains object or floor labels. If not, choose a class you can annotate correctly.

Hours 66-75: fourth practical, 10 hours

66-67: Task 7, manifest data splits and minimal head fine-tuning. 68-69: Task 8, a fixed test set, classical-versus-network comparison and five confidence bins. 70-71: Task 9, a combined terrain-and-table report and stage timing. 72-73: scenarios S1-S8, leakage checking and recording reprocessing. 74-75: acceptance and handover to M15. If training does not finish within the allocated time, state that limitation; preserve the baseline and checks.

Reducing scope if you fall behind

First remove a second network architecture, then detailed ICP tuning and porting to C++. Retain one learned baseline, a comparable pair of geometric methods, calibration, the cloud, the height map and the full failure suite. Save time by reducing dataset size and the number of parameters; retain a separate test set. Rust is only an optional exercise beyond the required deliverable here, without a third implementation of the pipeline.

Acceptance checks and the transition to localisation

A verifiable monthly deliverable

Include K , the distortion model, image size, physical or arbitrary board scale, original images, a held-out set and errors by viewpoint in the calibration package. Specify time, scale and pairing rules in the RGB-D manifest. Define transform directions and units for the frame chain; check forward and inverse conversion. Saved data must be sufficient to reproduce setup without verbal explanations from its author.

The pipeline builds a cloud, extracts a plane and object, estimates heights, preserves unknown and emits diagnostics. It includes at least one classical and one learned baseline, a shared test set and comparisons of quality and latency. Scenarios S1-S8 must be reproducible, rather than only described. Report limitations of the real sensor if one was used; clearly separate synthetic results from physical measurements.

Self-check without notes

1. Why is f_x measured in pixels, and what changes when image size changes?
2. How does a scale error differ from reprojection error?
3. Why must zero depth not be treated as a floor point?
4. How do you check the direction of $T_{\text{body_camera}}$?
5. When is a homography valid, and when is it not?
6. Why can ICP with a small residual give an incorrect position and orientation?
7. How does voxel size affect a step edge?
8. Why do neighbouring images in training and test sets create leakage?
9. What does high recall with low precision mean for the arm?
10. How does model confidence differ from the actual probability of error?
11. Why must a measurement timestamp not become a publication timestamp?
12. What additional checks are needed before allowing the real robot to place a foot in a selected cell?

What to pass to M15

Pass on short synchronised sequences, camera parameters, the coordinate-frame chain, ICP settings, reference displacements and descriptions of degenerate cases. Preserve reasons for lost point tracks and sensor dropouts: they become localisation test scenarios. Explicitly identify artificially corrected trajectories and excluded failed images.

The final demonstration takes 5-7 minutes: one normal scene, incorrect depth scale, a region without depth, a method comparison and a repeated launch. Separately state what the project does not yet measure: support load-bearing capacity, physical friction, actual contact and step stability. The next stage can use the described geometry, but must make motion decisions under its own constraints.

Assigned reading: a focused route through the sections

The literature is mainly in English; the programme and definitions are also in English. Read the selected sections before the relevant task, rather than entire books. Core reading is included in the existing 35 analysis hours; consult references as needed.

Core reading

R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. (2022). [Author's page](#). Ch. 2 Image Formation; selected parts of Ch. 7 Feature Detection and Matching and Ch. 9 Motion Estimation. For Tasks 1-4: camera rays, robust correspondences and Lucas-Kanade; record the conditions under which a method ceases to be meaningful.

R. Hartley, A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. (2004). [Book and authors' contents](#). Ch. 6 Camera Models; only two-view geometry from Ch. 9 Epipolar Geometry and the Fundamental Matrix; triangulation from Ch. 12 Structure Computation. Before Tasks 1-2, learn to distinguish calibration, baseline, depth and uncertainty.

For the small deep-learning experiment

I. Goodfellow, Y. Bengio, A. Courville, Deep Learning (2016). [Authors' online book](#). Ch. 9 Convolutional Networks: convolution and pooling; Ch. 11 Practical Methodology: metrics and model selection. Before Tasks 7-8, understand the backbone and head, data splitting and the experiment's time limit; the remaining chapters are not needed yet.

Three implementation references

[OpenCV: Camera Calibration](#) - Calibration, Undistortion, Re-projection Error; Task 2, fixed K and distortion with separate held-out images.

[Open3D: ICP registration](#) - Point-to-point and Point-to-plane ICP, evaluation. In Task 6, compare the residual with the error of the known transform.

[PyTorch: Transfer Learning](#) - ConvNet as fixed feature extractor and Train the model; Task 7, a small CPU experiment with saved preprocessing settings and weights.

After reading, explain why a small reprojection error does not establish scale, why ICP can fail despite a small residual, and why model confidence does not equal the probability of a correct decision without checking calibration.

Terms and definitions

Pinhole projection. An ideal camera model in which image coordinates are determined by the ratios of a point's transverse coordinates to its depth.

Intrinsics. The focal scales and principal point in K ; they determine the mapping from camera rays to pixel coordinates.

Extrinsics. Rotation and translation between the camera and another coordinate frame, with an explicitly stated transform direction.

Distortion. A systematic deviation from the selected ideal projection caused by optics and described by a calibration model.

Reprojection error. The distance between a measured pixel and the projection of an estimated point; usually expressed in pixels of the selected image.

Disparity. The coordinate difference between corresponding points in a rectified stereo pair; together with baseline and focal length, it determines depth.

Stereo baseline. The distance between camera optical centres in metres; it establishes the metric scale of stereo depth.

Depth scale. A factor converting a stored depth code into physical units; the data format determines the direction of conversion.

Homography. A projective mapping of a plane between images; one homography is generally insufficient for a scene with different depths.

Feature descriptor. A vector describing a local image region for matching; descriptor similarity requires subsequent geometric verification.

RANSAC. A model-estimation method using random minimal subsets, checking consistency of the remaining data and rejecting outliers.

Inlier. An observation whose model residual is within a threshold; this does not yet establish that the correspondence is correct.

Optical flow. A field of apparent image displacements between times; by itself, it does not define metric body velocity.

Point cloud. A set of spatial coordinates of measured or reconstructed surfaces, expressed in a common frame and physical units.

Voxel downsampling. Reduction of cloud density by combining points within spatial cells; cell size determines the loss of detail.

Surface normal. A direction perpendicular to the local tangent plane; its estimate depends on the neighbourhood and may be ambiguous at an edge.

ICP. Iterative refinement of a transform through cloud correspondences; it requires suitable initialisation and may converge to an incorrect solution.

Segmentation. Assignment of a label to each pixel; distinct from whole-image classification and detection of rectangular object regions.

Transfer learning. Use of pretrained weights while adapting selected layers to a new task and dataset.

IoU. The ratio of intersection area between predicted and reference regions to their union area; the rule for empty regions is specified separately.

Equipment and setup for this month

Use existing equipment

MacBook Pro M4, Jazzy in UTM, M13 data and geometry, a ruler, callipers and mounting hardware. Recorded RGB-D and synthetic data are suitable for algorithm work. [TUM RGB-D: File Formats](#) specifies depth scale, timestamps and the ground-truth format; saved images can be analysed on macOS.

Purchase before recording your own data

Select 1 RGB-D camera providing metric depth and RGB, accessible intrinsics and extrinsics, timestamps, a mask of missing and invalid depth, and a driver for the selected Jazzy/Ubuntu ARM64 environment. Orbbec Gemini 2 is one candidate whose compatibility still requires confirmation. Before ordering, check the exact model, firmware, SDK branch and ARM64 support against the [official Orbbec ROS 2 table](#). Separately verify USB in UTM using an available sample or a confirmed configuration.

Prepare 1 suitable USB data cable and 1 stable mount; use a powered hub only if testing establishes a need. Make 1 flat calibration board and measure its square size. Choose range and field of view for the workbench and scene, rather than maximum specification values. If compatibility is unconfirmed, continue with recordings and defer the purchase.

Install now

Install OpenCV and Open3D in a separate Python environment. [Open3D: installation and platforms](#) - select a wheel for your Python version and architecture; open3d-cpu targets x86_64 Linux. If no suitable Linux ARM64 wheel exists, analyse saved clouds natively on macOS ARM; a source build requires separate verification.

In the Ubuntu guest, configure the camera ROS driver and USB access according to the specific SDK instructions. For Task 7, add PyTorch/torchvision CPU and Jupyter in a separate environment; fix a small backbone, head and weights. The Mac M4 is not a CUDA device; purchasing a GPU or Jetson is not required here.

Check the installation

Import cv2, open3d and torch; read an RGB/depth pair, check scale against a measured plane and build a cloud. The camera must provide real, non-empty images, CameraInfo and consistent timestamps, and USB recording must be stable. Test ICP on a known transform and run the network once to verify its expected output shape. Setup is included in the 75 hours; delivery is outside study time.

Motion fundamentals: from pixel to target

Required: why a visible point is not yet a leg target

A pixel defines a ray rather than a unique point: without depth, distance along that ray remains unknown. After reconstructing XYZ, you need the camera's pose on the robot and the body's position and orientation at image time. Check the prerequisite: explain the difference between rotating a vector and translating a point, and label the optical axes C and body axes B.

Model and worked example

Use a rectified image without distortion. u, v are pixels; f_x, f_y, c_x, c_y are parameters in pixels; Z is depth along the optical axis in metres, not range along the ray. Then $X = (u - c_x)Z/f_x$, $Y = (v - c_y)Z/f_y$. With $f_x = f_y = 600$, $c_x = 320$, $c_y = 240$, $(u, v) = (370, 240)$ and $Z = 1.2$ m, we obtain $p_C = (0.1; 0; 1.2)$ m.

For an error of 1 pixel in u and exactly known Z , the transverse error is $\delta X = Z \cdot \delta u / f_x = 0.002$ m. A coordinate-frame rotation preserves the length of this error vector; depth and extrinsic-parameter errors are added separately. This is how pixel noise becomes uncertainty in a future movement.

In this example the camera faces forwards: its X points right, Y down and Z forwards; B has x forwards, y left and z up. Therefore $R_{BC}(X, Y, Z) = (Z, -X, -Y)$. The camera centre is $t_{BC} = (0.2; 0; 0.3)$ m: $p_B = R_{BC} p_C + t_{BC} = (1.4; -0.1; 0.3)$ m. Thus the object is to the right of the body, not the left; camera depth is not height above the floor.

At image time, let the body have $\text{yaw} = \pi/2$ and $p_{WB} = (1; 2; 0)$ m in world frame W . Then $p_W = R_{WB} p_B + p_{WB} = (1.1; 3.4; 0.3)$ m. In homogeneous notation, $T_{WC} = T_{WB} T_{BC}$: the rightmost transform acts first. If the camera is on a moving link, T_{BC} depends on the joints; when the body moves, use transforms at measurement time.

Two exercises within the existing laboratory

1. In Task 5, save three points: on the optical axis, to its right and below it. Draw frames C, B and W and tabulate coordinates; check the inverse path $W \rightarrow C$ and reprojection. For synthetic data, set computational tolerances of 10^{-6} m and 10^{-6} pixels. $Z \leq 0$ and missing or invalid depth must cause rejection; they are not points at the origin.

2. Keep the object fixed and move the body 0.1 m along its x -axis without rotation. In C, Z is now 1.1 m and $u \approx 374.55$; the world point must remain unchanged. Substitute the old body position and orientation and demonstrate a 0.1 m error. Save replays of both variants and explain how a timing error displaces a proposed grasp point or foothold; reachability and load-bearing capacity are checked later.

Time accounting: 1 hour from sessions 5-6 for coordinate frames and formula derivation; 2 hours from 51-52 for these checks as a concrete implementation of Task 5. This remains within the original 75 hours, with no new equipment or separate project.

Assigned reading: Lynch/Park, [Modern Robotics §3.3.1](#) - composition and inverse transforms; Tedrake, [Geometric Pose Estimation: Representations for geometry](#) - depth, clouds and coordinate frames. Compare the camera equations with Task 1.

Monthly outcome: where the robot is

M15 • 19 Dec 2027 - 20 Jan 2028 • 75 hours

Compare motion-based position estimation, localisation on a known map and joint reconstruction of a map and trajectory. Deliver a reproducible setup for 2D and visual SLAM, a trajectory-error table and an analysis of conditions that make estimates unreliable. This introduces global navigation for the future walking robot; a complete global-navigation implementation does not become a prerequisite for stable walking.

Connecting the body, legs and camera

The robot moves its body, measures motion with an IMU, observes surfaces with a camera and sometimes knows its support positions. Wheel odometry uses wheel geometry; leg odometry uses kinematics and an assumption of stationary contact; visual odometry uses an image sequence; inertial odometry integrates angular velocity and acceleration. For each source, record an assumption that can fail: slip, loss of contact, lack of texture or sensor bias.

What to bring from earlier modules

You need transformation matrices, a basic EKF, ROS 2 tf/rosbag, RGB-D loading and the M14 understanding of ICP. Check the yaw sign, quaternion component order, time units and camera-to-body transform. If these checks fail, first correct them using three known positions and orientations. Do not try to compensate for a coordinate-frame error by tuning filter noise.

Minimum experimental setup

The main 2D experiment uses a short simulator recording containing LaserScan, odometry, IMU and exact ground truth. Use one route: a straight section, a turn, a return to the start and a repetitive corridor. For the visual method, use an RGB-D sequence from the same simulator or a TUM recording with a reference trajectory. If the scenes differ, compare failure causes and normalised measures. Incomparable ATE values do not establish that one sensor is more accurate than another.

Continue in Ubuntu 24.04 and ROS 2 Jazzy. Use compatible versions of slam_toolbox and the chosen visual method; save versions and parameters. You do not need to write a second complete SLAM system from scratch: integrate an existing baseline or build a small pose graph from RGB-D odometry and verified loop closures. Save a map, trajectory and quantitative evaluation; an RViz visualisation alone is insufficient.

Organising 75 hours and comparing methods

Time allocation

1-20: odometry, coordinate frames, EKF and error growth; 10 hours of analysis and 10 of practice. 21-40: occupancy grids, inverse sensor models, scan matching and AMCL; 10+10. 41-60: pose graphs, nonlinear least squares, covariance and loop closure; 10+10. 61-75: ATE/RPE, degraded data, LiDAR-versus-vision comparison and acceptance; 5+10. Total: 35 hours of theory and analysis and 40 of practice.

Each 10-hour theory block comprises five 2-hour sessions; split the final 5 hours as 2+2+1. Place four 10-hour practicals in Sunday work intervals or divide them without increasing the total time. The numbers below denote module study hours. Overlapping module-boundary dates do not double the available time; retain holidays, rest and separate integration checkpoints from the main roadmap.

Fix the protocol before running algorithms

Save the sequence duration and boundaries, input sensors, frequencies, exact reference-trajectory source and map resolution. State whether the algorithm knows the initial position and orientation. Ground truth is only for error calculation; SLAM must not receive it through an accessible coordinate frame. Accidentally supplying ideal odometry invalidates the experiment.

Create bags/manifest, configs/lidar, configs/visual, trajectories, maps, evaluation and failures directories. For each run, save run_id, random-number seed, commit, noise parameters, data and trajectory-alignment method. Include matched timestamps, excluded intervals and the tracked-route fraction in evaluation, alongside aggregate metrics. Otherwise, a method that loses half the route can appear more accurate than a reliable baseline.

Experiment without searching every combination

Change one parameter class at a time: odometry noise, scan dropouts, initial offset or loop-closure threshold. Use three interpretable levels and retain one route. Tune on a separate sequence and evaluate the final result on a held-out one. You do not need to study all of Nav2: use only localisation and the map in M15. First verify visual SLAM on a short valid recording, then on one degraded-data episode; leave analysis time after installation.

Odometry and EKF: estimating motion with uncertainty

Essential concepts

Dead reckoning accumulates small displacements: for a planar body, $x_{\text{new}} = x + v \cdot \cos(\text{yaw}) \cdot dt$, $y_{\text{new}} = y + v \cdot \sin(\text{yaw}) \cdot dt$, $\text{yaw}_{\text{new}} = \text{yaw} + w \cdot dt$. Here v is in m/s, w in rad/s and dt in seconds. This discretisation suits a teaching calculation with small steps; for rapid turns, check the midpoint method or exact model integration. Bias is a constant offset, whereas noise is a varying deviation; they affect the trajectory differently.

In odom, the local trajectory should remain continuous even as drift accumulates. The map frame allows global map-based correction. Study the map $\rightarrow$ odom $\rightarrow$ base_link chain and the sensor's fixed pose relative to the body. One agreed source must publish each transform. A jump in the global estimate must not silently become a jump in the local controller's command.

Task 1. Body drift on a closed route

Generate a square trajectory with stops and saved true positions and orientations. Separately add velocity noise, a constant yaw-rate bias and slip on one turn. Integrate odometry and plot error against time and distance. Output: three trajectories and a parameter log. Test zero noise, $dt=0$ and timestamp discontinuity; no motion must mean no progress along the route.

Task 2. Fuse measurements and check covariance

Use state $[x,y,\text{yaw}]$, a motion model and periodic absolute position measurements in simulation. Write out prediction and correction steps, residual and covariance. Implement a compact EKF or adapt a verified example from earlier modules; compare with configured robot_localization on ROS data. Do not count the same information twice: position and velocity derived from the same encoder are correlated.

Output: the estimate, uncertainty ellipses and an innovation plot. Test missing absolute measurements, an anomalous jump and an artificially small R . For the leg, separately explain how an incorrect stationary-foot assumption produces false confidence. The task criterion is explainable error growth and filter responses, rather than a promise that EKF always improves the estimate. A poor model or wrong units can make it worse than simple odometry.

Maps and scan matching: what measurements provide

This week's topics

An occupancy grid stores the probability that a cell is occupied, not floor height. The inverse sensor model uses a ray: cells before a reliable return were observed free, cells at the return are probably occupied, and cells behind it remain unknown. Study log-odds $L = \log(p/(1-p))$, which is convenient for updates. Limit confidence accumulated from old observations. Define separate rules for missing measurements and maximum range.

2D scan matching estimates a transform between scans or between a scan and a map. ICP depends on correspondences and its initial estimate; a uniform long corridor poorly constrains motion along its length. AMCL represents a distribution of possible positions and orientations as particles on a known map: it localises rather than constructs a map. Compare a broad initial distribution across the map with a precisely specified initial pose.

Task 3. Build a map from known poses

Input: 200-500 simulated LaserScan messages and true poses, available only to this reference task. Transform laser points into the world, update free cells along rays and occupied cells at endpoints. Output: a grid, an occupied/free/unknown legend and the resolution setting. Check one ray and a wall by hand; do not mark the unseen side of a wall as free. Repeat with twice the cell size and compare wall thickness.

Task 4. LiDAR recording and localisation

Run `slam_toolbox` on the same route with ordinary odometry; save the pose graph and map. Then stop mapping and localise on the fixed map using AMCL or a supported localization mode. Do not run two `map→odom` publishers simultaneously. Save launch and config files, two trajectories, the map and a list of TF checks. Compare with your reference map from Task 3.

Test an incorrect initial pose, loss of 20% of scans and a laser rotation in the extrinsics. For scan matching, create a small pair with a known displacement and measure transform error, then use an ambiguous corridor pair. Separate mapping error from localisation error on an already known map in the report. Successful loop closure does not establish correct local geometry along every section.

Pose graphs: constraints, loop closure and false links

Sections to study

A graph vertex represents the robot's position and orientation, or pose; an edge represents measured relative motion or another constraint. Optimisation adjusts poses to reduce weighted residuals. Weights relate to uncertainty: using identical numbers for metres and radians without justification creates an arbitrary trade-off. Fixing one initial pose removes ambiguity in overall translation and rotation, but adds no measurements of the surroundings.

Loop closure recognises a previously visited place and adds a constraint. The optimiser reconciles constraints but does not verify correspondence truth. A robust loss reduces the influence of large residuals; it does not replace geometric verification and may fail to protect against a consistent false closure. Treat covariance as a local estimate within model assumptions, rather than the probability that the whole map is correct.

Task 5. A small graph you can understand

Create 20-30 planar poses around a square, noisy odometry edges and one correct loop-closure edge. Use GTSAM Pose2 or a small educational solver. Show initial and optimised trajectories, residuals and the method used to anchor the common coordinate frame. Save graph.json, noise settings and plots before and after optimisation. Test zero noise and a disabled prior; explain why absolute coordinates are undetermined without an anchor.

Task 6. False loop closure in a similar corridor

Add an edge between different but visually similar places. Compare ordinary squared error, a robust loss and geometric rejection of the incorrect link before optimisation. Measure ATE at the same poses and changes in residuals of correct edges. Save three results and the loop-acceptance rule. Show a case where the robust loss still allows map deformation; do not promise a universal threshold.

A minimal visual-odometry experiment

Bring calibration, RGB-D recordings and pair-registration checks from M14; in M15, assemble a short RGB-D odometry sequence. Build a graph of successive poses and add one revisit confirmed by image or cloud geometry. Use ground truth only for error evaluation. You may use an existing compatible visual SLAM system. If a loop closure is specified manually to test the optimiser, label it as a manual constraint: that experiment does not test automatic revisit recognition.

Evaluation: trajectory, map and tracking loss

What ATE and RPE measure

ATE compares estimated absolute poses with a reference after explicitly described alignment. For translation, report the RMSE of distances in metres in the usual formulation. RPE compares relative displacements between pairs of poses separated by a fixed interval, revealing local drift. Specify the interval in seconds, frames or metres and the timestamp-matching rule. Different intervals answer different questions and produce incomparable values.

For metric RGB-D and LiDAR, do not rescale trajectories during alignment except in a separate scale-error study. For a monocular baseline, state whether Sim(3) was used and which errors scale fitting conceals. Alongside aligned error, save the original trajectory, route coverage and the time tracking was lost. Do not evaluate only a successful short fragment while omitting the lost return leg.

Task 7. An evaluator you can trust

Create a reference trajectory, a copy with a known translation, a copy with slow drift and a time-shifted sequence. Run evo and independently calculate several pairs. Save commands, alignment settings, matched timestamps and an ATE/RPE table. After SE(3)/SE(2) alignment, overall translation and rotation should disappear while local drift remains. Specify the maximum timestamp mismatch and show the number of rejected poses.

Task 8. Map quality and robustness

In the simulated scene, compare reconstructed walls with known geometry: distances to reference lines, passage width and occupied/free-cell fractions in a predefined region. Do not count unknown as free. For each pipeline, report latency, tracking-recovery time and route coverage. Choose tolerances from map resolution and the clearance required by the future body; this is a teaching requirement, not a regulatory standard.

Offset sensor and odometry timestamps, then change the laser extrinsics; repeat with algorithm parameters unchanged. Explain why a synchronisation error can resemble a scan-matching error. Add a blurred section and loss of texture to the visual experiment. The overall conclusion must identify operating conditions for each method rather than declare one algorithm best under every condition.

Laboratory comparison of LiDAR and visual SLAM

Inputs and outputs

Inputs: recording manifest, calibration/frames and configs for three variants - odometry only, LiDAR SLAM and visual RGB-D SLAM. Outputs: trajectory with timestamp and pose fields, tracking status, a map or aligned keyframes, a loop-closure log and metrics. A separate error-evaluation programme receives the reference trajectory. For each variant, state available sensors, whether the starting pose is known and the source of metric scale.

Task 9. One question set for three methods

1. Select a valid sequence and a held-out difficult route. 2. Check TF and time bases without running SLAM. 3. Compute odometry alone. 4. Run the LiDAR variant and save the map and graph. 5. Run the visual method with fixed parameters. 6. Evaluate matching available intervals and full coverage separately. 7. Repeat with one fault. 8. Assemble a report explaining the causes of differences.

If the camera and laser were recorded in the same simulator, compare identical reference-trajectory sections. If RGB-D comes from TUM, create two tables: metrics within each dataset and a qualitative comparison of failures. Claims such as "LiDAR is N times more accurate" are invalid for different routes. Tabulate comparison conditions so scene differences remain explicit.

Task 10. Robot response to lost localisation

Create a component that receives pose with status, timestamp and uncertainty. For stale data or lost tracking, it must output `localization_unavailable` and reject new global goals in simulation. Do not restart the running local stability loop. Recovery requires a new valid estimate, reconciliation of map/odom and an explicit return transition. Save an event log and a missing-data test.

Deliverable package

The README must describe recording processing and error evaluation; the manifest documents data provenance; configs contains every parameter. Include ATE/RPE with definitions, map quality, latency, route coverage and failure scenarios in the report. Add a TF-tree snapshot and example input/output messages. For the future hexapod, state the applicability boundary: global localisation helps follow a route but does not validate leg stability or contact reliability.

Eight failure and sensitivity scenarios

Faults to introduce

S1 - No absolute observations: disable EKF corrections and loop closures along part of the route. Expect growing drift and increased uncertainty if the model accounts for it. Compare with zero noise and check for false confidence despite nonzero error. S2 - Slip: add odometry error on one turn. Save the fault time, local RPE and correction after observing the surroundings.

S3 - Corridor ambiguity: walls repeat and transverse structure is sparse. Measure errors along and across the corridor separately; a small ICP residual does not yet establish correct displacement. S4 - False closure: connect similar places with an incorrect edge. Evaluate map deformation, ATE and changes in correct-edge residuals with and without a robust loss.

S5 - Kidnapped robot: in simulation, instantaneously move the body elsewhere on the known map without a corresponding odometry change. Test AMCL with local and broad initial distributions and measure recovery time under the chosen criterion. Manual reinitialisation must not be called automatic relocalisation. S6 - Data loss: remove scans or images for 1 and 3 seconds. The consumer must detect estimate age and unavailability.

S7 - Time and extrinsics: shift sensor timestamps by one period, then separately rotate the sensor by a known angle. Compare errors on straight sections and turns. S8 - Image degradation: blur, a smooth wall or a covered camera. Measure the fraction of tracked images and test recovery; errors on the surviving images alone are insufficient. Do not remove failed sections before preparing the final table.

Choosing levels and conclusions

Fault levels are illustrative parameters chosen to suit the measurement frequency and scene scale; they are not allowable limits for a physical robot. Change one cause per experiment and preserve the seed and original bag. Evaluate sensitivity at three levels; state whether improvement repeats or occurs only on one route.

Each scenario needs an expected-result file, actual trace, numerical criterion or manual-check rule, and a decision: handled, detected without recovery, or undetected. The last outcome is acceptable as an established limitation of this introductory SLAM work, but a consumer must not present an unreliable pose estimate as current and accurate.

Sessions 1-40: motion, the filter and the map

Hours 1-10: theory in 2-hour sessions

1-2: wheel, leg, visual and inertial odometry models, with one failure-prone assumption each. 3-4: planar-motion integration and bias; calculate a turn and return by hand. 5-6: map/odom/base_link, transform directions and TF sources. 7-8: EKF prediction and correction, Q/R and measurement correlation. 9-10: ground truth, error units and the Task 1 protocol. Record these principles as implementation assumptions.

Hours 11-20: practical 1

11-12: square-route generator and an error-free check. 13-14: Task 1 - noise, bias and local slip. 15-16: Task 2 - EKF, covariance and innovation plots. 17-18: process a ROS recording, compare with the chosen filter and check TF. 19-20: missing measurements, excessively small R and an error report. Understand the source of drift before connecting SLAM.

Hours 21-30: theory in 2-hour sessions

21-22: occupancy, free/occupied/unknown and log-odds. 23-24: inverse laser-ray model, resolution and invalid range. 25-26: scan matching, ICP, local minima and the corridor. 27-28: particles, measurement likelihood and AMCL resampling through an understandable example. 29-30: slam_toolbox modes, the difference between a map and a graph, and state saving. Select the manual parameters needed for your scenario.

Hours 31-40: practical 2

31-32: Task 3, a map from known poses and a manual ray check. 33-34: a lower-resolution map and the unknown/free distinction. 35-36: Task 4, process the LiDAR SLAM recording and save the graph and map. 37-38: localisation on a fixed map, initial offset and TF transform sources. 39-40: corridor, scan loss and a comparison report against the reference map.

By this stage, there should be three clearly understood files: a reference map for evaluation, a reconstructed map and pose estimates. Identify their roles in filenames and the manifest. If map processing continues after playback stops, allow the queue to finish within the practical and record processing lag; do not compare one method's partial result with another's completed result.

Sessions 41-75: graphs, comparison and acceptance

Hours 41-50: theory in 2-hour sessions

41-42: graph vertices and edges, residuals, noise and units. 43-44: nonlinear least squares, linearisation and initial estimates; one worked manual example is enough. 45-46: the prior, common-frame ambiguity and covariance meaning. 47-48: loop closure, geometric verification and robust losses. 49-50: visual-method structure; choose an existing baseline or small RGB-D graph before the practical.

Hours 51-60: practical 3

51-52: Task 5, a graph of 20-30 poses, noise and correct closure. 53-54: Task 6, a false link and three handling methods. 55-56: visual odometry on a short M14 sequence. 57-58: a keyframe map and verified closure, or running an existing visual SLAM system. 59-60: save trajectories, tracking status and a reproducible launch. If you used a manually specified closure edge, label it in the experiment name.

Hours 61-65: theory 2+2+1

61-62: ATE/RPE, intervals, alignment and scale; calculate three pairs by hand. 63-64: route coverage, timestamp matching, map quality and limits of cross-dataset comparison. 65: prepare the final fault protocol and localisation-availability decision. Justify numerical thresholds in terms of scene scale rather than copying them from another sensor.

Hours 66-75: practical 4

66-67: Task 7 - check the error-evaluation programme with a known transform and drift. 68-69: Task 8 and the three-method comparison from Task 9 on held-out data. 70-71: S1-S4 with fixed parameters and a sensitivity table. 72-73: S5-S8 and Task 10, including status recovery. 74-75: reproduce the final package and hand the map and status to M16. All intervals together total 75 hours.

If time is limited

First reduce the number of long recordings and parameters, then omit extra optimisers and detailed 3D visualisation. Retain processing of one LiDAR recording, one visual baseline, a small understandable pose graph, ATE/RPE, route coverage and consumer rejection of stale estimates. Do not spend the rest of the month porting SLAM to Rust. State unresolved integration issues: introductory coverage does not require results matching leading academic benchmarks.

Completion criteria and data for the planner

Acceptance checks

Quantitatively compare at least uncorrected odometry and SLAM on one sequence. Add a separate visual method with a map or keyframes and failure analysis. For ATE/RPE, state definitions, units, interval, alignment method, matched-sample fraction and route coverage. Tabulate available sensors, reference data and condition differences. Incomparable scenes cannot establish sensor superiority.

Find and explain examples of drift, degenerate scan matching, false closure, timing error and sensor loss. Save settings for the valid run and modified versions. The map must contain resolution, origin and frame_id, with unknown/free/occupied distinguished. Recalculating errors must not require manual trajectory edits. Any component using pose must respond to stale data and unconfirmed recovery.

Self-check

1. How does map differ from odom, and who publishes their transform? 2. Why does leg odometry fail during slip? 3. What happens with an incorrect dt? 4. Why can an excessively small R worsen EKF performance? 5. How does an unknown cell differ from a free one? 6. Why does scan matching constrain some motion directions better than others? 7. How does a false loop edge change the whole trajectory? 8. What does a prior fix, and what does it not measure? 9. How can ATE improve after failed frames are removed, and why does this distort quality evaluation? 10. Why calculate RPE at several intervals? 11. When is scale correction permissible? 12. Why does good SLAM not establish hexapod stability?

Handover to M16

Pass on the map, coordinate-frame chain, start and goal in one frame, an example localisation status and a data-age limit. Add two maps: one valid and one with a known error or a passage through an unknown region. The planner must reject a task when the pose estimate is unavailable. A map alone does not establish whether an area is suitable as a foothold.

Demonstrate a normal circuit, a section with odometry error, a false closure and missing data. Show numbers, then explain one failure cause without saying merely "poorly tuned". Frame the final conclusion in terms of operating conditions: "in this scene, with these sensors and parameters". Global navigation broadens the topics covered; local body and leg control can develop using other state-estimation sources.

Assigned reading: a focused route through the sections

The literature is mainly in English; the programme and definitions are also in English. Read the selected sections before the relevant task, rather than entire books. Core reading is included in the existing 35 analysis hours; consult references as needed.

Core reading

S. Thrun, W. Burgard, D. Fox, Probabilistic Robotics (2005), accompanying teaching materials. Authors' slides: kalman.ppt - Bayesian filtering and EKF; mapping-occupancy.ppt - occupancy updates; slam.ppt - joint map and trajectory estimation. Before Tasks 1-4, record states, observations and assumptions. The MIT Press book provides detailed reference material on these topics.

F. Dellaert, Factor Graphs and GTSAM: A Hands-on Introduction, teaching notes. Author's GTSAM notes. §1 Factor Graphs, §3 Robot Localization, §4 PoseSLAM; for the prior example, §2.2-2.4. In Tasks 5-6, build a small Pose2 graph with a prior and relative constraints, then add one false closure.

Reference reading

R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. (2022). Author's book. Ch. 11 Structure from motion and SLAM: selected material on pose estimation, bundle adjustment and SLAM. For visual experiments in Tasks 5-7, connect correspondences, camera motion, error accumulation and map correction.

Documentation for your experiment

ROS REP-105 - odom, map, Relationship between Frames. In Tasks 1-2, explicitly assign each TF source and the location of global correction.

slam_toolbox - Parameters, Localization and serialization. For Task 4, learn to distinguish a saved map image from a graph that can support continued operation.

evo: Metrics - Absolute pose error, Relative pose error, pose relation and alignment. Before Task 7, choose units, time association and permissible trajectory alignment.

Minimum outcome from reading

Explain using your own route: where odom drifts, how map corrects global error, what connects two poses in the graph, and why a plausible-looking map can conceal a false closure. Downloading large amounts of KITTI and writing a second full SLAM system from scratch are not required.

Terms and definitions

Localisation. Estimating the robot's position and orientation relative to a known coordinate frame or map using available measurements.

Odometry. Estimating relative motion from successive measurements; accumulated error usually grows without external correction.

SLAM. Joint estimation of trajectory and map, linking motion uncertainty to uncertainty in observed objects.

Prior. Information about the state before incorporating the current measurement; it may come from a motion model or initial condition.

Likelihood. A function describing the probability of an observation for a given state and sensor model; by itself, it is not a state distribution.

Posterior. The state distribution after combining prior information and a measurement under the adopted probabilistic model.

EKF. A filter that approximates nonlinear models through local linearisation and represents its estimate by a mean and covariance.

Innovation. The difference between a measurement and its prediction, interpreted with the uncertainty of both prediction and sensor.

Covariance. A matrix of variances and joint deviations of estimate components; its units depend on products of the corresponding component units.

odom frame. A local frame with a continuous trajectory that can gradually drift relative to the surrounding world.

map frame. A global map frame that allows correction jumps when accumulated drift in the local estimate is detected.

Occupancy grid. A map of spatial-cell occupancy probabilities; an unobserved cell must differ from one confirmed free.

Log-odds. The logarithm of the probability ratio between an event and its absence; convenient for occupancy updates.

Scan matching. Estimation of a transform from spatial measurements; it may be ambiguous with sparse or repetitive geometry.

Pose graph. A representation of a trajectory as pose vertices and constraints between them, obtained from motion or repeated observations.

Factor. A local likelihood or cost function relating a subset of unknown variables to a specific measurement or condition.

Loop closure. A constraint between the current and a previously visited place, capable of correcting drift if the correspondence is recognised correctly.

Gauge freedom. Ambiguity in the overall pose under relative constraints; removed by anchoring or an absolute observation.

ATE. Error of estimated poses relative to reference poses after explicitly specified alignment; it depends on time association and the selected component.

RPE. Relative-motion error between pose pairs over a declared temporal or spatial interval; characterises local drift.

Equipment and setup for this month

Use existing equipment

MacBook Pro M4, Ubuntu 24.04 ARM64 in UTM, Jazzy and Harmonic, the M14 RGB-D camera if available, and [saved sequences](#). Simulated LaserScan with ground truth is sufficient for the required 2D experiment; purchasing LiDAR is not required this month.

Purchase for a physical LiDAR experiment

If desired, prepare 1 two-dimensional LiDAR with a ROS 2 Jazzy driver for your computer and documented range, angles, frequency and timestamps. Before ordering, check its ARM64 SDK, power and complete USB/serial/Ethernet communication path; select one supported unit. You need a compatible data cable and 1 mount. The purchase extends physical data collection but is optional for the teaching comparison.

Install now

Add [slam_toolbox](#) from a Jazzy branch or package and [robot_localization](#) for the EKF task to the existing Ubuntu workspace. Use `ros-jazzy-slam-toolbox`; configure `robot_localization` only for fields actually supplied. For the known-map experiment, install the Jazzy [nav2_amcl](#) component now if absent; the full Nav2 workflow is deployed in M16.

For trajectory evaluation, add `evo` in a separate Python environment and record its version. You may use [GTSAM](#) for the small pose graph: install an ARM64 package or build a fixed release if you choose this option. Alternatively, write a small least-squares graph using the existing NumPy/SciPy. For [RGB-D odometry](#), use Open3D from M14.

Check the installation

Replay a short bag containing LaserScan, odom and IMU. Check the single map → odom → base_link chain, build a map and save the pose graph. Reload the graph and check continued operation. For EKF, provide a known displacement with correct covariances; for AMCL, provide a known map and initial position and orientation.

Check metrics on a trajectory identical to ground truth, then on a known displacement; explicitly specify alignment and scale. A small three-pose graph must solve with a prior and detect an added false link. Check new peripheral USB connections separately; successful recording processing does not establish actual synchronisation.

Preparation and initial functional checks fit within the existing 75 hours in place of repeated installations; optional LiDAR delivery is outside sessions. Do not repurchase the camera, IMU, MCU, motor or computer.

Motion fundamentals: support and odometry

Required: how a leg measures body motion

When the body moves above a stationary foot, joint angles change even though world-frame contact velocity is zero. Encoders measure relative leg geometry; body velocity can be derived by adding leg kinematics and IMU angular velocity. Before deriving the equations, check foot FK and the cross product: rotation produces a linear velocity $\omega \times r$ that depends on the lever arm.

From position to a velocity constraint

W is the world, B the body and F a fixed material point on the foot. $p_{WF} = p_{WB} + R_{WB} p_{BF}(q)$; q contains only leg joint angles in radians. $J_p = \partial p_{BF} / \partial q$ is the $3 \times n$ position Jacobian in B , so relative foot velocity is $J_p \dot{q}$. This is not the foot's complete world-frame velocity.

Let v_{WB} be the linear velocity of the B origin expressed in W , and ω_B the body angular velocity expressed in B . Differentiating gives $v_{WF} = v_{WB} + R_{WB}(\omega_B \times p_{BF} + J_p \dot{q})$. All linear velocities are in m/s; ω_B is in rad/s. Only a stationary, non-slipping support gives $v_{WF} = 0$, hence $v_{WB} = -R_{WB}(\omega_B \times p_{BF} + J_p \dot{q})$. Contact alone does not justify setting this velocity to zero.

Example with body rotation

At one instant, $R_{WB} = I$, $p_{BF} = (0.2; 0; -0.3)$ m, $\omega_B = (0; 0; 0.5)$ rad/s and $J_p \dot{q} = (-0.04; 0; 0)$ m/s. The rotational contribution is $\omega_B \times p_{BF} = (0; 0.10; 0)$ m/s. This gives $v_{WB} = (0.04; -0.10; 0)$ m/s. Omitting rotation loses the lateral motion of 0.10 m/s, even though FK and encoders are correct.

If the same foot actually slips at $v_{WF} = (0.02; 0; 0)$ m/s, the true $v_{WB} = (0.06; -0.10; 0)$ m/s. The incorrect zero-velocity assumption gives an x-error of -0.02 m/s. Several reliable supports allow residual comparisons; a shared geometry error or simultaneous slip can bias estimates from all legs.

Exercises and evidence of understanding

1. Add a short synthetic stance episode to Task 1: calculate world-frame velocity in two ways, by differentiating p_{WF} and by summing the three terms. Save a table and a residual plot; halving the finite-difference step should reduce error until rounding error dominates. Ground truth is available only to metric calculation, not to the state estimator.

2. In Task 2, repeat the example with slipping and loss of contact. Save the innovation and support-admissibility flag; disabling an unreliable constraint must be visible. Explain why integrating velocities produces drift and does not by itself determine absolute position and orientation in map.

Time accounting: 1 hour from 1-2 for the leg-odometry model; 1 hour each from 13-14 and 15-16 for examples within Tasks 1-2. The total remains 75 hours; do not add physical walking or a new filter beyond these tasks.

Assigned reading: Lynch/Park, [Modern Robotics §3.2.2 Angular Velocities](#) and [§5.1.2 Body Jacobian](#). Distinguish the book's six-dimensional spatial velocity (twist) from the point-position Jacobian used here.

Monthly outcome: paths, trajectories and behaviour

M16 • 21 Jan 2028 - 15 Feb 2028 • 75 hours

Prepare comparative planner tests and a small autonomous task with a behaviour tree. The robot receives a goal, finds a valid path, builds a speed-limited trajectory along it and monitors execution, explicitly responding when it cannot continue. This month's examples are a body route among obstacles and a manipulator approach to an object. These foundations precede foot-placement planning.

Three different questions

Search determines which states connect the start and goal. A geometric path $q(s)$ describes a configuration sequence without time. A trajectory $q(t)$ adds time, velocity and acceleration. Task logic decides when to plan, wait, cancel and recover. A* success does not mean an actuator can execute the path; successful arm planning does not mean the object has already been grasped.

What to use from earlier months

Use the M15 map with unknown cells, M14 scene geometry, and link kinematics and actuator limits from earlier modules. Check that start and goal use one frame and that angular and linear units are consistent. The prerequisites are reading URDF, running ROS 2 actions, constructing a graph and explaining a derivative. Review how a joint-angle trajectory produces rad/s and rad/s^2 .

Two small experimental setups

Setup A is a 2D room map and a body with a finite planar footprint. You may use a simulated wheeled base to learn Nav2. Setup B is a ready-made manipulator model in MoveIt 2, with a table and a box. Choose one prepared package for each, compatible with Ubuntu 24.04 and ROS 2 Jazzy. Do not build a complex robot model and a new environment from scratch this month.

The final package contains your Dijkstra/A* implementations, comparison of at least two methods, a configuration-sampling experiment, trajectory checks, BT and FSM logs, and recovery scenarios. Foothold planning, hexapod kinodynamics and contact optimisation belong to later specialisation. Explain now why a circular body footprint and a free floor cell do not establish that a particular multilegged step is feasible.

75 hours: from the map to controlled execution

Four work blocks

1-20: graphs, BFS, Dijkstra, A*, heuristics and costmaps; 10 hours of theory and analysis +10 of practice. 21-40: configuration space, collision checking, PRM/RRT/RRT* and the MoveIt scene; 10+10. 41-60: time parameterisation, splines, velocity, acceleration and jerk limits; 10+10. 61-75: FSM, BT, recovery, timeouts and MDP/POMDP foundations; 5+10. The total budget is 35 hours of analysis and 40 hours of practice.

Use 2-hour sessions for the first three theory blocks; divide the last as 2+2+1. Sunday practical blocks of 10 hours include implementation, measurement and reporting; breaks do not count as study hours. The numbering below specifies study-hour order, not calendar dates.

Module-boundary dates may overlap; holidays, rest and separate checkpoints remain in the original calendar without counting hours twice.

Repository structure

Store synthetic and input maps in `maps/`; algorithms and the common interface in `search/`; OMPL settings in `sampling/`; MoveIt objects and robot parameters in `scenes/`; `q,t` files and limits in `trajectories/`; the tree, FSM and adapters in `autonomy/`; and scenarios, seeds and results in `benchmark/`. Tie every result to a map version and body footprint. After an obstacle moves, do not silently continue using the old geometry.

One hypothesis per experiment

To test "A* expands fewer vertices", retain the same graph, cost and tie-breaking order. To test "RRT* improves the path", compare time and cost under the same budget, including failed runs with different seeds. To test "smoothing improves motion", recheck collisions: a spline can cut a corner through a wall. First state the reason for the comparison, then choose the metric.

Finish GUI work by saving a machine-readable scene and launch commands. Do not rely only on dragging a goal manually in RViz. Prepare a small reference example for each task that another person can check without the complete setup: a known optimal path, a segment through an obstacle, a deliberately over-fast trajectory or an expected state trace.

Graph search and the cost of a body route

What to study

BFS finds the shortest path by edge count when edges have equal cost; Dijkstra handles nonnegative costs; A* adds a heuristic estimate h of cost to the goal. For optimality, study admissibility and consistency of h , vertex reopening and termination criteria. Manhattan distance is suitable for a 4-connected grid with unit steps; recheck admissibility for different motions and costs.

A costmap stores assigned movement costs, while an occupancy map stores occupied, free and unknown cells. For a finite-size body, account for its footprint, or inflate obstacles by the circular-model radius with a justified margin. Clearance must account for geometry and uncertainty. Explicitly define unknown handling: absence of observations does not permit motion.

Task 1. Dijkstra and A* on a robot map

Create an open field, a U-shaped trap and two corridors with different costs. Implement a common `solve(start, goal, map)` interface, path reconstruction and an expanded-vertex counter. Output: path, cost, runtime, expansions and `no_path` status. Test `start=goal`, a goal inside an obstacle, an unreachable goal and equal A*/Dijkstra cost with admissible h . Use BFS as a reference only for equal edge costs.

Task 2. A narrow passage and a dynamic obstacle

Use a circular body model and a passage around a box. Build two costmaps for different footprint sizes and compare passage availability. After planning, add an obstacle in the middle of the route. Mark the old path stale, stop the simulated execution process and replan. Save map and plan versions, the plan-invalidation log, and the new path or a correct rejection.

Check every segment against the current map; a diagonal must not pass between touching corners of two occupied cells. Save a case where the body centre passes but the footprint hits a wall. Explain the future distinction for a leg: the body-centre route does not define a contact sequence, and height variation requires another state model. In M16, demonstrate this boundary and do not call a 2D path a complete walking plan.

Sampling-based search and the manipulator scene

Configuration space

Configuration q contains joint angles, body position or other system coordinates. A workspace obstacle becomes a set of forbidden q values. Checking one endpoint does not establish the validity of motion between points. Study joint limits, self-collision and collisions with the environment and attached object. The configuration-space (C-space) metric must account for units and periodic angles.

PRM builds a configuration network for repeated queries; RRT grows a tree; RRT* improves connections to reduce cost as sampling grows. Probabilistic completeness and asymptotic optimality hold under assumptions and a limiting budget; they do not guarantee a solution within two seconds. A narrow passage may rarely be sampled. Post-search path simplification also changes the result and requires checking.

Task 3. Compare sampling-based search

Start with a 2D point or disc model among rectangular obstacles in OMPL. Compare RRTConnect with RRT* or PRM using equal time per query: at least 10 random-number seeds for three scenes. Save the exact-solution fraction, time to first solution, length and minimum clearance; account for PRM construction cost separately. Prepare configs, seed-list and a table of every run, including timeouts. A missing solution must not be counted as a zero-length path.

Task 4. A MoveIt planning scene

Use a ready-made arm, table and box. Specify object dimensions, positions and orientations in one coordinate frame; specify start and goal, and plan an approach in simulation. Then attach the object to the gripper and replan. Save the scene, a valid path and a failure example. Check that the attached object participates in collision checks and that its frame and scale are correct.

Separately check the path with sufficiently fine discretisation and compare with a coarse check: a thin obstacle between waypoints must be detected. Document the limits of discrete checking; without continuous collision checking, it is not a mathematical proof of no contact along the entire continuous path. The future arm uses this setup to approach an object, while a future leg uses similar checks for link motion.

Trajectories: time, smoothness and actuator limits

Core topics

Study linear interpolation, cubic and quintic profiles, and continuity of position, velocity and acceleration. Jerk is the derivative of acceleration, in rad/s^3 for a revolute joint. A geometric spline may be smooth in parameter s but become infeasible after incorrect time parameterisation. Even within velocity limits, a trajectory may require excessive torque; distinguish dynamic constraints explicitly.

For a simple motion from q_0 to q_1 , use $q(t)=q_0+(q_1-q_0)\cdot s(t/T)$, where $s(\tau)=10\tau^3-15\tau^4+6\tau^5$, $0\leq\tau\leq 1$. Velocity and acceleration are zero at both endpoints. Study the derivatives: velocity scales as $1/T$, acceleration as $1/T^2$ and jerk as $1/T^3$. This gives an understandable way to slow an educational movement, but does not automatically check collisions or dynamics.

Task 5. A leg or finger joint profile

Specify q_0, q_1 and teaching limits in the configuration, explicitly labelling them as simulation parameters. Calculate the profile for three T values, plot q, v, a, j and find the smallest tested T that satisfies the limits with a margin. Output: trajectory.csv, plots, assumptions and a validator. Test $q_0=q_1$, $T\leq 0$, non-numeric inputs and an excessively short movement. For a real actuator, obtain limits from specifications and load tests.

Task 6. Check a path after smoothing

Take the Task 4 path, apply time parameterisation with an available MoveIt tool, and check the complete result: joint limits, strictly increasing times, derivatives and collisions. Establish whether the chosen library changes geometry during processing. Output: original and processed paths, times and rejection reasons. Construct a corner near an obstacle where smooth interpolation intersects it.

For an introduction to trajectory optimisation, focus on the formulation: variables are states and controls; constraints are dynamics and obstacles; the objective is time, energy or smoothness. Solve a small scalar example or study a worked solution without building a full optimiser. Selecting path topology and continuously optimising a trajectory solve different parts of the overall task; both outputs require checking.

Autonomous behaviour: states and recovery

Minimal task logic

Define idle, planning, executing, waiting, recovering, failed and completed states. For each, specify inputs, actions, exit conditions and permitted cancellation. An FSM supports explicit transition accounting; a behaviour tree supports reusable conditions and actions returning SUCCESS, FAILURE and RUNNING. Study Sequence, Fallback, reactive condition checks and interruption of an asynchronous action.

Recovery needs a cause and an attempt limit. For a stale map, refresh observations and replan; endless retries do not make an unreachable goal reachable. Cancelling an active ROS action requires acknowledgement and an unambiguous state, otherwise motion may continue after the goal changes. Measure local timeouts with a monotonic timer; describe simulation time and recording replay separately.

Task 7. A behaviour tree for a route

Build "check localisation → plan path → follow path", with a branch that waits for fresh data and bounded replanning. First implement deterministic action stubs, then connect a Nav2 action in simulation. Output: BT XML or code, a transition table and an event log. Test planner FAILURE, endless RUNNING and user cancellation; configuration must bound the number of attempts.

Task 8. Measure recovery time

Block the path in simulation, then clear it; measure detection time, time to a new plan and time until motion resumes. For a permanent obstacle, expect failed or waiting for the operator, rather than a false completed. Output: a recording, BT settings, events and two plots. Do not transfer commands from the wheeled Nav2 example directly to a walking body: it needs a separate executor with contact checks.

MDP and POMDP through a concrete example

Describe a choice between a short unknown section, a detour and an inspection using state, action, transition and reward. In a POMDP, the true state is hidden; decisions use a belief distribution updated from observations. Tabulate two traversability hypotheses and update their probabilities after a camera observation. Reinforcement learning is not required here. Account for map uncertainty before permitting motion.

Integration setup: plans have a validity period

Request and plan requirements

Include request_id, current starting position and orientation, goal, frame_id, map_version, robot model and limits in the request. The response contains exact/approximate/no_path/timeout status, the path, its validity conditions and checking parameters. approximate status does not establish goal attainment. Before accepting a trajectory, the executor checks map version, localisation availability and whether the initial state is current.

Task 9. Connect search, trajectory and behaviour

1. Validate inputs. 2. Calculate an A* or Movelt path for your setup. 3. Independently check collisions. 4. Set motion timing and check limits. 5. Send the trajectory to the simulation executor. 6. Monitor map and status changes. 7. On a violation, block acceptance of the next segment, cancel the action and move the BT into recovery. 8. Save the final state and its cause.

For the body route, use a path between two map points. For the arm, approach a pregrasp pose and return to the initial state; the grasp cycle itself belongs to M17. A common validator checks shared data requirements, but the setups have different geometry. Do not present the same point list as a universal executor for wheels, legs and manipulators.

Task 10. Reproduce an asynchronous fault

Create this situation: plan A is delayed, the user has already requested B, and the old successful response A then arrives. The executor must reject A using request_id and version, and the log must show why. Repeat with delayed cancel acknowledgement and completion of the old action. Output: a reproducible GUI-free test and an event sequence. This is an asynchronous race in robotics: a mathematically correct path may belong to a task that is no longer current.

Final package contents

Save comparative tests under identical conditions with every seed, Movelt and Nav2 scenes, path and trajectory checks, the tree and FSM table, eight failure scenarios, and a README with commands, versions and limitations. Two short videos must show normal execution and correct recovery. Measure time from event logs rather than edited video. This software setup does not validate the physical safety of an actuator or real workspace.

Scenarios the planner must handle

Geometry and data

S1 - start or goal inside an obstacle: return `invalid_request` before search and do not begin execution. S2 - A passage narrower than the body footprint: a point path may exist, but reject the body plan. Check the narrowest point by hand. S3 - A thin wall between waypoints: independent collision checking must find the intersection; compare two checking resolutions.

S4 - An unknown strip across the map: forbid mode produces a detour or failure; allow is permitted only as an explicitly labelled teaching experiment. Save both settings without silently changing them after failure. S5 - A new obstacle during motion: cancel the old plan, increment version and use the current scene for the new plan. Measure detection time and record the FSM response.

Timing and logic

S6 - An infeasible trajectory: shorten its duration until velocity or acceleration limits are violated. The check must reject the result and identify the segment and measured quantity. Separately add non-monotonic timestamps. S7 - Timeout and a stuck action: a stub returns `RUNNING` beyond the limit. The tree must bound recovery attempts and finish with an understandable status. Record attempts and waiting time even if a path is eventually found.

S8 - Cancellation and a late response: request B replaces A, then result A arrives. No segment of A may be accepted after the change of current request is confirmed. Also test M15 localisation loss: block new global goals, and do not mark an ended action successful merely because updates stopped.

Metrics and comparison rules

For search, measure expansions, the number of expanded vertices; cost, the sum of specified edge costs; and runtime, the duration of the selected code segment. For sampling methods, measure exact-solution fraction, time to first solution, final cost, minimum clearance and number of collision checks. Report timeout separately; show median time for successful runs together with success fraction.

For the autonomous task, measure success fraction, recovery count, time to detect a violation and build a new plan, and rejected stale-command count. State clocks, tick frequency and hardware. Body-path length is the sum of segment lengths in metres. For a joint path, the sum of angular-increment norms means something different: specify joint weights and do not compare it with metres. Minimum clearance is calculated between geometric bodies at checked configurations; state discretisation and units. A map may penalise wall proximity, so the shortest path need not have the lowest cost. No collisions in a finite simulation set describes that set; it does not prove their absence for every input, parameter and physical contact.

Sessions 1-40: graphs and configuration space

Hours 1-10: five 2-hour sessions

1-2: distinguish paths, trajectories and task decisions; draw input/output diagrams. 3-4: BFS/Dijkstra/A*, queues and cost; expand a small map by hand. 5-6: heuristic admissibility, consistency and A* termination. 7-8: body footprint, obstacle inflation, unknown and costmap units. 9-10: comparison protocol, dynamic obstacle and version checks. Use your algorithm-problem experience, adding real robot size and the consequences of unknown cells.

Hours 11-20: practical 1

11-12: Task 1, common interface and Dijkstra. 13-14: A*, optimal-cost checks and BFS for unit-cost edges. 15-16: three maps, metrics and expanded-vertex display. 17-18: Task 2, different footprints and a narrow passage. 19-20: map changes, invalidation of the old plan and reproducible no_path. By the end of the block, prepare a baseline for comparison with subsequent algorithms.

Hours 21-30: five 2-hour sessions

21-22: configuration space of a two-joint arm, self-collisions and limits. 23-24: PRM/RRT/RRT*, metric, sampling and time limits. 25-26: probabilistic properties and narrow passages; record conditions that cannot be attained in finite time. 27-28: the MoveIt scene, attached objects and collision matrix. 29-30: independent path checking and motion discretisation.

Hours 31-40: practical 2

31-32: Task 3, OMPL and scenes for two planners. 33-34: 10 seeds, timeouts and a complete comparison. 35-36: Task 4, a ready-made arm, table and box. 37-38: attached-object movement, frame and size checks. 39-40: a thin obstacle, an unreachable goal and a report on sampling-search limits. Keep total time equal and account separately for PRM preprocessing.

If the MoveIt GUI does not start, record environment limitations and try opening the prepared scene without a graphical interface within the allocated time. Do not spend the entire practical on a different operating system. If necessary, simplify the arm while retaining joint limits, an obstacle and attached-object checks. Analysing a specific collision-checking error is more useful than adding models.

Sessions 41-75: trajectories and the autonomous task

Hours 41-50: theory in 2-hour sessions

41-42: $q(t)$ derivatives, units, linear and cubic interpolation. 43-44: quintic profiles and T scaling. 45-46: velocity, acceleration, jerk, boundary conditions and actuator limits. 47-48: MoveIt time parameterisation and geometry rechecking. 49-50: trajectory-optimisation foundations, objective and constraints; comparison with path-topology search.

Hours 51-60: practical 3

51-52: Task 5, one-joint profile and three durations. 53-54: analytical and numerical derivatives, validator and edge-case inputs. 55-56: Task 6, time parameterisation of an arm path. 57-58: smoothing near an obstacle and independent checking. 59-60: trajectory and limit files, plus rejection reasons; prepare the simulation-executor interface.

Hours 61-65: theory 2+2+1

61-62: FSM and BT, RUNNING/SUCCESS/FAILURE, Sequence/Fallback. 63-64: cancellation, timeout, recovery and late responses; draw the Task 10 event sequence. 65: MDP/POMDP through detour, inspection or motion choices and a traversability belief distribution. These hours do not include an additional RL course: understand which information is unavailable to simple logic.

Hours 66-75: practical 4

66-67: Task 7, a tree with deterministic action stubs and transition checks. 68-69: Task 8, Nav2 simulation with temporary and permanent obstacles. 70-71: Task 9, requirements for handing off plans, trajectories and execution, and map age. 72-73: Task 10, S1-S8, cancellation and late responses. 74-75: reproduction from a clean start, comparison table and final report. Every task fits within the 75 hours.

What to cut first

First omit a third planner, an extended smoothing-method comparison and additional interfaces. Then reduce scenes to three and shorten simulations while retaining every failure type. Dijkstra/A*, one sampling experiment, a MoveIt collision scene, trajectory limits and BT recovery timing remain mandatory. Keep stale-plan checks: an asynchronous race can negate the correctness of the search algorithm.

Acceptance: correct rejection is also a result

Required verification results

Compare at least two planners under one protocol; verify Dijkstra/A* cost on reference graphs. In the sampling experiment, use multiple seeds, count timeouts and check the resulting path. The MoveIt scene must include an obstacle and an attached object. Reject an infeasible trajectory with a specific reason; after smoothing, recheck collisions and limits.

The behaviour tree completes the task in a normal scene, recovers from a temporary obstacle and ends its attempts when the goal remains unreachable. Cancellation works during RUNNING, and a late result does not replace the current task. Logs connect map_version, request_id, plan and execution. All eight scenarios are reproducible; launch instructions do not depend on the author's manual GUI actions.

Self-check

1. When do BFS and Dijkstra produce the same cost? 2. How does an incorrect h remove A*'s expected optimality? 3. Why does footprint size affect whether a path can be found? 4. How does PRM usage differ from RRT? 5. Why does RRT* not promise an optimal path in fixed time? 6. What does checking only waypoints miss?
7. How do v, a, j change when T doubles? 8. Why can a smooth spline collide with a wall? 9. How do velocity constraints differ from torque constraints? 10. Why is RUNNING neither success nor failure? 11. How do you cancel an old task and reject its late response? 12. What additional information does a POMDP require compared with a fully observed state?

Handover to manipulation and legs

Pass the arm scene, collision objects, pregrasp path, verified executor, limits and event log to M17. For later walking, retain the map with unknown, footprint assumptions and constraints absent from the 2D route. Contact selection, stability, foot reachability and body dynamics will require a new model. Cancellation and task-freshness checks remain applicable.

The final demonstration must include no_path, an excessively fast trajectory and recovery, as well as a successful route. Explain the precise level of assurance your test suite provides. "No valid plan found" is more useful than moving on an uncertain result. Do not claim physical safety of a real system based on one month of simulation.

Assigned reading: a focused route through the sections

The literature is mainly in English; the programme and definitions are also in English. Read the selected sections before the relevant task, rather than entire books. Core reading is included in the existing 35 analysis hours; consult references as needed.

Core reading

S. LaValle, Planning Algorithms (2006). [Author's online book](#). Ch. 2 Discrete Planning - graph search; Ch. 4 The Configuration Space - C-space obstacles; Ch. 5 Sampling-Based Motion Planning - PRM/RRT. Before Tasks 1-4, write out the formulation, validity criterion and termination condition.

K. Lynch, F. Park, Modern Robotics (2017). [Book and learning resources](#). §9.1-9.2 Point-to-Point Trajectories, §9.3 Polynomial Via Point Trajectories, §9.4 Time-Optimal Time Scaling. In Tasks 5-6, distinguish $q(s)$ and $q(t)$, calculate velocity and acceleration, and check limits.

Task-management reference reading

M. Colledanchise, P. Ögren, Behavior Trees in Robotics and AI: An Introduction. [Authors' book version](#). §1.3 Classical Formulation of BTs, §2.1 Finite State Machines; in Ch. 3 Design principles, §3.1-3.2 on success conditions and reactivity. For Tasks 7-8, draw a tree for planning, following and recovery; check RUNNING returns, stopping and condition reevaluation.

Three implementation references

[OMPL: How to Benchmark Planners](#) - Writing benchmarking code and Processing the benchmark log. In Task 3, save exact/approximate, timeout, run count and validity of the solution found.

[MoveIt: Planning Scene](#) - Collision checking, Constraints, State validity. In Task 4, include self-collision and environment collisions; check a changed scene before execution.

[Nav2: Detailed Behavior Tree Walkthrough](#) - Navigation Subtree and Recovery Subtree. For Tasks 7-8: a minimal planning, following and recovery sequence; check XML and plugins against Jazzy.

MDP/POMDP are currently studied only through the distinction between state and observation. After reading, explain why an exact path in a model can be unexecutable and why timeout failure must remain visible in comparative test results.

Terms and definitions

Configuration q . A set of independent coordinates defining the robot's geometric arrangement; velocities usually belong to an expanded state.

Configuration space (C-space). The space of all robot configurations; collisions between geometric bodies define a forbidden region in this space.

Footprint. The planar projection of the area occupied by the robot, used in mobile-base planning and passage checks.

Costmap. A discrete representation of obstacles and movement penalties; a cell's cost need not be its occupancy probability.

Geometric path $q(s)$. A configuration sequence parameterised by progress s ; time, velocity and acceleration are not yet specified.

Trajectory $q(t)$. Motion specified over time, allowing calculation of velocities, accelerations and actuator requirements.

Dijkstra. Shortest-path search with nonnegative edge costs, processing the smallest current cost successively.

A*. Search selecting a node by the sum of accumulated cost g and the estimated remaining cost h .

Admissible heuristic. A remaining-cost estimate that never exceeds the true optimum; search guarantees also depend on implementation and graph conditions.

PRM. A roadmap method using valid random configurations and checked connections between neighbouring samples.

RRT. A search tree extended towards samples in the space; the basic algorithm aims to find a path rather than optimise it.

Probabilistic completeness. A property in which the probability of finding an existing solution approaches one as resources increase and assumptions hold.

Collision checking. Checking robot geometry for intersection with itself and the world; sparse point checks may miss collisions between them.

State validity. Satisfaction of specified configuration, scene and task constraints; it does not establish validity of an entire connecting trajectory.

Time parameterisation. Assignment of path progress as a function of time, accounting for velocity, acceleration and other constraints.

Jerk. The time derivative of acceleration, expressed in m/s^3 or rad/s^3 ; describes how abruptly the applied motion changes.

Behaviour tree (BT). A hierarchical structure executing actions and conditions that return success, failure or ongoing-execution status.

Tick. A signal to evaluate a tree node; tick frequency determines the timeliness of condition checking and action cancellation.

FSM. A finite state machine with explicit states and transitions driven by events or conditions; defines permissible behaviour order.

Recovery. A bounded procedure following a detected failure, with a completion condition, retry limit and a route to safe stopping.

Equipment and setup for this month

Use existing equipment

MacBook Pro M4, UTM with Ubuntu 24.04 ARM64, ROS 2 Jazzy, Gazebo Harmonic, and M14-M15 maps, scenes and recordings. The two teaching setups require simulated mobile-base and manipulator models. No new hardware purchases are mandatory; do not order a physical arm, wheeled base or additional actuators.

Install now

In the existing Ubuntu guest, install Nav2 navigation2/nav2_bringup, MoveIt 2 and OMPL with dependencies for the Jazzy version in use. MoveIt: Binary Install specifies ros-jazzy-moveit for Ubuntu 24.04; the official Nav2 repository is the starting point for package installation and compatible bringup. Record package versions and arm64 availability; do not copy Rolling commands without checking.

Install MoveIt 2 in M16 because the manipulator-scene laboratory runs here; reuse the stack in M17. Initially use OMPL from MoveIt dependencies, adding a separate C++ benchmark build only for metrics required by the task. Python bindings are not mandatory dependencies for your own A*/Dijkstra implementation.

Use a BehaviorTree.CPP version compatible with Nav2; Reactive and Asynchronous behaviors explains Sequence/ReactiveSequence and halt. A BT GUI editor is optional. Retain one RMW, the RViz configuration and prepared demo packages; do not update the entire workspace merely for a new documentation page.

Check the installation

1) In Nav2, send a reachable goal on a small map, then an unreachable one; check the action result and cancellation. 2) In MoveIt, load a prepared model, table and box; obtain a path and reject a collision. 3) Check availability of the chosen OMPL planners, exact/approximate statuses and timeout. 4) Record RUNNING, tree cancellation and recovery, and final cessation of the command.

Test RViz and Gazebo in UTM with a separate small run: an installed package does not establish working graphics. If problems occur, save console logs and paths for analysis; visual verification remains open until the output is actually viewed. These simulation tasks do not need USB, but reused physical peripherals still require their previous checks.

Reading and setup fit within the 75 hours. An onboard Pi/x86, GPU and physical arm kit belong to later conditional stages; after M17, the separate OPT120 has its own CAD/BOM and single-finger validation.

Motion fundamentals: timing along a path

Required: why geometry is insufficient

The same path can be traversed slowly or abruptly. Configuration geometry determines collisions, while velocity, acceleration and required torques also depend on time. Prerequisite: take one revolute leg or finger joint, with q in radians, and explain $\dot{q}$ and $\ddot{q}$ in rad/s and rad/s².

Two functions rather than one point list

A path $q(s)$, $0 \leq s \leq 1$, specifies configuration order; a monotonic $s(t)$ specifies progress. The chain rule gives $\dot{q} = q'(s)\dot{s}$ and $\ddot{q} = q''(s)\dot{s}^2 + q'(s)\ddot{s}$. The first acceleration term comes from path curvature and the second from changes in the rate of progress. Thus smoothness of $q(s)$ does not replace checks of time derivatives.

For a straight joint-space path $q(s) = q_0 + \Delta q \cdot s$, choose $s(t) = 3\xi^2 - 2\xi^3$, $\xi = t/T$. Then $\dot{q} = (\Delta q/T)(6\xi - 6\xi^2)$, $\ddot{q} = (\Delta q/T^2)(6 - 12\xi)$. Velocity is zero at both endpoints; acceleration there is nonzero and jumps when transitioning to rest. The quintic profile in the existing Task 5 adds zero boundary accelerations.

Numerical example and duration selection

Let $q_0 = 0$, $\Delta q = 0.6$ rad and $T = 2$ s. Maximum velocity at $t = 1$ s is $1.5\Delta q/T = 0.45$ rad/s. Maximum $|\ddot{q}|$ at the endpoints is $6\Delta q/T^2 = 0.90$ rad/s². With example limits of 0.30 rad/s and 0.60 rad/s², this profile is too fast.

The velocity limit gives $T \geq 1.5 \cdot 0.6 / 0.30 = 3$ s; the acceleration limit gives $T \geq \sqrt{(6 \cdot 0.6 / 0.60)} = \sqrt{6} \approx 2.45$ s. Thus this cubic profile requires $T \geq 3$ s. At $T = 3$ s, the maxima are 0.30 rad/s and 0.40 rad/s²; choose a larger T for a margin. This result applies to the chosen profile shape, not the globally shortest time for any motion.

Foot velocity equals $J_p(q)\dot{q}$ and varies with configuration; one joint limit does not specify constant foot speed. Moreover, the torque needed to support weight does not disappear with slow motion. After checking velocity and acceleration constraints, check dynamics, contacts and geometry; calculating torque does not set the control mode of a position-controlled servo.

Two exercises in Tasks 5-6

1. Save $q, \dot{q}, \ddot{q}$ for $T = 2, 3, 4$ s in trajectory.csv. Compare analytical extrema with a dense grid and numerical derivatives; check $q(0), q(T)$, zero velocities and rejection of $T \leq 0$. Then use the same validator on the Task 5 quintic profile: its maximum coefficients differ.

2. Take the arm path near the box: a monotonic change of time parameterisation alone preserves path geometry, while waypoint-based path smoothing may change it. Show both versions and repeated collision checking; include one correct rejection and a plot of the violated constraint.

Time accounting: 1 hour from 41-42 for the derivation; 1 hour each from 51-52 and 53-54 for the numerical example and validator. Geometry checking remains within the existing 57-58. The budget stays at 75 hours, with no additional setup required.

Assigned reading: Lynch/Park, [Modern Robotics §9.1-9.2, part 1](#) - path and time; [part 2](#) - cubic and quintic time scaling and boundary conditions.

Monthly outcome: a manipulator under human supervision

M17 • 15 Feb 2028 - 19 Mar 2028 • 75 hours

Build a small, reproducible testbed for grasping and transferring an object. Treat it as an integrated robotic system connecting kinematics, planning, operator actions, data, commands, updates and engineering responsibility. The M17 deliverables are a simulated transfer of a simple object, a study of repeatability and failures, an interface for selecting modes and cancelling actions, and a documented assessment of threats and hazards.

Four areas of work

This month covers manipulation, human-robot interaction, ethics and cybersecurity, industrial applications and teamwork. Allocate the manipulator work so that every topic receives attention. A complex anthropomorphic hand, tactile skin and force-controlled grasping are not required here. Under the curriculum, the advanced OPT block on hand mechanics and tactile sensors takes 120 hours after M17 and is separate from these 75 hours.

What to bring from M14-M16

From M14, use the object-observation format and the descriptions of coordinate frames and uncertainty; from M16, use the prepared manipulator scene, collision checks, pre-grasp pose and trajectory validation. In the main experiment, specify the object's position precisely in simulation to distinguish manipulation errors from vision errors. Then deliberately offset the estimated position and orientation. A complete vision-guided grasping system is outside this month's required project.

Testbed and working environment

Prepare an existing manipulator model with a simple parallel gripper, a table, a cube or cylinder, and a target region. Work in Ubuntu 24.04 with ROS 2 Jazzy and a compatible MoveIt 2 configuration. You may additionally test your own mechanism once the baseline simulation experiment works; rated limits and real contacts require separate tests. A cancel button in a graphical interface does not, by itself, constitute a hardware emergency stop.

By the end of the month, explain why the manipulator might fail to reach its goal, who is authorised to issue commands and how the operator knows the current mode. State which data are retained, what happens when communication is lost and what must be checked before changing a limit. For the future hexapod, the same questions concern body motion, leg contacts and handing control to a human.

75 hours and a separate hand project

Four blocks

1-20: FK and IK, grasp coordinate frames, object transfer, contact and repeatability; 10 hours of theory and analysis +10 of practice. 21-40: teleoperation, levels of autonomy, modes, operator feedback and a small interface study; 10+10. 41-60: privacy, data errors, threat modelling, authentication, encryption and updates; 10+10. 61-75: PLCs, industrial buses, standards, requirements, verification and planning a team exercise; 5+10. Total: 35 hours of analysis and 40 of practice.

In the first three blocks, schedule reading in 2-hour sessions; in the last block, use 2+2+1. Divide practical work into four Sunday blocks of 10 hours, or equal periods that suit your schedule. Page dates follow the original curriculum, while step numbers indicate the learning workload. Overlapping module boundaries, rest, holidays and separate checkpoints do not add hours; OPT120 is counted separately.

Keeping the scope manageable

Start with one robot, one object type and one scene, then introduce variations. Keep development of custom motor control, vision and a physical hand outside this experiment. MoveIt plans the geometric sequence of actions; assess contact and object retention using separate simulator observations. A simplified object attachment demonstrates planning, but does not establish that grasping force is sufficient.

Preparation in the first block

Create a model-description file: simulator and MoveIt versions, object dimensions, joint limits, table position, coordinate frames and the source of the pose estimate. For each experiment, save `run_id`, the random-number generator seed, parameters, initial object state and result. Provide observation without issuing commands, manual control, plan execution and stopping. Maintain a shared log of `command_id`, mode and actor, even while there is only one operator.

Working folders: manipulation/ for tasks and scenes; operator/ for the interface and study protocol; evaluation/ for repeated experiments; review/ for hazards, threats and ethical decisions; requirements/ for criteria and changes. Store secret keys and participants' personal data outside the repository. In the report, distinguish observations, proposed causes and ways to test them: this helps another engineering team understand the system.

Manipulation: poses, approach and contact

What to study

Review forward kinematics, FK: it determines end-effector position and orientation from joint coordinates. Inverse kinematics, IK, solves the reverse problem; there may be several valid solutions or none. Define the world, base, tool, object and grasp coordinate frames. Define the grasp pose relative to the object and the pre-grasp pose as an offset along the approach direction. Before connecting the planner, verify transform composition and inversion using three known poses.

Transfer sequence: move into position → approach → close the gripper → confirm retention → lift → transfer → lower → open → withdraw. Define geometric and logical conditions for each stage. The attach operation in the planning scene changes the collision model, but does not measure contact. When introduced to force and impedance control, establish how position control differs from specifying a desired interaction; a complete force-control loop is studied separately.

Task 1. FK, IK and the grasp coordinate frame

Use an existing manipulator model and three reachable object poses. Specify the grasp offset, calculate the tool pose and solve IK while checking joint limits and self-collisions. Calculate FK for the solution and compare its position and orientation with the target. Save a table of poses, solutions and rejection reasons. Test an object outside the workspace and a rotation of 180°; if no valid IK solution exists, the limits must remain enabled.

Task 2. A complete grasp-and-transfer cycle

Use MoveIt Task Constructor or a compatible sequence of ROS actions. Save each stage and its result, add the table and object to the scene, and check attached geometry during transfer. Prepare a task description, configuration and reproducible demonstration. Separately verify that the object reached the target region and was released, and that the arm withdrew: successfully producing a plan without executing it must not count as task success.

Interpreting contact checks

For a physical simulation of retention, specify mass, friction and the gripper model; check whether the object can fall. When using a simplified attach, label the mode a kinematic demonstration (kinematic demo). Consider a virtual spring $F=K \cdot (x_{\text{des}} - x)$, with K in N/m, and a damper. Explain why high stiffness and delay can increase contact forces. Specified forces and speeds are educational parameters and do not replace checking rated specifications and conducting physical tests.

Repeatability and operator interaction

Task 3. Defining a successful transfer

Perform 10 runs with the same initial scene and 10 with small, predefined object offsets. Save success status and failure stage, final pose, cycle time and retention state. Before the experiments, specify the target region in metres and acceptable orientation. Present every result, mean error and dispersion; if deterministic simulation produces identical results, explain why.

Repeatability describes how close results are under identical conditions; accuracy describes closeness to a specified reference. An incorrect grasp offset can make the arm place the object in the same wrong location every time: test this case. In separate experiments, introduce an incorrect mass, friction or calibration offset and distinguish these causes. In a kinematic demonstration, evaluate the plan and pose; such an experiment does not measure holding force.

Human-robot interaction (HRI)

Study direct teleoperation, assisted control, autonomous task execution and explicit transfer of control. The operator needs the current mode, active goal, age of state data, reason for rejection and available action. A mode error occurs when a person expects manual control while an autonomous plan is active, or the reverse. Spatial interaction requires clearly indicated access zones and the robot's future path; low speed alone does not demonstrate contact safety.

Task 4. Modes and operator intervention

Create a simple panel or CLI with view, manual, auto and stopped states. Include `command_id` and mode in commands. Before a manual↔auto transition, complete or cancel the previous ROS action; after stopped, the old queue must not resume automatically. Save the transition table, interface and test log. Test a manual request during auto and a repeated start after communication loss.

Give the operator a scenario: position the gripper, start a transfer and stop it when the object is wrong. Display the planned path and current stage; present debugging diagnostics separately. The same principle applies to transferring control of gait: a human action must bring the system into a clear state, otherwise competing commands may enter a shared topic.

Interface testing and privacy

Task 5. Test the interface on a clearly defined task

Conduct a pilot study in simulation with two interface versions: for example, buttons alone and buttons with explicit mode and state indications. Use the same task: start a transfer, notice an intentionally incorrect goal and cancel the action. Explain the task and the data to be recorded to the volunteer beforehand. Conduct this experiment without physical robot motion near a person. If no participant is available, follow a predefined scenario yourself and state that no independent study took place.

Prepare 3-5 attempts for each version, alternate their order and provide the same initial training. Record the time until correct cancellation, mode errors, unnecessary actions and a brief answer to “what is the robot doing now?”. Save the protocol, anonymised table and three observations. So few attempts cannot establish statistical superiority; use them to correct specific unclear indications.

Task 6. Camera data and model errors

List the testbed's data: table images, object pose, operator actions, identifiers and logs. For each type, state its purpose and retention period, access rights and deletion method. In the demonstration, retain only the necessary table region or use synthetic data. If people or personal belongings appear in the frame, exclude unnecessary regions from the educational dataset in advance; transfer to an external service requires an appropriate basis.

Return to the baseline CNN from M14 and compare errors under two lighting conditions and with two object materials. Consider practical ethical questions: for whom and under which conditions does the system perform worse, who knows its limitations and who can correct the result? If the data contain no people, evaluate gaps in operating conditions and interface accessibility; there is no basis here for demographic statistics. Good average performance does not establish performance under unstudied conditions.

A documented decision

Write a short record: “question → alternatives → chosen decision → verification → remaining limitation”. For example, events and anonymised errors suffice to measure cancellation time, so operator video need not be retained. If pose-estimation confidence is low, request another observation before grasping. Include these decisions and supporting results in the final engineering review.

Command security: a threat model for your testbed

Distinguish hazards from threats

A hazard is a potential source of harm, such as unexpected gripper motion. A cybersecurity threat is the possibility of malicious interference, such as an unauthorised command or a substituted update. A code error, malfunction or attack may lead to the same consequence. For each case, record the cause, protection and test. Encryption does not correct an invalid joint limit, and a limit switch does not determine access rights to video recordings.

Task 7. A threat model for your own testbed

Draw the testbed boundaries: operator, ROS and DDS network, control process, storage and update channel. For each flow, identify the protected asset, sender, recipient, trust boundary and possible violation. Analyse unauthorised commands, replay of old commands, calibration changes, corrupted updates and log leaks. Prepare a table of 5-8 specific threats with priorities and owners of protective measures.

In an isolated experiment, test commands containing an unknown actor, expired timestamp, repeated command_id, NaN and out-of-range values. The expected result is rejection with an explanation and no simulated motion. DDS certificates, identity and permissions govern admission to the system and its channels; the application checks command freshness and prevents repeated execution. Enabling SROS2 alone does not protect against all these threats.

Task 8. Minimum protection and an update

Study authentication, authorisation, encryption and trust in updates. On the local testbed, configure minimum permissions for one publisher and subscriber using SROS2, or reproduce the official example in a separate namespace. Test permitted and prohibited publication. Save a public description of roles and permissions and verification logs; exclude private keys from the final package.

Prepare an educational configuration-update scenario. A new version of the limits requires a contents description, a hash and engineering review; a deliberately modified file must be rejected, and the previous working version must be restored in simulation. Explain that a hash detects change, but verifying authorship requires a trusted signature or channel. Parameters may change during motion only through a defined state transition.

Industrial applications: the cell and its requirements

What to understand at an introductory level

A PLC executes industrial cycle logic and processes inputs and outputs; the robot controller controls drives and the programme; a supervisory computer may plan actions and exchange data. An industrial bus connects devices and defines communication properties. Learn the purposes of EtherCAT, PROFINET and CAN; implementing these protocols is outside this month's scope. Distinguish ordinary control from separate protective functions.

ISO 10218 concerns industrial robots and their applications; ISO 13849 addresses safety-related parts of control systems; IEC 61508 provides the context of functional safety and its lifecycle. In M17, read official descriptions of scope and terminology, without claiming compliance with a standard. Applicability, the required level and verification of a real cell require separate input data and competent assessment.

Task 9. Sketch an educational cell

Draw the robot, table, object feed, presence sensor, operator panel and a notional PLC. Define a ready/request/busy/done/fault state exchange with cycle_id and test it using software process simulators. Save the diagram, I/O table and event sequence. Test a repeated request, a missing part, communication loss and reset during busy. The done state belongs to a particular cycle_id: an old event must not confirm processing of a new part.

Task 10. Requirements, changes and teamwork

Formulate 8-10 requirements: start conditions, acceptable scene, cancel behaviour, pose-estimate age, operator rights, logging and rejection on recovery. For each, provide a criterion and a link to its test. Change the grasp offset or speed and prepare a change request: reason, affected requirements, checks and restoration of the previous version. Support engineering review with data; label your own verification as self-review.

Plan how to gain teamwork experience: which part of the work can be submitted for review, which questions to ask a control or mechanical specialist, and which small contribution to a robotics project would demonstrate collaboration. Prepare a review package and a list of missing experience, such as industrial equipment, initial hardware commissioning and working with someone else's requirements. An educational testbed does not replace an internship or a verified professional qualification.

Eight checks for manipulation, the operator and threats

Manipulator and interface

S1 - unreachable pose: IK or the planner reports failure, and the system does not enter executing. S2 - the object touches the table during transfer: the collision model includes the attached object and rejects the plan. S3 - incorrect grasp offset: the robot misses repeatedly, and the report distinguishes repeatability from accuracy. In a physical simulation, add slipping; in a kinematic demonstration, attach does not confirm retention.

S4 - mode change during auto: a manual request triggers cancel, waits for coordinated completion and only then allows a manual command. Mode outputs are not mixed. S5 - loss of communication or pose estimates: data age increases, and the task receives the defined stopped or uncertain status. Network recovery does not start the old queue. Describe the simulator's actual response; this experiment does not demonstrate a certified protective stop.

Commands, data and the cycle

S6 - repeated command or unknown actor: the application rejects the request without repeating the cycle. Check `command_id`, mode and validity period separately from DDS participant identification. S7 - configuration change: integrity checking detects substitution, the update is not activated and the previous version can be reproduced. Separately state that without a trusted signature, forged authorship remains possible.

S8 - late done from the previous PLC cycle: the new request has a different `cycle_id`, so the old event does not count as success. Add an absent object after a claimed grasp: the hold-check stage must report the corresponding failure. Inspect the final archive manually: it must contain only necessary frames and events, without participants' personal data or secret keys.

Recording criteria

For each scenario, record the initial scene, triggering event, expected states, observed response, timing and remaining risk. Justify pose and timing tolerances as parameters of the educational task. This table does not define safe speeds for physical interaction with a person. The absence of errors in 10 repetitions does not prove zero failure probability.

Indicate how each result is supported: simulation, logic or document review, or a required physical test. Show the operator the reason for failure, whether motion has been cancelled and whether restarting is possible. If the execution outcome is still unknown, state that explicitly: hidden uncertainty prevents an accurate assessment of robot state.

Sessions 1-40: object transfer and the operator

Hours 1-10: five sessions of 2 hours

1-2: review FK and IK, coordinate frames and grasp offset; calculate three transforms by hand. 3-4: pre-grasp pose, approach, attach/detach and scene geometry. 5-6: transfer stages and criteria for planning and execution. 7-8: contact, friction, fundamentals of force and impedance control, and differences between kinematic and physical demonstrations. 9-10: repeatability, accuracy, test protocol and simulation limitations. Prepare the scene for the practical session.

Hours 11-20: practical session 1

11-12: Task 1, reachable and unreachable poses with FK verification. 13-14: Task 2, initial stages using Task Constructor or ROS actions. 15-16: complete transfer with the correct scene and target-region checks. 17-18: Task 3, repeated trials and pose variations. 19-20: failures S1-S3, grasp offset and cause analysis. If the physical grasp model is not ready, save the basic kinematic demonstration and state its limitations explicitly; the remaining weeks are needed for other topics.

Hours 21-30: five sessions of 2 hours

21-22: teleoperation, automated assistance, autonomy and command authority. 23-24: mode FSM, operator intervention, cancel and transition confirmation. 25-26: operator feedback, data age and mode errors. 27-28: spatial interaction, zones and limitations of the educational simulation. 29-30: pilot study, metrics, participant preparation and limitations of a small sample.

Hours 31-40: practical session 2

31-32: Task 4, a simple interface and mode table. 33-34: operator intervention during auto, communication loss and command_id log. 35-36: Task 5, two interface versions and pilot attempts. 37-38: analyse events, cancellation time and mode errors; fix one identified problem. 39-40: repeat the scenario and write a short HRI report stating its limitations.

If no volunteer is available, follow a previously written scenario yourself and list questions for external testing. Study results must reflect the attempts actually conducted. A simple version can use a CLI with prominent mode and state indications: this experiment focuses on making robot control understandable.

Sessions 41-75: data, threats and project handover

Hours 41-50: theory in 2-hour sessions

41-42: necessary data, privacy and retention periods. 43-44: machine-learning limitations, incomplete data and responsibility for decisions; connection to M14. 45-46: hazards, cybersecurity threats, protected assets and trust boundaries. 47-48: authentication, authorisation, encryption and command-freshness checks. 49-50: update integrity, signatures, restoration of the previous version and documentation of remaining risks.

Hours 51-60: practical session 3

51-52: Task 6, data inventory and checks across lighting conditions and materials. 53-54: Task 7, a flow diagram and five specific threats. 55-56: test commands with repetitions, NaN and an unknown actor; rejection logs. 57-58: Task 8, permitted and prohibited communication in an isolated SROS2 example. 59-60: update and restore the previous version, inspect the archive and review responsible engineering decisions together.

Hours 61-65: theory, 2+2+1

61-62: PLC, robot controller, industrial bus and cycle coordination; channel purposes. 63-64: official scopes of ISO 10218, ISO 13849 and IEC 61508, requirements and supporting results. 65: change management, engineering review and planning teamwork experience. Introductory hours cover terminology and the limits of standards' applicability; a full compliance analysis requires separate work.

Hours 66-75: practical session 4

66-67: Task 9, cell sketch, I/O and cycle coordination. 68-69: checks for cycle_id, missing parts, communication loss and late done. 70-71: Task 10, requirements and a change request for one parameter. 72-73: scenarios S1-S8, repeat the transfer after the change and restore the previous version. 74-75: final materials package, demonstration and handover to OPT120 and subsequent modules. The intervals total 75 hours.

How to reduce scope

If time is short, first remove the second object, complex contact model and interface decoration; then reduce the number of configuration variants. Keep a reproducible experiment with one manipulator, repeatability testing, operator intervention, the HRI study, threat and hazard assessment, industrial diagram and a requirement with its test. Move hand development, tactile sensors and force experiments to the separate OPT120 block, preserving M17 time for ethics and teamwork.

Acceptance checks and preparation for the hand project

Evidence for the four areas

Manipulation: scene, configuration, complete cycle or explicitly labelled kinematic demonstration, FK and IK verification, and a table of repeated trials. HRI: modes, cancellation, indications and a protocol for the participant study or self-conducted walkthrough. Ethics and cybersecurity: data inventory, use decisions, threat model and results of isolated tests. Industrial applications: cell diagram, I/O, requirements, change request and a plan to develop teamwork experience.

Prepare a README and a unified set of commands to reproduce the work. State in the report what was measured physically, what was tested in simulation and what was considered only theoretically. These experiments do not establish ISO/IEC certification, safe collaborative operation or industrial readiness. Save observed outcomes for the eight scenarios, and supporting results or a description of the remaining limitation for every significant protective measure.

Self-check questions

1. How does the tool pose differ from the object pose?
2. Why can IK have several valid solutions?
3. How does an attached object affect collision checking?
4. Why does attach not measure grasping force?
5. How does repeatability differ from accuracy?
6. How does the operator know who currently controls the robot?
7. What happens after cancel and communication loss?
8. How does a hazard differ from a cybersecurity threat?
9. Why does encryption not replace limit checks?
10. What does a hash prove about an update, and what does it not prove?
11. How does cycle_id prevent a false done?
12. What external experience do you still need to work in a robotics team?

What to hand over to OPT120

Hand over coordinate-frame descriptions, the baseline arm model, scene with objects, grasp parameters, reproducible action sequence, operator-intervention requirements and failure log. With this foundation, the separate 120-hour block after M17 develops hand mechanics, tactile perception, retention and physical testing. Its hours, equipment and criteria are accounted for separately; a completed anthropomorphic hand is not an M17 deliverable.

For the walking robot, retain the rules for mode selection, command authority, command-freshness checks and review of limits. They will be needed for external control of the body and legs. Before use on a real robot, test these decisions with the relevant mechanics, loads and actuator electronics. M17 is complete when you can explain and reproduce the entire testbed's operation, including failure and recovery.

Assigned reading: manipulation and human-robot interaction

Read the specified extracts before the corresponding tasks. Core reading explains principles; reference reading supports implementation. Sources may be in English; prepare your explanations and report in English. Reading is included in the 75 hours.

Core reading

1. Russ Tedrake, Robotic Manipulation (MIT, course notes). [Basic Pick and Place](#): Monogram Notation; Pick and place via spatial transforms. For Tasks 1-2: write down the object → grasp → tool transforms and one pre-grasp pose; other chapters are not needed now.

2. Christoph Bartneck et al., Human-Robot Interaction: An Introduction, 2nd ed., 2024. [Chapters Design, Research Methods, Robots in Society](#). For Tasks 4-6: one mode-indication principle, a pilot-study protocol and a data decision; skim Spatial Interaction to define the boundaries of the motion zone.

3. Ross Anderson, Security Engineering, 3rd ed. [Ch. 4 Protocols](#); [Ch. 6 Access Control](#). For Tasks 7-8: selected examples of authentication, message replay and permissions. After reading, distinguish an unauthorised command, an old command and an incorrect but permitted parameter.

Mechanics reference reading

4. Kevin Lynch, Frank Park, Modern Robotics. [§12.1.2 Contact Types](#); [§12.2.1 Friction](#); [§11.5 Force Control](#). For Task 3 and the transition to OPT120: distinguish touching, slipping and holding, and position and force. Leave the full theory of force-based grasping to the separate block.

Laboratory documentation

[MoveIt Task Constructor: Pick and Place](#) - stages and scene for Task 2; check main documentation against your pinned Jazzy version. [SROS2: README and tutorials on the Jazzy branch](#) - participant identification and permissions in Task 8, with permitted and prohibited communication.

Official ISO/IEC scope descriptions for Tasks 9-10 are listed on the preparation page. After reading, prepare a cell diagram, one verifiable requirement and applicability limits. Reading the standards in full is not required this month.

Fundamental concepts in M17

Pose. A body's position and orientation relative to a specified coordinate frame; both components are needed to describe a grasp.

Grasp frame. A coordinate frame on the object relative to which the desired end-effector position and orientation are specified.

Inverse kinematics (IK). Finding joint coordinates for a specified pose; a valid solution must respect limits and be checked using forward kinematics.

Pre-grasp pose (pregrasp). Position and orientation before approaching the object; it defines the approach direction and space for the open gripper.

Attached object. Geometry linked to the robot in the planning scene; this link alone does not establish physical retention of the object.

Repeatability. Closeness of repeated transfer results under identical conditions; a systematic offset may persist in every attempt.

Accuracy. Closeness of a result to a specified reference; it is evaluated relative to a defined goal, coordinate frame and measurement conditions.

Impedance. A specified dynamic relationship between motion and interaction force; it usually includes virtual stiffness and damping.

HRI. The study and design of human-robot interaction, including state understanding, control, errors and social context.

Mode error. An operator action based on an incorrect understanding of the current mode or of who controls the robot.

Hazard. A potential source of harm; the presence of a hazard alone does not describe an event's probability or the severity of its consequences.

Security threat. A possible violation of protected system properties, such as command substitution, data disclosure or loss of control availability.

Authentication. Verification of a participant's claimed identity or identifier by a trusted method; it does not itself define permitted actions.

Authorisation. A decision to permit a particular action by an identified participant, considering role, resource and access policy.

Encryption. Transforming data to conceal their content from those without the key; it does not replace checks of command authority, freshness and admissibility.

Hash. A short computed representation of content; comparison detects change relative to a trusted reference, but does not establish authorship.

Digital signature. Cryptographic verification of integrity and of a message's association with the signer's key, assuming trust in the corresponding public key.

Idempotency. A processing property whereby repeating the same logical command does not repeat the physical action or change.

PLC. A programmable logic controller that executes industrial cycle logic and processes inputs and outputs according to its configuration.

Cycle handshake. An exchange of readiness, request and completion states under defined rules; cycle_id links acknowledgement to a particular operation.

Equipment and software for M17

Use existing equipment

MacBook Pro M4; Ubuntu 24.04 ARM64 in UTM; ROS 2 Jazzy, Gazebo Harmonic and the MoveIt 2 configuration from M16. A virtual arm, table and object suffice for simulation. The M14 camera is needed only for the existing data in Task 6; do not repurchase sensors or the printer.

No mandatory purchases

M17 requires no purchases. Simulation is sufficient for all ten tasks; a physical arm, PLC, industrial bus and tactile sensors are unnecessary. The operator interface uses the keyboard and screen of the existing computer. A volunteer participant also needs no additional devices.

Install now: additions to the existing environment

In Ubuntu ARM64, add a Jazzy-compatible MoveIt Task Constructor only if it is absent and selected for Task 2; see [Pick and Place](#). Pin the package, commit and dependencies, and check main against Jazzy. A custom sequence of ROS actions is acceptable.

For Task 8, add [SROS2 tools and tutorials from the corresponding branch](#) in a separate ROS namespace. Save the chosen DDS implementation and its security settings. The mode FSM can be implemented in the existing code; a separate BehaviorTree library is needed only if a BT implementation is selected.

Installation checks

Launch the previous module's MoveIt scene, solve IK for one reachable pose, and execute a short transfer and cancel. In SROS2 with security enabled, check permitted communication and rejection of a participant without permission; keep keys out of the report. Test the graphical interface and networking inside UTM on your own computer, and save versions and logs.

Only after M17: the separate OPT120 block

Before building and testing the hand, prepare CAD, a BOM and requirements. Include one actuator and one selected contact or tactile sensor in the first order for a single-finger prototype, with suitable load capacity, power supply and interface. Determine quantities for the full hand after testing the finger. Purchases and the 120 hours are accounted for separately.

For introductory Tasks 9-10: [ISO 10218-1](#), [ISO 13849-1](#), [IEC 61508-1](#) - official scopes. This overview does not assign a PL/SIL or establish installation compliance.

Preparation and checks use time already allocated within the original 75 hours to review reading and setup. Delivery of possible OPT120 equipment is outside the learning workload. Check the availability of equipment and installed components.

Motion fundamentals: fingers and grasping

Required: touching is not yet retention

Kinematics establishes whether contact has been reached; statics establishes whether admissible forces can balance the object's weight and moments. First draw the object and finger separately, showing pairs of equal and opposite forces and the moment arm. For the finger, f_{ext} always denotes the force exerted by the object on the robot; the robot's force on the object points in the opposite direction.

A simple retention calculation

Consider a stationary object between two symmetric vertical fingers. Each contact has a normal force N and upward friction no greater than μN ; the weight mg acts downward. For a centred object without an external rotational moment, $2\mu N \geq mg$ is necessary. This checks that particular vertical equilibrium; it does not prove retention against every possible disturbance.

With $m=0.20$ kg, $g=9.81$ m/s² and $\mu=0.50$: $mg=1.962$ N and $N \geq 1.962$ N per finger. Choosing $N=2.5$ N gives a maximum total friction force of 2.5 N. If μ falls to 0.25, the limit becomes 1.25 N and can no longer support the weight. Equality at the boundary provides no margin; μ here is an illustrative model parameter.

For a massless vertical link with a normal-force moment arm of 0.04 m, the holding torque magnitude is $0.04 \cdot 1.962 \approx 0.0785$ N·m; friction in this geometry acts along the link. A real finger requires its Jacobian, mass and configuration: in statics, $\tau_{\text{act}} + \tau_g + J^T f_{\text{ext}} = 0$, where τ_g denotes joint torques due to gravity. Changing the external force direction also changes the sign of the compensating torque.

Why one motor does not determine two angles

For a finger with two independent joints, one taut, inextensible tendon can impose only $\Delta l = r_1 q_1 + r_2 q_2$ for constant moment arms. With $r_1 = r_2 = 0.01$ m and $\Delta l = 0.005$ m, $q_1 + q_2 = 0.5$ rad: the pairs (0.1;0.4) and (0.3;0.2) both satisfy the command. The actual pose depends on springs, contact and dynamics. This illustrates underactuation; when an object stops one link, the distribution of motion in the other changes.

Two exercises for Tasks 2-3

1. In the object-transfer report, save a force diagram and a table for three values of μ and two masses. With $N=2.5$ N, predict by hand whether the object will be held or will slip. Compare with contact simulation if available; with simplified attach, label the result a model calculation. Planner success does not establish this equilibrium.

2. Plot the line $q_1 + q_2 = 0.5$ and add the contact condition $q_1 = 0.2$ rad: this leaves $q_2 = 0.3$ rad. Save the plot and explain which physical condition determined the configuration. A full-size hand will need separate mechanics and sensors; a calculation without purchases suffices for now. Do not send torque commands to position-controlled servos.

Time allocation: 1 hour from 7-8 for statics; 1 hour from 17-18 to check the table as part of Task 3, and 1 hour from 19-20 to explain failure. These are within the original 75 hours. Building the hand and working with tactile sensors belong to the separate OPT120 stage after M17.

Assigned reading: Lynch/Park, [Modern Robotics §5.2 and §12.2.1](#) - torque and friction; Tedrake, [Fully-actuated vs Underactuated Systems: Definitions](#) - independent coordinates and available control inputs.

Why a robot needs a learned model

M18 • 30 Apr 2028 - 04 Jun 2028 • 75 hours

Monthly outcome

Create a reproducible classifier for foot contact with a surface and compare it with a simple rule and logistic regression. Feed it joint and IMU measurements available to the robot, together with leg-motion features. Its outputs are contact probability, a decision of “contact”, “no contact” or “insufficient data”, and the time taken to obtain that decision. Save the data, dataset membership, training configuration, model, error analysis and CPU execution-time measurements.

Learning here addresses a narrow perception task. The classifier helps determine which foot can be trusted when estimating body state. For now, it operates on recordings or alongside a classical algorithm in simulation. A false confirmation of contact can make the estimator treat a slipping or lifted leg as stationary; average accuracy alone therefore does not establish the model's suitability.

What must be ready before starting

Prepare synchronised recordings from previous simulations: time, episode and leg identifiers, joint positions and velocities, body angular velocity, foot position or velocity and an available force estimate. Save simulator contact ground truth only for labelling and evaluation. If recordings are insufficient, perform controlled motions of one leg: free swing, touchdown, stable support, liftoff and slipping. Data collection does not require a complex gait.

Check that you can read CSV and Parquet, plot a time series, explain the mean and standard deviation, calculate a dot product and differentiate squared error. Practise missing skills in the first block. Use your development experience to describe the data format, ensure reproducibility and investigate causes of model errors; there is no need to repeat all Python fundamentals.

What the result establishes

The main outcome is a justified educational experiment. The network is not yet permitted to control a physical robot. The original curriculum mentions G3, and the overall calendar allocates separate time to integration checks. In M18, prepare the data, model and limitations for that stage. Checks are required before any physical use, regardless of the date of G3. In later modules, the classical estimator and controller must also work without machine learning.

The connection to the hand is direct: the same process applies to estimating finger-object contact. However, a positive “touch” label does not mean a “reliable grasp”. Use one main foot-contact task this month; describe the differences in features and errors for transfer to a finger, without requiring a second training project.

75 hours: from data to a decision

Four learning blocks

Hours 1-20: 10 hours of theory and analysis plus 10 hours of practice. Study regression, classification, loss functions, bias and variance. Define features, labels and training, validation and test sets comprising whole episodes. Prepare a dataset description, check synchronisation and establish the first classical baseline method. The deliverable is data that do not expose the correct answer as a model input.

Hours 21-40: 10 hours of theory and 10 hours of practice. Study tensors, automatic differentiation, layers, optimisers, normalisation and overfitting control. Write a small PyTorch training loop and run a compact network. The outcome is a reproducible run, rather than a lengthy architecture search. Change one hypothesis in each experiment; preserve the original configuration unchanged.

Hours 41-60: 10 hours of theory and 10 hours of practice. Study confusion matrices, precision and recall, ROC and PR curves, calibration, class imbalance and data shift. Investigate failures, remove feature groups and test abstention. Choose the contact-confirmation threshold using the validation set; keep test episodes closed until the final comparison.

Hours 61-75: 5 hours of theory and analysis plus 10 hours of practice. Study export, numerical precision, quantisation and the difference between latency and throughput. Measure the complete path from an available measurement to a decision, reproduce the model in a separate process and prepare a computing-platform decision. Total: 35 hours of theory and 40 of practice; unattended waiting for training does not replace learning work.

Scheduling sessions

Divide each 10 hours of theory into five two-hour sessions. Complete practical work in Sunday blocks of 10 hours; each may be divided into four periods with breaks excluded from learning time. Divide the final 5 hours of analysis as $2 + 2 + 1$. Numbering denotes the sequence of learning workload. Boundary dates may overlap adjacent modules; rest, holidays and integration blocks remain as in the curriculum, and no hour is counted twice.

At the start of each session, write one testable question, such as “does foot velocity reduce false contact reports at liftoff?”. At the end, save the configuration, plot, conclusion and next step. Limit the active list to three tasks: the current hypothesis, a known data error and the next check. This keeps the module within 75 hours and helps distinguish a model improvement from a change to the experiment itself.

Data, labels and splits without leakage

Task 1. Describe the contact data

For each field, state its name, units, frequency, source, delay and availability on the future robot. Separate the features and ground_truth tables. Features must not contain true contact force, scenario identifier, a future frame or foot coordinates relative to the exact surface when the robot lacks the corresponding sensor. This distinguishes measured quantities from information known only to the simulator. Save data_contract.md and ten rows checked by hand.

Define the positive label from simulator contact and a minimum load chosen for this model. Mark ambiguous touchdown moments uncertain and describe how they are evaluated. Do not adjust the labelling threshold to suit network results. Manually check several intervals before, during and after contact: the label must match the event without a one-packet shift. Report the number of uncertain rows separately.

Task 2. Split by complete experiments

First assign episode_id to complete independent runs. Then create an explicit list of episodes for the train, validation and test directories: for example, 60/20/20 percent of episodes; the number of independent groups matters more than the exact proportions. Adjacent windows from one experiment must stay in the same subset. Group consecutive files from one robot recording together. Separately save an OOD set with new friction, noise or load.

Check that identifiers and source-recording ranges do not overlap. Determine normalisation, missing-value imputation, balancing and feature-selection parameters from the training set only. With few episodes, use grouped cross-validation within it, while retaining separate final test groups. Show the class and condition distributions across subsets. Randomly shuffling time-series rows makes evaluation unjustifiably easy.

Task 3. Check causal availability

For a window of length W , use only measurements timestamped no later than the decision time. Construct the mean, dispersion and maximum velocity over the past window, plus a missing-packet flag. Change future values in the source file: the prediction for the current time must remain unchanged. Deliver a causality test and a plot of window delay. If good metrics require future information, that version is suitable for analysing recordings, but not for online contact estimation.

Start with 6-12 understandable features. Millions of nearly identical frames do not add new conditions. Several episodes with different surfaces and motion phases are more useful than another hour of stationary-foot recordings.

Baseline models and loss functions

What to study before a neural network

Regression predicts a number, such as normal load in newtons; classification predicts a category or contact probability. For an educational comparison, plot squared error against the numerical prediction and binary cross-entropy against the probability of the correct class. Do not conflate objectives: low force-estimation error does not guarantee reliable touchdown detection, particularly near zero.

In the binary task, y is 0 or 1 and p denotes contact probability. The loss is $-[y \log(p) + (1-y) \log(1-p)]$. It penalises confidently wrong answers more strongly. Use a numerically stable implementation operating on logits; avoid manually taking the logarithm of zero or applying sigmoid twice. The resulting probability is used for analysis, not as a multiplier for actuator commands.

Task 4. Two baseline methods for comparison

Build a simple rule from available features: low foot velocity together with a sufficient load estimate, if one exists. Choose thresholds on the validation set. If load is not measured, state that and use kinematics and the commanded phase. The second baseline is logistic regression with normalisation calculated on the training set. Both methods and the network must use identical episodes, inputs and window delay.

Save `predictions_baseline.csv`, a confusion matrix and errors by episode. Test a stationary leg hanging freely: low velocity alone does not mean contact. Test a slipping support: contact can exist at nonzero velocity. These two cases explain the need for a feature set and why even a reasonable rule is not physical proof of support.

Task 5. Bias, variance and regularisation

Train logistic regression on a small and the full training set, then a compact network with and without regularisation. Compare training and validation losses. High error in both subsets suggests possible problems with features, the model or labels. Low training error with high validation error calls for checks of overfitting and differences in conditions. These observations help investigate a cause but do not establish it automatically.

After the first baseline result, test at most three settings: for example, two regularisation values and one hidden-layer width. Set an overall epoch limit and early stopping based on the validation set. Save a table of hypothesis, change, observation and decision. Claiming improvement requires more than one successful run. Do not repeatedly use the final test set to select an architecture.

PyTorch: training a small model

Minimum topics

Study tensor shape: a row corresponds to a measurement window and a column to a feature. For N examples with D features, the input shape is $N \times D$; the binary model outputs one logit per example. Check dtype, device and target dimensions. Use normalisation and a small multilayer perceptron, MLP, with one or two hidden layers. Complex convolutions, transformers and architecture search are unnecessary for this task.

Task 6. One reproducible run

Create a Dataset from the episode list, a DataLoader, model, loss function, optimiser and separate evaluate function. During training, sequentially zero the gradients, perform the forward pass, calculate loss, perform the backward pass and take an optimiser step. For evaluation, disable gradient accumulation and enable eval mode. Save normalisation, feature order and schema version alongside the weights. Loading must explicitly reject an input-schema mismatch.

Check the loop on a small set of correctly labelled examples: the model should substantially reduce training error. This tests functionality, not generalisation. Then restore the normal data split. Deliberately swap two columns and check that the schema detects the error before prediction: degraded answers alone reveal a format change too late.

Task 7. Gradients and optimisers

For a model with one linear layer, compare one autograd gradient component with a central finite difference for a small weight perturbation. Check the update direction and change in loss after a small step. Then compare SGD and Adam using the same number of epochs, batch size and number of examples. Record the rate of loss reduction and final validation metric; this experiment does not establish the universal superiority of an optimiser.

Study learning rate, weight penalties and a simple step-size reduction schedule. Run one experiment with a deliberately excessive step size to diagnose divergence, then restore a working setting. For each run, save the random-number generator seed, library versions, device, commit, episode list, optimiser settings and best weights. A seed improves repeatability but does not guarantee bitwise agreement across platforms and versions; compare within a pinned environment.

The baseline platform is the CPU. Apple Silicon can use MPS when the required operations are supported, but the MacBook M4 is not a CUDA platform. No GPU purchase is planned for this small tabular network; first measure actual data-preparation and training time.

Performance, confidence and unfamiliar conditions

Metrics must reflect the robot's errors

Let contact be the positive class. Precision = $TP/(TP+FP)$ is the fraction of reported contacts that are correct; recall = $TP/(TP+FN)$ is the fraction of actual contacts detected. A false contact, FP, is particularly harmful to contact odometry, while a missed contact, FN, reduces available information. State the rule for a zero denominator. Additionally evaluate touchdown and liftoff detection delay relative to labelled events, in milliseconds.

Task 8. Thresholds and imbalance

On the validation set, plot a confusion matrix, PR curve and ROC curve. Choose a threshold considering the acceptable number of false contacts in the educational scenario. Define this criterion before the final test; it is not a universal safety limit. Show the number of events and examples in each class. If support dominates the recording, an “always contact” rule may achieve high accuracy: include it to show this metric's limitations.

For rare liftoff events, compare ordinary and weighted losses; calculate class weights on the training set. Recheck calibration after changing example sampling or weights. The test-set distribution must match the stated scenario, without artificial balancing. Show performance for each episode and the average across episodes, so a long recording cannot conceal complete failure on a short one.

Task 9. Calibration and abstention

Group validation probabilities into bins. Compare the mean predicted probability with the actual contact fraction, and show the number of examples in each bin: this produces a reliability diagram. Calculate the Brier score as the mean of $(p-y)^2$. To fit calibration, reserve separate groups from training or validation data; do not use test data. With a small dataset, report estimation uncertainty and the number of observations.

Define reasons for abstention: missing mandatory fields, a stale window, values outside known ranges and insufficient confidence. Range checks detect only some OOD cases; high network probability does not prove that conditions are familiar. Under new friction and noise, measure abstention frequency and remaining errors. On abstention, send unknown to the classical estimator, which applies verified rules and does not automatically accept contact.

Laboratory: contact recognition

Specification for contact_model_v1

Replay leg recordings by timestamp. First check the schema and data freshness, then construct features solely from measurements available at that moment. The model outputs a probability, the threshold block produces a status, and a separate check compares it with contact ground truth. No actuator commands are sent here. On one plot, show contact ground truth, prediction, unknown, foot velocity and rejection reason; mark touchdown, liftoff and slipping with lines.

In the input-data description, list files, hashes, robot model, simulator, durations, classes and conditions. Specify window, features, seed, model, epoch limit, thresholds and abstention rules in the configuration. Save predictions for every valid time instant, episode metrics and timing measurements. Account for rejected rows too: their number, reasons and proportion of all measurements must remain visible, otherwise removing difficult cases artificially improves the metric.

Task 10. Final comparison on the test set

After choosing the configuration, pin the commit and open the test groups. Evaluate the rule, logistic regression and chosen MLP in the same way. Compare precision, recall, event delay, unknown fraction and processing time. Perform paired comparisons on identical episodes. To estimate variability, resample whole episodes rather than neighbouring frames. If there are too few episodes for a confident conclusion, state that.

The curriculum criterion is improvement over a simple baseline on held-out data. Before testing, choose the primary metric and a false-contact limit. If the network does not win, retain the negative result and mark the criterion unmet; collect new data in the next iteration. Reproducible evaluation is still useful: it may provide sound grounds for choosing the simpler model.

Task 11. Feature ablation

Remove commanded-phase information, kinematic velocity and IMU features in turn. Train each version on the same training set and select settings on the same validation set. Use a separate analysis set, preserving the independence of the final test. Compare errors during liftoff, slipping and a stationary leg in the air. Formulate a hypothesis about the value of each information source and save two plots of failed episodes.

Finally, reproduce the work from a clean directory using the README. Run training, performance evaluation and timing through separate commands, so checking a saved model does not require retraining.

Tests: eight ways to go wrong

Data and behaviour checks

1. **Adjacent windows in different subsets.** Prepare a deliberately invalid list in which one `episode_id` appears in train and test. The check must reject it before training. On diagnostic data, compare random row splitting with grouped splitting. Explain the inflated estimate through correlation between neighbouring observations. Save an automated overlap report and both estimates, identifying the split method.
2. **A stationary leg in the air.** Replay a halted swing without contact. A velocity-only rule may produce FP; test the network and combined baseline under the same conditions. Record the duration of false contact and explain which measurements can distinguish the cases. If the sensors are insufficient, use unknown.
3. **A slipping support.** Use contact with nonzero tangential velocity. The contact label remains positive; “non-slipping support” is a different task. Check that the analysis does not call all slipping frames mislabelled. Provide a separate support-quality indicator for the future estimator, without substituting it for the current objective.
4. **Missing data and delay.** Delete a sequence of packets and delay one channel. Record the age of the oldest measurement used. After the limit specified in the configuration, the decision must become unknown. Check instants before and after the threshold, and recovery after a full window has accumulated from data already received.

Model and execution checks

5. **New load and friction.** Test physically valid OOD episodes outside the training ranges. Compare errors with the ordinary test set and measure the fraction of OOD cases detected. Describe a confidently incorrect answer and the remaining risk: perfect detection of unfamiliar conditions cannot be expected.
6. **NaN and a broken schema.** Supply a non-numeric value, reordered columns and a different velocity unit. NaN and schema-version mismatches must be rejected before the network. Check unit errors using explicit metadata and ranges: not every unit change is detectable from a single number. Record the precise reason code.
7. **Repeatability.** Repeat the fixed configuration in the same environment, then compare three seed values. Report variability in the primary metric and training time, including all runs. Evaluate agreement across platforms using a numerical tolerance chosen beforehand.
8. **Export and a slow consumer.** Compare the original model with the saved or exported model on identical inputs, then artificially slow processing. Reading a result later must not change its age. Measure the number of missed processing deadlines and the observer's response; save the actual timestamp sequence.

Session sequence: hours 1-40

Block 1: task and data

Hours 1-2: draw the chain “leg motion → measurement → feature → label → decision”. Study how contact differs from stable support and permission to transfer weight. List errors that the experiment must detect. Hours 3-4: study regression, classification, loss functions and overfitting; calculate two losses and a confusion matrix for ten predictions by hand. Do not move to PyTorch until the values in this table are clear.

Hours 5-6: study grouped splitting and causal windows. On paper, split six short experiments so that one scenario is not fragmented into neighbouring rows in different sets. Hours 7-8: describe features, units and sensor availability. Hours 9-10: establish the baseline models, primary metric and test protocol. These five sessions provide the required 10 hours of analysis.

Hours 11-20 - first laboratory: 2 hours for extraction and manual label checks; 2 for episode lists and leakage checks; 2 for causal feature preparation; 2 for the rule and logistic regression; 2 for failure plots and recording the baseline result. If importing data is difficult, reduce the number of scenarios while retaining both classes and independent episodes. Calculate normalisation after splitting the data, and preserve the test set.

Block 2: training a small model

Hours 21-22: tensors, shapes and types; a dimension table from input to loss. Hours 23-24: autograd and the gradient step; manual derivative and finite-difference checks. Hours 25-26: Dataset, DataLoader, train/eval modes and disabling gradients for evaluation. Hours 27-28: SGD, Adam, learning rate, regularisation and early stopping. Hours 29-30: saving the model, configuration, seed values and experiment log.

Hours 31-40 - second laboratory: 2 hours for the training loop and tiny-dataset check; 2 for the first full run; 2 for debugging gradients and data preparation; 2 for two bounded hypothesis comparisons; 2 for restoring saved weights and checking reproducibility. Retain unsuccessful runs. If the network does not learn, check labels, dimensions, gradients and normalisation before changing model size.

By the end of hour 40, retain the original classical method unchanged and one working learned-model configuration. A runs directory, metrics in CSV and a README with the reproduction command suffice. Every number must be traceable to specific data and settings; an additional interface, cloud logging and complex automation are optional.

Session sequence: hours 41-75

Block 3: testing hypotheses

Hours 41-42: precision, recall and PR; check denominators and the meaning of the positive class by hand. Hours 43-44: compare ROC and PR under class imbalance, and select a threshold on validation data. Hours 45-46: calibration and its diagram, with examples of confidently wrong answers. Hours 47-48: unfamiliar OOD conditions - noise, load, friction and time shifts. Hours 49-50: feature-ablation plan and error inventory.

Hours 51-60 - third laboratory: 2 hours for confusion matrices and thresholds; 2 for calibration and unknown; 2 for missing data and shifts; 2 for limited feature ablation; 2 for the final comparison and analysis of the two worst episodes. Identify the dataset for every number. Once the final test is opened, a new setting requires an explicitly declared next iteration and new independent evaluation.

Block 4: measurements when applying the model

Hours 61-62: study export and the difference between a weights format and an executable graph; check that the entire preprocessor transfers with the model. Hours 63-64: study batch size, single-decision latency, warm-up and memory. Hour 65: describe remaining deployment limitations and what the later G3 check will require. These are the final 5 theory hours, divided as 2 + 2 + 1.

Hours 66-75 - fourth laboratory: 2 hours for loading, export and output comparison; 2 for CPU measurements; 2 for reproducing the complete measurement-to-decision path; 2 for reproduction from a clean directory; 2 for the report and computing-platform choice. Measure after warm-up, separately from cold start. State repeat count, batch=1, thread count, CPU model, power mode and competing workload.

Task 12. Is an accelerator needed?

Separate total latency into window accumulation, preparation, prediction computation and result transfer. Measure p50, p95 and p99 - values below which 50, 95 and 99 percent of observations lie - plus the maximum and deadline-miss count. These measurements do not prove an upper time bound. Throughput in examples/s at a large batch size does not replace latency for one fresh measurement. A GPU can accelerate the network while adding transfer overhead.

Justify any need for Jetson and CUDA using latency, memory, energy consumption and supported operations. Purchasing hardware and authorising learned walking control are considered at the separate G3 checkpoint. If the CPU meets requirements, an accelerator need not be purchased. Otherwise, first identify the latency source and reduce the model. Test quantisation separately: repeat accuracy, calibration and OOD evaluation. Reduced memory use does not guarantee acceleration.

Acceptance checks and handover

Materials that support the result

Create a `contact_model_v1` directory containing README, data_contract, manifests, configs, baseline, model, reports and measurements. In the README, explain how to obtain the source logs and provide train, evaluate, replay and profile commands. The report must state the objective, measured features, label provenance, operating limits, environment versions and device specifications. Keep exact configurations even for rejected models; otherwise, the reasons for preferring the final choice cannot be reconstructed.

Criterion 1: no overlapping episodes, future frames or preprocessing fitted on test data; automated and manual checks are attached. Criterion 2: the baseline is compared with the network on a set kept closed until the final evaluation, using a preselected metric and FP limit; if there is no improvement, the criterion remains unmet. Criterion 3: repetition with the same config and seed gives explainably similar results in one environment. Criterion 4: the complete data path is measured, including processing before and after the network.

Criterion 5: OOD errors, unknown, missing data and delays are shown; the classical branch remains available. Criterion 6: a review of “data, model and limits of use” is prepared for integration testing. A saved weights file does not mean readiness for control. Before physical use, sensors, the actuator chain, independent limits and stopping are checked separately; the calendar placement of G3 does not replace these actions.

Self-check without notes

Explain: 1) why row-based splitting inflates performance; 2) why normalisation must not be fitted on all data; 3) how logits differ from probabilities; 4) why a stationary foot may not touch the floor; 5) why contact and absence of slipping are different objectives; 6) how confidently confirming a false contact affects body-state estimation. Support your answers with your own plots.

Also answer: 7) how calibration differs from classification; 8) why ROC can conceal the importance of rare errors; 9) where window-accumulation delay arises; 10) what a seed provides; 11) why export requires retesting; 12) which measurements justify a GPU. If an answer depends on an unknown robot characteristic, record its clarification as a requirement for the next stage.

If time is short

If time is short, remove additional architectures, then the MPS/GPU comparison, then the practical quantisation experiment while retaining its theory. Keep the CPU, small network, simple features and grouped data split. Retain baselines, leakage checks, OOD, full-latency measurement and reproducibility. Hand the probability and unknown formats to M19 with their limitations. Contact estimation for the classical gait must have a verified path without the network.

Assigned reading: learning from robot data

Read the specified extracts before the corresponding tasks. Core sources explain principles; references assist implementation. Reading is included in the 75 hours. Sources may be in English; prepare your explanations and report in English.

Core reading

1. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor, An Introduction to Statistical Learning with Applications in Python, 2023. Chapters 2, 4, 5: authors' materials. Statistical Learning - error and generalisation; Classification - logistic regression; Resampling Methods - validation and cross-validation. For Tasks 1-5: justify episode splitting and baseline selection; not all laboratory exercises need to be completed.

2. Aston Zhang, Zachary Lipton, Mu Li, Alexander Smola, Dive into Deep Learning. §2.5 Automatic Differentiation; §3.7 Weight Decay; §5.1-5.3 (Multilayer Perceptrons; Implementation; Forward/Backward Propagation). For Tasks 6-7: one small MLP, gradients and weight updates; perform the exercises using your own contact features.

Reference for an unclear result

3. Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, 2016. Ch. 5 Machine Learning Basics; Ch. 7 Regularization for Deep Learning. Find explanations of model complexity, overfitting and your chosen regularisation for Task 5. Ch. 8 Optimization for Training Deep Models - only the selected optimiser for Task 7, without reading the whole chapter.

Documentation needed for the experiment

PyTorch: Learn the Basics - Tensors, DataLoaders, Autograd, Optimization, Save/Load for Tasks 6-7.
scikit-learn: Common pitfalls - Data leakage and preprocessing for Tasks 1-3; fit transformations only on the training set.

scikit-learn: Probability calibration - Calibration curves and Calibrating a classifier for Task 9. Explain FP, FN and the chosen threshold using your own confusion matrix; supplement accuracy with error analysis.

First read the core source for the topic, then test your understanding on the task. Consult the third book for explanations of a specific difficulty; reading all three textbooks in full is not required this month. CPU measurements and conditions for accelerator use are described on the preparation page.

Fundamental concepts in M18

Feature. A measured or computed quantity available at decision time; future values are inadmissible.

Label. The target answer for training; here it denotes contact under the adopted rule, not absence of slipping.

Training set (train). Data used to fit model parameters and feature transformations; these parameters are learned only from this set.

Validation set (validation). Separate data for selecting hyperparameters and the threshold; used before final evaluation.

Test set (test). Separate episodes for final evaluation; tuning on them compromises the evaluation's independence.

Data leakage. Information used in training or evaluation that is unavailable during application; for example, adjacent frames placed in different subsets.

Standardisation. Subtracting a mean and dividing by a scale; statistics are obtained from the training set and fixed.

Logistic regression. A sigmoid applied to a linear combination of features, yielding a class probability; here, for binary contact.

Loss function. A measure of disagreement between prediction and label that training minimises; it is not the same as operational usefulness.

Gradient. A vector of loss derivatives with respect to parameters; it indicates the direction of local increase for weight updates.

Backpropagation. Calculating gradients using the chain rule; an optimiser changes weights in a separate step.

Regularisation. A constraint or penalty against fitting peculiarities of training data; its strength is selected on validation data.

Overfitting. Improvement on training data by fitting their specific features, accompanied by deterioration on new episodes from the same distribution.

Precision. The fraction of true contacts among predicted contacts: $TP/(TP+FP)$; it reflects false confirmations.

Recall. The fraction of detected contacts among all positive labels: $TP/(TP+FN)$; it reflects misses at the chosen threshold.

Probability calibration. Agreement between probabilities and frequencies on independent data; accurate classification does not guarantee it.

Decision threshold. A boundary for class selection or abstention; set according to error costs before the final test.

Distribution shift. A change in conditions relative to training; new materials can reduce performance while confidence stays unchanged.

Ablation. Removing a feature or component under an unchanged protocol to assess its contribution and possible leakage.

Inference latency. The time required for prediction; total latency also includes preparation, transfer and subsequent processing.

Equipment and software for M18

Use existing equipment

Use the MacBook Pro M4, previous synchronised leg and IMU recordings, and the simulator. Perform training and timing primarily on the CPU in macOS ARM. If necessary, export ROS recordings from Ubuntu 24.04 ARM64 in UTM; preserve episode_id, time, units and the separation of features from ground truth.

No mandatory purchases

The required classifier needs no Jetson, NVIDIA GPU, new camera or new contact sensor. First measure the entire data-processing path of a small MLP and logistic regression. Obtain contact labels through the adopted simulator protocol and store them separately from model input features.

Install now: a separate Python environment

In the uv/venv already used on macOS ARM, add torch, scikit-learn and JupyterLab if absent; reuse NumPy/Matplotlib/pytest. Choose the Python version for compatibility with the pinned packages, and save the lock and manifest. Instructions: [PyTorch Start Locally](#), [scikit-learn install](#), [Jupyter install](#).

Select CPU execution; CUDA is not installed for Mac M4. Add pandas and pyarrow only if needed to read input tables; the established loader suffices for CSV. Do not install training dependencies in ROS's system Python. Transfer data between environments using files in an agreed format.

Installation and reproducibility checks

Import torch and sklearn, check CPU tensor operations, perform forward and backward passes through a small network and take one optimiser step. Train LogisticRegression on a small training set, then save and reload it. After restarting the notebook, reproduce the saved [episode split](#) and [classification metrics](#).

Record seed values, versions, thread count and chosen device; repeat a short run following [PyTorch Reproducibility](#). For Task 12, measure warmed-up repetitions and cold start, including features and subsequent processing: [PyTorch Benchmark](#).

Optional, after measurement

You may compare the built-in [MPS](#) with CPU after checking operation availability; retain the CPU result. An accelerator purchase is considered only for a demonstrated computational limitation and a separate G3 decision. G3 on 27.11-10.12.2028 is still ahead; M18 does not complete it.

Installation and checks use time already allocated to setup within the original 75 hours. This month's result is a model and measurements from recordings or simulation. The classical algorithm must operate independently of machine learning.

Motion fundamentals: a model learned from data

Required analysis: what regression learns about a leg or finger joint

From physical law to learned parameters

Before Tasks 4 and 7, explain the distinction between an unknown parameter and an unknown state. Consider a revolute joint in a vertical plane without contact: q is the angle from the downward vertical in rad, $\dot{q}$ is velocity in rad/s and $\ddot{q}$ is acceleration in rad/s². Positive torque τ acts in the direction of increasing q . Geometry and axis direction have already been checked.

$I \cdot \ddot{q} + b \cdot \dot{q} + k \cdot \sin(q) = \tau$. I is the moment of inertia about the axis, kg·m²; b is the viscous-friction coefficient, N·m·s/rad; $k = m \cdot g \cdot l_c$ is the known gravity coefficient, N·m; m is link mass in kg, $g = 9.81$ m/s² and l_c is the distance from the axis to its COM in m. Assume a rigid link, constant I, b and no backlash. Take τ from the educational simulation; a commanded servo angle and supply current are not measured joint torque.

Move the known gravitational torque term: $y = \tau - k \cdot \sin(q)$, $\phi = (\ddot{q}, \dot{q})$, $\theta = (I, b)$. This gives $y = \phi \cdot \theta$: motion is nonlinear in q , but the regression is linear in the unknown I, b . Minimise the sum of squared residuals in N²·m². Conditions $I > 0$ and $b \geq 0$ express positive kinetic energy and power dissipation $b \cdot \dot{q}^2$; low error alone does not ensure them.

A small calculation: parameters and a new pose

Use the educational value $k = 0.20$ N·m. Two independent samples at $q = 0$ are $(\dot{q}, \ddot{q}, \tau) = (1, 2, 0.14)$ and $(2, -1, 0.03)$, in the stated SI units. Thus $2I + b = 0.14$ and $-I + 2b = 0.03$, giving **$I = 0.05$ kg·m², $b = 0.04$ N·m·s/rad**. For a separate sample $q = \pi/6$, $\dot{q} = 0.5$, $\ddot{q} = -2$, predict $\tau = -0.10 + 0.02 + 0.10 = \mathbf{0.02}$ N·m.

Two exercises with the baseline model

A. In the existing notebook, solve this system using NumPy and save `id_check.csv` containing the inputs, units and prediction for the third row. Check the answer by substitution and power balance: in the second row, input power $0.03 \times 2 = 0.06$ W equals the kinetic-energy rate -0.10 W plus friction losses 0.16 W. This checks the calculation on known data; generalisation is evaluated on separate episodes, as in Tasks 1-5.

B. Set $\ddot{q} = 2 \cdot \dot{q}$ in every row. The feature matrix has rank 1: only $2I + b$ can be identified, not both parameters separately. More identical motions do not resolve this. In a free-motion recording, check synchronisation of τ and q ; differentiation amplifies noise. For a current estimate, use only measurements already received. Contact forces and simulator ground truth remain in labelling and evaluation; M18's rules still apply.

Hours: 1 hour from session 3-4 on regression and loss functions, and 1 hour from the first laboratory, 11-20, allocated to baseline analysis. The physical example replaces the generic regression example. Retain the main classifier, data split, OOD and timing measurements. The total remains $35 + 40 = 75$ hours.

Selected reading: Russ Tedrake, [System Identification](#): “Estimating inertial parameters (and friction)”, “Lumped parameters for the simple pendulum”, “Equation error vs simulation error”. Explain why a small equation residual does not guarantee an accurate prediction over a long interval.

From individual sensors to a slow gait

M19 • 04 Jun 2028 - 06 Jul 2028 • 75 hours

What you will build this month

Create a quadruped simulation with state estimation from the IMU and stance-leg kinematics, a deterministic planner for a slow gait that swings legs one at a time (crawl), reachable foothold selection and controlled stopping. The robot must stand for at least 60 seconds and execute a short, reproducible sequence of steps. For every attempt, save contacts, slip indicators, support margin, state errors, failure reasons and whether a fall occurred.

The central engineering task is to connect confidence in sensors with permission for the next step. Even accurate inverse kinematics cannot rescue a gait if the controller uses contact from a lifted leg, confused IMU axes or stale body position. Therefore, first verify the standing robot's state, then swing one leg, and finally execute a full cycle. Increased speed and trotting are outside the required implementation.

What to bring from previous modules

Prepare a model of the body and four legs, forward and inverse coordinate transforms, joint limits, verified actuator simulation and telemetry format. Save masses, centres of mass and the initial stance in a separate configuration. Check simulation stability with the chosen integration step and a floating body. Fixing the body is acceptable in a leg-kinematics experiment, but such an experiment does not demonstrate robot balance.

The M18 ML classifier may run as an observer for comparison with classical contact detection. The required gait operates without it. Simulator ground-truth coordinates and contact forces are used for evaluation and diagnostics; do not silently feed them to an estimator claimed to use only IMU and encoders. List every component's inputs explicitly.

Connection to the future robot

Four legs provide a clear intermediate stage between one mechanical leg and walking; the move to six legs comes later, after the baseline testbed succeeds. M19 does not require rebuilding the physical body or starting push experiments on a real robot. Reproduce all described disturbances in software. The result supports stance-force allocation in M20 and staged transfer to hardware in M22.

The same principle applies to the hand: a supporting finger imposes a kinematic constraint only when contact is reliable. Compare a foot on the ground with a finger on an object: what is known about surface motion, and when does the stationary-surface assumption fail? Gait remains this month's primary task; the grasp state machine is studied separately.

A 75-hour work plan

Four blocks with observable outcomes

Hours 1-20: 10 hours of theory and 10 of practice. Study coordinate frames, IMUs, sensor biases, orientation prediction, kinematic contact constraints and time alignment. Prepare a standing-body state estimate, compare it with ground truth and test exclusion of an unreliable leg. Describe observability limitations. First check signs and units in a simple case, then move to a filter with more states.

Hours 21-40: 10 hours of theory and 10 of practice. Study stance and swing phases, allowed transitions, stance-time fraction, starting, pausing and stopping. Create a slow-gait state machine that swings only one leg and prohibits the next swing when support is insufficient. For now, use a prescribed foot trajectory above a flat surface; add terrain after checking transitions.

Hours 41-60: 10 hours of theory and 10 of practice. Study swing trajectories, clearance, reachability, collisions, surface height, foothold selection with a margin and fallback mode. Implement selection from a small finite set of points, checks along the complete trajectory and a log of rejection reasons. Local selection suffices for a short step; a global terrain planner is unnecessary.

Hours 61-75: 5 hours of theory and 10 of practice. Topics are support margin, body levelling, simplified load distribution, small repeatable disturbances and degraded mode. Deliver a test harness, a 60-second stand, a crawl under declared conditions and a verified pause. Total: 35 hours of analysis and 40 hours of your own experiments.

Organising sessions

Five two-hour sessions prepare each of the first three 10-hour practical blocks. Divide the final 5 analysis hours as 2 + 2 + 1. Split a Sunday practical block into preparation, one implementation, measurement and analysis; schedule breaks separately. Hours below indicate workload sequence. Boundary dates may overlap the next module: place sessions in free periods of the original calendar without duplicating hours; rest, holidays and integration blocks remain unchanged.

Before each run, record a hypothesis and restore an identical initial state. Store the scenario in the configuration: slope, noise, packet loss, leg order and body commands. Video reveals a failure; numerical logs help explain it. Change controller gains, simulation step and geometry separately so that the cause of improvement can be identified.

Orientation, velocity and trust in contact

A minimum state model

Let W denote the world frame, B the body frame, r_i the position of foot i relative to the body in metres, and R the rotation from B to W . The state for initial experiments comprises body orientation, velocity and position, an estimated gyroscope bias and contact-quality flags. A gyroscope measures angular velocity in rad/s with bias and noise; an accelerometer measures specific force, not ready-to-use linear acceleration in the world frame. On a stationary body, readings depend on orientation and the chosen gravity convention.

Task 1. Axis and time conventions

Draw body and leg axes, specify the sign of z and quaternion component order. Record a stationary body, a small positive rotation about each axis and one foot moving forward. Compare measurement signs with the expected motion. Save frames.md and three plots; detect sign errors before introducing the filter. Convert timestamps to a common time base and retain each measurement's age separately.

Task 2. Predict orientation

Estimate the initial gyroscope bias from a stationary interval. Subtract it and integrate angular velocity using the actual dt and a chosen library rotation operation. After each step, check quaternion norm and finite values. Compare with ground truth using the relative rotation angle. Quaternion components cannot be subtracted like ordinary coordinates: q and $-q$ represent the same rotation.

Introduce a constant bias and missing packets. Plot orientation error over time and compare uncompensated and compensated versions. Do not claim that an accelerometer always corrects tilt: during dynamic acceleration, its direction is not simply aligned with gravity. M19 requires only a restricted slow-motion regime and an explicit reduction in confidence when this assumption is violated.

Task 3. Kinematic velocity from a support foot

For a stationary, non-slipping foot, v_B in $W = -R(\omega_B \times r_i + J_i \cdot \dot{q}_i)$, where J_i maps joint velocities to foot velocity in B . Check dimensions: $\text{rad/s} \times \text{m}$ and $J \cdot \text{rad/s}$ give m/s . Standing should yield velocity near zero; when the body moves with the foot fixed, the estimated velocity sign must match ground truth. Combine several supports using a weighted mean only after rejecting unreliable contacts.

Study the structure of a full invariant EKF as a possible extension. Without an external reference, absolute position and yaw generally drift: contact does not create a global reference. State in the report which local tasks the estimate supports and which absolute-accuracy limitations remain.

Contact, slipping and sensor alignment

When the contact constraint fails

A touching foot is not necessarily stationary relative to the world: it may slip, the surface may move and joints may deflect under load. The estimator needs different confidence weights for different legs. Store planner mode, measured contact evidence, confidence and the time of the last reliable update separately. A scheduled stance phase does not confirm actual contact.

Task 4. Velocity residual during slipping

For each presumed support, calculate the foot-velocity residual: $r_v = v_{est} + R(\omega \times r_i + J_i \cdot \dot{q}_i)$. Its norm is expressed in m/s and should be small for a consistent stationary support. First obtain its distribution in a quiet stance, then during known slipping in simulation. Choose a threshold from these diagnostic data and state its applicable conditions; there is no universal number for every robot.

Where possible, compare a leg with an estimate formed without its own contribution, provided enough other supports exist. Otherwise, explicitly state the residual's dependence on the combined estimate. Test slipping of each leg in turn, a brief impact and loss of all reliable contacts. Save plots of residuals, detection delays and each leg's weight. A brief spike must not cause continuous state chattering.

Task 5. Hysteresis and data loss

Implement a classical contact state machine with separate activation and deactivation conditions, a minimum confirmation time and a timeout. Choose durations from sensor rate, noise and permissible response delay. At low confidence, reduce the constraint weight; without fresh measurements, exclude it entirely. The unknown state means insufficient information and therefore cannot be treated as a confirmed swing phase.

Introduce a known encoder time offset relative to the IMU. Compare residuals and velocity error before and after correct alignment. Then delete an IMU packet and introduce a long gap. A short gap may be handled by prediction within a declared horizon; a long gap puts the system into degraded mode. Record actual data age and transition reason, not only the final flag.

Analysis to document

In the same scenario, compare four versions: IMU without bias compensation, IMU with all contacts unchecked, contact-based estimation with rejection, and estimation with a delayed channel. Use one ground truth. Show velocity root-mean-square error, RMS, maximum tilt error and time without a reliable estimate. Rejection discards some information; explain why error may increase over particular intervals.

Gait as a state machine of permitted transitions

Stance and swing phases

During stance, the foot supports the body; during swing, it moves to the next point. The duty factor is stance duration divided by the leg's period. It is usually high in a slow gait with legs swinging one at a time; specify intervals in a schedule with additional pauses. Trotting swings diagonal pairs and requires a different dynamic stability analysis. Study the distinction this month; enabling trot would require new checks.

Task 6. A deterministic state machine

Define IDLE, SHIFT_BODY, LIFT, SWING, TOUCHDOWN, VERIFY_CONTACT and PAUSE states. In IDLE, first check estimate freshness and sufficient support. SHIFT_BODY moves the centre-of-mass projection into an admissible region of the remaining legs. Lift a leg only after confirming the margin. After SWING, confirm contact by measurement, then transfer load gradually; only after this select the next leg.

Define conditions for every transition: trajectory completion, contact, timeout, insufficient margin or stale state estimate. Errors take priority over ordinary transitions. The state machine receives time, state estimate, contact statuses and a velocity command; it outputs leg modes, target points, a body command and a pause reason. Identical inputs must yield the same sequence regardless of rendering frequency.

Task 7. Starting, stopping and a new command

Run the sequence: rest → slow motion → direction change → stop request during swing → restart. Stopping does not mean instantly setting every joint position to zero. Choose a defined policy: complete an admissible swing and confirm contact, or perform a controlled return if reachable. With insufficient information, maintain the declared conservative mode and do not begin a new step.

Check these rules: at most one leg swings in the normal slow gait; the next leg lifts after the previous contact is confirmed; a command change does not reset the phase backwards; identical messages do not restart the timeout. Save transitions.csv with transition conditions. Reprocessing the recording must produce the same state sequence.

Define the response to losing two supports and being unable to complete a step. Holding a pose may be physically impossible in an unstable configuration, so a stop command alone does not guarantee safety. Mark failure in simulation, stop issuing further gait commands and save the final seconds for analysis.

Foot trajectory and foothold selection

Swing geometry

A trajectory specifies foot position in metres and its time derivatives. Start with smooth interpolation and zero velocity at both ends; define swing height with a separate smooth curve. Explain why a position discontinuity demands high velocity, and a velocity discontinuity demands acceleration. Choose height from the prescribed terrain and its estimation error.

Task 8. Clearance and reachability

On a flat surface, swing one leg along a short arc. Check inverse kinematics throughout the trajectory, joint position and velocity limits, and absence of intersections with the body or other legs. Measure minimum clearance relative to the surface below the foot at every intermediate step. Save plots of height, joint velocities and minimum distances. Even when the endpoint is reachable, the intermediate arc may be inadmissible.

Add a low obstacle in simulation and an error in the height map. Derive the height limit from model geometry and available range of motion. Compare swings with sufficient and insufficient clearance, and retain the collision cause. Raising the foot does not always help: it may move the knee beyond its angle limit or bring the leg closer to the body.

Task 9. Choose a foothold with a margin

Generate a finite candidate grid around the nominal foothold. First filter out candidates that are unreachable, too close to an obstacle or on unknown surfaces. For the remaining candidates, evaluate reachability margin, local slope, surface quality and change in support margin. Normalise quantities into dimensionless scores; otherwise, millimetres may accidentally outweigh degrees purely because of numerical scale.

Save candidates and rejection reasons: `outside_workspace`, `collision`, `insufficient_clearance`, `unknown_height`, `poor_support`. Check the entire trajectory to the selected point. If no suitable point exists, reduce step length under predefined rules; if that also fails, maintain support and enter PAUSE. The fallback algorithm must respect the original limits.

Compare planned and actual touchdown points in one coordinate frame, in metres. Increasing error may come from terrain, delay or actuator tracking accuracy. The candidate-selection algorithm can be retained for a six-legged robot, but step admissibility must be recalculated for its support set.

The crawl_estimator_v1 laboratory

Testbed components and data

Use one simulator and one quadruped model. Processing sequence: time checks → IMU and kinematic estimation → contact checks → gait state machine → foothold selection → foot trajectory → actuator controller. An independent observer saves simulator ground truth and failure indicators. Ground truth must not secretly correct the estimate; otherwise, operation depends on data unavailable to a real robot.

Specify masses, inertias, geometry, physics step, controller and sensor rates, initial stance, leg order, noise, timeouts and motion limits in the configuration. At every tick, record time, true and estimated state, contacts, confidence, residual, measurement age, phase, selected point, failure reason and commands. Save events separately so an incorrect transition can be replayed quickly.

Task 10. Standing and a full cycle

First stand for 60 seconds on a level surface without swings. Check for no fall, finite state values, compliance with joint limits and absence of hidden body fixation. Then perform one cycle that swings each leg, followed by a stop. Only afterwards repeat several cycles at a low commanded speed. Define path length and tilt limits for the specific model before running, and record them as criteria for the educational scenario.

Define a fall explicitly: for example, body contact with the floor, or orientation or height outside selected limits. Justify the boundaries for the model; they are not universal physical standards. Fall rate is the number of episodes containing a fall divided by the number of runs. Account separately for distance travelled and stops caused by deteriorating conditions: stopping after one second does not demonstrate route completion.

Task 11. Compare estimators

Compare identical scenarios in three modes: ground-truth state as a diagnostic reference, IMU-only estimation and contact-aided estimation. Present results for each mode separately. Compare orientation and velocity errors, stable-walking duration and pause count. If the gait works only with ground-truth state, contact-based estimation is not ready: identify how estimation error causes an incorrect transition.

For every failure, save a short recording before and after the event, the configuration and a reproduction command. Then repeat the scenario without rendering and compare contacts: control must keep working when graphical-interface load changes.

Balance, disturbances and deteriorating conditions

What support margin means

For slow motion on a horizontal surface, construct the convex polygon of reliable support points and project the centre of mass onto the plane. The margin is the minimum signed distance from the projection to the polygon edges, in metres. A positive value inside is useful for quasi-static planning; it does not fully guarantee dynamic stability. Sloping or uneven surfaces and appreciable accelerations require additional force conditions.

Task 12. Weight transfer and levelling

Calculate the centre of mass from link masses and positions, rather than just the body's geometric centre. Before lifting a leg, recalculate the polygon without it and smoothly move the body into the admissible region. Check the instant before liftoff, mid-swing and the instant after touchdown. If the margin is insufficient, the step does not begin. Deliver a plot of COM, supports and margin, marking state-machine transitions.

Add a small roll and pitch correction through the desired body pose and stance-leg kinematics. Respect limits on height, rate of change and joint angles. Distribute vertical loads using a simple quasi-static model, checking nonnegativity and total weight. A full QP with friction and torques is studied in the next module; for now, identify conditions in which the simplified model is insufficient.

Task 13. A repeatable disturbance

In simulation, apply a force pulse of known force and duration at a declared point on the body. Impulse equals force multiplied by time, in N·s; torque depends on the moment arm. Start with a small fraction of the model's permissible deviation and increase it only within the educational series. Compare identical disturbances with all four supports and during one swing. Save amplitude, phase and application time.

Measure maximum tilt, minimum support margin, time to return to a defined range and number of new steps after the deterioration signal. Count recovery only after remaining within the range for a specified duration, rather than briefly crossing its boundary. Record demands beyond model capability as failures; falls must remain in the series results.

When conditions worsen, reduce commanded speed, prohibit new swings and choose an admissible way to finish the current swing. If the estimate is unreliable or required supports are lost, stop planning motion. Test this transition separately: it must not produce a command jump or an unreachable pose.

Session plan: hours 1-40

Hours 1-20: state estimation

1-2: review transforms between W, B and the foot, quaternions and angular velocity; label units in five log rows. 3-4: study IMU bias, noise, specific force and gravity. 5-6: derive body velocity from the stationary-foot constraint and check a one-dimensional case. 7-8: study contact modes and observability, and sketch yaw drift. 9-10: plan synchronisation and comparisons of estimator variants.

11-20 - first laboratory: 2 hours for axis conventions and ground-truth recordings; 2 for IMU prediction and bias; 2 for kinematic velocity; 2 for contact confidence and slipping; 2 for comparing missing packets, bias and time offsets. Save a plot and explanation after each experiment. Deliver a body-state estimate with confidence and freshness indicators.

If the formula disagrees with ground truth, check the frames of ω and r , the Jacobian definition and agreement between dt and timestamps before tuning the filter. Increasing filter gains does not resolve a coordinate-frame error. First check a simple known motion, then tune noise smoothing.

Hours 21-40: gait state machine

21-22: draw a stance/swing phase diagram and calculate the schedule's duty factor. 23-24: construct the support region with each leg removed. 25-26: describe transition conditions, timeouts and error priorities. 27-28: work through starting, pausing, stopping and command changes. 29-30: record expected events for three short scenarios, including unconfirmed touchdown.

31-40 - second laboratory: 2 hours for the state machine without physics; 2 for synthetic inputs and transition tests; 2 for swinging one leg in simulation; 2 for a four-leg sequence; 2 for stopping and reprocessing a recording. Start with a known plane and a short step. If necessary, postpone turning and backward motion until the next block, while retaining every stopping mode.

At the end of the block, show leg states on one plot and count simultaneous swings. Even one tick with two legs incorrectly swinging may be invisible in video yet cause a fall. Save the working `classical_crawl_baseline` configuration separately from later terrain experiments.

Sessions and checks: hours 41-75

Hours 41-60: footholds

41-42: smooth trajectories and endpoint velocities; 43-44: clearance and height error; 45-46: workspace and collisions; 47-48: normalised foothold criteria; 49-50: rejection, step reduction and reason logging. Practice 51-60: 2 hours each for the trajectory, IK checks at intermediate points, candidate grid, terrain with error and integration with the state machine. Save an explanation for every rejected step.

61-62: study COM, quasi-statics and support margin. 63-64: body balance and disturbance protocol. Hour 65: stopping criteria, result limitations and handover to M20. Practice 66-75: 2 hours for the 60-second stand; 2 for weight transfer; 2 for a disturbance series; 2 for failure checks; 2 for the results summary, video and reproduction from configuration. The total is exactly 75 hours.

Required test scenarios

1. Zero command. Stand for 60 seconds with the body unfixed. Record estimation errors, minimum height and false-contact count. **2. Unconfirmed touchdown.** Remove the surface below the target foot. On timeout, prohibit the next swing even when the trajectory is complete. Check the pause reason and absence of a second swinging leg.

3. One slipping support. Change friction beneath one foot while keeping other conditions fixed. Measure detection delay, number of incorrectly trusted samples and effect on body velocity. **4. IMU/encoder gap.** First introduce a short gap, then one exceeding the freshness limit. Check prediction within the permitted window, then degradation and recovery without a jump.

5. Unreachable foothold. Place all candidates outside the workspace or near an obstacle. Expect rejection reasons, bounded step reduction and PAUSE. **6. Height error.** Change the surface relative to the map. Check clearance and early and late touchdown; the next step must await contact confirmation.

7. Stopping during swing. Request a stop at the start, middle and end. Compare transition admissibility and continuity of target positions. **8. The same push in different phases.** Repeat a known impulse in standing and on three supports. Show maximum tilt, recovery time and fall rate; include both cases in the report.

Acceptance checks, limitations and the transition to M20

What to present as the result

The `crawl_estimator_v1` package includes the model, configurations, estimation code, state machine, foothold checks, test scenarios, source recordings and a reproducible report. Attach video of a 60-second stand and a short gait, with plots. State physics parameters, the floating body and the exact conditions for successful operation, falling or entering a pause.

State estimation: coordinate frames, units, effects of bias, missing data and delays, contact residuals and operation without machine learning have been checked. Orientation error is measured by relative rotation, and velocities are compared in a common frame. Gait: transitions are admissible, the next swing depends on actual support, and starting and stopping have been tested in different phases. Footholds: rejection reasons are saved, and both endpoint and complete trajectory satisfy geometric limits.

Scenario acceptance: stable standing for at least 60 seconds, with episode count, falls, slips, pauses and errors recorded. At least three repeats of each selected nominal and disturbed scenario are useful as an educational minimum; they do not establish real-robot failure probability. If 60 seconds are not achieved, do not declare the module complete after one successful step: save a minimal reproducible case and fix the baseline control loop.

Questions to explain independently

1) Why does touching not imply a stationary support? 2) Why can the commanded phase not be the sole contact ground truth? 3) How does the sign of $\omega \times r$ relate to body motion? 4) Why do yaw and absolute position drift? 5) How does brief packet loss differ from a stale estimate? 6) Why does a positive support margin not guarantee stability during fast motion?

7) How do link centres of mass affect weight transfer? 8) Why does endpoint reachability not guarantee an admissible trajectory? 9) What happens after late touchdown? 10) Why can increasing swing height cause a collision? 11) How does stopping during swing differ from a zero velocity command? 12) How can recordings alone distinguish an estimator failure from a controller failure?

Reduce scope while preserving the objective

If time is short, remove trotting, complex terrain, learned confidence estimation and the full invariant EKF. Retain contact-aided estimation, a flat surface, a limited set of footholds, a slow gait and failure tests. A 60-second stand, actual contact, replay and stopping are mandatory. Hand active supports, COM and orientation estimates, joint limits and a verified classical mode to M20 for force allocation.

Assigned reading: estimation and slow gait

Read the specified extracts before the corresponding tasks. Core reading explains principles; reference reading assists implementation. Sources may be in English; prepare your explanations and report in English. Reading is included in the 75 hours.

Core reading

- 1. Kevin Lynch, Frank Park, Modern Robotics.** [§3.2.1 Rotation Matrices](#); [§3.2.2 Angular Velocities](#); [§5.1.1-5.1.2 Jacobians](#). For Tasks 1-3: reconstruct coordinate-frame transforms and foot velocity from joint motion. The key outcome is to derive the stationary-support constraint including body motion.
- 2. Simo Särkkä, Bayesian Filtering and Smoothing, 1st ed., 2013.** [§4.3 Kalman filter](#); [§5.2 Extended Kalman filter](#). For Tasks 2-5: write the prediction, measurement residual and correction for a small example. EKF is reference material for extension; a full invariant filter is unnecessary.
- 3. Russ Tedrake, Underactuated Robotics (MIT, course notes).** [Planning and Control through Contact: Autonomous Hybrid Systems](#); [Given a fixed mode sequence](#). For Tasks 6-7: describe stance and swing through modes, transition conditions and events. From [Simple Models of Walking and Running](#), read Simple Models of Walking and Stance Dynamics: explain how your slow gait differs from dynamic walking.

Documentation for implementation checks

[SciPy Rotation: from_quat, apply, inv, composition](#) - independent rotation checks in Task 1; explicitly specify quaternion component order and transform direction.

[MuJoCo Computation: Contact, friction, constraint model](#) - only if MuJoCo is chosen: meaning of contact in ground truth and friction parameters. If retaining Gazebo, use documentation for its pinned physics engine.

After reading

Draw the supports, centre of mass and one unreliable foot. Explain which measurements the estimator has lost and why the state machine prohibits the next swing. Copying a gait from someone else's SDK does not replace this reasoning; the required deliverable remains your own small crawl_estimator_v1.

Fundamental concepts in M19

State. The set of model quantities needed to predict motion; here it includes orientation, body velocity and sensor-error parameters.

Orientation. Rotation of the body coordinate frame relative to the world frame; specify the matrix or quaternion together with its transform direction.

Specific force. Acceleration relative to free fall, measured by an accelerometer; in the ideal model $f = R^T(a - g)$, in m/s^2 .

Sensor bias. A systematic measurement-error component that may vary over time; integrating it causes the motion estimate to drift.

Prediction. Propagating a state estimate forward using the dynamic model and inputs, while accounting for model and measurement uncertainty.

Measurement update. Refining a prediction with a new measurement according to its reliability; false contact can corrupt the body-state estimate.

Innovation. The difference between measured and predicted values; it helps detect inconsistent sensors or a model error.

Covariance. A matrix describing error dispersion and pairwise relationships; it characterises uncertainty in the chosen state coordinates and units.

Observability. The ability to distinguish states through measurements and motion; unobservable components cannot be uniquely recovered from those data.

Leg kinematic odometry. Estimating body motion from joint measurements and leg geometry under an explicitly stated assumption about support motion.

Contact. Interaction between a foot and a surface; touching alone confirms neither absence of slipping nor sufficient load-bearing capacity.

Slip. Relative foot motion along a surface; it violates the stationary-support assumption and may distort estimated body motion.

Support polygon. The convex hull of support points in a chosen plane; a geometric guide to quasi-statics when contact assumptions hold.

Static support margin. The minimum distance from the COM projection to the polygon boundary, positive inside; it does not fully account for dynamics and friction.

Stance phase. The part of a leg cycle in which it should support the body; actual contact suitability is checked using measurements.

Swing phase. The part of a cycle that moves the foot to a new support; a swing command does not itself confirm actual absence of contact.

Duty factor. The fraction of the gait period allocated to stance for a leg; it describes time, not force.

Gait phase. A normalised coordinate within the cycle that specifies leg-action order; actual transitions also require event and condition checks.

Transition guard. A logical condition permitting a state-machine mode change, such as reliable contact and an admissible configuration of the other supports.

Foothold. The selected location for the next foot contact, checked for reachability, surface conditions, trajectory clearance and stability margin.

Equipment and software for M19

Use existing equipment

MacBook Pro M4, the four-leg model, simulator contacts, actuator parameters and the Python environment from previous months. Keep the existing physical IMU, MCU and assembled leg; simulation suffices here for disturbances and slow gait. The main ROS route is Ubuntu 24.04 ARM64 in UTM, Jazzy and Gazebo Harmonic.

No mandatory purchases

Do not order a quadruped body kit, additional actuators, a new IMU or an onboard computer. The 60-second stand and step sequence are tested in simulation; purchases for the physical quadruped stage are deferred to M22.

Install now: only what is missing

No new software is mandatory: reuse pinned NumPy, SciPy, Matplotlib, pytest and the current simulator. For transforms, use the installed [SciPy Rotation](#) or your own verified implementation. Maintain one timestamp source and one coordinate-frame description format throughout the testbed.

Optional: a separate MuJoCo route

Only if MuJoCo has already been selected for this laboratory, add mujoco in a separate macOS ARM environment following the [official Python bindings instructions](#). Check build availability for your Python version and ARM64; do not replace a working Gazebo without a reason. On macOS, passive viewing requires launching through mjpgpython.

Before using contact ground truth, read [Computation: contact and constraints](#). Begin model checks without a graphical interface, then enable viewing. You may read CHAMP and other SDKs to compare architectures; they are not mandatory dependencies for your own FSM.

Installation checks and model limits

Restart the scene and check the floating body, masses and gravity. Perform a 90° rotation, inverse transformation and quaternion-order check. Record 60 seconds of standing, then remove one contact. The log must show the change in confidence and whether swing is permitted or prohibited.

Repeat with a stale timestamp, gyroscope bias and foot slipping. On the CPU, measure loop rate and deadline misses separately from UTM graphics load. Simulator ground-truth position and orientation are used only for quality evaluation; the estimator receives the declared sensor data.

The next purchasing decision

A GPU and Jetson are unnecessary for slow gait. The separate G3 on 27.11-10.12.2028 remains a future decision; having ML from M18 does not replace it. Preparation and checks use part of the original 75 hours; equipment-delivery time is excluded.

Motion fundamentals: weight transfer

Required analysis: why admissible support changes before a leg is lifted

What supports the body during slow gait

First check the whole robot's centre of mass: $\text{COM } c = \sum m_j c_j / \sum m_j$. It may differ from the body's geometric centre. All positions below are in world frame W : x forward, y left and z up; units are metres and kilograms. In quasi-statics, neglect accelerations and external loads other than gravity. Feet remain stationary on one horizontal plane; ground reaction forces act on the robot.

Let $F_i \geq 0$ denote the vertical support reaction acting on a foot in N , m the total mass and $g = 9.81 \text{ m/s}^2$. Force balance gives $\sum F_i = mg$; moment balance gives $\sum F_i \cdot p_{i,xy} = mg \cdot c_{xy}$. Thus $c_{xy} = \sum \lambda_i p_{i,xy}$, where $\lambda_i = F_i / (mg) \geq 0$ and $\sum \lambda_i = 1$. This explains the convex support polygon: a negative reaction would require an ordinary foot to pull on the ground.

Three supports: calculate before lifting the fourth

Use an example robot mass of 10 kg; lift foot $D = (0.20, 0.15) \text{ m}$. The remaining feet are $A = (-0.20, -0.15)$, $B = (0.20, -0.15)$, $C = (-0.20, 0.15) \text{ m}$. After `SHIFT_BODY`, let $c_{xy} = (-0.05, -0.03) \text{ m}$. From the coordinates: $\lambda_B = 0.15/0.40 = 0.375$, $\lambda_C = 0.12/0.30 = 0.400$ and $\lambda_A = 0.225$. Loads **$F_A = 22.07$, $F_B = 36.79$, $F_C = 39.24 \text{ N}$** sum to 98.10 N.

The diagonal boundary BC is $3x + 4y = 0$; the expression is negative inside the triangle. Distances to the three boundaries are 0.15, 0.12 and $|3(-0.05) + 4(-0.03)|/5 = 0.054 \text{ m}$. **The margin is 5.4 cm.** With the previous $c_{xy} = (0.05, 0.03)$, $\lambda_A = -0.225$: this three-support stance requires a negative load, even though the projection may still lie inside the four-support rectangle.

What to check in the state machine, beyond the diagram

A. In Task 12, calculate c from the links before lifting each leg and recalculate the region without that leg. Save `support_case.csv`: points, c , λ , loads and signed margin. Checks: $\sum \lambda = 1$, reconstruction of both coordinates of c , $F_i \geq 0$ and weight balance; the negative case must prohibit LIFT. Attach the diagram to the existing transitions plot.

B. In Task 6, use an example gait period $T = 4 \text{ s}$ and stance time of 3.2 s: the duty factor is 0.8. Schedule four swings of 0.8 s without overlap, leaving pauses on four supports. The timer specifies the plan; actual contact, a fresh estimate and support margin authorise transition to the next leg. Test unconfirmed touchdown: the schedule must not start the next step.

A positive margin checks geometry under the stated assumptions. Sufficient friction and actuator torques, a load-bearing surface and leg reachability are also needed. Under acceleration, $\sum f + m \cdot g_W = m \cdot c_{dd}$, where $g_W = (0, 0, -9.81) \text{ m/s}^2$; moments change angular momentum. The same polygon therefore does not guarantee balance under a push, during running or slipping, or with non-coplanar contacts.

Time: 1 hour from 23-24 on excluding a leg, and 1 hour from 66-75 allocated to weight transfer. This calculation supports Tasks 6 and 12 within the existing laboratory; all previous tests and 75 hours remain. Read: Tedrake, [Highly-articulated Legged Robots](#), “The special case of flat terrain”; [Planning and Control through Contact](#), “(Autonomous) Hybrid Systems”.

Controlling the body through support forces

M20 • 07 Jul 2028 - 31 Jul 2028 • 75 hours

Final outcome

Implement stance-leg force allocation as a quadratic programming problem, QP. From the desired total body force and moment, current contact points and constraints, it determines admissible support forces and a diagnostic status. Compare the result in simulation with the verified classical mode from M19. Test slopes, small disturbances and solver failure. Prepare `qp_stance_v1` with a working fallback mode and solve-time measurements.

Determine when requested support forces are physically feasible. The achievable body moment depends on leg placement: contact cannot pull on the ground, friction limits lateral force and actuators have limits. If a command is unreachable, the controller must report it and select a verified mode. A small optimisation residual is meaningful only when the model, constraints and result timing are correct.

Prerequisites

From M19, bring the slow gait, classical standing and pause modes, orientation and velocity estimates, foot coordinates, reliable supports and contact-loss log. Earlier modules supply masses, centres of mass, kinematics, the Jacobian, joint limits and actuator model. Check units and parameter sources. Label illustrative simulation currents and friction as educational values; they do not replace measured hardware characteristics.

Start with four unchanged contacts and a stationary body, then move to three supports and contact switching. This separates force-allocation errors from step-planning errors. If the gait is unstable, keep the legs in stand mode, retain the floating body and verify standing before connecting QP to swing. Fixing the body in the world does not demonstrate force-control quality.

Whole-body control and MPC

Study the centre of mass, momentum, desired force-and-moment vector (wrench), friction pyramids, conditioning, incompatible constraints, warm starting from the previous solution, mapping through the transposed Jacobian, impedance and task priorities. Study the structure of a complete inverse-dynamics controller. For MPC, conduct a small finite-horizon experiment; for NMPC, examine the problem formulation.

This month's result is classical support control under specified simulation conditions. Running, jumping and push tests on a real robot are outside its scope. Preserve the verified mode unchanged for comparison in M21: a learned policy must build on completed checks of the baseline force-control loop.

75 hours: from the model to verified failure handling

Work plan

Hours 1-20, theory 10 / practice 10: dynamics about the centre of mass, momentum, forces and moments, contact changes, units and balance. Build the matrix linking foot forces to the body's force-and-moment vector, and analytical check cases. Compare M19 quasi-statics with a centre-of-mass acceleration model; list effects still excluded.

Hours 21-40, theory 10 / practice 10: QP formulation, friction and normal-force constraints, conditioning, regularisation, incompatible constraints and reuse of the previous solution. Prepare a solver with independent output checks, status logging and timing measurements. First make a small problem correct: increasing the horizon does not eliminate sign errors.

Hours 41-60, theory 10 / practice 10: converting contact forces into joint torques, impedance, priorities, torque and current limits, saturation and inverse dynamics. Connect the actuator model and check limits on the complete command. The sum of force and position controllers can exceed the limit even when each is individually admissible.

Hours 61-75, theory 5 / practice 10: finite-horizon MPC, the idea of NMPC, repeatable slopes and disturbances, and return to verified standing. Compare the baseline with QP, test solver failures and describe operating limits. Total: 35 hours of study and analysis plus 40 of practice. Waiting for a solver does not count as a separate session.

Organising the exercises

The first three theory sections each comprise five 2-hour sessions. Practice comprises four Sunday blocks of 10 hours, with breaks excluded from counted time. The final 5 hours of analysis are 2 + 2 + 1. Numbered hours specify workload and order, not an additional calendar. Date boundaries may overlap the next module; rest, holidays and separate integration checks follow the curriculum, without duplicating hours.

Keep two configurations: the unchanged classical baseline and the current QP. For each experiment, record the change, expected effect, result and grounds for accepting the command. Include expected failures handled by the fallback mode in the failures log: they demonstrate protective behaviour even if the robot travels a shorter distance.

Do not spend time installing OSQP, Crocoddyl and acados simultaneously. The required work needs one QP solver and the familiar simulator; other tools illustrate possible next steps. Python suffices for formulation and analysis; add C++ only to an existing control loop if measurements demonstrate a need.

Centre of mass, momentum and force requests

Meaning of the model

Let c be the centre of mass in metres, m the total mass in kilograms and v the centre-of-mass velocity. Linear momentum is $m \cdot v$, and its change is determined by external forces. For ground forces on the robot f_i , $m \cdot a = \sum f_i + m \cdot g$, where g is the gravitational-acceleration vector. In a stationary stance, support force balances weight; a controller-requested acceleration requires the corresponding physical forces.

The moment of contact forces about the COM is $\sum(r_i \times f_i)$, where $r_i = p_i - c$ is the foot's moment arm in metres. Force is measured in newtons and moment in $N \cdot m$. This moment determines the change in angular momentum about the centre of mass. If the body is approximated as one rigid body, specify its inertia and assumptions. Leg motion also affects total momentum, especially at high accelerations.

Task 1. Check mass and centre of mass

Express every link's centre of mass in the common world frame and collect the link masses. Calculate $c = \sum m_j c_j / \sum m_j$. Compare a symmetric stance, a raised leg and a payload at a known point. Save the expected shift direction and calculated coordinates. Zero or negative mass and inconsistent units must cause explicit rejection by the model check.

Task 2. Assemble the force matrix

For n supports, assemble a force vector f of size $3n$ and a matrix A of size $6 \times 3n$: $A \cdot f = [\sum f_i; \sum(r_i \times f_i)]$. The first three rows sum forces; the next three sum moments. Manually check a vertical force with a nonzero moment arm, then two equal and opposite forces producing a nonzero moment. Express r and f in the same coordinate frame.

Task 3. From pose error to a request

Use COM position and velocity errors to request a bounded acceleration a_{des} . The required contact force is $m(a_{des} - g)$. Obtain the desired moment from a small orientation error and angular velocity, stating the frame and approximation. Limit the rate of change of requests before optimisation. At zero error, check weight compensation; at a small positive error, check the direction of the restoring action.

Compare quasi-statics, $a_{des}=0$, with a small nonzero acceleration. Show the force changes and explain why achievable moments differ between three and four supports. Increase controller gains gradually for your own simulation; these values are not universal settings for all actuators.

QP: admissible forces take priority over an ideal request

Problem formulation

Minimise the weighted error $\|A \cdot f - w_{\text{des}}\|^2_Q$, adding $\lambda \|f\|^2$ and, if needed, $\gamma \|f - f_{\text{prev}}\|^2$. Here w_{des} is the desired total force and moment, Q contains the selected weights, and λ and γ are nonnegative regularisation coefficients. The last term smooths solution changes but must not retain force at a lost contact. Reconcile force and moment scales through normalisation or a characteristic length; otherwise, weights depend implicitly on units.

Task 4. Contact constraints

For each foot, define local tangential axes and a surface normal. Normal force f_n is nonnegative and bounded above by the scenario limit. During swing, every contact-force component is zero. Take the friction coefficient μ from the surface model and investigate its uncertainty. Negative μ and an invalid normal must be rejected by input validation.

For a simple conservative pyramid, use $|f_{t1}| + |f_{t2}| \leq \mu f_n$, represented by four linear inequalities $\pm f_{t1} \pm f_{t2} \leq \mu f_n$. This diamond-shaped region lies inside the circular Coulomb cone. The two independent constraints $|f_{t1}| \leq \mu f_n$ and $|f_{t2}| \leq \mu f_n$ form a square whose corners extend outside the circular cone; do not call this approximation conservative without a correction.

Task 5. First solutions with hand-calculated answers

Solve the static symmetric problem on a horizontal plane. With symmetric geometry and equal weights, expect equal vertical loads summing to $m \cdot g$ and near-zero lateral forces. Disable one support, update A and constraints, and compare the distribution. Check Af , normal forces and limits yourself, independently of solver status.

Request a deliberately excessive lateral force. With a soft penalty on desired force-and-moment deviation, the problem may remain feasible while tracking error grows. Incompatible hard equalities produce infeasible status. Distinguish these cases: solved does not mean that the body's request is met exactly. Set an acceptable residual for your scenario.

Save `qp_cases.json` with initial conditions, solutions, independent constraint checks and explanations of physical meaning. State the coordinate frame for every force. The ground reaction on the leg and the leg's force on the ground are opposite.

Numerical stability and solver failures

What to study about numerical computation

Poor conditioning arises from disparate variable scales or geometry that can barely produce force or moment in a particular direction. Regularisation smooths a solution but does not add physical support. Compare widely spaced and nearly coincident contacts. Identify which moments are difficult to produce; a condition number alone is insufficient explanation.

Task 6. Scaling and the initial guess

Solve an example in consistent SI units, then deliberately supply a moment arm in millimetres instead of metres. Unit checking must detect the data-format error. If the solver still outputs a number, show the change in physical moment. On correct data, compare normalised and unnormalised problems. Record residual and iteration count alongside the final forces.

For nearby successive states, compare starting from zero with starting from the previous solution. When contact changes, zero or transform invalid components of the old force: a swinging leg must not retain load. A fixed vector of 12 components can be retained, with inactive contacts set to zero by constraints. If dimensionality changes, explicitly update the leg-index mapping.

Task 7. Monitor status and deadlines

Handle solved, inaccurate, infeasible, maximum iterations, time limit and internal errors according to the pinned OSQP version's API. Define accepted statuses and tolerances in the configuration. Independently check every solution for finite values, inequalities, tracking error, current contacts and result age. A support may disappear during computation, making even a feasible but late result unusable.

Test an iteration limit, artificial delay, contradictory constraints and NaN input. Save reject events with precise reasons and the transition to classical mode. Computation in the critical loop requires external timing supervision: a hung call must not prevent the observer from detecting a missed deadline and prohibiting a new step.

Measure solve time

On the CPU, measure setup, update and solve separately. After warm-up, save a series of solutions with a specified range of contact configurations; report observation count, p50/p95/p99, maximum and deadline misses. Repeat under moderate background load and at cold start. Percentiles describe the measured sample but do not guarantee maximum time; account for this when selecting loop rate.

From foot force to actuator torque and current

Establish the sign conventions first

With the body fixed, the leg Jacobian relates $\dot{q}_i$ to foot velocity. In the full model, J_c maps generalised velocity v to contact velocities; the environment's force on the robot contributes $J_c^T f_{\text{ext}}$. The dynamics are $M(q) \cdot \ddot{q} + h = S^T \tau + J_c^T f_{\text{ext}}$, where h includes gravity (p. 15).

Therefore, $+J_c^T f_{\text{ext}}$ cannot be sent directly to motors: the equation determines its sign and the other terms. The leg's force on the ground is opposite to the ground reaction.

Task 8. Check virtual power

For a leg with the body fixed, choose a configuration, a small $\dot{q}$ and a known external force. Calculate $v_F = J \cdot \dot{q}$ and compare $f \cdot v_F$ with $(J^T f) \cdot \dot{q}$. With consistent frames and dimensions, the powers agree within numerical error. Then check the static torque of a single-revolute-joint model using the force moment arm. Save the sign and N·m unit checks before connecting the whole robot.

Connect QP through a mapping consistent with gravity compensation and the dynamics model. Show contact, gravitational and impedance contributions to the command separately. For an existing controller, first study its sign conventions; changing gain signs does not replace this check. The same rule applies to a finger: the object's force on the finger and the finger's force on the object are opposite.

Task 9. Limits on the complete command

At a fixed configuration and with the other terms known, express joint-torque limits as constraints on f . Determine them from the actuator, transmission, permissible current and load regime. In a simplified model, $\tau_{\text{motor}} \approx k_t \cdot I$; joint torque also depends on transmission ratio and efficiency. Do not assume that the controller's supply current equals winding current without verification. Peak torque at a stalled rotor is not the permissible continuous torque.

Before execution, check the sum of commands. Saturation after QP changes the resulting forces and moments; measure this change and include it in tracking error. Reduce the torque available to one leg and compare loads, body errors and transitions to fallback mode. Define simulation tolerances in advance. Record unknown real-actuator characteristics as required data for later transfer.

Impedance determines the response to pose and velocity errors, but stiffness is limited by the actuator and discretisation. Contact and actuator constraints must be satisfied first, then the body-holding task, then additional tasks. A weighted sum of penalties in one QP does not always enforce strict priorities. Study how this differs from hierarchical whole-body control.

The qp_stance_v1 laboratory

Interface and independent checking

Inputs: a fresh body estimate, COM and feet in a common coordinate frame, surface normals, contact set, desired forces and moments, and actuator limits. Outputs: forces, predicted torques, status, residual, solve time and acceptance or rejection reason. Retain the contact generation: a result for old supports cannot be accepted solely because its timestamp is fresh.

The testbed includes a request generator, QP construction, solver, independent validation and a switch between classical standing and pause modes. Start with M19 recordings without controlling the robot, then close the loop in simulation. Check constraints against the original physical data independently of solver diagnostics. This detects an incorrectly assembled matrix that the solver itself regards as a valid problem.

Task 10. Close the loop

Test, in sequence, four supports with weight compensation, a small COM shift, a small roll and pitch request, three supports and restoration of the fourth. Compare the desired force-and-moment vector, achievable A_f and simulation response. State delays, actuator effects and model limits. Before motion, check signs and constraints in standing.

Compare the baseline and QP in identical scenarios. Measure RMS height and tilt errors, maximum deviation, fraction of time at a limit, solver-failure count, recovery time, falls and correct pauses. Reducing one error through continuous saturation does not constitute an overall improvement. Plot commands before and after every limit.

Task 11. Test fallback mode

If a solution is unavailable or rejected, prohibit new swings and use the M19 classical policy verified for the current supports. Do not replay old forces on a leg already in the air. The transition limits command jumps as far as current contacts allow; loss of support takes priority over smoothing. Holding may become impossible; then record scenario failure and stop further motion planning.

To return from fallback mode, require a stable, reliable estimate for a specified duration, several valid solutions and explicit state-machine permission. Oscillating solver status must not cause frequent switching. Save a diagram of rejection reasons, modes, contacts and commands. Demonstrate fallback operation for every planned artificial fault.

MPC and whole-body control

A finite horizon on a simple model

An ordinary support QP allocates forces for the current request. MPC predicts several future states and selects a command sequence subject to constraints. Execute the first step, then solve again using new measurements. Computational cost and accuracy depend on horizon, time step and model. With an incorrect model or data, increasing the horizon does not guarantee improvement.

Task 12. A conceptual MPC experiment

Use a one-dimensional COM model: $x_{\text{next}} = x + v \cdot dt$, $v_{\text{next}} = v + a \cdot dt$. The acceleration constraint represents available lateral force. Compare choosing only the current step with planning over a short horizon to stop before a boundary. Use identical initial x and v , boundary and constraints. Plot predicted and actual trajectories, then repeat with a mass error or command delay.

The experiment illustrates future constraints and replanning, but does not describe every quadruped contact. Record the assumptions: fixed contact, simplified dynamics, known boundary and no full rotational motion. The linear educational model is not NMPC. For a nonlinear formulation, list states, actions, contact sequence, constraints and required derivatives.

How whole-body control is structured

Study full inverse dynamics: floating-base state, mass matrix, Coriolis and gravitational terms, contact Jacobians, joint torques, and body and foot tasks. A floating base has no separate motors for each coordinate; contact forces produce body motion. Its desired pose must therefore be consistent with the legs and friction constraints.

Compare three levels: the quasi-static baseline distributes load during slow motion, QP accounts for current force constraints, and MPC adds prediction. State what the project implements and what is studied theoretically. Pinocchio, Crocoddyl and acados can help explain how these systems work; installing all three libraries is unnecessary.

Slope and disturbance

Specify a small slope with a known normal and an identical disturbance at one body point. Compare nominal μ with lower actual friction. Show when the model overestimates admissible forces and the solver accepts them while actual slipping occurs. Save the baseline result before adding complexity. Simulation comparison does not replace physical testing.

Step-by-step sessions: hours 1-40

Block 1: model and check cases

1-2: review COM and mass weighting, and collect model parameters in SI. 3-4: study linear and angular momentum, force and moment arm. 5-6: derive A for two contacts by hand and check cross-product signs. 7-8: construct a force-and-moment request from bounded body error. 9-10: compare quasi-static and dynamic assumptions, and prepare four and three supports.

11-20 - first practical block: 2 hours for centre of mass and parameters; 2 for the contact matrix; 2 for analytical force and moment examples; 2 for the w_des generator; 2 for comparison with simulation and description of assumptions. Explain the physical meaning of every row of A. If Σf balances weight but the body accelerates downward, check how force is applied and how gravity is modelled before tuning gains.

Save a numerical example: mass, coordinates of four feet, COM, specified forces and expected total force and moment. Label these values as parameters of a particular simulation. Use the example for repeated checks after changing leg order or coordinate frames. Verify its correct answer by hand.

Block 2: QP and computation

21-22: quadratic objective and linear constraints; convert a simple error function into the solver's form. 23-24: friction cone and conservative pyramid. 25-26: a soft force-and-moment error penalty and incompatible hard constraints. 27-28: scaling, regularisation and starting from the previous solution. 29-30: OSQP statuses, independent result checking and the computation-time limit.

31-40 - second practical block: 2 hours for static QP; 2 for three supports and friction; 2 for an unreachable request and independent checking; 2 for cold start and starting from the previous solution; 2 for timing and artificial faults. For a failure, save the forces and original formulation, including active constraints and solver parameters.

By hour 40, the allocator operates on recordings and returns explainable solutions or rejection. Do not connect it to physical motors yet. Closing the loop in simulation adds actuator dynamics, update period and the combined command as new error sources; the next block is specifically allocated to studying them.

Step-by-step sessions and tests: hours 41-75

Integration blocks

41-42: Jacobian and virtual power; 43-44: contact-force signs and dynamics; 45-46: torque, transmission, current and continuous limits; 47-48: impedance, saturation and priorities; 49-50: whole-body control and independent mode switching. Practice 51-60: 2 hours for $J^T f$; 2 for the complete command; 2 for actuator constraints; 2 for closed-loop standing; 2 for contact changes and fallback mode.

61-62: finite horizon and a small MPC example; 63-64: NMPC formulation and slope comparison; hour 65: operating limits and final criteria. Practice 66-75: 2 hours for one-dimensional MPC; 2 for comparing the baseline and QP on a slope; 2 for identical disturbances; 2 for all fault scenarios; 2 for reporting and reproduction. Total: 75 hours, including necessary library installation.

Eight required scenarios

- 1. Symmetric stance.** Equal admissible loads balance weight and produce near-zero moment. Check the force sum independently.
- 2. A lost support.** Disable a contact after the solver starts. Reject the old result based on contact generation, zero the lost support's force and record the new mode.
- 3. Unreachable lateral command.** Request a force outside the friction cone. Compare solved with high tracking error against infeasible under hard requirements; these are different outcomes.
- 4. Insufficient actuator torque.** Lower one leg's limit. Check the sum of contact, gravitational and impedance contributions: the complete command must respect the specified limit.
- 5. A late result.** Insert a delay longer than permitted. A new step is prohibited, the mode switch uses the verified policy, and a late answer is not accepted as fresh.
- 6. Invalid inputs.** NaN, negative mass, an invalid normal and mixed-up units must produce a specific validation error, not an arbitrary command.
- 7. Low friction on a slope.** Actual μ is lower than the model assumes. Measure slipping and body error even with solved status; contact checking must detect the violated assumptions.
- 8. Return from fallback mode.** Alternate valid and invalid answers near the threshold. Hysteresis must prevent frequent switching while preserving immediate mandatory rejection. Check command continuity and absence of spontaneous gait restart.

What it means to complete the module

Contents of the final package

In `qp_stance_v1`, save coordinate-frame and sign conventions, model parameters, QP formulation, force-and-moment request generation, actuator adapter, independent validation, classical fallback mode, scenarios and source measurements. The README must include commands for analytical checks, replay, simulation comparison and timing. Attach at least one analysis of a rejected solution with all constraints and the physical reason.

Criterion 1: force and moment balance and units are checked by hand and in code. Criterion 2: accepted QP solutions obey contact constraints and limits on the complete joint command within the specified numerical tolerance. Criterion 3: rejection responses are defined for error statuses, incompatible constraints, iteration limits, timeout and NaN. Criterion 4: fallback mode is tested on current supports and does not use force from an absent leg.

Criterion 5: the timing distribution is measured; CPU, solver version, example count, start methods and deadline misses are reported. Criterion 6: the baseline and QP are compared on identical slopes and disturbances, including failures and saturation. Criterion 7: for whole-body control, MPC and NMPC, implemented work is distinguished from theoretical study. This protocol does not test dynamic running.

Questions without prompts

1) Why must force and moment scales be reconciled in a shared norm? 2) How is the moment of a foot force about the COM calculated? 3) Why can ordinary unilateral contact not pull the foot towards the ground? 4) How does the inner friction diamond differ from the outer square? 5) Why does solved not confirm exact achievement of w_{des} ? 6) When is an initial guess from the previous solution physically invalid?

7) How can virtual power check the sign of $J^T f$? 8) Why are gravitational and impedance contributions considered together? 9) How does winding current differ from supply current? 10) Why does p99 not guarantee an upper time bound? 11) Why must the previous force not be retained after contact loss? 12) What does MPC add to the current support QP? Give an example from your testbed for every answer.

If the time budget is exhausted

If time is short, remove transfer to C++, extensive parameter tuning, acados installation and full inverse dynamics. Retain a working QP, simple model, explicit constraints, fault checks and classical fallback mode. If the QP gait is unstable, restrict the result to standing and slow contact changes, stating this in the report. Hand M21 the unchanged working mode, bounded-correction interface and independent observer; baseline checks must be complete before learning begins.

Assigned reading: forces, constraints and QP

Read the specified extracts before the corresponding tasks. Core reading explains principles; reference reading assists implementation. Sources may be in English; prepare your explanations and report in English. Reading is included in the 75 hours.

Core reading

- 1. Stephen Boyd, Lieven Vandenberghe, Convex Optimization, 2004.** [§4.4 Quadratic optimization problems](#); [§5.5.1](#) and [§5.5.3](#) (stopping criteria; KKT optimality conditions). For Tasks 4-7: write the variables, objective and constraints of your QP; understand feasibility and numerical accuracy.
- 2. Kevin Lynch, Frank Park, Modern Robotics.** [§5.2 Statics of Open Chains](#) - derivation through virtual power for Task 8. [§11.5 Force Control](#); [§11.6 Hybrid Motion-Force Control](#) - selected reading for Tasks 9-11: distinguish motion control, force control and contact constraints.

Notes for the short MPC experiment

- 3. Russ Tedrake, Underactuated Robotics.** [Trajectory Optimization: Direct transcription](#); [Model-Predictive Control](#); [Receding-horizon MPC](#). For Task 12: a constrained one-dimensional COM model, finite horizon, execution of the first action and replanning. Do not implement full nonlinear body dynamics.

Three sections of solver documentation

[OSQP: The solver](#) - Problem statement, convergence and infeasibility for formulating Tasks 4-5. Map every matrix to the testbed's physical quantities by hand.

[OSQP: Python interface](#) - setup, solve, update, warm start for Tasks 5-7. [Status values and errors](#) - table of statuses and rejection reasons; additionally check physical constraints and result age.

What to retain in your notes

Keep the standard QP formulation, a diagram of admissible forces, the $\tau = J^T f$ check with consistent signs and the rule for accepting a result. Study the structure of Pinocchio, Crocoddyl and acados later; installation is unnecessary for the required force allocation and small MPC example.

Fundamental concepts in M20

Centre of mass (COM). The mean position of a system's mass: $c = \sum m_i r_i / \sum m_i$; it depends on link poses and mass distribution.

Wrench. Total force and moment about a specified point; components are expressed in N and N·m.

Momentum. A system's linear momentum $p = m \cdot v_{\text{COM}}$; its change is determined by the sum of external forces. Angular momentum is described separately.

Normal force. The reaction component along the surface normal; for ordinary unilateral contact, it points away from the surface and is nonnegative.

Friction cone. Admissible contact forces in the Coulomb model: tangential-force norm does not exceed μ times normal force.

Friction pyramid. A polyhedral approximation of a cone using linear constraints; a conservative approximation places its boundaries inside the original cone.

Quadratic programme (QP). Optimisation of a quadratic objective subject to linear equalities and inequalities; here, variables describe stance-leg forces.

Convexity. A property of the objective and feasible set under which a local minimum is global; uniqueness requires additional conditions.

Feasibility. The existence of variable values satisfying every constraint in the formulation; mathematical feasibility does not establish physical-model correctness.

Objective function. The criterion for choosing admissible forces; it usually combines desired force-and-moment error with a penalty on command magnitude.

QP regularisation. An additional penalty on forces or other variables that reduces ambiguity; its magnitude affects the selected solution.

Scaling. Bringing quantities to comparable numerical scales; physical constraints are checked after converting results back to original units.

Residual. A numerical measure of equation or optimality-condition violation; a small value does not prove that axes, parameters and timing are correct.

Incompatible constraints (infeasible). The problem has no solution satisfying all constraints; this differs from a computation deadline miss or input error.

Warm start. Reusing the previous approximation; after contact changes, check component mappings and constraints.

Deadline. The time limit for a control result to remain applicable; beyond it, even a correct answer may rely on stale state.

The mapping $\tau = J^T f$. The relationship between force and joint torques through equality of virtual power, with consistent coordinates, signs and force definition.

Impedance. A specified dynamic relationship between motion error and force; actuator capability, delay and loop discretisation limit achievable behaviour.

MPC. Control through repeated finite-horizon optimisation: after executing the first action, measure state and solve the problem again.

Whole-body control. Coordinated control of a mechanism accounting for dynamics, contacts and actuators; force allocation is only one part.

Equipment and software for M20

Use existing equipment

Use the MacBook Pro M4, the M19 crawl_estimator_v1 testbed, NumPy, SciPy and the existing simulator. ROS runs in Ubuntu 24.04 ARM64 in UTM with Jazzy and Gazebo Harmonic; QP analysis may run directly in macOS ARM. Before integration, preserve classical standing and pause modes and comparison scenarios.

No mandatory purchases

The force allocator requires no new actuators, force sensors, Jetson or NVIDIA GPU. Use previous measurements for physical-actuator limits; explicitly label illustrative simulation parameters. The full leg set is purchased at the physical M22 stage, not for solving the QP.

Install now: one solver

Add osqp to a separate analysis environment or an explicitly designated Python environment for the testbed. Reuse NumPy and scipy.sparse. The official [OSQP Python installation and setup/solve/update interface](#) provide the minimum route. Record Python, versions, architecture and solver settings; check ARM64 build availability before choosing the environment.

On macOS ARM, perform calculations on recordings; before closing the ROS loop, test the same problem separately in Ubuntu ARM64. Install Python packages in a compatible environment. If a prebuilt wheel is unavailable, use the documented source build or a verified environment with measured communication latency.

Installation and result checks

Solve a small convex QP with a known answer, then a symmetric stance: four vertical forces balance weight with consistent signs. Check [OSQP statuses](#) for feasible and deliberately contradictory formulations. An independent validator checks constraints, finite values, freshness and contact generation.

In a solve/update series, measure cold and warmed-up starts, long delays and deadline misses. On contact loss, exclude that leg's old force. Timeout and infeasible statuses must activate verified stand/pause modes. Check equality of virtual power for $\tau = J^T f$.

Optional, only for a specific need

Pinocchio is needed only if its dynamics algorithms are used; Crocoddyl and acados are unnecessary for the required QP and small MPC example. Reuse MuJoCo only if already selected in M19. An accelerator may be purchased after measurements and the G3 decision on 27.11-10.12.2028; this future checkpoint is not treated as completed here.

Preparation and short functionality checks are included within the original 75 hours instead of reinstalling the environment. For M21, retain the classical baseline unchanged with its configuration and fault-test results.

Motion fundamentals: a floating body

Full dynamics model and the implementation limits of whole-body control

Configuration, velocity and available actuators

For rigid links with 12 revolute joints, $q=(p_B, Q_{WB}, q_j)$: body position in W , a unit quaternion for rotation $B \rightarrow W$ and joint angles. Here $n_q=3+4+12=19$, but $v=(v_B, \omega_B, \dot{q}_j)$ contains $3+3+12=18$ components. Express v_B and ω_B in W , in m/s and rad/s. The quaternion has a norm constraint, so $\dot{q}=N(q)v$, where N has size 19×18 ; do not substitute the 19-component $\dot{q}$ for v .

$M(q) \cdot \dot{v} + h(q, v) = S^T \tau + J_c^T f_{\text{ext}}$. M is the 18×18 inertia matrix; h includes velocity-dependent and gravitational terms. $S=[0_{12 \times 6} \ I_{12}]$ selects the 12 actuated velocities: $S^T \tau$ has six zeros for the base. The first six equations cannot be satisfied by separate “body motors”. Generalised forces have units N for translational and N·m for angular components.

Contact links motion and forces

For k stationary, non-slipping point supports, J_c has size $3k \times 18$ and maps v to foot velocities in W . While this mode persists, **$J_c v=0$, $J_c \dot{v} + \dot{J}_c v=0$** . f_{ext} denotes environment forces on the robot, in N; unilateral-contact and friction constraints also apply. During swing, the corresponding force is zero; during slipping, the zero-foot-velocity condition cannot be retained.

In M20, the wrench is ordered [force; moment]: $w_C=[\Sigma f_i; \Sigma((p_i-c) \times f_i)]$ about COM c , all in W . A point contact transmits no moment of its own; force acting at a moment arm produces the moment about COM. A contact patch could also transmit its own moment, but the current QP uses point feet.

A small example: check the sign before optimisation

Let $c=(0,0,0.40)$ m, $p=(0.20,0.15,0)$ m and $f=(0,0,30)$ N. Then $r=p-c=(0.20,0.15,-0.40)$ m and **$r \times f=(4.5, -6, 0)$ N·m**. For a symmetric stance with a mass of 10 kg, four vertical forces of 24.525 N balance the 98.1 N weight and produce zero total moment. This checks equilibrium, not compliance with joint limits.

In statics, the actuated rows give **$\tau=h_j - (J_c^T f_{\text{ext}})_j$** . If $h_j=3$ N·m in one joint and the external force contributes +5 N·m, the required torque is -2 N·m. Therefore, $+J_c^T f_{\text{ext}}$ must not automatically be sent to motors. Virtual power $f_{\text{ext}}^T (J_c v) = (J_c^T f_{\text{ext}})^T v$ checks the consistency of the mapping.

Connect the model to the existing experiment

In Tasks 2 and 8, create `shapes.md` for q, v, M, S, J_c . Check that $S^T \tau$ does not act directly on the base. In the analytical example, save `force_balance.csv` with total force, moment and the static sign of τ . Disable a support: the old forces must fail the new constraints. Do not send computed torque to a position-controlled actuator; the adapter must match its available interface.

Hours: 1 hour from 43-44 on dynamics and 1 hour from 51-60 for the force-mapping test. The model is required; a new inverse-dynamics solver is unnecessary. Existing QP, fallback mode, deadlines and 75 hours are retained. Read: Tedrake, [Multi-Body Dynamics](#), “The Manipulator Equations” and “Bilateral Position Constraints”; [Centroidal dynamics / Generalization to multibody](#).

One bounded learning experiment

M21 • 01 Aug 2028 - 03 Sep 2028 • 75 hours

What you will achieve this month

The deliverable is `learning_eval_v1`: a comparison of imitation learning and a small PPO experiment against unchanged classical control. The report includes tests under conditions absent from training and verification of the return to the baseline algorithm. The main task is a small correction to the body orientation of a standing quadruped within specified ranges of tilt and disturbance. The learned component produces only a bounded high-level correction; the actuator loop, contact constraints and independent monitor remain classical.

In 75 hours, study demonstration collection, behaviour cloning, MDPs, rewards, PPO, random-number generator initialisation (seed), environment parameter randomisation and a safety filter. Learning a complete dynamic gait from scratch, jumping and transfer to real actuators are outside this module. All imitation learning (IL) and reinforcement learning (RL) experiments here take place in simulation. An increase in reward alone does not justify running a new policy on hardware.

Prerequisites

From M19-M20, you need stable standing (stand) or a slow-gait pause mode (pause), a verified QP or conservative force-control mode, the actual model limits, and independent detection of stale state data, loss of support and unacceptable commands. From M18, bring data-handling rules, grouped dataset splits, a small network, an experiment log and a CPU workload profile. Save the working baseline with its commit identifier and configuration; it must remain unchanged in the comparison with ML.

If classical posture holding fails its own checks, investigate that defect first. Check the data, describe the MDP and train on the valid demonstrations available; postpone closed-loop trials with the learned policy until the cause is resolved. Until verification is complete, learning remains an optional branch and hardware use is prohibited. G3, moved into the overall calendar, remains a separate integration checkpoint and does not replace approval to conduct the experiment.

Connections to your projects

For the walking robot, the result will show whether a small learned correction can improve body recovery under bounded uncertainty. For the hand, the equivalent is a bounded correction to position, orientation or grip force added to the command of a working classical controller. Do not train both systems this month: describe how the formulation would transfer to a finger as a separate exercise, specifying contact features and the different meaning of failure.

A successful outcome may be that the baseline is already adequate and ML does not improve the selected metrics. This conclusion is acceptable when comparison conditions are identical and results are reproducible. A new policy does not have to replace the verified algorithm; measurements must establish its usefulness and its limitations must remain explicit.

Time, computing resources and experiment limits

The 75-hour plan

Hours 1-20: 10 hours of theory and 10 of practice. Study demonstration datasets, behaviour cloning, error accumulation, shifts in the distribution of visited states, the DAgger concept and data auditing. Prepare a small demonstration dataset split by episode and a first imitation model. Evaluate it on recorded data and in closed-loop operation. The baseline controller must already work: redeveloping it is outside this allocation.

Hours 21-40: 10 hours of theory and 10 of practice. Study MDPs, observations, actions, rewards, episode termination, PPO, learning curves and variation across seeds. Prepare a valid environment and complete three small, reproducible runs. For SAC, understanding its differences from PPO is sufficient; implementing a second RL algorithm is not required.

Hours 41-60: 10 hours of theory and 10 of practice. Examine the effects of mass, friction, delay, noise and terrain, progressively harder training and randomisation limits. Prepare separate training and evaluation profiles, test each parameter family and conduct bounded ablations. Hours 61-75: 5 hours of theory and 10 of practice. Study OOD, the safety filter, independent constraints, fallback operation and recovery after a push. Compile the final test table and a protocol for future transfer from simulation to hardware.

A realistic CPU budget

The default platform is a CPU and the existing simulator running without graphics. Start with one or two small network layers and two actions: roll and pitch corrections. For each of the three main PPO runs with different seeds, set an upper limit of 50 000 policy steps or 30 minutes of training, whichever comes first; this is an educational limit, not a convergence guarantee. Record actual steps and time. If the environment is too slow, reduce the step count equally for all seeds, retain all three runs and explain how the limit constrains your conclusions.

For the five short ablations, use one seed selected in advance and a maximum of 10 minutes per variant. This analysis is preliminary: it does not replace the three-seed comparison for the main result. Total machine time for these training runs is bounded; while they execute, analyse earlier logs within the same practice block without counting the same time twice. No GPU is assumed. Apple M4 is not a CUDA platform; consider Isaac Lab separately only with compatible hardware.

Placement in the calendar

Theory: five 2-hour sessions in each of the first three blocks, then 2 + 2 + 1. Practice: four 10-hour blocks, with breaks accounted for separately. Date boundaries may coincide with the adjacent module; retain rest, holidays and integration blocks from the overall plan. Hour numbers define the work sequence; they do not create a new calendar or imply double-counting time.

Demonstrations: what the model should learn

Two roles for the learned component

In behaviour cloning (BC), the expert is an unchanged classical function: it selects a bounded target correction to body orientation from the observation. Choose scenarios with small, non-zero tilts and disturbances so that actions are not almost always zero. The low-level controller from M20 tracks this target. PPO later uses a separate residual-correction mode: an even smaller learned correction is added to the classical command. Label the modes separately and do not combine them into one metric.

Task 1. Audit the expert and data

Collect independent standing episodes with small deviations and disturbances known in advance. Record observation, expert_action, applied_action, contacts, data age, scenario identifier, seed and termination reason. Save the true simulator parameters and ground-truth state for analysis, but do not expose them to the policy if the future robot cannot measure them. List every expert input: explicitly identify any hidden privileged information.

Split whole episodes into training (train), validation and final test sets. All windows from one trial must belong to the same set; calculate normalisation parameters from training data only. Check the action and state distributions, saturation fraction, erroneous commands and episodes containing failures. Do not silently discard invalid demonstrations: specify the exclusion rule before training and retain the exclusion count. Exclude samples following a physically invalid environment reset from training.

Task 2. Behaviour cloning

Train a compact network to minimise the mean squared deviation from the expert correction in normalised units. For angles, also report MAE in radians or degrees, labelled explicitly. Start with the simplest method, the mean action in the training set, and a linear model; compare the neural network against them on the same dataset. Check large expert corrections separately: average error can hide poor behaviour precisely when conditions become difficult.

Save weights, normalisation parameters, feature list, configuration and predictions. For verification, replay a separate episode and plot expert and model actions. Boundary cases are a zero command, an unknown schema, a stale observation and an action at its limit. Pass only independently checked actions to the controller; replacing NaN with an admissible zero must be recorded in the log.

The first step aims to reproduce a bounded decision from available observations, rather than establish superiority over the expert. If the expert uses information unavailable to the model, some error may be unavoidable; identify this before increasing network size.

Error accumulation and the DAgger concept

Why error on recorded data is insufficient

In recorded data, the model receives states visited by the expert. When it controls the system itself, even a small action deviation changes the next state; it then encounters less familiar states and makes further errors. Low error on demonstrations therefore does not guarantee stable closed-loop control. Compare commands, visited states, safety interventions and episode outcomes.

Task 3. Evaluation on recordings and in closed-loop operation

First replay test-set observations and calculate action error without allowing the model to influence simulation. Then, in a separate evaluation environment, let BC select the high-level correction through the safety filter, preserving the same initial conditions. Compare body trajectories, maximum tilt, recovery time, command clamping and rejection rates (clamp and reject), and falls. Filter parameters must be identical for all controllers being compared.

Choose one episode in which error grows gradually. Locate the first substantial divergence and compare the observation against the training range. Do not attribute everything to OOD: check delay, normalisation, privileged expert features and the actual sensitivity of the dynamics to the action. Save a plot of the causal chain: command error → state change → next error.

Task 4. A minimal DAgger iteration

Study the principle: collect states visited by the current policy, query the expert for actions in those states and add the pairs to the training set. One bounded iteration in simulation is sufficient for this educational experiment. The expert labels new states while the independent filter supervises execution. Do not continue an episode beyond admissible states simply to diversify the data.

Add new episodes only to the training set; do not use the validation or final test sets for training. Retrain by the same method and compare results on a separate diagnostic set of control episodes. Show changes in state distribution, action errors and intervention counts. Record a lack of improvement if that is the result. One iteration does not establish all theoretical properties of DAgger; unlimited data collection is unnecessary here.

Transferring the concept to the hand

Consider a finger pressing against an object: a small error can move the object, causing subsequent states to diverge from the demonstration. Describe available measurements, the expert command and the condition for ending contact. Complete this written exercise within the analysis allocation; do not launch a second learning project. For both robots, assess closed-loop behaviour separately from imitation accuracy along another controller's trajectory.

Freeze BC as a separate version. When moving to PPO, retain its results: imitation of an expert and improvement through a residual correction solve different problems, and both sets of results must remain available for inspection.

An MDP and an environment with explicit actions

The learning-task contract

An MDP comprises a state, action, transition, reward and termination rule. The real agent receives an observation that may omit parts of the state, creating partial observability. For the small experiment, use available roll and pitch, angular velocities, desired-pose errors, contact statuses, measurement age and previous action. The exact friction coefficient and a future push are not observation inputs.

Task 5. A Gymnasium environment

Implement `reset(seed)` and `step(action)` around the existing simulator. The action comprises two normalised values in $[-1,1]$, scaled to a small residual roll and pitch correction added to the classical command. Derive the maximum correction and its rate of change from the previously verified simulation range. The policy does not control currents directly. The low-level controller and monitor retain their existing limits.

Follow this step order: check observation freshness, calculate the raw action, apply the filter, execute several physics steps with the filtered command, then obtain the new observation, reward and termination flag. Save raw and applied actions separately. If the filter frequently replaces an action, both policy and protection determine the result; reflect this in the metrics. Exceeding numerical limits and falling terminate the episode under the task conditions; label an administrative time limit truncated.

Task 6. Reward and undesirable strategies

Start with a negative weighted sum of squared body-orientation error, angular velocity, residual magnitude and action change. Normalise quantities by declared characteristic scales so that weights have a clear meaning. If the reward approximates an integral cost, account for dt consistently when changing frequency. Add a penalty for violating conditions, but do not expect it to replace a hard safety filter.

Manually check four sequences: holding posture, an oscillating correction, rapid episode termination and passive waiting. Calculate the reward components. An early fall must not appear advantageous merely because it stops negative costs accumulating. Save reward components and a separate success flag based on engineering criteria. Increased return without reduced tilt or fewer failures is not an improvement to the robot.

Check `reset`: the same seed reproduces initial conditions, different seeds actually change the declared parameters, and filter state and previous command are cleared. State leakage between episodes can create a hidden dependence on test order.

PPO: a small reproducible experiment

What to understand about the algorithm

PPO updates a policy from environment interactions while limiting changes in the objective. Study trajectories, returns, discounting, the value function, action advantage and clipping of the probability ratio. Use Stable-Baselines3 and a simple multilayer perceptron (MLP). Full proofs and a custom RL framework are unnecessary; understand observations, actions and episode termination before running training.

Task 7. Check the environment and run the first trial

First run the automated environment API check, trials with random admissible actions and trials with zero residual correction. With zero correction, behaviour must match the frozen baseline using the same low-level controller. Then run a very short training session to check implementation: rewards remain finite, episodes end correctly, monitor weights remain unchanged and the step counter advances. Fix environment errors before tuning PPO.

Freeze the rollout step count, batch size, learning rate, discount factor and total step limit. Start from the configuration in the documentation, adapting only what is necessary for your episode frequency and duration; explain every change. Do not run dozens of variants and select the best on test disturbances. One selected PPO experiment matches the agreed scope of the month.

Task 8. Three runs and reproducible curves

Complete three independent runs with seeds listed in advance, identical limits and separate directories. Three evaluations of one trained network do not count as three independent training runs. Save learning curves, checkpoints and timing logs for each. For a model stopped by the time limit, state the actual number of interactions: a faster run may have seen more data. For a strict comparison by step count, use a common achievable limit.

After training, evaluate all three policies on the same list of independent episodes. Report each result, the aggregate mean and spread, and the result for the worst-performing seed. Do not show only the successful instance in the final plot. Select checkpoints using validation data; use the final test set after the choice is frozen. If all models are worse than the baseline, this is a valid negative result for the bounded experiment.

When to stop training

The limit is 50 000 steps or 30 minutes per seed; reduce it equally when physics is slow. This budget may be insufficient to produce improvement; state that explicitly. Do not compensate for limited time by disabling the monitor or supplying ground-truth state as observations without declaring a different formulation. Check environment quality, action scale and reward first; allocate a new computing budget in a future experiment.

The required branch runs on CPU. Decide whether to use an accelerator after measuring runtime costs: a small MLP may take less time than the physics calculation. Isaac Lab and large batches of GPU environments are a separate choice and are not required to complete M21.

Parameter randomisation with physical meaning

Five families of uncertainty

Mass and inertia change the required forces; friction affects contact feasibility; delay changes when a command takes effect; sensor noise and bias distort observations; terrain height and slope change support. Do not choose arbitrary ranges without a stability rationale. For each family, record the nominal value, training and validation ranges, a separate OOD range, units and source: measurement, specification or an assumed range for the educational simulation.

Task 9. A randomisation register

Sample mass and friction at environment reset, and noise according to an explicit temporal model; delay may remain constant within an episode or vary by a declared rule. Save the actual sampled parameters for each episode. Keep inertia consistent when changing mass, and avoid negative friction or impossible geometry. Mark physically invalid examples as invalid simulation; do not count them as valid OOD tests.

Start with a narrow range that the classical baseline can handle. Increase training difficulty progressively when a predefined criterion is met, such as a sufficient fraction of validation episodes without falls or excessive interventions. Choose the threshold for the educational scenario before running it. Progression must not depend on the final test set; save stage durations and boundaries in the configuration.

Task 10. Parameter-family ablations

For the main result, train PPO with the chosen randomisations. Then run five short preliminary variants, disabling mass, friction, delay, noise and terrain randomisation in turn while retaining all other conditions and one seed selected in advance. The budget is up to 10 minutes per variant. Evaluate them on the same diagnostic dataset for the relevant family. This is a limited training ablation; one seed cannot establish a robust causal effect.

Separately evaluate the trained model while varying parameters systematically: change one family at a time and hold the others at nominal values. Plot failures and errors against the magnitude of change. The two experiments answer different questions: whether randomisation is needed during training, and how the finished policy responds to changed conditions. Varying evaluation parameters alone does not demonstrate the effect of training randomisation.

Limits of transfer from simulation to hardware

Randomisation broadens familiar conditions, but does not guarantee coverage of every hardware condition. Backlash, elasticity, heating, communication delays and contact errors may be absent from the model. List these discrepancies and how to measure them later. For the hand, object friction, finger compliance and tactile-sensor properties would also matter; renaming variables cannot transfer the current body policy to that task.

Independent monitoring, OOD and fallback control

Separate policy evaluation from authorisation to execute

A policy shield checks or limits the proposed correction against known conditions. It is not trained with PPO and receives no reward for passing an unsafe command. Check finite values, amplitude, rate of change, observation age, pose, contacts and available limits throughout the actuation chain. The QP and M20 controller retain their constraints on the complete command, including torque and modelled current.

Task 11. An OOD indicator and its reliability limits

Start with simple checks on standardised feature ranges and distance from training data. Choose thresholds using validation data, then freeze them. Document that this indicator misses some unfamiliar states and may reject familiar ones. Action probability, PPO policy entropy and network confidence do not guarantee physical reliability. Measure false alarms and undetected bad episodes for each OOD scenario.

Inject test faults: sensor bias, a delayed observation, a new slope, low friction and a missing foot. Record the first unreliable input, monitor activation and transition to fallback. Calculate response latency on one time base. Also test a new combination of conditions whose individual values all lie within admissible numerical ranges: componentwise checks may miss this combination.

Task 12. Transition to fallback control

When a command is rejected (reject), disable the learned residual according to a predefined rule; classical standing (stand) or gait pause remains available. During ordinary deactivation, the correction may decrease smoothly within admissible limits. When a critical constraint is violated, protection takes priority over smoothness. Do not continue applying a stale learned action for a few more samples after its validity period expires.

In BC mode, fallback restores the expert high-level command; in residual PPO mode, it zeros the correction added to baseline control. Both modes use the same low-level controller. Resuming ML requires reliable data for a specified duration and an explicit state-machine condition; one normal sample is insufficient. Record the fraction of time under baseline control: if protection almost constantly replaces the policy, successful episodes cannot all be credited to it.

Force a fallback transition while the body is displaced and compare it with deactivation during quiet standing. Show raw and applied actions, state, reason and constraints. The ability to fall back does not prove that every already dangerous state is recoverable; outside the verified region, record a failure rather than a guaranteed recovery.

Integrated evaluation: learning_eval_v1

Comparison modes

Compare four explicitly labelled modes: classical baseline control; cloning of the high-level command; baseline control with a PPO residual; and the same PPO with mandatory monitoring and fallback. If unprotected PPO is needed for diagnosis, run it only in simulation with external stopping conditions; that experiment does not authorise hardware use. BC and residual PPO solve different tasks, so action-prediction error is not a common measure of control quality.

The primary candidate for further use is the protected scheme. For a fair comparison, the baseline uses the same sensors, delays, constraints and scenarios. Evaluation shares initial states, disturbances and seeds; list training seeds separately. If the baseline is modified after ML fails, repeat the complete comparison and explicitly identify its new version.

Task 13. Compare recovery after a push

Apply a small reproducible impulse at a known body location and episode time, within the previously verified simulation range. Vary direction and intensity on a fixed grid. For each mode, measure maximum tilt, time to return to a declared band and remain there for the required duration, RMS error, saturations, command clamps and rejections, falls and successful completion. Retain episodes without a push as well: the policy must not degrade ordinary standing in exchange for an occasional successful recovery.

Report reward in a separate column. If reward rises while interventions or falls increase, reconsider the objective instead of claiming success. State the episode count in each cell and show all three PPO runs with different seeds. Variation across seeds does not replace statistics across diverse scenarios; show both sources of uncertainty.

Task 14. A hardware-transfer protocol without operating actuators

Prepare the sequence of future checks: unit and schema verification; processing real recordings without control; shadow operation without affecting the robot; a constrained test rig with known loads; independent limit and stopping checks; only then consideration of physical use. Specify required data and progression criteria for each step. M21 produces the protocol; it does not perform automatic deployment.

Allowable current, temperature and torque, together with mechanical limits, must come from the specific actuators and earlier-stage tests. Missing values remain unresolved dependencies. The separate G3 includes data, model and deployment audits and a decision on Jetson and learned motion control; moving its date does not permit bypassing checks. The M21 CPU path does not depend on buying an accelerator. Pass a working classical algorithm to M22 whether or not learning proved useful.

Session-by-session schedule: 1-75

Block 1: hours 1-20

1-2: verify the baseline and available observations. 3-4: imitation learning and the action loss function. 5-6: grouped data splitting and demonstration labelling. 7-8: error accumulation and state-distribution shift. 9-10: the DAgger concept and control-episode protocol. Practice 11-20: 2 hours each for data collection and auditing, dataset splitting and normalisation, BC training, closed-loop evaluation, and one bounded data-aggregation iteration. Save results before moving to RL.

Block 2: hours 21-40

21-22: MDPs and partial observability. 23-24: observation, action and policy frequency. 25-26: reward, task termination and truncation by an external limit. 27-28: return, action advantage and PPO clipping. 29-30: seeds, checkpoint selection and independent evaluation. Practice 31-40: 2 hours for the environment and reset; 2 for reward and zero correction; 2 for a short PPO check and logging setup; 2 for three bounded runs with concurrent log analysis; 2 for comparison and archiving.

Block 3: hours 41-60

41-42: physical meaning of mass, inertia and friction. 43-44: delays, noise and temporal correlation. 45-46: terrain, training ranges and OOD. 47-48: progressively harder training and acceptance limits. 49-50: the distinction between training ablation and evaluation parameter sweeps. Practice 51-60: 2 hours for the parameter register; 2 for the main randomised configuration; 2 for bounded ablations; 2 for parameter-family sweeps; 2 for failure analysis. When retraining the main randomised variant, retain three seeds and the same per-run limit; this is included in this block.

Block 4: hours 61-75

61-62: OOD indicators and their reliability limits. 63-64: the independent safety filter, complete-command constraints and fallback. Hour 65: the written hardware-transfer protocol and decision criteria. Practice 66-75: 2 hours for monitoring and input-fault injection; 2 for fallback transitions; 2 for the common push test; 2 for evaluating all models and seeds; 2 for reproduction from a clean directory and the final report.

What to save after each session

Each theory session ends with a diagram, a worked example or a decision on interface requirements. Each practice session ends with a configuration, raw log, numerical metric, failure case and next question. Do not extend training without new checks: the learning objective is to understand why a policy works or fails within the agreed budget. Queueing, loading and waiting must not consume the time needed to analyse sensors, constraints and comparison quality.

Acceptance: eight checks and the final decision

Required scenarios

1. Zero residual correction. Behaviour matches the baseline exactly; any difference indicates a wrapper error. **2. Faulty reset.** Repeat the seed and verify that previous actions, filters and timers are cleared. The test must detect hidden memory between episodes.

3. Invalid observation. Supply NaN, an incorrect schema and expired data. The monitor rejects the action and logs the reason. **4. Action at the limit.** Supply a large step in policy output. Check amplitude, rate of change and the complete command after the low-level controller; report every command limitation.

5. OOD friction or mass. Move one parameter family beyond the training range while preserving physical validity. Measure errors, falls and OOD detection; high network confidence does not exclude failure. **6. A combination of familiar extremes.** Combine delay, noise and slope within their individual ranges. Show whether a simple detector can miss a dangerous combination.

7. Forced fallback. Disable the policy during quiet and displaced standing. Measure the command jump and recovery; check that automatic reactivation does not occur. **8. An identical push.** Run the common episode list for the baseline, BC and three PPO runs with different seeds. Retain poor results and compare engineering metrics against reward.

Completion criteria

Prepare an unchanged baseline version, a demonstration description, dataset splits without leakage, a BC model, a verified environment and bounded PPO runs with at least three seeds, or a documented reason for stopping before training. Save configurations, normalisation, checkpoints, actual steps and runtime, OOD profiles, ablations and the fallback report. If baseline or monitor checks fail, use of the learned policy is prohibited; state this explicitly in the deliverable.

The final decision selects one outcome: retain the baseline; continue research in simulation; or prepare a separate next verification stage. This table does not automatically authorise a physical run. Claim improvement only against metrics selected in advance, without degrading mandatory constraints. For a policy requiring frequent interventions, state separately how much success is provided by the classical loop.

Self-check and scope reduction

Explain the distinctions between recorded-data and closed-loop evaluation, BC and a residual correction, state and observation, terminated and truncated, reward and success, training randomisation and condition shifts during evaluation. Show the expert's hidden information, seed selection, safety-filter operation and OOD-detector limitations. If time is short, omit Isaac Lab, new architectures and extended curriculum learning. Retain the small CPU environment, independent runs, classical comparison, OOD and fallback. No improvement is an acceptable outcome; work without verification is incomplete.

Assigned reading: imitation and one RL experiment

The references may be in English; the exercises and definitions are presented here in English. Read only the named sections before the associated work, within the existing 75 hours.

Core reading: three sources

1. R. Sutton, A. Barto - Reinforcement Learning: An Introduction, 2nd ed. [Publisher's contents](#). Sections 3.1-3.5: interaction, reward, episode, policy and value; 13.1, 13.3-13.5: policy gradient, REINFORCE and actor-critic. For tasks 5-8, write out your environment's MDP and the meaning of action advantage; reading the whole book is unnecessary.

2. S. Levine - CS 185/285, Supervised Learning of Behaviors. [Lecture 2, parts 2, 3 and 5](#): the BC algorithm, distribution shift and DAgger. Before tasks 1-4, explain why error on recorded frames differs from error during the model's own closed-loop control.

3. R. Tedrake - Underactuated Robotics, MIT notes. [Model-Free Policy Search → Policy Gradient Methods](#). Selected reading: REINFORCE and Sample efficiency. Before the three runs with different seeds, estimate the cost of one experiment and explain why a small CPU budget limits conclusions rather than proving that RL fails.

Reference sections for practical tasks

Stable-Baselines3. [PPO: Notes, Example, Parameters](#) and [RL Tips: Evaluation, Custom environments](#). For tasks 7-8: one pinned implementation, identical comparison scenarios and multiple seeds.

Gymnasium. [Env: reset, step, spaces](#) and [Handling Time Limits](#). For task 5: the environment contract, the distinction between terminal and truncated, and correct estimation of future return.

Optional: original papers

[Ross et al.: DAgger, the aggregation algorithm](#); [Schulman et al.: PPO, clipped objective](#); [Tobin et al.: Domain Randomization](#). The last paper mainly concerns visual transfer; ranges for mass, friction and delay need a separate rationale.

Reading deliverable: one page explaining the MDP, the distinction between BC and PPO residual correction, and the criteria for returning to the frozen controller.

Terms and definitions

Behaviour cloning (BC). Learning a mapping from observations to actions using labelled expert examples and an ordinary prediction-error loss.

Dagger. Iteratively adding states visited by the current policy to the training set, with actions freshly labelled by the expert in those states.

MDP. A model of sequential decisions comprising state, action, probabilistic transition and reward; the next state depends on the current state and action.

Observation. Information available to the agent at a step; it may omit part of the physical state, so the input may lack the Markov property.

Policy. A rule for choosing an action from available information; a stochastic policy defines a probability distribution over admissible actions.

Reward. A number received for one transition; it expresses the selected learning objective but is not by itself an engineering success criterion.

Return. The sum of future rewards with specified discounting from a chosen point in the episode; a random variable until the trajectory is observed.

Discount factor γ . A dimensionless multiplier of future rewards; values of γ closer to one increase the relative contribution of more distant consequences.

State value V . Expected return when starting in a given state and subsequently following the specified policy in the given environment.

Advantage. The difference between the expected value of a particular action $Q(s,a)$ and the average state value $V(s)$; it measures the action's relative usefulness.

PPO. A policy-update algorithm that clips the probability ratio in a surrogate objective; this moderates changes but does not guarantee safe motion.

Residual policy. A bounded learned addition to a controller command; its scale, rate of change and deactivation are specified separately.

Rollout. A sequence of policy-environment interactions: observations, actions, rewards and terminations; distinct from replaying a fixed recording.

Termination and truncation. Ending because a task condition is met and interruption by an external limit affect estimation of future return differently.

Distribution shift. A change in the frequency or region of observed states relative to the training set, including effects of the agent's own incorrect actions.

Domain randomization. Sampling simulation parameters from specified distributions during training; the ranges match the experiment's physical assumptions.

OOD. Conditions outside the established training distribution; a detector may miss them, so independent constraints remain in force.

Fallback. A verified transition to a classical mode or stop when the learned component fails; it needs its own switching condition and test.

Relationships: $G_t = r_{t+1} + \gamma G_{t+1}$; $A(s,a) = Q(s,a) - V(s)$. Check: high reward and low BC loss do not yet establish stable behaviour.

Equipment, purchases and installation

USE EXISTING EQUIPMENT

Use the MacBook Pro M4, the M18 CPU environment, the M19-M20 model and simulator, the unchanged classical baseline and experiment logs. Physical actuators do not participate in training episodes; the MCU, IMU and camera remain available for earlier tasks.

PURCHASES NOW

No new mandatory purchases. Do not order a Jetson, graphics card or additional actuators to complete M21. Three short PPO runs with different seeds and BC training fit the established educational limit; achieving a particular reward is not guaranteed.

INSTALL NOW

Add **stable-baselines3** and **gymnasium** to a separate M21 environment; use a compatible pinned version of PyTorch CPU from M18. [Official SB3 installation: Prerequisites, Stable Release](#). Save a lock file; the additional Atari and Box2D packages are unnecessary for this task.

System: train the model with the existing simulator in Ubuntu 24.04 ARM64 under UTM; BC analysis can run directly on macOS ARM. When working across systems, record versions, architecture and data format. Do not claim bitwise-identical results across platforms: [PyTorch: Reproducibility](#).

INSTALLATION CHECK

Import torch, gymnasium, stable_baselines3; confirm that the model runs on CPU. Run check_env, reset with a fixed seed, 100 admissible steps and a short PPO update. Check finite values, separate terminated and truncated flags, and policy saving and loading; then confirm that zero residual correction reproduces baseline behaviour.

OPTIONAL, AFTER THE PREREQUISITE IS MET

A second simulator is unnecessary: use previously installed MuJoCo only for an already selected comparison. Consider NVIDIA hardware and Isaac Lab after measuring a CPU shortfall and making a separate G3 decision, scheduled for 27.11-10.12.2028. [Isaac Lab: requirements and installation](#). Apple M4 does not support CUDA; this path is not a prerequisite for M21.

Preparation and checks fit within the existing 75 hours, replacing repeated reading and setup. Delivery waiting time does not count as study time.

Motion fundamentals: the physical meaning of a model action

Required analysis: action, physical effect and time step

Before training, explain the chain: observation → target correction → low-level controller → actuator → mechanical motion → sensor. A normalised network action does not have torque units. In M21, it specifies a roll and pitch correction; the existing controller calculates the force or torque applied. The same correction may produce different torques under different loads.

A minimal model: from cause to state change

For an individual revolute joint, q is the angle in rad and ω is velocity in rad/s. The model $I \cdot d\omega/dt = \tau_{\text{net}}$ relates moment of inertia I in $\text{kg} \cdot \text{m}^2$ to net torque τ_{net} in $\text{N} \cdot \text{m}$; the latter already includes actuator, gravity and friction contributions with their signs. If I and τ_{net} remain constant over a short interval Δt , then $\Delta\omega = \tau_{\text{net}} \cdot \Delta t / I$. Both position and velocity are needed: the same q alone does not determine the next state.

Example. $I = 0.02 \text{ kg} \cdot \text{m}^2$, $\tau_{\text{net}} = 0.10 \text{ N} \cdot \text{m}$, $\Delta t = 0.02 \text{ s}$, initial $\omega = 0$: acceleration is 5 rad/s^2 , final $\omega = 0.10 \text{ rad/s}$, and angle change is 0.001 rad . Double I at the same torque: velocity becomes 0.05 rad/s . This is an analytical physics test; it does not change the policy interface to torque control.

Practice: detect unit and frequency errors

1. Trace the action. In a separate test, normalised $a = 0.5$ with an example scale of 0.04 rad means a target correction of 0.02 rad . Save the raw action, target, applied command and observation. For the actual environment, use its previously verified limit. At zero residual correction, verify agreement with the baseline; under saturation, identify which control level changed the request.

2. Check the integral penalty. For constant error $e = 0.1 \text{ rad}$ and scale $e_0 = 0.1 \text{ rad}$, the integrated error penalty over 1 s is the sum $(e/e_0)^2 \cdot \Delta t = 1 \text{ s}$. At $\Delta t = 0.02$ and 0.01 s , the result is identical; without Δt , the sums become 50 and 100 . Repeat on a saved trajectory to separate a metric error from the effect of control frequency on the motion itself.

Acceptance and schedule allocation

Save `action_physics.md`, a unit table and tests. Explain why doubling mass does not always double every inertia component, and why delay changes closed-loop dynamics. Allocate 1 hour from the analysis in 23-26 and 2 hours from practice in 31-34 to defining actions and rewards. This refines tasks 5-6 within the existing 75 hours ; no new training or equipment is required.

Assigned reading: Russ Tedrake, [Underactuated Robotics, Definitions](#) - state and equations of motion; Lynch, Park, [Modern Robotics §8.9](#) - gearing and the physical meaning of actuation.

Quadruped robot: from components to a controlled system

M22 • 03 Sep 2028 - 01 Oct 2028 • 75 hours

This month's deliverable

Assemble and commission a quadruped with 12 degrees of freedom: four legs, each with three actuated joints. The deliverable is **an assembled robot ready for final acceptance**: a specific hardware build, reproducible configuration, calibrations and test results demonstrating at least 60 seconds of standing and a slow gait on a prepared level floor. This stage verifies the architecture before the six-legged version, which remains the programme's final goal.

The software model, mechanical assembly and visualiser must describe the same physical unit. For every actuator, record its physical location, identifier, positive movement direction, zero, operating limits and firmware version. Save raw measurements and conditions for every trial: battery position, mass, surface, IMU mounting, communication frequencies and initial temperature.

What must be ready at entry

Prepare the outputs of earlier modules: a tested leg and actuator rig, CAD and drawings, power diagram, URDF, standing and gait controller, state estimator, diagnostics and failure scenarios. The entry check is to run a saved simulation test and explain the origin of every actuator command. If an axis sign or encoder conversion is unclear, resolve it on one leg first.

List the components actually available and the missing parts. Do not schedule a full-machine trial before the necessary support, wiring and measurement equipment are available. This month's main tools are the existing C++ and ROS 2 software, Python for analysis, a multimeter and suitable current and temperature instruments; choose ranges for your system. There is no need to introduce a new programming platform at this stage.

Scope and connection to the hand

M22 covers assembly, measurement, calibration and transfer of control to real hardware. Running, jumping, difficult terrain and neural-network control are outside the minimum deliverable. Actuator records, current monitoring, hardware motion stopping and calibration logs can be reused for the hand. The hand remains a separate device: successful robot standing does not verify it.

Distinguish three evidence levels: simulation, a supported test rig, and the physical robot on the floor. A suspended test does not establish stable standing. If a hardware problem cannot be resolved this month, save a release candidate with its blockers and continue the repair. Progression to 18 actuators requires the M23 entry check.

Organising 75 hours and the workspace

Four sequential blocks

Hours 1-20: 10 hours studying construction, power and commissioning procedures; 10 hours of assembly, inspection and initial power-up. Deliverables: the unit record and power-verification protocol. **21-40:** 10 hours on calibration and communication; 10 hours measuring one actuator, one leg and then the remaining legs. Deliverables: a machine-readable calibration file and actuator mapping table.

41-60: 10 hours analysing HIL, contact, IMU and delays; 10 hours comparing the model with real measurements and rerunning the controller. **61-75:** 5 hours analysing standing and the test programme; 10 hours practising standing, slow stepping, controlled stopping and preparing the candidate. Total: 35 hours of theory and analysis and 40 of practice; protocol preparation and data processing are included.

Arranging sessions

Organise each 10-hour analysis block into five 2-hour sessions: begin with a specific question, read the relevant section, calculate or inspect a log, then decide what to do on the rig. Split the final 5 hours into 2 + 2 + 1. Each practice block comprises 10 study hours in the Sunday practical, with breaks and alternating assembly, measurement and analysis. Do not operate actuators continuously for 10 hours: use cooling time to process recordings.

The dates on the first page are the calendar window from the original plan. A boundary date may belong to two modules; allocate available sessions without double-counting. Retain rest, holidays and separate integration G blocks from the overall plan. The hour numbering below specifies workload and order; it does not add evening sessions to the calendar.

Rig organisation

Separate the workspace into unpowered assembly, measurement and potential-motion areas. Prepare a stable support that holds the body when actuators are disabled, and provide access to the hardware E-stop. Before starting, specify the permitted motion and stopping condition. Do not use your hands to hold the legs or body as the primary motion restraint.

Each attempt's log contains run_id, author, Git commit identifier, CAD revision, configuration hash, initial conditions, expected behaviour and actual outcome. Record interventions separately: retightening fasteners, replacing a cable or moving the battery. A condition changed during a trial defines a new experiment. Record a missing measurement as no data, not zero current or zero error.

Mechanics, mass and load distribution

What to study

Identify the connections that transmit load from body to foot: frame, hip-actuator mounts, links, shafts, bearings and threaded inserts. Read the insert and fastener manufacturers' guidance for the material used, together with the test results for your printed parts. Check print orientation, local cracks and deformation after tightening. No universal tightening torque applies to every printed connection.

Review centre of mass: $r_{COM} = \Sigma(m_i r_i) / \Sigma m_i$, with masses in kilograms, coordinates in metres and all r_i expressed in one frame. CAD provides an initial estimate, but measure the mass of cables, battery and actual actuators. For quasi-static standing, the centre-of-mass projection and actual contacts help estimate support margin; they do not prove dynamic stability.

Task 1. Record the actual assembly

Inputs are CAD, the parts list, the assembled body and four legs with actuator power disconnected. 1) Assign leg names LF, RF, LR, RR and mark body axes. 2) Weigh assemblies and the complete unit in one configuration. 3) Compare measured mass with the assembly sum and CAD. 4) Inspect every loaded connection, insert and clearance throughout the admissible manual movement range, where the actuator permits manual movement.

Save assembly_manifest.csv, photographs of connections and a discrepancy list. Check that every loaded fastener and insert has an inspection status, and explain the difference between total and summed masses considering scale resolution. Boundary case: a cable is trapped only with the leg bent. Record the collision configuration, reroute the cable and repeat the complete inspection.

Task 2. Check centre of mass and deflection

Use supports or scales with suitable ranges and known contact coordinates. Measure reactions in a repeatable static configuration and estimate horizontal centre-of-mass position from moment balance. Repeat after moving the battery between two intended locations. Do not infer centre-of-mass height from this horizontal-plane measurement; use a model estimate and state its uncertainty.

Save mass_properties.md with calculations, battery positions and a frame-deflection estimate from markers or a measuring instrument. Check that centre-of-mass shift agrees with battery movement and the reaction sum agrees with weight. If a support carries no load, do not count it as a confirmed contact. Reposition supports and repeat the measurement, then update model inertial parameters only from established quantities.

Power, cable harnesses and independent motion stopping

What to review before power-up

Trace the power circuit from the battery or supply through protection, switching and distribution to each actuator. Identify computer and sensor power, common grounds, signal interfaces, cable strain relief and connectors separately. For each branch, list wire specifications, connector ratings, allowable conditions, protective device and its time-current characteristic. Comparing a single current rating is insufficient.

Distinguish controller current limiting, wiring protection, BMS and emergency energy removal. They serve different purposes and do not replace one another. If the circuit uses a contactor, check its suitability for DC, voltage, load characteristics and coil-transient suppression against component documentation. Develop a procedure for the selected circuit here; do not assign generic fuse values.

Task 3. Wiring checks and staged power-up

Inputs are the de-energised assembly, circuit diagram and component instructions. 1) Check continuity of intended connections, absence of unwanted connections between power rails, and connector polarity. 2) Check harness restraint throughout the leg's permitted range. 3) Power the low-power section and measure voltages. 4) Connect power branches using the approved procedure, beginning with one supported actuator; do not enable motion yet.

Save `powerup_checklist.csv` and an oscilloscope trace or log of available voltage and current measurements. Check that measured polarity and ranges match specifications; explain unexpected consumption before the next stage. Boundary case: computer supply voltage sags when an actuator is powered. Return to disconnected actuator power and check inrush current, DC-DC conversion, ground and connections; increasing a software timeout does not repair a power fault.

Task 4. Demonstrate E-stop operation

Define the physically achievable state after an emergency stop: support by an independent structure, removal of actuator energy and prevention of automatic reactivation. On a protected rig, test the circuit with the programme running and with the control process stopped. Record the button-press instant and an independent indication that force or torque application has ceased, using a circuit or actuator measurement appropriate to the design.

Save `estop_test.md` with the circuit, measured response time and residual-motion description. Derive the timing criterion from admissible displacement and mechanism characteristics. Releasing the pressed button alone must not resume an earlier motion command: test recovery separately. If torque removal makes the robot fall, test with support; improve how the safe state is achieved instead of calling free fall a successful stop.

Calibrating 12 actuators and reliable communication

The single-joint model

For each joint, record $q = s \cdot k \cdot (n - n_0)$, where q is angle in radians, n is the encoder count, n_0 is the calibration zero, s is $+1$ or -1 , and k is radians per count including the transmission ratio. The formula applies only to your encoder mode; count wraparound and multiple turns need separate rules. Do not apply another actuator's conversion factor merely because its connector looks similar.

Separate the mechanism's physical limit from the software operating boundary. The latter must account for calibration error, backlash, deformation, stopping distance and cable clearance. A limit recorded in the model still requires verification in the command path. For current and temperature, state measurement source, scale, offset and interpretation: built-in housing or board temperature may differ from winding temperature.

Task 5. Identifiers, signs and zeros

Connect one actuator or one leg with support that constrains unexpected motion. 1) Read ID, model and firmware and match them to the physical label. 2) In the permitted small region, command a known increment and compare motion with the positive URDF axis. 3) Establish a repeatable mechanical reference pose and calculate n_0 . 4) Repeat approaches to the reference from both directions, then apply the procedure to the remaining joints.

Save calibration.yaml and a table of repeats. Check that all 12 IDs are unique, commands reach the observed joint, and residual error and backlash measured in radians remain within a tolerance justified in advance. Boundary cases are a mirrored leg and a swapped cable. The script must detect disagreement with the hardware map; bus response order alone does not identify which actuator to enable.

Task 6. Frequency, errors and bus utilisation

First calculate communication time from byte count, bit rate and framing for the actual interface; add response and direction-switching delays. Then measure the real cycle for 1, 3 and 12 devices with the same command frequency and telemetry content. For DYNAMIXEL, study Sync Read/Write and Status Packet; for another bus, use its primary specification. Do not count a missing reply as packet loss when the protocol specifies no reply to that command.

Save bus_budget.csv, the cycle-duration distribution and counters for timeouts, CRC errors and retries. Calculate margin to the communication deadline using a measured adverse case and list untested modes separately. In the driver simulator, suppress one reply: only that joint's data must become stale, and an old sample must not be republished as new.

To calibrate current, compare the built-in channel with an independent measurement of the same physical quantity in several permitted modes; record scale, offset and error. Check temperature against a reference at a comparable location and in steady conditions. A board sensor cannot be calibrated as a winding sensor by simple subtraction. If the register cannot be corrected, save the verified conversion model and its limitations in the actuator record.

HIL and comparison with real measurements

What HIL means here

HIL is a test in which part of the real computing or hardware chain operates with a simulated plant. Draw the boundary: identify which process, controller, driver, bus and sensor are real and which are replaced. If control hardware is absent, call processing a file on a laptop replay. Match interface versions, units and timestamps; the HIL label alone does not make a test more accurate.

Tabulate discrepancies between model and robot: mass, link lengths, zeros, command delay, filters, friction, backlash, stiffness and foot contact. For each, specify its expected effect and measurement method. Correct geometry and signs first, then timing and dynamics. Simultaneously tuning many coefficients hides the root cause and does not produce reproducible improvement.

Task 7. Time alignment and control verification

Take a low-amplitude target sequence already verified on the leg and one log of its physical response. Record command-generation and transmission times, data-reception time and measurement timestamp if available. Establish the relationship between clocks first; subtracting timestamps from unsynchronised devices does not measure delay. Plot q_cmd , q_meas and interface events together, estimate the shift, then replay the same input in the model.

Save `fidelity_gaps.csv` and a before-and-after report for one justified parameter correction. Check that input, initial pose and frequency are identical; calculate RMSE on a common time grid and label it in radians. Boundary case: several samples are lost. Mark the gap and do not present interpolated values as complete measurements. Check controller changes by reprocessing the recording dataset before restoring actuator power.

Task 8. IMU and foot contact

With the body secured, check physical axes against `frame_id`, then test known static tilts using an external reference. Verify roll and pitch signs and IMU orientation relative to the body. Under translational acceleration, do not treat tilt calculated from the accelerometer alone as ground truth. For each foot, compare detected contact against observed touching and unloading, using the chosen sensor or an indirect estimate with stated uncertainty.

Save `imu_contact_validation.md`: reference, reference uncertainty, orientation errors and a table of correct and incorrect contact classifications. Control case: a leg touches the floor but slips; contact exists, but there is no stationary support. Remove one contact from estimator input and check the degradation flag. Accompany simulation friction tuning with a surface description; one fitted value does not characterise all real terrain.

Retain the M19 classical algorithm and M20 force allocator; check QP failure and timeout. Before operation, establish whether the actuator supports the required torque or current mode. $J^T f$ is not a ready-to-use position command: a position interface requires a justified inner and outer control-loop design. Keep the M21 policy disabled during initial integration.

Integrated laboratory: standing and the first step

Entry conditions and protocol

The assembly may enter this laboratory after tasks 1-8: known calibrations, verified independent stopping, working diagnostics, and a recording and time-alignment procedure consistent with measurements. Prepare a level area, foot and body markers, a backup support that carries no weight during the qualifying stand, a way to observe displacement, and all measurable limits. Record initial temperature, charge, voltage and battery configuration.

Before starting, specify operating boundaries: body-position region, joint limits, allowable data age, current, temperature and trial-termination conditions. For every threshold, provide its calculation or specification, unit and response. Criteria must use measurable quantities rather than descriptions such as not very hot or a plot colour. Removing torque and controlled lowering are different actions, selected according to the stopping cause and mechanism capability.

Task 9. Stand for at least 60 seconds

1) On the support, verify coordinated movement of all four legs to the permitted starting pose. 2) Place feet on their marked locations and gradually transfer load to the robot using your procedure. 3) Run a short trial and inspect currents and contact. 4) Only then run a continuous qualifying interval of at least 60 seconds. Record roll and pitch, body height and displacement where measurable, joint errors, currents, temperatures and events.

Save `stand_report.md`, video showing the backup support and the raw log. Success means independent standing throughout the interval without crossing declared boundaries or hiding telemetry losses. Calculate mean and maximum absolute roll and pitch errors separately and state units explicitly. If the backup support carries weight or an operator supports the body, the attempt does not establish standing. Do not remove that moment from the evidence.

Task 10. Slow gait and stopping

After standing, perform a limited crawl sequence: transfer the load centre, lift one foot, move it and confirm touchdown. Begin with one cycle, then attempt a short marked route. Compare initial and final poses, completed steps and contact losses. A commanded stop must reach the previously described state; an emergency cause uses the independent stopping path.

Save `crawl_run` and `stop_run` with video aligned to logs. Repeat a small, bounded change in tilt or load under identical conditions, first on the rig; derive method and amplitude from allowable loads instead of applying a hand strike. Check for recovery into the declared region within a measured time, or a timely stop. One successful traversal is a demonstration, not proof of reliability.

Separately from the qualifying stand, command a small permitted sequence of body roll and pitch changes, save target and measurement, and calculate error and settling time. Set amplitude from available kinematics and loading. This tests command tracking as well as holding one pose; stop the experiment on joint saturation according to the adopted criterion.

Failure tests and data validity

Six required tests

T1 - incorrect ID or sign. Substitute a configuration in the test environment, then verify it by reading hardware on the rig with actuator power disconnected. Activation must be refused with the exact joint name. **T2 - stale sample.** Repeat the last value with its original timestamp in the driver simulator. The controller must distinguish a missing new measurement from a stationary joint.

T3 - missing reply. Simulate one lost packet, then a sequence of losses; check counters and state transitions without an automatic hidden reset. **T4 - command beyond the limit.** Supply an inadmissible target only to the software path or HIL; verify rejection or documented clamping before writing to the actuator. This test does not require contact with the mechanical stop.

T5 - emergency stop with a stalled programme. With the robot on a support, stop the control process and check the independent path for ending force or torque application. **T6 - inconsistent contact.** During replay, supply contact during swing or loss of expected support; the system must report degradation and enter the intended state. Save the input, expected event and observed sequence for every test.

Heat, current and physical repeats

Also test slowly rising temperature and current indication beyond the set threshold by substituting telemetry in HIL. This establishes response logic; a separate circuit-specific rig test establishes the protection's physical ability to remove energy. Do not create a short circuit or overheat a motor to check a software flag. Load tests remain within the previously calculated range.

State which current is measured: supply current, branch current, phase current or the actuator's internal estimate. Quantities with different definitions cannot be summed without a model. For temperature, identify sensor location and sampling interval. Failure to record a peak with infrequent sampling does not prove there was no peak. Telemetry losses make the relevant metric incomplete.

Documenting verification

One test_matrix.csv row contains test ID, revision, evidence level, preconditions, stimulus, expected result, measured result, PASS/FAIL/INCOMPLETE and log link. Run required scenarios first in the simulator, then only those necessary and permissible on the rig. Do not reproduce physical faults by dangerously destroying components.

Do not exclude a failed attempt from statistics without a reason defined before the trial. A startup failure caused by your programme is a result, not bad data. An external-instrument error may invalidate a trial, but retain the record. Fully analyse at least one real failure or deviation and its correction: symptom, hypothesis, check, change and repeat.

The first 40 hours: detailed sequence

Hours 1-10: five 2-hour analysis sessions

1-2: reread the M22 objective, run the original algorithm in simulation or replay and inventory earlier-module outputs. 3-4: trace load transmission in CAD and list questionable connections and task 1 measurements. 5-6: calculate centre of mass and plan task 2 reaction measurements; state method limitations. 7-8: annotate the power diagram, ratings and unknown data, and prepare continuity testing. 9-10: describe the E-stop, support and restart prevention; rehearse without motion.

Use the final 15 minutes of each session to record decisions. If a component rating is unknown, name who will clarify it and the document required. An assumption must not become permission to power up. Before the practical, prepare a short action sheet with power-up and stopping sequences, available beside the rig.

Hours 11-20: the first practical

11-13: assembly, labels, inspection and weighing for task 1. 14-15: reaction measurements and centre-of-mass estimation for task 2. 16-18: wiring, continuity checks and staged power for task 3. 19-20: independent-stop verification for task 4 and evidence recording. These intervals include instrument setup and documenting results; repeats after finding a defect use time from the same block.

If the E-stop fails verification, do not start the next motion stage. Use remaining time to repair the circuit and make passive measurements. The month may end with a candidate that has not yet passed; retain that status instead of recording an unsupported pass. Calendar pressure does not relax the mandatory deliverable.

Hours 21-40: calibration and the bus

21-22: encoder conversion and zero definition. 23-24: mirrored legs, axis signs, operating limits and uncertainty. 25-26: physical meaning of current and temperature readings for the selected actuator. 27-28: calculate frame size and bus utilisation. 29-30: hardware-component lifecycle, read-error handling and pre-activation checks. These are five 2-hour analysis sessions.

31-34: calibrate one leg and then the others; record zero-repeatability results. 35-37: compare communication with 1, 3 and 12 devices and check timeouts in the simulator. 38-39: parameterised hardware-map and limit tests. 40: inspect after first activation, freeze the calibration version and prepare a log for model comparison. By the end of this block, physical names and software indices must map unambiguously.

Hours 41-75: from HIL to a release candidate

Hours 41-50: transfer analysis

41-42: define the HIL boundary and tabulate timing in command and measurement paths. 43-44: estimate delay and comparison limitations from task 7. 45-46: check measured masses, lengths and contact parameters; choose one significant model correction. 47-48: review IMU conventions, the tilt reference and contact-trust rules. 49-50: plan estimator and controller regression checks using original and corrupted logs.

At this stage, read only the required primary-source sections on page 12. Reading must produce a revised diagram, test or calculation. If dynamics material extends substantially beyond your control stack, document the model limitation and retain the previously verified controller; this month integrates material already studied.

Hours 51-60: the third practical

51-53: obtain a low-amplitude physical response from one leg and align it in time. 54-55: correct one parameter, repeat the same input and record improvement or deterioration. 56-57: measure the IMU in reference orientations and verify foot contact. 58-59: run T1-T6 in software or HIL and verify recovery. 60: prepare the complete standing configuration and reinspect harnesses after movement.

Do not spend the entire block seeking visually perfect simulation-hardware agreement. Establish the magnitude of significant discrepancies and verify that the controller respects limits in their presence. Record at least two explained discrepancies and one still unexplained, if present. An incomplete model is a limitation that can be investigated experimentally.

Hours 61-75: completion

61-62: define standing and roll/pitch tracking criteria, stopping conditions and measurement methods. 63-64: specify the slow gait, controlled stop and repeatable small disturbance. 65: review readiness from documentation; state which result permits or blocks M23. These 5 hours are split into 2 + 2 + 1.

66-68: preliminary and qualifying standing for task 9, with analysis between attempts. 69-71: a slow gait cycle, short route and normal stop for task 10. 72-73: permissible disturbance repeats and checks for new defects. 74-75: assemble the candidate package, verify plot reproduction and complete the progression decision. If time is short, omit video editing, additional routes and optional optimisation; calibrations, E-stop, standing and raw trial records remain mandatory.

M22 acceptance and handoff to the six-legged robot

What complete means

The release candidate contains source code and the exact assembly revision; a circuit with ratings and selection rationale; mass and centre of mass with measurement descriptions; the 12-actuator map; calibrations and operating limits; hardware-stop documentation; a consistent URDF; HIL, replay, IMU and contact records; and evidence of independent standing for at least 60 seconds, slow walking and controlled stopping. During the completed trials, currents, temperatures and communication parameters remain within operating ranges specified in advance.

Turn the original criterion, no unbounded growth of current, temperature or bus errors, into an observable check: every signal has a source, sampling interval, limit, response and test duration. A short normal run does not authorise continuous operation of unknown duration. An unresolved cause of control loss or an unreliable emergency stop blocks physical scaling.

Engineering self-check

Answer without running the programme: 1) Which joint moves when a chosen ID is addressed, and why? 2) How was its zero measured, and what is the spread? 3) What happens after emergency torque removal? 4) Which part of stopping is independent of the main computer? 5) How does a measurement timestamp differ from recording time? 6) Why does supply current not equal torque at each joint? 7) How do a new measurement from a stationary sensor and a stale sample differ? 8) Which simulation-reality discrepancy constrains the next trial?

Practical defence: choose one failed run, open its raw log, reproduce the plot and identify the root cause or explicitly state the hypothesis still untested. Show which change tests it. Then explain the power-disconnection chain on the diagram and the axis direction using a physical label. A screenshot of a successful simulation cannot replace this check.

The M23 package and the separate hand workstream

Create handoff_M23.md: verified quadruped capabilities, test boundaries, problematic parts, maximum observed bus utilisation and a projection for 18 actuators. Attach the progression decision PASS/HOLD. PASS means the scaling entry requirements are met; it does not establish that battery, frame and bus are already suitable for six legs.

For the hand, pass on the calibration.yaml template, diagnostic scheme and method of linking evidence to a physical unit. Do not copy numerical leg limits to fingers. The hand and tactile-equipment work in OPT 120 hours remains a separate workstream begun after M17. M22 does not consume its time or present its results as results of this assembly. Completion of both systems is checked at the end of the overall programme.

Assigned reading: quadruped assembly and measurement

The references may be in English; the exercises and definitions are presented here in English. Read only the named sections before the associated work, within the existing 75 hours.

Core reading

1. K. Lynch, F. Park - Modern Robotics. Sections 4.1.1 and 8.9: forward kinematics in the space frame; actuation, gearing and friction. For the 12-joint map, connect physical zero and sign to the model; for current tests, explain the effects of transmission and friction.

2. NASA - Systems Engineering Handbook. 5.2 Product Integration; 5.3 Product Verification; 5.4 Product Validation. Before HIL and floor trials, separate evidence of assembly, compliance with requirements and suitability of the selected mode. Deliverable: the power-up sequence and entry criteria for each rig.

Reference book: power problems

3. P. Horowitz, W. Hill - The Art of Electronics, 3rd ed. 9.4 Heat and power design; 9.6 Switching regulators and dc-dc converters. Read selected heating and power-conversion calculations for task 4. The aim is to understand losses and limits of a ready-made DC-DC converter; designing a mains supply is unnecessary.

Documents before specific checks

ros2_control, Jazzy. Hardware Components: interface types, lifecycle, errors. For the driver, specify which states and errors permit command output; relate this to independent stopping.

ROS REP-103 / REP-145. Units, Axis Orientation, Rotation Representation; IMU: Data Reporting, Common Parameters. For axis and IMU calibration, state the coordinate frame, units and meaning of acceleration; perform static verification before standing.

The actual bus specification. For DYNAMIXEL Protocol 2.0: Status Packet, Sync Read/Write, Bulk Read. For another actuator, use its specification. Calculate communication volume with 12 devices and the cycle deadline; a similar connector does not establish compatibility.

Separately save specification pages for your actuators, battery, BMS, charger, DC-DC converter and protective devices: voltage, current, temperature, mode and duration. A textbook example does not establish this assembly's operating limits.

Terms and definitions

Commissioning. Connecting and verifying an assembled device in stages, progressing to the next mode only after declared criteria are met.

Hardware revision. An identifier for the exact composition and implementation of an assembly: parts, wiring, sensors and board versions to which measurements apply.

Actuator map. A table linking physical joints to bus addresses, directions, zeros, units and limits; verified by real movement of every assembly.

Zero calibration. Determining the offset between a sensor reading and a selected geometric zero, recording the method, conditions and repeatability spread.

Backlash. Relative movement within a transmission when load direction reverses, during which input motion is not transmitted to the output.

Continuous torque. Torque admissible at specified speed, cooling and sustained thermal conditions; it must not be equated with short-duration stalled-shaft torque.

Centre of mass. The mass-weighted position of the complete assembly; it changes as legs move or the battery is repositioned.

Inertia tensor. A matrix describing mass distribution about a selected point and axes; it relates angular motion to torques, with units $\text{kg}\cdot\text{m}^2$.

Duty cycle. The fraction of time under active load within a declared cycle; together with peaks and cooling, it affects actuator heating.

Power tree. A diagram of sources, converters, protective devices and loads showing current paths and points where individual branches can be disconnected.

Voltage sag. A drop in voltage under load caused by source and wiring resistance or supply limiting; it can reset electronics.

BMS. A system monitoring battery cells and providing specified protection; it does not replace a compatible charger, fuses and emergency stop.

DC-DC. A DC voltage converter selected for input range, output voltage and current, losses, cooling and response to transient loads.

E-stop. A specified independent means of ending hazardous motion; the structure may fall after torque removal, so verify the final state.

Watchdog. Monitoring timely confirmation of correct operation; exceeding the timeout triggers a predefined response independently of the normal command cycle.

Communication deadline. The latest time for obtaining required data and issuing a command; exceeding it requires a response even without a packet error.

HIL. Testing in which real hardware interacts with a simulated part of the system; explicitly document which channels are real.

Specific force. The ideal accelerometer reading associated with a $-g$ in the sensor frame; it is not directly equal to body acceleration in world coordinates.

Relationships: $P = UI$; conductor losses $P = I^2R$. Check: disconnecting actuator power and reaching a safe final state are different facts.

Quadruped purchases and setup

USE EXISTING EQUIPMENT

Use the tested 3-degree-of-freedom leg, its bus interface, MCU and IMU, RGB-D camera, multimeter, CC/CV supply, analyser, tools and Mac M4. For the existing 3D printer, select a profile for its exact model; replenish consumables and fasteners according to released CAD and BOM.

BUY BEFORE ASSEMBLY - AFTER LEG ACCEPTANCE

9 identical working actuators plus **2 separate spares**: $3 + 9 = 12$ installed. The family, voltage, bus, thermal regime and continuous loading have already been verified on the leg. Add parts for three legs, frame, harnesses and fasteners according to the BOM; the M7 bench DC motor is not included in these 12.

1 onboard computer: Pi 5 with 8 GB or an Ubuntu-compatible x86 computer after measuring CPU, RAM and USB loads. Reuse a suitable computer already purchased. The Pi needs 1 boot medium, cooling and power according to [Pi documentation](#); account for camera consumption separately. The model name does not guarantee every driver will work.

1 power system: battery, compatible charger and BMS, hardware stop and power disconnection, branch fuses, DC-DC converter, connectors and wiring. Select ratings after measuring operating and peak currents, duration, voltage sag and heating; the bench supply limit is not the power budget for 12 actuators. The number of protective devices matches the branch count.

1 support or backup frame with verified load capacity; **1 IR thermometer** with a suitable range and allowance for surface properties; spare harness components. If choosing contact-sensing feet, provide **4 channels, one per foot**, repeatable activation and calibration for robot mass; these sensors do not establish absence of slip.

INSTALL AND VERIFY

Onboard, use [Ubuntu 24.04 ARM64 for Pi](#) or [24.04 x86_64](#); [ROS 2 jazzy](#), existing `ros2_control`, bus driver and diagnostics from the pinned working environment. Retain the existing environment on the Mac and in UTM.

Check the build, software simulator and replay without motion; 12 unique IDs; units and sign of one supported channel; heartbeat loss; current and temperature recording; and independent stopping. Test USB, network and camera under load on the chosen computer. Do not enable torque automatically when the OS starts.

For the model: [URDF inertial/collision](#); for logs: [Foxglove Plot](#); for the ID table: [pytest parametrization](#). Preparation and checks fit within the existing 75 hours, replacing repeated reading and setup. Delivery waiting time does not count as study time.

Motion fundamentals: loading before the first step

Required analysis: measured reactions, centre of mass and model limits

Kinematics shows where a foot can be placed. Forces and torques are needed to determine whether the robot can withstand standing and acceleration. Begin with measurable statics: it reveals errors in mass, centre-of-mass and load-transmission estimates before gait tuning. The number of supports alone does not imply equal weight distribution.

Free-body diagram and equilibrium

The example body model assumes a horizontal surface, a stationary robot, gravity and vertical reactions from two support groups. The x axis points forwards; the rear group is at $x=-a$, the front at $x=+a$; z points upwards. F_r and F_f are ground forces on the robot in N, and x_c is centre-of-mass position. Force and moment balance give $F_r+F_f=mg$; $a(F_f-F_r)=mg \cdot x_c$. Therefore, $F_f=mg(a+x_c)/(2a)$, $F_r=mg-F_f$.

Example. $m=4$ kg, $a=0.20$ m, $x_c=0.05$ m, $g=9.81$ m/s²: $F_f=24.525$ N, $F_r=14.715$ N. Adding a 0.5 kg load at $x=0.25$ m gives total mass 4.5 kg and $x_c=(4 \cdot 0.05 + 0.5 \cdot 0.25)/4.5=0.07222$ m. The new reactions are 30.043 N and 14.102 N. Rear loading decreases: the payload adds a moment as well as weight.

Practice: verify the calculation on the assembled robot

1. Measure mass and grouped reactions in task 2 using existing fixtures and instruments. If an instrument displays equivalent mass in kg, convert it to N using g. Recover $x_c=a(F_f-F_r)/(F_f+F_r)$, compare against CAD and include the mass of cables, battery and mounts. Two sums cannot determine the force at each of four feet.

2. Relate load to the joint. For one adopted pose from M8, calculate J and the force in the same coordinate frame; the force contribution is $\tau_{\text{ext}}=J^T F_{\text{ext}}$. During holding, the actuator must compensate for it while accounting for link weight; compare with the M9 model. Convert current to torque only through a calibrated transmission model. Do not send a calculated torque to a position-controlled servo as an angle.

What counts as evidence

Save load_balance.csv: pose, payload, reactions, calculation, residuals, units and instrument uncertainties. Check reaction sum, moment balance and direction of centre-of-mass displacement. Set tolerance from scale resolution, repeatability and geometry; do not impose 1% without justification. A negative calculated reaction means that support is infeasible, not that the foot is attracted to the floor.

Within 75 hours: 1 hour of analysis in 5-6 and the practical in 14-15 (task 2), without a new motion mode. This is a quasi-static check: accelerations, slopes and external disturbances require the full model. Read [Modern Robotics §5.2](#) - the relationship between external force and joint torques.

Six legs: scale only what has been verified

M23 • 01 Oct 2028 - 01 Nov 2028 • 75 hours

The completed robot

M23 aims to produce **a six-legged robot v1.0 with 18 degrees of freedom, slow walking and active body levelling**: three actuators per leg. It must stand independently for at least 120 seconds, perform wave and ripple gaits under declared conditions, and diagnose loss of data, contacts and individual components. The central question is what changes when moving from four legs to six in mechanics, power, communication timing and support control.

18 DOF refers to the number of actuated joint coordinates, not the number of independent body motions available on the ground. Additional legs provide more contact-allocation options, but increase mass, energy consumption, wiring and possible faults. Simply appending six IDs to an array does not validate previous calculations. Analysing the architectural change is a major practical task.

Entry check before scaling

Open the M22 package: independent quadruped standing, slow gait, functioning independent stop, calibrations, current and bus budgets, and analysis of significant failures. Record the entry decision PASS/HOLD in writing. If E-stop, power, actuator identification or the control cycle remain unverified, physical expansion stays on HOLD. In that case, complete M23 calculations and simulation tasks while resolving hardware blockers.

The six additional actuators and parts for two legs are planned only after this check, as in the original plan. Purchasing is not automatic: first verify compatibility, calculate the budget and obtain the project owner's decision. Changing actuator family for the last two legs requires new calibrations, a delay model and verification of joint operation.

The operating scope of the result

The $\pm 10\text{-}15^\circ$ range in the overall plan is a design target for body levelling and slope tests, admissible only where the particular assembly's geometry, stability, traction and loads permit. Do not adopt it as an operating range before calculation. Start with a smaller verified angle and expand only when criteria are met. State surface angle and desired body angle separately: they are different quantities.

The robotic hand develops along its own OPT workstream after M17; M23 leg verification does not replace its limits or results. Shared logging and diagnostics benefit both systems. The final goal is a walking robot and hand by programme completion, so pass six-legged-version limitations and unfinished hand items to M24 and the separate G4 without adding hidden hours.

Work blocks, measurements and scope management

The monthly plan: 35 hours of analysis + 40 hours of practice

1-20: 10 hours recalculating mechanics, power and communication; 10 hours assembling, calibrating and testing 18 devices. **21-40:** 10 hours on contact sequences and admissible transitions; 10 hours implementing wave and ripple in simulation and on the prepared test area.

41-60: 10 hours on surface, contact and uncertainty estimation; 10 hours on active levelling, bounded terrain and operation without a camera. **61-75:** 5 hours analysing reliability and energy; 10 hours conducting a test series, maintenance and preparing v1.0.

Organise the first three analysis blocks as five 2-hour sessions each; the last is 2 + 2 + 1. Practice comprises four Sunday blocks of 10 study hours. Alternate movement, cooling, inspection, resetting initial conditions and analysis. Charging time alone is not study time; use that interval for calculations and data processing.

One experiment, one question

For every run, record gait type, phase order, cycle duration, swing height, target body tilt, surface, load, battery condition and temperatures. Begin by comparing two configurations differing in one parameter. Simultaneously changing gait, surface and controller gain prevents you from identifying why error changed.

The first-page dates repeat the overall plan's calendar window. Shared boundary dates do not double hours; holidays and integration G blocks retain their place in the overall schedule.

Work-hour numbers specify task order and do not require extra evening work. Print parts and arrange components in advance after entry-check authorisation, allowing for actual delivery time.

Reducing scope while preserving the objective

If behind schedule, postpone tripod first, then sideways motion on difficult terrain, then optional depth-camera comparisons and additional surfaces. Retain wave and ripple, standing, independent stopping, admissible-slope assessment, operation when the camera is unavailable and transparent repeated-test results. Without hardware verification of wave and ripple, call the result a candidate, not a completed six-legged robot v1.0.

Do not consume the hand reserve or G4's 35 hours on minor gait refinements without replanning the programme. The main deliverable is a justified operating envelope. Python is useful for support geometry and reports, C++ for the existing control loop. An additional log analyser may be written in Rust; a third implementation is optional.

The six-legged frame and the 18-actuator budget

What to study and recalculate

Examine the arrangement of six hip assemblies, fastener access, harness routing and centre-of-mass changes. Check overlap between adjacent leg workspaces, load transmission from the middle pair to the frame and whether one actuator can be removed without dismantling the body. New legs use the existing naming conventions and units but have their own calibrations. Estimate stiffness for the selected design load case and compare it with admissible foot-position change.

Build the power budget for actual load combinations: standing, initiating swing, a low body posture and stopping. Do not assign each leg exactly one sixth of the weight in every phase. List specification limits and cooling conditions for branches, connectors, BMS and DC-DC converters. Summed rated peaks represent only one conservative scenario; separately measure simultaneous currents and their duration.

Task 1. Design review: 4 → 6

Inputs are the M22 candidate, six-legged-frame CAD and component specifications. 1) List changes to parts, masses, axes, harnesses and power distribution. 2) Check fasteners, clearances and service access for every new hip frame. 3) Calculate the new configuration's centre of mass and predict support reactions. 4) After unpowered assembly, measure mass and verify geometry using reference dimensions.

Save `scaleup_design_review.md` and an updated `assembly_manifest`. Check that every change traces to a CAD revision and that software renaming does not conceal workspace collisions. Boundary case: a middle foot collides with a front link during turning. Correct the trajectory or geometry and save the forbidden region as a test before permitting physical motion.

Task 2. Power and communication after expansion

Calculate the 18-actuator communication cycle for your protocol and telemetry content, then compare with M22 measurements for 12 devices. With the robot supported, add new branches in stages, checking IDs, polarity, signs, zeros and idle current. Measure simultaneous communication at the actual frequency, computer load and enabled diagnostics. Record supply and branch currents separately where instruments permit.

Save `power_bus_18.csv` and plots of margin to the communication deadline. Acceptance: new devices create no duplicate IDs, and voltage sag and delays remain within justified operating ranges. Boundary case: two branches work separately but overload a shared DC-DC converter together. Revisit calculation and measurement of the common circuit; reducing gait speed is not a universal solution to an electrical problem.

Wave, ripple and admissible contact transitions

Gait model and transition conditions

Describe each leg with the state machine STANCE → LIFT → SWING → TOUCHDOWN → STANCE. In STANCE, the foot is an assumed support; in SWING it moves; TOUCHDOWN requires contact confirmation. Transitions depend on more than time: check target reachability, state freshness and remaining supports. Phase ϕ from 0 to 1 identifies position in the cycle, while duty factor is the fraction of the cycle assigned to stance.

For slow motion, calculate the convex polygon of confirmed supports and the centre-of-mass projection along gravity onto their plane. Support margin is the signed minimum distance from this projection to the polygon boundaries. On a horizontal floor, this is the ordinary vertical projection; retain the projection direction on a slope. Positive geometric margin alone does not check friction, actuator torque or dynamics.

Task 3. A wave table and verifiable support margin

Inputs are the six-legged model, level floor, calibrated axes and initial standing pose. 1) Specify sequential single-leg swing and a phase table for all six legs. 2) Before lift-off, recalculate remaining supports and body-position feasibility. 3) Visualise contacts, polygon and margin in metres. 4) Run a complete cycle, startup and stopping, first in simulation and then with a small permitted step on the test area.

Save gait_wave.yaml, the phase diagram and a log of minimum margin. Check that no more than one leg swings at a time, feasible support exists before lift-off and commands cannot skip touchdown. Boundary case: the foot has not touched down but the timer requests the next lift. Block the transition and emit an event; the state machine must not continue without contact confirmation.

Task 4. Ripple, turning and sideways motion

Construct ripple with more overlap between swings, an explicit phase table and allowed combinations. Do not rely on the gait name alone: ripple implementations use different conventions. For every combination, check reachability, collisions and support. Then add turning about body z and sideways translation separately; positive directions follow the frozen coordinate frame.

Save gait_ripple.yaml, allowed-transition tests and three short scenarios: forwards, turning and sideways. Check that an identical command has the same meaning for right and left legs and that mode changes pass through an admissible state. Boundary case: switching wave → ripple during swing. Permit it only after phase alignment or through a stop. If the assumed contact is physically impossible, the test fails even when motion appears smooth.

Six contacts, the surface plane and uncertainty

What to study

Review transforming foot coordinates from leg frames to a common body or world frame. For an approximately planar patch, estimate $z = a \cdot x + b \cdot y + c$ from confirmed supports: matrix A contains rows $[x, y, 1]$, vector z contains foot heights in metres, and the solution minimises squared residuals. The normal is proportional to $(-a, -b, 1)$. Use weights for measurements with different reliability, explaining their origin.

A plane model cannot represent arbitrary stairs or separate rocks without examining residuals. Three nearly collinear points provide poor geometry even with a small computed error. Six coordinates are not six independent measurements if all derive from one erroneous body orientation. Contact, a stationary foot and estimator confidence are three different indicators.

Task 5. Estimate a plane from six supports

Generate synthetic data for a horizontal plane and two inclined planes with known parameters, then add noise, one outlier and contact loss. 1) Transform every foot into one frame. 2) Select confirmed supports. 3) Solve by least squares and check rank and conditioning. 4) Calculate residuals and a validity flag. 5) Compare with an independent reference measurement of the prepared area.

Save `terrain_estimator_tests` and `plot_plane_error`. Check that ideal data recover the specified plane within numerical precision; with noise, compare error to the declared tolerance, not zero. Boundary cases are two supports, nearly collinear contacts and one foot on a protrusion. Return invalidity or a limited estimate in these cases; an arbitrary normal without a flag is unacceptable.

Task 6. Depth-based foothold selection and camera failure

Use a saved scene depth map and camera calibration from earlier modules. Construct a small region of admissible footholds: reachable kinematics, clearance, known surface, height uncertainty and image age. Compare selection against a method using only confirmed contacts. With the camera disabled, the robot remains in a previously tested mode or stops; an IMU alone does not authorise movement over unknown terrain.

Save `foothold_candidates.json` and two identical scenarios with and without the camera. Check that unknown depth does not become zero height, a stale image is not treated as fresh, and the selected foothold passes reachability checks. Boundary case: a shiny or dark surface creates a hole in the depth map. Show uncertainty and explain rejection of the placement while retaining the classical fallback mode.

Active levelling and the terrain operating boundary

Relating body tilt to leg configuration

Levelling maintains a selected body orientation relative to gravity by changing leg positions within reachability and load limits. The IMU estimates orientation, contact estimation describes support, and kinematics converts desired body position into joint commands. Zero roll and pitch do not mean the feet lie at the same height; on a slope, leg configurations may differ.

Map the constraints for each pose: joint travel, proximity to singularity, loaded torque, body clearance, traction, contact quality and support margin. Estimate quasi-static joint torque using $J(q)^T \cdot f$, where J is the foot Jacobian, f is force in newtons and the result is in N·m. Force allocation must satisfy equilibrium and contact constraints; equal forces on all six legs are only one special case.

Task 7. Verify 120 seconds of standing

Start on a level surface in a previously verified configuration. 1) Run a short current and contact check. 2) Stand independently for at least 120 seconds without the backup support carrying weight. 3) Record roll and pitch errors, data freshness, minimum support margin and temperatures. 4) Repeat after cooling and restoring initial conditions, explicitly identifying every repeat.

Save stand_120s with raw log and video. The criterion is a complete continuous interval within declared limits, without body support or unaccounted measurement losses. This verifies a particular standing condition, not machine lifetime. Boundary case: tilt error decreases by saturating one joint. Body-orientation error is small, but a constraint is violated, so the trial fails.

Task 8. Map slopes and uneven terrain

Before the physical trial, calculate admissible surface angles and commanded body orientations, then test them in simulation with mass, friction and delay variations. On an adjustable rig with suitable backup support, increase slope in steps from a verified value. For each angle, save achieved body roll and pitch, currents, joint poses, contacts and stopping reason. Introduce unevenness as a separate factor with measured height.

Save operating_envelope.csv: tested, untested and forbidden regions. Mark the $\pm 10\text{-}15^\circ$ design target as achieved only from actual tests of the specific configuration. If the limit is smaller, identify the limiting mechanism. Boundary case: traction is lost while torque remains admissible. Reducing controller error does not solve a contact problem. Return to level ground and investigate foot-surface interaction.

When expanding the M20 force allocator, update contact-set size, constraints and solver timing measurements. In HIL, test infeasible constraints and solver-deadline overruns: retain the previously tested classical fallback. The M21 policy is outside the mandatory physical result; transfer from four legs to six requires separate verification of observations, actions and applicability.

Reliability, energy and maintenance

A fault tree with a clear top event

Choose the top event: the robot loses admissible support or fails to stop on request. Divide causes into power, bus, actuator, mechanical connection, state estimation and control logic. AND denotes a combination of required events; OR denotes alternative sufficient paths. Do not assign branch probabilities without data; a qualitative tree already helps select tests. A common power source or configuration error links several legs, so do not silently assume independent failures.

With only five functioning legs, the system must first recognise the condition and restrict behaviour. Continued walking depends on geometry, load and how the sixth leg failed. A freely dangling leg, a leg jammed in the air and a leg jammed while supporting load require different responses. The basic fallback to verify is stopping or lowering according to the design procedure. Five-legged walking requires a separate justification.

Task 9. Component or leg failures and recovery

From the tree, select telemetry loss, a stopped node and incorrect contact. First inject them in simulation or HIL, recording the stimulus time, detection and resulting state. Then reproduce only permissible rig variants with the robot supported. Check that a recovered node cannot receive an old command without state verification and an explicit resumption procedure.

Save `fault_tree` and `fault_runs` with a timeline. Check that every stimulus has an expected outcome and measured detection time; behaviour must fit the remaining contacts and limits. Boundary case: a common bus failure produces six identical warnings instead of recognising a shared cause. Add aggregate status while retaining primary errors. Do not unplug a loaded connector by hand to demonstrate a fault.

Task 10. Energy and maintainability

On a marked route, measure voltage $U(t)$, supply current $I(t)$ and distance d . Energy $E = \int U \cdot I \, dt$ is in joules; $E/3600$ is Wh, and E/d is J/m for a valid non-zero distance. Compare wave and ripple at comparable speed, surface, mass and temperature. For a stationary robot, report power and energy over a selected interval; do not divide by a near-zero distance.

Save `energy_comparison.csv` with integration boundaries, measurement gaps and repeats. Then, with the system de-energised, replace an accessible service assembly according to its instructions, measure the time, and repeat inspection and calibration. Save `maintenance_runbook` and a reasonable spare-parts list based on actual failures. Boundary case: the mechanics are restored but zero is lost. Restart is prohibited until the joint record is updated.

For thermal balance, tabulate every actuator's temperatures on the same route: initial, maximum observed, rise and load duration. Identify a consistently overloaded leg and check zero, friction and support allocation. Change only one admissible configuration or phase parameter and repeat; reduced heating in one leg must not conceal increased heating elsewhere or compromised support.

The test series and six required boundary cases

A repeat plan with a defined trial unit

Plan 40 attempts as a working target within the original 30-50+ guideline. Example allocation: 10 wave trials on level ground, 10 ripple trials on the same ground, 10 levelling trials on a verified slope, and 10 transitions into restricted operation without a camera. Each attempt includes restoring the starting pose, checking conditions, executing the specified scenario and recording the outcome. Sensor samples and steps within one traversal are not independent attempts.

If equipment availability or cooling duration prevents this plan, justify a smaller sample and narrower conclusion in advance. Do not replace unavailable difficult trials with many identical easy ones while claiming the original coverage. Mark untested conditions explicitly. For comparisons, fix the order or alternate modes so battery discharge is not mistaken for a gait advantage.

Control scenarios

T1. Camera loss during an admissible short route: enter a verified fallback or stop, without selecting new unknown footholds. **T2.** One false contact during replay: the estimator accounts for inconsistency and does not automatically expand support. **T3.** Nearly collinear surface points: return poor-conditioning or invalidity status; do not present an arbitrary slope as a reliable estimate.

T4. Stale state from one leg: do not authorise the next phase using an old pose. **T5.** Wave/ripple switching in an inconvenient phase: defer the transition or pass through a stop, retaining an admissible contact order. **T6.** A stopped node or bus loss in HIL: detection, restricted behaviour and recovery proceed without unintended resumption. A physical variant requires impact analysis and support verification first.

What the results can establish

Report N, successes, failures and unusable records separately by condition. Calculate a confidence interval for success rate and explicitly state the assumption of independent trials. Even 40 successful tests do not prove absolute reliability or determine lifetime in hours. Temperature drift and a shared defect in one assembly make trials dependent; discuss this limitation instead of artificially increasing N.

The summary must include at least one negative scenario with the expected restriction of behaviour and one real failure if one occurred. Analyse causes, add a regression test and repeat the relevant test class. A version change during the series separates data into different configurations; do not combine them into one v1.0 estimate without explanation.

The first 40 hours: expansion and gaits

Hours 1-10: decisions and calculations

1-2: check the M22 package and record PASS/HOLD for physical expansion. 3-4: compare four- and six-legged CAD and map loads, service access and conflicts. 5-6: recalculate mass, centre of mass and power scenarios. 7-8: assess BMS, DC-DC converters, protective branches and communication margin for 18 actuators. 9-10: specify assembly, calibration and E-stop re-verification sequences.

Put unresolved specification parameters in a question table. For each, identify its decision impact: blocks power-up, limits the tested mode or concerns future expansion. Separately assess the effects of external covers or styling and unknown power-connection ratings. By the end of hour ten, have a concrete task 1 work list and expected task 2 measurements.

Hours 11-20: expansion on the rig

11-14: assemble new hip frames and two legs; measure mass, add labels and check clearances. 15-17: staged connection, identification, signs and calibration of new joints. 18-19: communication load and power measurements with 18 devices. 20: repeat independent stopping, inspect and freeze the revision. If the entry decision is HOLD, use these hours to repair the quadruped and test the six-legged model without physical expansion.

Post-assembly verification must include the original four legs: new harnesses or added mass can impair previously working components. Repeating known M22 scenarios checks preservation of earlier functions, but does not verify an untested six-legged gait. Update numerical limits only with supporting evidence.

Hours 21-40: contacts and movement

21-22: phase state machine and duty factor. 23-24: support polygon, centre-of-mass projection and permissible-lift criteria. 25-26: wave table and touchdown that does not occur when the timer expects it. 27-28: ripple and allowed overlapping swings. 29-30: turning, sideways motion and gait changes. For every 2-hour session, prepare one test or drawing needed in practice.

31-34: task 3 in simulation, support checks and a short hardware wave traversal after rig testing. 35-37: task 4, ripple and transition through an aligned phase. 38-39: separate turning and sideways scenarios within the verified region. 40: analyse worst support margin and correct one transition fault. Treat tripod as a possible extension; do not add it at the expense of unfinished touchdown control.

Hours 41-75: terrain and evidence

Hours 41-50: four important distinctions

41-42: foot coordinate frames, gravity and the support plane. 43-44: least squares, residuals, rank and nearly collinear contacts. 45-46: contact confidence, slip and body observability. 47-48: depth uncertainty, image age and selection of a reachable foothold. 49-50: load and kinematic-constraint maps on a slope, desired body orientation and conditions for restricting motion.

Explain four distinctions in writing: touching versus stationary support; the plane beneath the feet versus the horizontal; body tilt versus surface slope; and no camera versus no sensors at all. If these concepts are confused, postpone new-terrain trials and correct notation and tests. Terminology must agree across code, plots and protocol.

Hours 51-60: the surface experiment

51-53: task 5 on synthetic data and the measured area, including an outlier and contact loss. 54-55: task 6, the saved depth map and the same scenario without a camera. 56-57: preliminary and qualifying 120-second standing for task 7, with analysis. 58-60: stepped slope increases and small measured terrain variations for task 8, only within the admissible region. Save every attempt, but count it in the confirmation series only if it follows the predefined protocol.

A failed slope trial still yields a useful result: the operating boundary and its cause. Do not raise current limits to reach a desired plot value without recalculating the complete power circuit and thermal behaviour. Do not change foot size midway through a series without a new design revision either: traction and the contact model then change.

Hours 61-75: reliability and release

61-62: fault tree, five functioning legs and admissible recovery. 63-64: energy metrics, a 30-50+ series or justified N, accounting for repeats and dependence. 65: readiness of scenarios, instruments and criteria. These 5 analysis hours are split into 2 + 2 + 1.

66-68: task 9 and control faults first in HIL, then necessary rig checks. 69-71: the prepared series of short trials, including attempts already accepted; resetting conditions and cooling are included. 72-73: task 10 energy measurements and a service operation on the de-energised machine. 74-75: results table, success-rate interval, limitations and the six-legged robot v1.0 package, or a candidate with release blockers. Do not shorten checks merely to reach a repeat count: revise claimed precision and coverage.

Acceptance, additional gaits and the M24 handoff

Required evidence for v1.0

The package includes the six-legged-frame revision and 18-joint map; mass and centre of mass; power and communication calculations and measurements; repeated independent-stop verification; confirmed 120-second standing; wave and ripple with admissible phases; levelling within a defined region; responses to camera loss and node failure; a test series with justified N; an energy report; and inspection and maintenance instructions.

Every result has a raw file, configuration version and criterion. Holding level becomes a measured roll/pitch error, surface condition and duration. Traversing uneven terrain requires measured obstacle geometry and attempt count. Operation without a camera applies only to the tested restricted mode. An unmeasured quantity remains unknown even when the video appears convincing.

When to investigate tripod

Consider tripod only after verifying wave and ripple, actuator loading and feasibility of three-legged support. Typical alternating groups for a symmetric six-legged body are LF/RM/LR and RF/LM/RR; verify this convention against your names and geometry. For every phase, check contacts, margin and transition conditions. A triangle's existence does not guarantee admissible torques, friction or stability during swing.

Optional task if time permits: compare tripod and ripple phase tables in simulation with the same input, saving minimum margin and errors. A physical run requires a separate positive decision for that mode; it is outside the required M23 evidence. Similarly, five-legged walking remains research unless its operating envelope is calculated and verified.

Defence and handoff package

Answer: why do 18 actuators change the communication deadline; why is six-legged load not always shared equally; what does insufficient rank in plane estimation mean; how do loss of contact and slip differ; why does zero tilt not guarantee an admissible pose; what response is available for a jammed leg; what does E/d measure; and why do 40 successful attempts not mean 100% reliability? Demonstrate the answers using your data.

Pass to M24 requirements_candidates.csv, the attempt register, links to every raw log, assembly and calibration instructions, limitations, known defects and repair priorities. Record the hand's status and OPT results on a separate line. Do not rename an unfinished function as verified to meet the shared date. G4 remains a separate 35-hour integration checkpoint after the modules; it is not hidden within these 75 hours.

Assigned reading: six legs, contact and reliability

The references may be in English; the exercises and definitions are presented here in English. Read only the named sections before the associated work, within the existing 75 hours.

Core reading: contact and loading

1. R. Tedrake - Underactuated Robotics, MIT notes. Planning and Control through Contact → (Autonomous) Hybrid Systems. For tasks 3-4, write out foot modes, transition conditions (guards) and state changes on switching (reset); a phase transition requires a verified event. Retain the remaining contact-optimisation material as reference.

2. K. Lynch, F. Park - Modern Robotics. 12.1.2 Contact Types; 12.1.3 Multiple Contacts; 12.2.1 Friction. For tasks 4-6, distinguish touching, slipping and contact loss. Apply the contact model to feet; manipulation results do not directly establish six-legged stability.

A decision-making reference

3. NASA - Systems Engineering Handbook. 6.4 Technical Risk Management; 6.5 Configuration Management. For task 9 and PASS/HOLD, connect the fault event, consequence, test and assembly revision. Use a brief register without unsupported probabilities.

Documentation as needed

Actuator bus. DYNAMIXEL Protocol 2.0: Sync Read/Write, Bulk Read - only if it matches the real protocol. For task 2, recalculate communication volume and time for 18 devices; spares are excluded from active polling.

ROS diagnostics. README: diagnostic_updater, diagnostic_aggregator. For task 9, distinguish a missing reply from one leg, common-channel failure and stale status; check the branch against Jazzy.

NumPy. linalg.lstsq: rank, singular values, residuals. For task 5, assess geometric degeneracy in plane estimation. An empty residuals array does not establish a perfect fit.

Optional: architecture comparison

Kevin Ochs, ROS Hexapod Stack: Nodes and hexapod_controller. This historical ROS project is useful for comparing ripple and tripod; do not install its dependencies and gains into the current working environment. Reading deliverable: two transferable design choices and two incompatible assumptions.

Terms and definitions

Gait phase. Normalised position within a repeating cycle; it specifies the contact plan but does not confirm actual foot-surface contact.

Stance. An interval of planned leg contact during which admissible support force is constrained by traction, geometry and actuators.

Swing. Movement of a foot to its next support without planned support-force transmission; premature contact requires a separate response.

Wave gait. A slow sequential gait with legs swinging in turn; the specific order and admissible overlaps are defined by a phase table.

Ripple gait. A gait with phase-shifted waves of swing; the selected table, rather than the name, determines the number of simultaneous swings.

Tripod gait. A gait alternating two groups of three legs; group geometry alone does not guarantee admissible forces.

Guard. A testable condition for changing a discrete mode, such as confirmed contact or a timeout; activation is associated with a defined response.

Reset. A specified model or controller state change at a mode transition, such as clearing a timer and initialising a new support point.

Support polygon. The convex hull of selected contact points in a planar model; its applicability depends on assumptions about surface and forces.

Static stability margin. The minimum signed distance from the centre-of-mass projection to the support-polygon boundary, positive inside. Its use is limited to a planar quasi-static model.

Duty factor β . The fraction of a cycle a leg spends in planned stance; a dimensionless quantity between zero and one.

Surface normal. A unit vector perpendicular to a local plane; fix its direction and coordinate frame to prevent sign reversal of the estimate.

System rank. The number of independent data or constraint directions; insufficient rank means the selected plane parameters cannot be determined uniquely.

Slip. Relative tangential motion between a contacting foot and surface; a contact sensor may remain active during slipping.

Active levelling. Changing leg commands to reduce body-orientation error while respecting reachability, contacts and load limits.

Load redistribution. Changing forces among stance legs during movement or failure; additional contacts do not automatically equalise forces.

Bus utilisation. The fraction of available channel time occupied by communication, including overhead, pauses and retries; distinct from useful data throughput.

Specific energy E/d . Energy used per unit distance travelled, usually J/m; comparison requires identical conditions and a non-zero measured distance.

Relationships: $\beta = T_{\text{stance}}/T_{\text{cycle}}$; $E = \int U(t)I(t)dt$. Check: verified 120-second standing characterises the completed trial, not every possible duration.

Six-legged robot purchases and setup

USE EXISTING EQUIPMENT

Use the accepted M22 quadruped with 12 working actuators and 2 separate spares, onboard computer, MCU, IMU, camera, bus, instruments and support rig. For the existing 3D printer, choose the exact-model profile; print new parts to the six-legged-frame revision.

BUY BEFORE EXPANSION - AFTER M22 PASS

6 additional working actuators from the same verified family and **2 additional spares**. This gives **18 installed + 4 separate spares** if the earlier spares are unused. Update inventory when replenishing used spares. The M7 actuator rig is excluded from this count.

1 set of parts for two legs and a new or reinforced frame, bearings, inserts, fasteners and harnesses, with quantities from released CAD and BOM. If retaining M22 contact-sensing feet, add **2 channels for two feet**: $4 + 2 = 6$ total. Without the original four, select a separate six-channel set after testing one foot.

OPTIONAL OR SUBJECT TO CALCULATION

Recalculate power, thermal behaviour and communication for the new mass and 18 devices. Replace battery, BMS, charger, power disconnection, DC-DC converter, cables and protective devices only when they fail the measured budget; upgrading the battery alone is insufficient. Ratings depend on peaks and their durations, voltage drop and interruption currents, not the ratio 18/12.

Check the existing rig's load capacity and onboard-computer cooling. A second computer, CUDA GPU and new actuator family are optional. Hardware tripod operation requires a separate check of that mode after wave and ripple; do not expand while on HOLD.

INSTALL NOW

No new software stack is needed: pinned Ubuntu 24.04, ROS 2 Jazzy, `ros2_control`, NumPy and SciPy. If diagnostics are not connected, add `diagnostic_updater` and `diagnostic_aggregator` from [ROS diagnostics](#) for your version; check dependencies with `rosdep`. Retain the analysis and replay environment on the Mac and in UTM.

INSTALLATION AND CONFIGURATION CHECK

Before applying torque: check 18 unique IDs, 18 correct URDF joints, the software simulator, replay with a missing node and displayed diagnostics. Then measure communication cycle and stopping on the support. For terrain, check degenerate point sets, [REP-103 units and axes](#) and [SciPy proportion_ci: method, confidence_level](#) using the log of all attempts.

Preparation and checks fit within the existing 75 hours, replacing repeated reading and setup. Delivery waiting time does not count as study time.

Motion fundamentals: support transitions and equilibrium

Required analysis: support changes, body acceleration and actual contact

Walking changes both leg angles and the set of active contacts. A lifted foot no longer supplies support force; count a new support only after confirmed touchdown. The gait timer specifies intent; the sensor confirms the physical transition. Even six functioning legs do not guarantee stability when these transitions occur in the wrong order.

From static projection to a moving body

For slow standing on one horizontal plane, check that the centre-of-mass projection lies inside the contact polygon, friction is sufficient and loads are admissible. Under acceleration, this is insufficient. In a simplified planar model with constant height h and zero rate of change of angular momentum about the centre of mass: $x_{CoP} = x_c - (h/g) \cdot a_x$. CoP is the point of application of the resultant normal reaction; it must lie in the admissible support region.

Numerical counterexample. $x_c = 0.03$ m, $h = 0.25$ m, $g = 9.81$ m/s²; after a support transition, the admissible longitudinal CoP interval is $[-0.08; 0.12]$ m. At rest, $CoP = 0.03$ m. Requesting $a_x = 5$ m/s² gives $CoP = -0.09742$ m: the centre-of-mass projection is inside, but the required reaction lies outside. This model rejects the command. This is a calculation test, not an instruction to accelerate the physical robot.

For $\mu = 0.6$, the aggregate friction constraint gives $|a_x| \leq \mu g = 5.886$ m/s² when vertical acceleration is zero. It separately permits 5 m/s², but the CoP condition is already violated. Both conditions and actuator constraints must hold. A longitudinal interval does not check lateral stability; use complete contact geometry for the robot.

Practice: verify a wave → ripple transition

In the existing simulation scenario, select the instant before leg lift-off. Construct current and next support regions using confirmed contacts only and calculate static margin. Then estimate required CoP under the declared assumptions; for the general case, use M20 force constraints. Inject loss of one contact: defer swing or select a previously verified fallback.

Save `contact_transition.csv` and support diagrams: phase commands, touchdown confirmations, x_c , CoP estimate, boundaries and decision reason. Separately test delayed swing and a false early touchdown. Increasing duty factor provides more time in stance, but does not establish that every transition is feasible or friction sufficient.

Within 75 hours: 1 hour from 23-24 and 2 hours from the simulation portion of 31-34, tasks 3-4. Physical trials remain within the accepted M22 region. Read Russ Tedrake, [Highly-articulated Legged Robots: CoP and ZMP](#) - assumptions of simplified dynamics. Do not apply this formula to contacts at different heights or large body rotations without checking its validity.

Completing the programme: verified results and release

M24 • 02 Nov 2028 - 27 Nov 2028 • 75 hours

The M24 objective

Bring the learning outputs together into an engineering project: verified requirements, reproducible data, versioned code, CAD and configurations, a technical report and a defence. The original plan's outcome is **open release v1.0, a technical report and project defence**. This month, prepare a release candidate whose results can all be checked. The owner decides on public publication separately after reviewing the contents and comments.

The principal devices are the M23 six-legged robot and the robotic hand from the separate workstream after M17. Each retains its own requirements, limits, calibrations and evidence. A shared portfolio index links their software components but does not substitute one project for the other. If either device has not passed mandatory acceptance, state its exact status and completion blockers; presentation does not turn an unfinished device into a finished one.

What to gather before the first session

Prepare M1-M23 outputs: source code, CAD, drawings, circuits, calibrations, configurations, simulators, tests, datasets, failure logs and alternative solutions. Verify that files exist, rather than relying on links in an old report. For the six-legged robot, obtain standing, wave and ripple, slope, failure-response and energy data. For the hand, obtain evidence for the grasps, force and current limits, sensors and responses actually included in its OPT project.

Do not begin the month by rewriting the entire architecture or changing documentation tools. Select a pinned working version, describe its boundaries and fix problems that prevent verification of claimed functions. Your software-development experience helps with automation, but the number of unit tests cannot replace physical validity.

What constitutes a professional result

Every important claim traces to a requirement, procedure, configuration, raw measurement and reproducible calculation. State separately what simulation, rig tests, the physical machine and external review establish. Include unsuccessful approaches and limitations in the report: when causes and conclusions are justified, they demonstrate engineering judgement.

M24 ends on 27.11.2028; the original calendar then schedules G3 for 27.11-10.12.2028 and G4 for 10-22.12.2028, 35 hours each. These hours and the hand's separate 120 OPT hours are outside M24's 75 hours. By 22 December 2028, both systems must have an explicit final status. Study hours and a portfolio alone do not confer a degree, senior-engineer qualification or research mastery.

75 hours: verification campaign, documentation and defence

Time allocation

Hours 1-20: 10 hours on requirements, metrics and the verification plan; 10 hours auditing data and conducting the first reproducible runs. **21-40:** 10 hours on safe states, failures and release contents; 10 hours on repeated checks, versioning and clean reproduction. **41-60:** 10 hours on technical explanation and external-review preparation; 10 hours on the report, diagrams, defence and review comments. **61-75:** 5 hours choosing a professional focus and final criteria; 10 hours closing comments, checking the portfolio and preparing the G4 handoff.

Total: 35 hours of theory and analysis and 40 of practice. The first three theory blocks each contain five 2-hour sessions; the last is 2 + 2 + 1. Each 10-study-hour practice block includes setup, testing, processing, corrections and evidence recording. The 30-45-minute external meeting is included in the third practical, not added to the budget.

Organising the package

Create one index covering requirements, configurations, tests, raw data, calculations, report, instructions, comments, release and portfolio. Give requirements stable IDs, such as HEX-STANCE and HAND-GRASP, and configurations unique revisions. Do not link evidence only to a filename such as final.csv: include run_id, file hash and system version. Large data need suitable storage with integrity checks; DVC is useful if it already fits the project.

The first-page dates define the original plan's window. Shared boundary dates, holidays and separate G blocks do not create extra hours. Place numbered work within available sessions without double-counting. Use time awaiting a reviewer's response for independent tasks; external review is incomplete until a response arrives.

Priorities when time is short

First resolve invalid measurements and untested mandatory functions, then an unreproducible build, then ambiguous instructions and reviewer comments. Website design, complex animation and an additional demonstration video come last. A narrower verified feature set may be released with explicit limitations; lowering a mandatory criterion after failure requires declaring the requirement change.

Keep two lists: v1.0 release blockers and development after the programme. For each new feature, determine whether it is needed to verify behaviour already claimed. If not, add it to the future specialisation. This preserves time for both physical projects and final integration.

Requirements, metrics and links to measurements

What to study

Distinguish verification, checking fulfilment of a stated requirement, from validation, checking that the system solves the intended task under real conditions of use. A specific trial checks 120 seconds of standing; assessing whether the robot makes repeatable experiments convenient also requires instructions, preparation and user evaluation. Select a method for each requirement: analysis, inspection, demonstration or testing, sometimes in combination.

A good requirement identifies the object, action, conditions and criterion. Replace low latency with the events between which the interval is measured, the clocks used, the load and the limit. Derive the limit from project dynamics, frequency and loading; a round number alone is not justification. A specification citation identifies the page, units and applicable conditions.

Task 1. A requirements matrix for both systems

Inputs are the six-legged-robot objectives, hand objectives and actual release candidates. 1) Break broad claims into verifiable requirements with stable IDs. 2) For each, specify origin, method, criterion, admissible configuration and required log. 3) Assign PASS/FAIL/NOT TESTED/NOT APPLICABLE with an explanation. 4) Verify that every feature on the portfolio homepage appears in the matrix.

Save requirements_traceability.csv. Check that every mandatory requirement has a specific procedure and result source; NOT TESTED is not a pass. Boundary case: after actuator replacement, an old calibration and test remain linked to the new assembly. Update links and repeat affected checks. There is no need to retest an unchanged M1 parser merely to increase the test count.

Task 2. Audit metrics and units

Calculate latency as $t_{\text{end}} - t_{\text{start}}$ only after establishing clock relationships. Define jitter explicitly, for example as deviation of cycle period from its target; show its distribution, p50, p95, p99 and observed maximum with sample count. For angular error, state conventions and units. Obtain energy by integrating $U \cdot I$ over time; use E/d in J/m only for a valid non-zero distance. For the hand, define grasp, holding and release success separately.

Save metrics_spec.md and a calculation script applied to one reference dataset with a known answer. Check that values and units agree across code, table and plot; do not present small-sample quantiles as precise estimates of rare extremes. Boundary case: a current-recording interval is missing. Mark energy incomplete or use an explicitly justified estimate with an interval; do not fill the gap with zero.

Independent stopping, failures and recovery

What to verify

Verify the final physical state, not only a software enumeration changing to STOPPED. Removing torque may make a walking robot fall; for a hand, it may release or trap the object depending on mechanics. Each system needs a described path for normal stopping, emergency termination of force or torque application, and recovery. Support and motion restraint come from the design verified on earlier rigs.

Independence applies to specific failures. If a hardware path depends on the same board's power or firmware, show that in the circuit. A second thread in the same process does not make stopping independent of process failure. Test in the intended protected configuration with an observable physical indication of the result and a criterion established in advance.

Task 3. Reconfirm the final state

Use the final six-legged-robot and hand revisions. 1) Trace command and power paths on the circuit. 2) Prepare robot support and a safe test object or fixture for the hand. 3) Repeat normal stopping and the independent emergency path with the programme operating normally. 4) Repeat the permitted rig case with the control process stopped. 5) Check button release and restored communication: the previous command must not resume automatically.

Save `safe_state_evidence` for both devices with a timeline and observation of the mechanism. Acceptance: each claimed path actually reaches the described state within justified time and displacement limits. Boundary case: the log reports deactivation but the actuator still applies force or torque. The test is FAIL; investigate the hardware path and repeat. Software confirmation does not replace measurement.

Task 4. Failure, diagnostics and recovery

From the risk register, choose sensor loss, stale telemetry, a stopped node and a configuration error. Check them first in replay or HIL. For each, record stimulus time, detection, restricted behaviour and operator action. Physically reproduce only permissible variants using the established procedure; destructive battery failure and short circuits are unnecessary for verifying software responses.

Save `fault_recovery_matrix` and a minimal regression set. Check that after a node recovers, the system rechecks calibration, mode and data freshness before authorising motion. Boundary case: different errors need different responses, but one reset button hides both. Separate confirmation that the cause is resolved from permission for new motion. Retain failed cases and the rationale for protective behaviour.

A reproducible code, CAD and data package

What reproducible means

Another engineer must receive consistent source code, hardware documentation, configuration and sufficient data to reproduce the main results. Code, CAD, circuit and calibration have separate versions linked by a shared release manifest. An STL without its source parametric model and dimensions does not explain the design; a binary without its configuration version does not define controller operation.

Distinguish reproducing calculations from saved data from conducting a new physical experiment. The former can be checked in a clean environment without the robot. The latter requires an assembly, equipment, calibration and admissible conditions. Instructions must state which results readers can obtain without hardware and which are confirmed only on your unit.

Task 5. The v1.0 candidate manifest

Inputs are the pinned revisions of both systems. 1) List source code, CAD, circuits, BOM, calibrations, dependencies, tests, data, documentation and demonstrations. 2) State each item's version or hash and purpose. 3) Separate raw data from derived plots. 4) Check for dependencies on local paths and ensure required external files are accessible through the instructions.

Save `release_manifest.json` and a local candidate archive. Check that every link resolves, checksums match and release notes describe this exact version. Boundary case: calibration comes from a different hand despite matching file format. Include the physical unit's serial identifier and revision and add a compatibility check. A filename does not establish correct configuration.

Task 6. Reproduce in a clean environment

Create a separate working copy or isolated environment, restore pinned dependencies and only the dataset included in the candidate. Follow the instructions from the start: build, software tests, selected-log processing, table and plot generation. Documentation scripts must not automatically connect powered hardware. Hardware instructions require a deliberate mode choice and entry checks.

Save `reproduction_log.md` with commands, environment, runtime and omissions found in the instructions. Check that at least one key plot for each of the six-legged robot and hand reproduces the same substantive result within the stated tolerance. Boundary case: a notebook displays an old saved-cell plot while calculation fails. Clear saved outputs, run from scratch and fix the dependency. A missing raw-data file blocks the claim that relies on it.

For M20-M21 results, include the QP configuration, solver-failure check, unchanged classical baseline, random-number generator seeds and the OOD/fallback report. Identify learned components investigated only in simulation. Increased reward does not verify physical operation; explicitly record lack of hardware authorisation in the report and portfolio.

Technical report and external project defence

What you must be able to explain

Organise the report around the problem each system solves: objective, requirements, architecture, methods, experiments, results, limitations and future work. In the architecture, show physical components, command flow, measurement flow and power circuit. A separate timing diagram explains the measurement-to-action path; a state diagram explains motion authorisation, stopping, failure and recovery. Do not combine everything into one unreadable diagram.

In methods, describe procedure, instruments, calibrations, initial conditions and processing. In results, separate measured facts from hypotheses. Each plot has units, a caption, version and attempt count; photographs show the fixture. Limitations must name surfaces, loads, durations, uncertainties and untested modes rather than offer a generic paragraph about future potential.

Task 7. A report that stands on its own

1) Write a half-page account of each system's result. 2) Draw three diagrams: architecture; timing; states and stopping. 3) Select 4-6 main plots that substantiate requirements. 4) Trace each back to the raw file. 5) Analyse one unsuccessful design choice and one trade-off, such as speed versus heating or grip force versus protecting the object.

Save technical_report and diagram source files. Check that a reader can identify the operating envelope and locate data supporting the main claim within a few minutes. Boundary case: body-tilt and surface-slope plot axes share the same label. Correct notation and caption, then recheck the text. Every plotted quantity must have an unambiguous meaning.

Task 8. A 30-45-minute external review

Prepare a compact package in advance: objectives, diagrams, requirements matrix, one successful trial and one failed trial. Choose someone able to assess at least part of the mechanics, control or software system. During the meeting, allow 5 minutes for context, 10 for architecture and limitations, 10 for evidence and 5-15 for questions. Record each objection, affected requirement, required action and closure method.

Save design_review_minutes with the comments actually received. Check that at least one technical objection is addressed substantively and the response is supported by data or an explicit limitation. Polite praise does not replace engineering review. If the person has not replied, status is pending; you may rehearse independently, but cannot call it an external defence. The owner deliberately sends invitations and grants access; no automatic mailing is required.

A shared test campaign for two devices

Specification of the integrated work

Create campaign.yaml with conditions and run_id values. For the six-legged robot, test standing, wave and ripple, levelling, restricted operation without a camera and stopping. For the hand, test the project's intended grasps, holding, release, detection of an unreliable sensor and failure response. If the hand is not mounted on the body, do not claim integrated mobile manipulation: these are two completed devices sharing a verification method.

Freeze version, success definition and trial unit first. For the walking robot, the unit is a complete bounded scenario with restored initial conditions; for the hand, a specified cycle on a known object with defined position and load. Reprocessing one file checks calculation reproducibility; it does not add physical trials. A different hardware revision starts a separate result group.

The 30-50+ series and limits of inference

The overall plan suggests 30-50+ tests or a justified sample size. Plan the sample around the claim: comparable attempts to estimate success rate and separate boundary conditions for coverage. M23 data may count if configuration, procedure, conditions and criteria match the final ones; this reuses evidence rather than adding newly conducted experiments. Define the hand's series separately, without pooling grasp successes with walking.

An example arrangement is 40 attempts for one frozen six-legged-robot task and a separately agreed grasp series. If time is insufficient, narrow the claim and justify N, or move the open block to G4 within the overall schedule. Do not promise a reliability percentage from an arbitrarily small sample. Every mandatory mode retains its test status even if excluded from the main homogeneous series.

Execution and calculation

First check instruments on a known signal and run a pilot trial. Then follow a fixed order or alternate modes to control for discharge and heating. After every attempt, record outcome, initial conditions and interventions. Classify failures as mechanics, power, communication timing and bus, perception, control, procedure or data. Categories support repair but do not remove failures from the final table.

For success rate, report k/N, the confidence-interval method and applicability conditions. If heating creates serial dependence, do not call individual cycles independent; analyse blocks or restrict conclusions to descriptive statistics. For continuous metrics, show the distribution across attempts, units and actual observation duration. Observed maximum current and the hardware's admissible maximum are different numbers from different sources.

Tests of the evidence package itself

Six required scenarios

T1 - a different configuration. In a test copy, substitute calibration from another unit: hardware activation must be refused or the report must flag incompatibility. **T2 - corrupted raw data.** Change one byte or row in a copy: integrity checks or the parser must detect the problem and must not present the result as unchanged.

T3 - a missing time interval. Delete part of a log copy: duration, energy or latency calculations must indicate incompleteness. **T4 - an old notebook plot.** Generate the report after clearing saved cell outputs: the plot must be regenerated from data rather than retained from a previous version. **T5 - control-process failure.** On an admissible rig, repeat independent stopping and prevention of automatic resumption.

T6 - reproduction without undocumented local dependencies. In a clean copy, run the claimed recorded-data analysis without hardware: instructions must work with the specified files rather than an undocumented author directory. Explicitly identify any access requirement and provide an accessible example dataset. Save expected outcome, actual result and package version for each test.

Checking whether claims are justified

Take five main claims from the README and CV. Trace each chain: claim → requirement → test → configuration → raw data → calculation. If a link is missing, restore evidence, narrow the claim or remove it. A matching trial supports 120-second standing in this configuration; it does not support stable operation in every condition. A claim to have developed a controller and compared it with a baseline must identify your own contribution and comparison conditions.

Distinguish certification, educational verification and operational authorisation. Your tests establish specific project requirements; mentioning safety does not make them official certification. Name completed checks directly: independent disconnection, command limiting and response to data loss. Do not claim more than the methods support.

Closing defects

For every defect, record reproduction, consequences, correction and affected requirements. After changing the system, repeat the smallest set that demonstrates the correction and absence of related regressions. A caption edit does not require rerunning the entire two-year plan; a power-circuit or calibration change requires the corresponding physical check.

The change log identifies results belonging to rc1, rc2 and the final version. Label earlier-version data retained as comparisons. Release with a known limitation is acceptable only when it does not conceal an unmet mandatory requirement for a claimed function and is clearly documented in the instructions.

Hours 1-40: requirements, testing and reproduction

Hours 1-10: analytical preparation

1-2: inventory results, check both devices' status and choose test candidates. 3-4: requirements, verification versus validation and the start of task 1. 5-6: definitions of latency, jitter, energy, errors and grasp success from task 2. 7-8: sampling plan, failure classification and compatibility of earlier data. 9-10: trial order, stopping criteria, instruments and run-description fields.

By the end of hour ten, prepare campaign.yaml and the requirements-status table. Assess remaining work by open rows rather than plot count. If the physical hand is missing, this is a separate blocker to overall programme completion: M24's 75 hours are for verification and presentation, not an unannounced replacement of its 120-hour construction work.

Hours 11-20: the first practical

11-13: task 1, traceability checks and raw-file audit. 14-15: task 2, reference inputs and reproduction of main metrics. 16-18: pilot trials and planned campaign repeats on verified configurations. 19-20: failure classification, observation register and a decision on remaining data needs. Do not run a new physical mode merely because old data are incomplete; prepare its procedure first.

Some earlier-module results may be unusable because timestamps are absent or configuration changed. Do not retrospectively alter raw data. Retain the original, state the reason and collect new measurements or narrow the claim. Synthetic test data are useful for checking calculations but do not replace physical evidence.

Hours 21-40: stopping and the package

21-22: paths to final states for both systems and independence from specific failures. 23-24: recovery, stale commands and reactivation. 25-26: release contents and the links between code, CAD, firmware and calibrations. 27-28: data versions, checksums and a clean environment. 29-30: release notes and reproduction instructions without undocumented assumptions about the author's environment.

31-33: task 3, verify stopping of both systems in suitable fixtures. 34-35: task 4, failure and recovery in HIL or on the rig. 36-37: task 5, manifest and local rc1 archive. 38-40: task 6, clean reproduction and instruction fixes. The deliverable is a package a reviewer can use without spoken guidance at every step. Grant others access deliberately after selecting the package contents.

Hours 41-75: defence, corrections and release readiness

Hours 41-50: explaining the engineering result

41-42: report structure and each system's main claim. 43-44: architecture and paths for energy, commands and measurements. 45-46: timing diagram and failure/recovery states. 47-48: presenting results, uncertainties and unsuccessful approaches. 49-50: questions for the external reviewer and defence rehearsal. Each session must improve a specific part of task 7, not merely increase text length.

Arrange the external review with the chosen person in advance. Prepare a small package and a precise question in their area of competence. If the meeting moves, continue the report and independent tests; external-review status remains open. Correspondence and invitations are deliberate actions by the project owner; the plan does not generate them automatically.

Hours 51-60: report and review

51-54: task 7, report, plot captions and three diagrams. 55-56: task 8, the 30-45-minute meeting, comments and clarified actions. 57-58: reproduce one review issue from data, fix a significant problem or document a limitation. 59-60: repeat affected tests and update rc2. If the reviewer is unavailable, use these hours for independent preparation and rehearsal; move the actual external review to available time in the overall plan with explicit hour accounting.

Distinguish correction from response: saying addressed does not close an issue if the matrix lacks a new result. A comment may change a requirement, but retain the previous wording, reason and new status. This matters particularly for hand loading, admissible slope and claimed autonomous runtime.

Hours 61-75: presenting professional results

61-62: choose a work direction and map competencies to completed work. 63-64: select strong CV evidence, interview questions and the next technical gap. 65: check exit criteria and the G4 list. These 5 analysis hours are split into 2 + 2 + 1.

66-68: task 9 on page 11, portfolio index and CV adaptation. 69-70: task 10, professional feedback or a prepared open-source change with a test. 71-72: final rc2 reproduction and link/package checks. 73-74: close objections, agree limitations and plan G4. 75: decide RELEASE READY/HOLD and prepare the publication package. Public publication is a separate deliberate action after contents review; automatic deployment from a study script is unnecessary.

G3 after M24 is a separate final audit of perception and SLAM, planning recovery, manipulation, ML data and learned-model deployment. Its late date does not retroactively authorise learned-policy hardware control or a Jetson purchase. Retain the classical controller until technical checks pass, and explicitly identify learning tested only in simulation.

Acceptance, professional results and the separate G4

Task 9. A portfolio for the chosen role

Choose one direction: robotics software, embedded-systems and control integration, or machine perception. Match typical work in that direction to your own contributions: driver, estimator, controller, experiment, CAD integration or data processing. For each strong CV point, state a concrete action, conditions, measured result and link to the work. If a result is not yet available, do not invent an improvement percentage; describe the implemented and verified function.

Save portfolio_index and a CV version with 3-5 substantiated technical points. Check that every point has a file, test or documented decision behind it and makes your own contribution clear. Boundary case: a ready-made ROS package was used, but the wording credits you with its whole algorithm. Separate integration, your modifications and the third-party component. Prepare 10 interview questions from actual project difficulties and answer them using data.

Task 10. Feedback and the next focus

Choose one small useful change to an open-source project you use, or obtain additional professional feedback on your result. For a change, prepare issue reproduction, the change and a suitable test or documentation; submit separately after owner review. For feedback, save technical comments and actions. Do not make completion of the month depend on a maintainer accepting the contribution; state its actual status.

Save contribution_or_feedback.md and an 8-12-week specialisation plan: one gap, one project, a result criterion and time limits. Check that the direction follows an identified weakness or the chosen role. Boundary case: ten new technologies are listed without a practical question. Reduce this to one next verifiable result, such as improved contact observability or hand-force calibration.

M24 exit criteria

Complete the requirements matrix, valid metrics, raw-data package, failure regression checks, setup and maintenance instructions, versioned candidate, technical report and comments actually received from an external reviewer. Label the release READY only when mandatory requirements for the claimed operating scope are closed. Otherwise retain HOLD with a concrete action list. The release draft and index can be fully prepared before public publication.

Pass to the separate G4 the completion of test tables and statistical analysis, independent E-stop and final-state verification, external review with corrections, release archive and future plan. Its 35 hours comprise 7 for diagnostics, 12 for integration, 9 for failure checks and corrections, and 7 for review and decision. PASS permits continuation; CONDITIONAL requires a named correction and responsible person; FAIL means repeating failed work and revising the calendar. Do not silently transfer the hand's OPT work here. The final target for both systems is 22.12.2028; a date does not replace evidence.

Assigned reading: engineering release

The references may be in English; the exercises and definitions are presented here in English. Read only the named sections before the associated work, within the existing 75 hours.

Core reading: three guides

- 1. NASA - Systems Engineering Handbook.** Appendices C, D, E: requirement wording, verification and validation matrices. Then 5.3-5.5: Verification, Validation, Transition. For tasks 1 and 3-6, link requirements, procedures, results and the handoff package for both devices.
- 2. S. Chacon, B. Straub - Pro Git, 2nd ed.** 2.6 Tagging; 7.12 Bundling. For task 5, freeze the exact version and verify repository restoration. Reference: 7.4 Signing Your Work; a signature and verifiable identity differ from a matching hash alone.
- 3. NIST/SEMATECH - e-Handbook of Statistical Methods.** 7.2.4.1 Confidence intervals. For the test campaign, study a success-proportion interval, small samples and the edge cases 0/N, N/N. Record assumptions and check the calculation against the log of all attempts.

Documentation for the selected tool

pytest. How to use fixtures: teardown/cleanup. For package checks, eliminate dependence on test order and leftover files. Automated reproduction tests must not energise physical motors.

Sphinx. Getting started: sources, document structure. For the report, connect the index, instructions and API. Use only if it is already the chosen generator; changing a working stack at programme end is unnecessary.

DVC. Get Started: Tracking data, Retrieving data, Switching between versions. Reference for task 5 if using this data-storage method: restore the correct-version raw data into an empty directory. A .dvc file without accessible data does not complete the archive.

Reading deliverable

Create one matrix: claim → requirement → procedure → raw data → calculation → decision, and reproduce a selected result using the instructions without the old environment. Tests of your assembly establish physical limits; the v1.0 label does not.

Terms and definitions

Requirement. A verifiable mandatory system property with unambiguous conditions and a criterion; a wish without an evaluation method is not a complete requirement.

Acceptance criterion. A predefined PASS/HOLD/FAIL rule for a requirement, including limit, conditions, procedure and required evidence volume.

Verification. Obtaining evidence that a stated requirement is met; methods may include analysis, inspection, demonstration or testing of a specific configuration.

Validation. Obtaining evidence that a system suits the declared application under expected conditions; distinct from compliance with an internal specification.

Traceability. Linking a claim to a requirement, implementation, procedure and result, allowing the effects of changes and the source of conclusions to be checked.

Evidence package. A consistent set of requirements, configurations, procedures, raw data and calculations sufficient to verify specific project claims.

Calculation reproduction. Obtaining a result again from specified data, code and parameters in a documented environment; allowable numerical differences are declared in advance.

Raw data. Primary saved measurements before subsequent processing; they must include units, time, source and recording conditions.

Data provenance. A record of data sources, acquisition conditions and transformations, linking a final metric to the original measurement and processing versions.

Release manifest. A machine-readable list of files with purposes, versions or checksums, used to verify that release contents are complete.

Content hash. A deterministically computed data fingerprint for detecting change against a trusted value; it does not by itself establish authorship or measurement validity.

Digital signature. A cryptographic link between a message and a private key; attribution to an author also requires a trusted association between the public key and identity.

Regression check. Rechecking previously verified behaviour after a change to detect unintended deterioration in the covered scenarios.

Fault injection. Deliberately reproducing a specified fault in a controlled test to check detection, response and the system's final state.

Confidence level. The proportion of intervals covering the true parameter when the procedure is repeated under its assumptions; not the probability that the parameter lies inside an already computed interval.

Selection bias. A systematic difference between attempts included in analysis and the target population, such as excluding failures; increasing N does not automatically remove it.

Relationship: estimated success proportion $\hat{p} = k/N$; also report N, interval method and trial conditions. READY status does not replace this information.

Preparing for release and G4

USE EXISTING EQUIPMENT

Use the MacBook Pro M4, working Ubuntu 24.04 environment with ROS 2 Jazzy, onboard computer, M23 six-legged robot, tools and instruments. For the hand, use the assembly actually built and data from the separate OPT 120 workstream; record their status independently of leg success.

PURCHASES NOW

No new mandatory equipment. Consumables and replacements for defective parts may follow the fault log and existing BOM; additional actuators, a new camera or GPU are unnecessary for release. Use existing storage with verified restoration for the archive.

INSTALL NOW

No new mandatory stack: restore pinned Git, Python, pytest, SciPy and the selected documentation generator. Prepare the report on macOS ARM; run ROS tests in the existing Ubuntu ARM64 environment or on the agreed onboard x86 platform. Record the architecture for every reproducible result.

If the report already uses [Jupyter Book](#), install the pinned version and follow Execute & Compute and Export in the matching documentation without mixing generations. DVC and Sphinx remain conditional components of the chosen workflow; the existing storage and build methods may be retained.

INSTALLATION AND RESTORATION CHECK

In a clean directory, restore the Git-tagged version and data, install dependencies from the lock file, and run regression checks and replay with hardware-command output disabled. Recreate a key table and plot without saved cell outputs. Check [SciPy: proportion_ci](#), manifest contents, raw-data availability and checksums against the trusted manifest.

Open the final PDF, CAD and instructions in another viewer; check links and units. For substantive review, use [Google: Design, Functionality, Complexity, Tests](#). Prepare a [GitHub Release draft](#) with attachments and limitations; publication is a separate owner decision.

HOURLY BOUNDARIES AND FINAL CHECKPOINTS

M24 ends on 27.11.2028 and contains 75 hours. It is followed separately by **G3, 27.11-10.12.2028 - 35 hours**, and **G4, 10-22.12.2028 - 35 hours**. G4 receives verification of both systems, independent stopping, comments and the archive. The hand's OPT has another separate 120 hours; software installation and presentation do not close failed physical acceptance.

Preparation and checks fit within the existing 75 hours, replacing repeated reading and setup. Delivery waiting time does not count as study time.

Motion fundamentals: justifying the robot model

Required synthesis: explain geometry, forces, motion and measurement

By programme end, you should independently reconstruct the calculation chain: foot or finger target → admissible joint coordinates → trajectory over time → required torques → actual actuation → measured motion. Include one such analysis using leg data and one using finger data in the report; reuse completed experiments without creating a new test programme.

A reference example without a robotics library

A planar mechanism's revolute joint is at the origin, with x rightwards and y upwards. A point mass $m=0.40$ kg is attached to the end of a massless link $L=0.30$ m. Angle q is measured from the downward vertical towards +x; neglect friction. For $q=\pi/6$, $\dot{q}=0.40$ rad/s and $\ddot{q}=0.80$ rad/s²: $p=(L \sin q, -L \cos q)=(0.15000; -0.25981)$ m.

Differentiate the geometry: $\dot{p}=(L \cos q, L \sin q) \cdot \dot{q}=(0.10392; 0.06000)$ m/s. Moment of inertia about the joint is $I=mL^2=0.036$ kg·m². Dynamics $I \cdot \ddot{q} + m \cdot g \cdot L \cdot \sin q = \tau_{\text{act}}$ with $g=9.81$ give $0.0288 + 0.5886 = 0.6174$ N·m. The first term accelerates the mass; the second compensates gravity. A link with distributed mass has different I and gravity terms.

Add a vertical environmental force $F_{\text{ext}}=(0;2)$ N at the end. Its torque about the joint is $\tau_{\text{ext}}=p_x \cdot F_y=0.30$ N·m. The equation $I \cdot \ddot{q} + m \cdot g \cdot L \cdot \sin q = \tau_{\text{act}} + \tau_{\text{ext}}$ now requires $\tau_{\text{act}}=0.3174$ N·m. The environment assists motion; do not choose the sign of $J^T F_{\text{ext}}$ from memory without defining force direction.

Four defence questions

Geometry: where do FK/IK and the solution branch come from, and what does the Jacobian lose at a singularity? **Dynamics:** which forces are included, and what changes with mass or speed?

Control: which signal does the actuator actually accept, and why might the requested trajectory be infeasible? **Contact:** what confirms support and grasp, and when does a static model cease to describe the experiment?

Prepare `movement_defense.md`: an axis diagram, notation and units, derivation, numerical hand-calculation results, an independent software calculation, one real log and model limits. Recheck velocity using a central difference of FK, the external-force sign using $r \times F$ and substitution of torque into forward dynamics. This examines connections between modules rather than memorised formulas.

Within 75 hours: 1 hour from 43-44 for the model diagram, 1 hour from 49-50 for rehearsal, and 1 hour from report work in 51-54 for calculation and evidence (task 7). G3, G4 and OPT retain separate hours. Review Lynch, Park, [Modern Robotics §5.2](#) and [Tedrake: Definitions](#); derive this example's equations independently from the sum of torques.